DOP-C02

DEVOPS ENGINEER



COURSE SLIDES · WSQ

DevOps Engineering on AWS

WSQ Course Code: TGS-2023040474

Conducted by Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version v14 · 23 August 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

COURSE ADMINISTRATION

Welcome & Housekeeping

© 2026 Tertiary Infotech Academy Pte Ltd

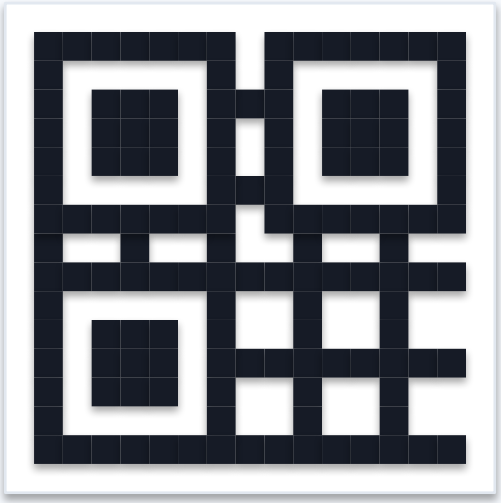
Digital Attendance (Mandatory)

Take the SSG digital attendance at three points on every funded day:

AM	PM	ASSESSMENT
Start of the morning session	Start of the afternoon session	Before the Day 5 assessment

1. The trainer/administrator displays the SSG digital-attendance QR code from the portal.
2. Scan the QR code with your mobile phone camera and submit your attendance.
3. It is mandatory for AM, PM and the Assessment sessions on WSQ-funded courses.

SCAN TO SUBMIT ATTENDANCE



SSG digital-attendance QR — displayed live from the portal by the trainer

Minimum 75% attendance on the SSG Digital Attendance record is required to be eligible for assessment and funding.

© 2026 Tertiary Infotech Academy Pte Ltd

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

© 2026 Tertiary Infotech Academy Pte Ltd

YOUR TRAINER

About the Trainer



Dr Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte Ltd

ROLE
Principal Trainer, Tertiary Infotech Academy Pte Ltd

QUALIFICATIONS
PhD; ACTA/DACE-certified adult educator.

EXPERTISE
AWS cloud architecture, DevOps, CI/CD automation, Infrastructure as Code and cloud security.

DELIVERS
WSQ courses on AWS, DevOps engineering, cloud and software engineering.

FOUNDER
Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

© 2026 Tertiary Infotech Academy Pte Ltd

ICE-BREAKER

Let's Know Each Other

- Your name and organisation / role.
- Your experience with AWS, DevOps or automation (if any).
- What you want to automate or improve on AWS after this course.

© 2026 Tertiary Infotech Academy Pte Ltd

HOUSEKEEPING

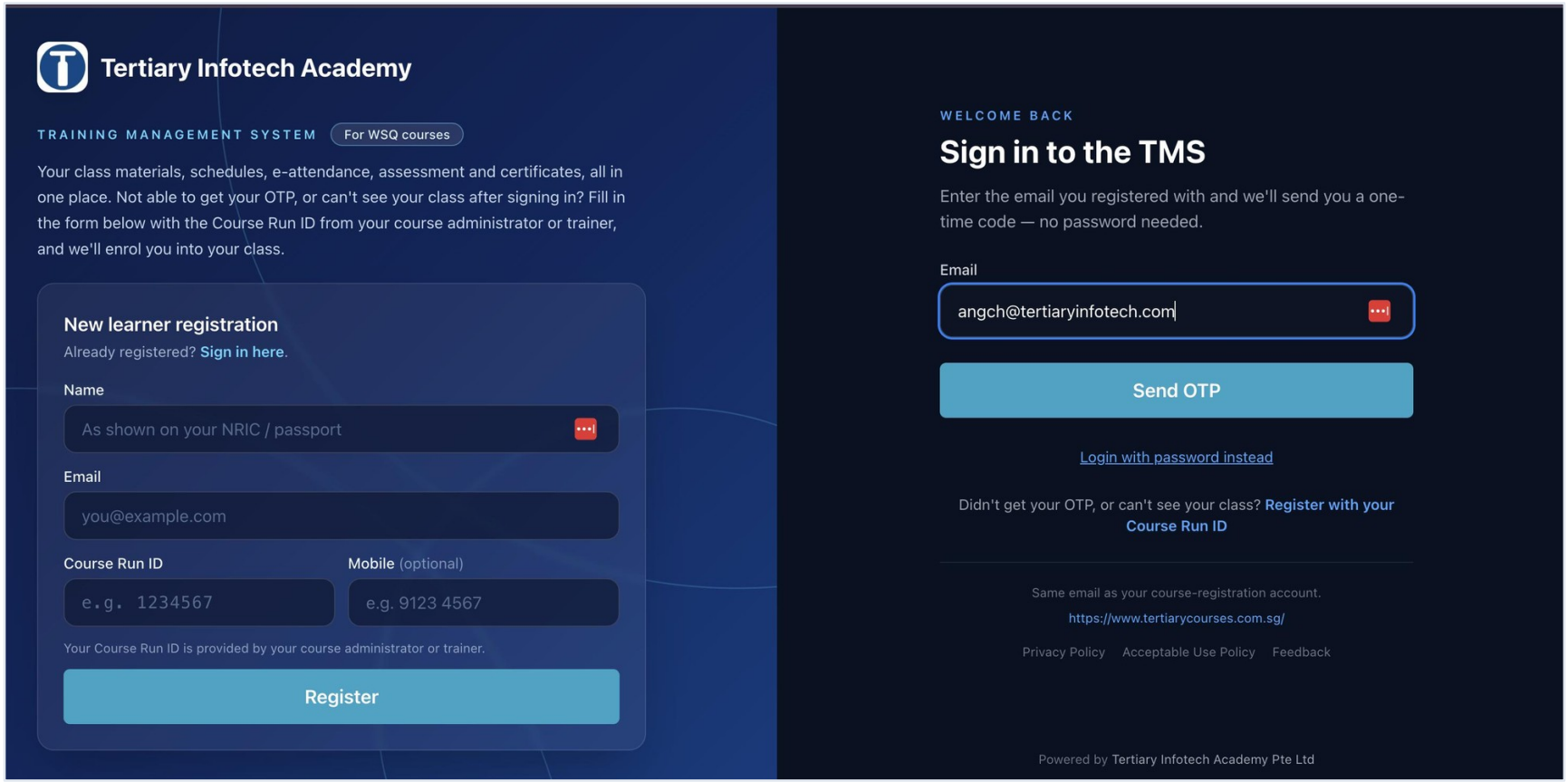
Ground Rules

1. Set your mobile phone to silent mode.
2. Participate actively — no question is too small.
3. Mutual respect: agree to disagree.
4. One conversation at a time.
5. Be punctual; return from breaks on time.
6. 75% attendance is required for funding.

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE PORTAL

Login to LMS and Download Course Material

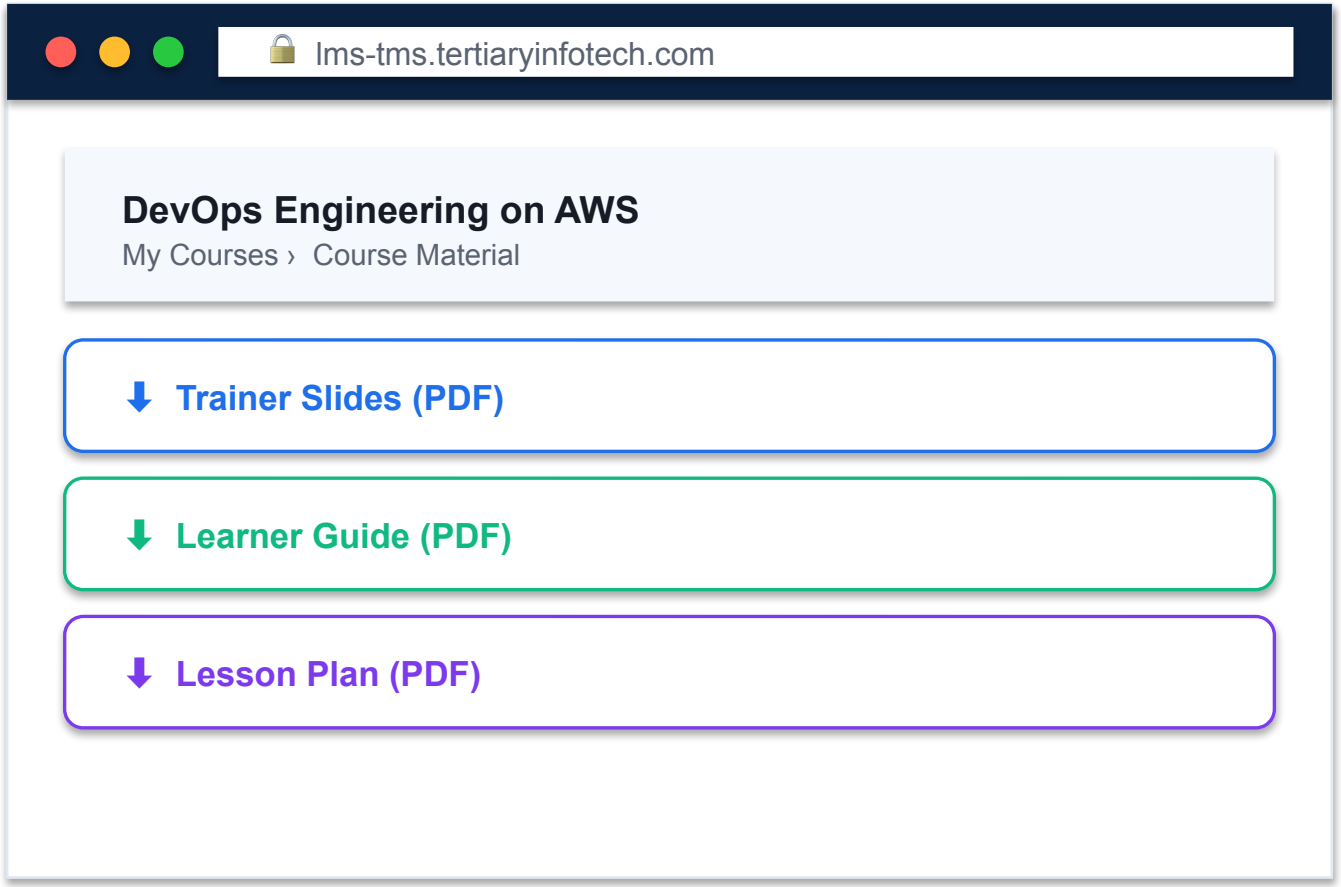


Log in at lms-tms.tertiaryinfotech.com with your registered email and the OTP sent to your inbox, then open this course to download the slides and Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE PORTAL

Download Your Course Material



1 Sign in
Log in to lms-tms.tertiaryinfotech.com with your enrolled account.

2 Open your course
Go to My Courses and select DevOps Engineering on AWS.

3 Course Material
Open the Course Material tab for this course.

4 Download
Download the Trainer Slides, Learner Guide and Lesson Plan for the open-book assessment.

© 2026 Tertiary Infotech Academy Pte Ltd

CRITERIA FOR FUNDING

Funding & Eligibility

1

Attendance

Minimum 75% attendance on the SSG Digital Attendance record.

2

Assessment

Complete the assessment and be assessed as ‘Competent’.

3

Open book

The slides and Learner Guide may be used during the assessment.

4

Course portal

All materials, attendance and assessment are on lms-tms.tertiaryinfotech.com.

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE SCHEDULE · 40 HOURS (37 INSTRUCTIONAL + 3 ASSESSMENT)

Lesson Plan — 5 Days · 5 Hands-On Labs per Day (25 Labs)

1

Day 1
Foundations & SDLC Automation (Labs 1–5)

2

Day 2
Configuration Management & IaC (Labs 6–10)

3

Day 3
Resilient Cloud Solutions (Labs 11–15)

4

Day 4
Monitoring, Logging & Incident Response (Labs 16–20)

5

Day 5
Security, Compliance & Capstone (Labs 21–25) + Final Assessment

6

Daily timing
9:30 am – 6:00 pm · 30-minute lunch · tea breaks within

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL ACHIEVE

Learning Outcomes

1

LO1

Develop an AWS CI/CD pipeline proof of concept.

2

LO2

Identify the technical and practical requirements in an AWS cloud infrastructure setup.

3

LO3

Develop a resilient cloud solution based on AWS cloud components.

4

LO4

Implement regular AWS monitoring and perform necessary troubleshooting.

5

LO5

Demonstrate solutions to incidents and business problems on the AWS cloud.

6

LO6

Prepare a technical blueprint to address security and compliance.

© 2026 Tertiary Infotech Academy Pte Ltd

TSC MAPPING · K1–K7 · A1–A6 · LO1–LO6

Skills Framework Alignment (ICT-DES-4006-1.1)

Domain	Learning Outcome	Knowledge	Ability
01 SDLC Automation	LO1	K1	A1
02 Configuration Management and IaC	LO2	K3	A3
03 Resilient Cloud Solutions	LO3	K2, K4	A2
04 Monitoring and Logging	LO4	K5	A6
05 Incident and Event Response	LO5	K7	A5
06 Security and Compliance	LO6	K6	A4

© 2026 Tertiary Infotech Academy Pte Ltd

BEFORE THE ASSESSMENT

Briefing for Assessment

- Place phones and other materials under the table or on the floor unless the assessment is open book.
- No photos or recording of assessment scripts.
- No discussion during the assessment.
- Use a black/blue pen for hard-copy assessments.
- No liquid paper / correction tape.
- Scripts are collected when time is up.

© 2026 Tertiary Infotech Academy Pte Ltd

FINAL ASSESSMENT

Assessment

- Written Assessment (WA) — Short-Answer Questions (SAQ), 1 hour, open book.
- Practical Performance (PP) — hands-on AWS DevOps tasks, 2 hours, open book.
- Format: Open Book — course slides, Learner Guide and approved materials only.
- Conducted on Day 5 after the training content (3 assessment hours in total — WA 1 hour + PP 2 hours).
- A minimum of 75% attendance is required to be eligible for assessment and funding.
- An appeal process is available if required.

© 2026 Tertiary Infotech Academy Pte Ltd

ON ASSESSMENT DAY

Assessment Flow

1. TRAQOM survey — scan the QR code on the LMS
2. Assessment digital attendance — scan the SSG QR
3. Sit WA (SAQ) then PP — open book
4. Submit your answers on the LMS
5. Sign the Assessment Summary Record

© 2026 Tertiary Infotech Academy Pte Ltd

FOUNDATIONS

DevOps Engineering on AWS

© 2026 Tertiary Infotech Academy Pte Ltd

THE MINDSET

What is DevOps?

1

Culture + practice

DevOps unites development and operations to deliver value faster and more reliably.

2

Automate everything

Build, test, deploy and infrastructure become code — repeatable and reviewable.

3

Fast, safe feedback

Small, frequent changes with monitoring make failures cheap and recoverable.

4

Shared ownership

Teams own their service end to end — you build it, you run it.

© 2026 Tertiary Infotech Academy Pte Ltd

CULTURE FRAMEWORK

The CALMS Model

1

Culture

Collaboration and shared responsibility across dev and ops.

2

Automation

Remove manual, error-prone steps from the delivery pipeline.

3

Lean

Small batches, fast flow, eliminate waste.

4

Measurement

Metrics and feedback drive continuous improvement.

5

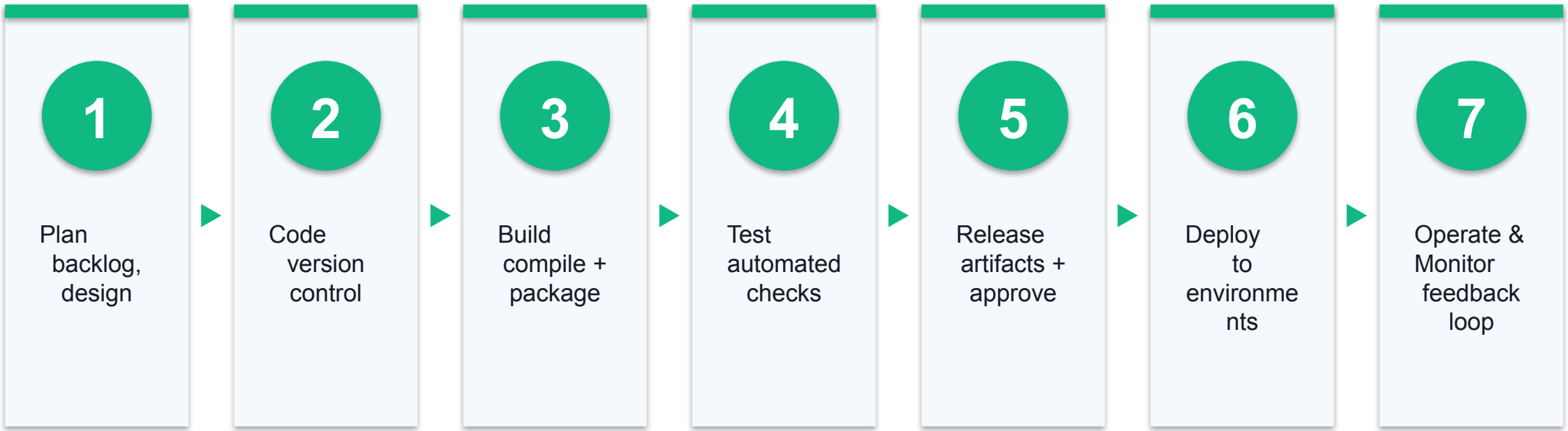
Sharing

Transparency, blameless learning and reusable patterns.

© 2026 Tertiary Infotech Academy Pte Ltd

A CONTINUOUS LOOP

The DevOps Lifecycle



© 2026 Tertiary Infotech Academy Pte Ltd

OUTCOMES THAT MATTER

Measuring DevOps — The DORA Metrics

1

Deployment frequency

How often you ship to production — elite teams deploy on demand.

2

Lead time for changes

Commit → production time — minutes/hours, not weeks.

3

Change failure rate

% of deployments causing a failure — lower is healthier.

4

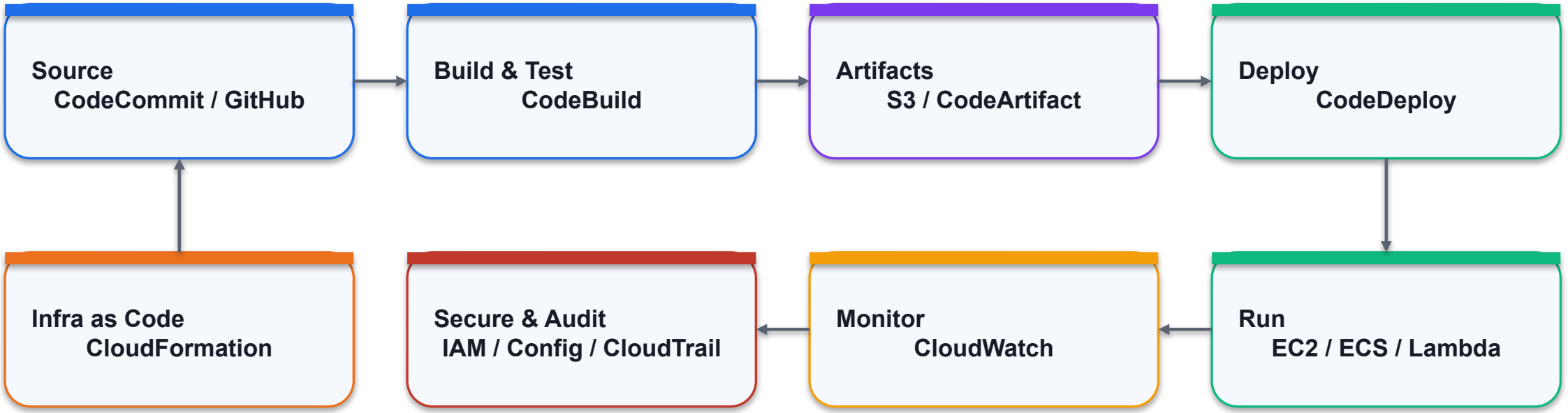
Mean time to recover

How fast you restore service after an incident — minimise it.

© 2026 Tertiary Infotech Academy Pte Ltd

END-TO-END DELIVERY

The AWS DevOps Toolchain



- CodePipeline orchestrates the top row; IaC, monitoring and security wrap the whole lifecycle.

© 2026 Tertiary Infotech Academy Pte Ltd

SIX PILLARS

AWS Well-Architected Framework

1

Operational Excellence

Run and monitor systems; improve processes continuously.

2

Security

Protect data, systems and assets with least-privilege and audit.

3

Reliability

Recover from failure automatically; meet availability targets.

4

Performance Efficiency

Use resources efficiently as demand and technology change.

5

Cost Optimisation

Deliver business value at the lowest price point.

6

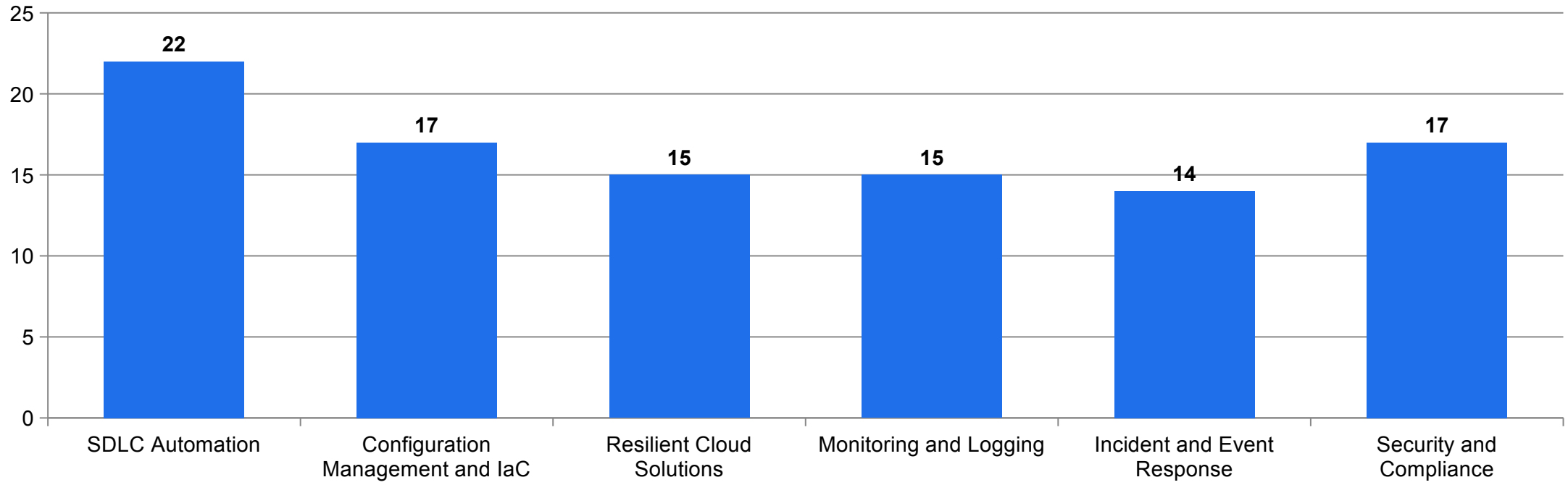
Sustainability

Minimise the environmental impact of running workloads.

© 2026 Tertiary Infotech Academy Pte Ltd

HOW THE COURSE IS WEIGHTED

Exam / Skills Domain Weightings



WHAT THE DATA SHOWS

SDLC Automation (22%) and the two 17% domains — IaC and Security & Compliance — carry the most weight; plan revision time accordingly.

© 2026 Tertiary Infotech Academy Pte Ltd

WHO SECURES WHAT

AWS Shared Responsibility Model

AWS responsibility

- AWS — security OF the cloud
  - Physical data centres & hardware
  - Global network & virtualisation layer
  - Managed-service infrastructure
  - Availability Zones and Regions

Your responsibility

- You — security IN the cloud
  - IAM, identities and permissions
  - OS, network & firewall configuration
  - Application code & data encryption
  - Patching guest OS and dependencies

© 2026 Tertiary Infotech Academy Pte Ltd

THE SERVICES YOU'LL USE

Core AWS Building Blocks

1

Compute

EC2, ECS/EKS, Lambda, Auto Scaling.

2

Networking

VPC, subnets, security groups, ELB, Route 53.

3

Storage & data

S3, EBS, RDS, DynamoDB.

4

Identity & security

IAM, KMS, Secrets Manager, Config.

5

Developer tools

CodePipeline, CodeBuild, CodeDeploy, CloudFormation.

6

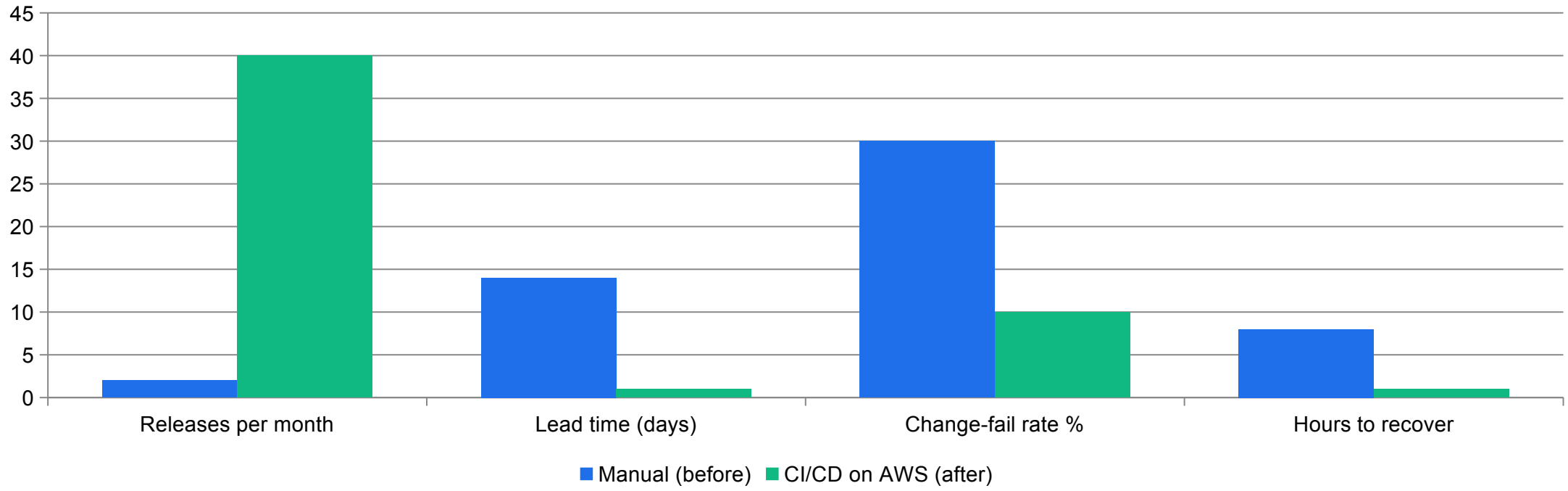
Observability

CloudWatch, X-Ray, CloudTrail, EventBridge.

© 2026 Tertiary Infotech Academy Pte Ltd

THE BUSINESS CASE

Why CI/CD — Release Frequency & Lead Time



WHAT THE DATA SHOWS

Automating the pipeline turns a few risky monthly releases into many small, low-risk ones that recover in hours, not days.

© 2026 Tertiary Infotech Academy Pte Ltd

WHY IT MATTERS

Build, test, deploy — automatically and safely.

The six AWS DevOps domains follow the delivery lifecycle: automate the SDLC, codify infrastructure, make it resilient, observe it, respond to events, and secure it.

© 2026 Tertiary Infotech Academy Pte Ltd

DORA Metrics — The Delivery/Stability Balance

Move faster

- Throughput
  - Deployment frequency — how often you ship
  - Lead time for changes — commit to production
  - Elite: on-demand deploys, <1 day lead time

Stay stable

- Stability
  - Change failure rate — % of deploys that hurt
  - Time to restore service — MTTR after failure
  - Elite: <15% CFR, restore in <1 hour

© 2026 Tertiary Infotech Academy Pte Ltd

HOW TEAMS BRANCH

Version Control Strategy — Trunk-Based vs Long-Lived Branches

1

Trunk-based

Short-lived branches merged to trunk daily — keeps integration continuous.

2

Feature flags

Ship unfinished work dark; decouple deploy from release.

3

Gitflow (heavier)

Release/develop branches — more ceremony, slower integration.

4

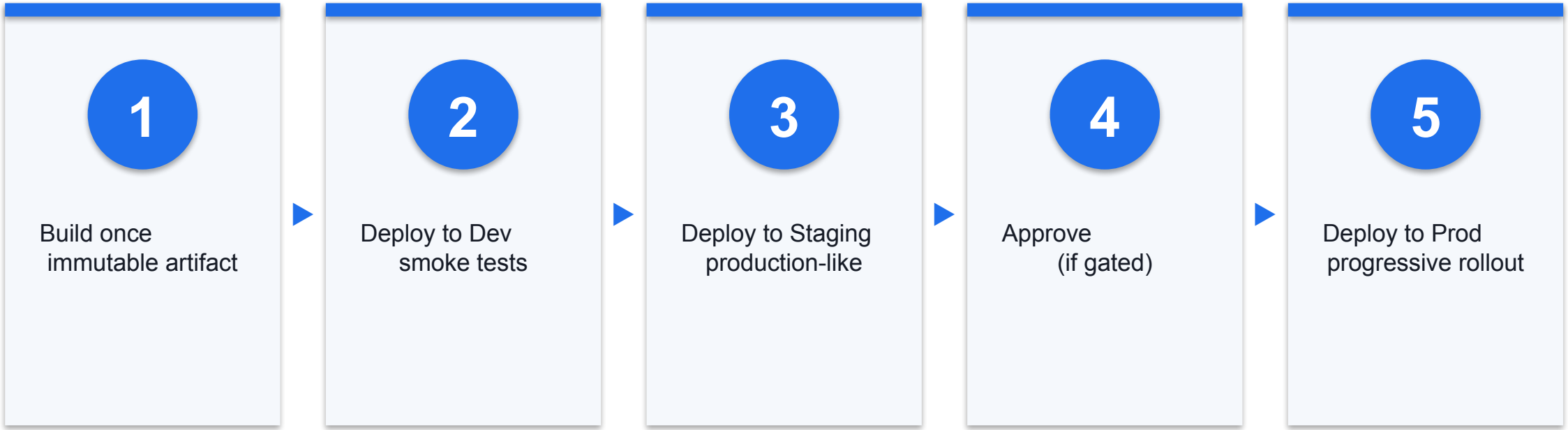
Branch protection

Required reviews and green checks before merge, enforced in GitHub.

© 2026 Tertiary Infotech Academy Pte Ltd

ONE ARTIFACT, MANY ENVIRONMENTS

Promotion Across Environments



© 2026 Tertiary Infotech Academy Pte Ltd

SOURCE INTEGRATION

GitHub via AWS CodeConnections — The Modern Source Path

1

Managed connection

Connect GitHub/GitHub Enterprise/Bitbucket/GitLab without long-lived Git credentials.

2

Webhook triggers

A push starts the pipeline instantly — no polling latency.

3

Keep your workflow

Existing pull requests, reviews and branch protection still apply.

4

CodeCommit optional

AWS-native Git remains available, but is not required for new work.

© 2026 Tertiary Infotech Academy Pte Ltd

A CLOUD-NATIVE MINDSET

Immutable Infrastructure — Cattle, Not Pets

1

Replace, don't patch

Deploy a new, tested image instead of mutating running servers.

2

No configuration drift

Every instance is built the same way from the same artifact.

3

Fast, safe rollback

Roll back by launching the previous known-good image.

4

Bake with a pipeline

EC2 Image Builder / golden AMIs produce reproducible images.

© 2026 Tertiary Infotech Academy Pte Ltd

DEVOPS MEETS FINOPS

Cost Awareness in DevOps — Build It In

1

Right-size & auto-scale

Match capacity to demand; scale to zero where you can.

2

Tag everything

Cost allocation tags make spend visible per team/service.

3

Clean up as code

Tear-down steps in every lab/pipeline stop orphaned resources.

4

Budgets & alarms

AWS Budgets alert before spend surprises the finance team.

© 2026 Tertiary Infotech Academy Pte Ltd

SECURITY IS EVERYONE'S JOB

DevSecOps — Shift Security Left

1

Scan in the pipeline

SAST/SCA on code and dependencies; scan IaC templates.

2

Least-privilege pipelines

Scope build and deploy roles; no secrets in source.

3

Policy as code

Guardrails as SCPs and Config rules, versioned like code.

4

Continuous compliance

Detect and remediate drift automatically, always-on.

© 2026 Tertiary Infotech Academy Pte Ltd

MATCH THE RUNTIME TO THE WORKLOAD

Choosing Compute — EC2 vs ECS/EKS vs Lambda

Option	Best for	Ops overhead	Scaling
EC2 + ASG	Lift-and-shift, full OS control	Higher (patching, AMIs)	Instances via ASG
ECS / Fargate	Containerised services	Medium (Fargate = serverless)	Tasks/services
EKS	Kubernetes-standard workloads	Higher (cluster ops)	Pods via HPA
Lambda	Event-driven, spiky, short tasks	Lowest	Automatic per request

© 2026 Tertiary Infotech Academy Pte Ltd

01

DOMAIN 01

SDLC Automation

Exam weighting 22% · CI/CD pipelines · automated testing · artifacts · deployment strategies

© 2026 Tertiary Infotech Academy Pte Ltd

DELIVERY AUTOMATION

SDLC Automation



- Automating the software-delivery lifecycle on AWS

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 01 · WEIGHTING 22%

Key Concepts — SDLC Automation

1

Continuous Integration

Developers push often; CodeBuild runs the build and tests on every commit so bugs are found early, not at release.

2

Continuous Delivery / Deployment

Every green build is releasable; CodePipeline and CodeDeploy move it through dev → test → prod with approvals.

3

The AWS developer tools

CodeCommit / CodeBuild / CodeDeploy / CodePipeline / CodeArtifact / CodeGuru compose the managed CI/CD toolchain.

4

buildspec & appspec

buildspec.yml tells CodeBuild what to build and test; appspec.yml tells CodeDeploy what and how to deploy.

5

Deployment strategies

In-place, rolling, blue/green and canary trade release speed against blast radius and rollback safety.

6

Artifacts

Build outputs are versioned in S3 / CodeArtifact so any release is reproducible and traceable.

© 2026 Tertiary Infotech Academy Pte Ltd

THE MANAGED CI/CD TOOLCHAIN

The AWS Developer Tools

1

CodeCommit

Managed, private Git repositories (or bring GitHub).

2

CodeBuild

Fully managed build & test service driven by buildspec.yml.

3

CodeDeploy

Automated deployment to EC2, on-prem, ECS and Lambda.

4

CodePipeline

Orchestrates Source → Build → Deploy with approvals.

5

CodeArtifact

Secure artifact & package repository.

6

CodeGuru

ML-powered code reviews and profiler.

© 2026 Tertiary Infotech Academy Pte Ltd

KNOW THE DIFFERENCE

CI vs CD vs Continuous Deployment

Integrate

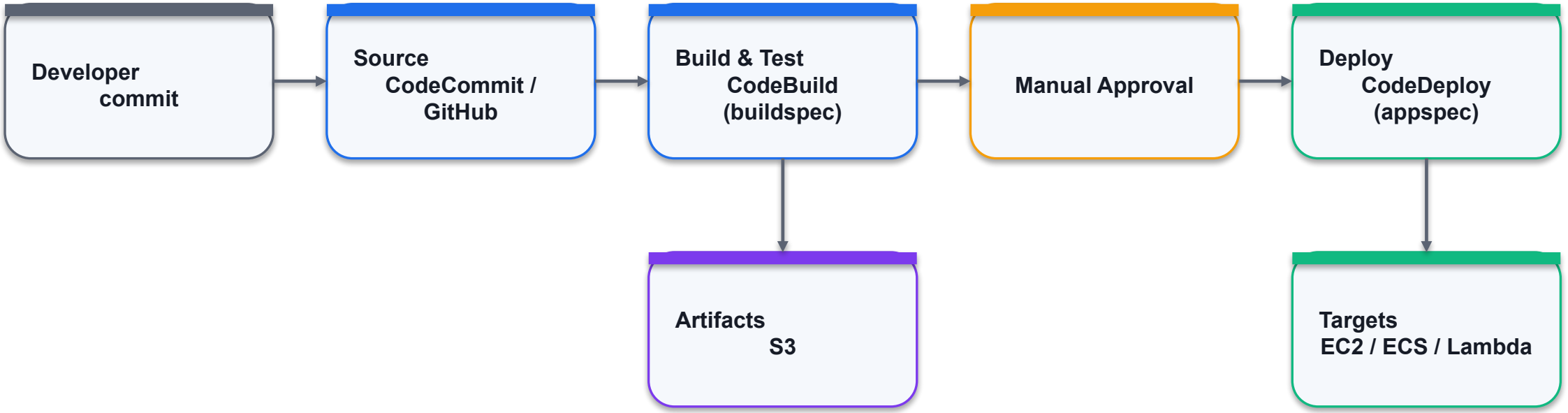
- Continuous Integration
  - Merge & test every commit automatically
  - CodeBuild runs unit tests on push
  - Fast feedback, fewer merge conflicts

Deliver / Deploy

- Continuous Delivery / Deployment
  - Every green build is releasable
  - Delivery = one approval to promote to prod
  - Deployment = fully automatic to prod

© 2026 Tertiary Infotech Academy Pte Ltd

CI/CD Pipeline on AWS



▶ A commit flows Source → Build → Approve → Deploy; a failed test stops the pipeline before production.

© 2026 Tertiary Infotech Academy Pte Ltd

CHOOSING A RELEASE PATTERN

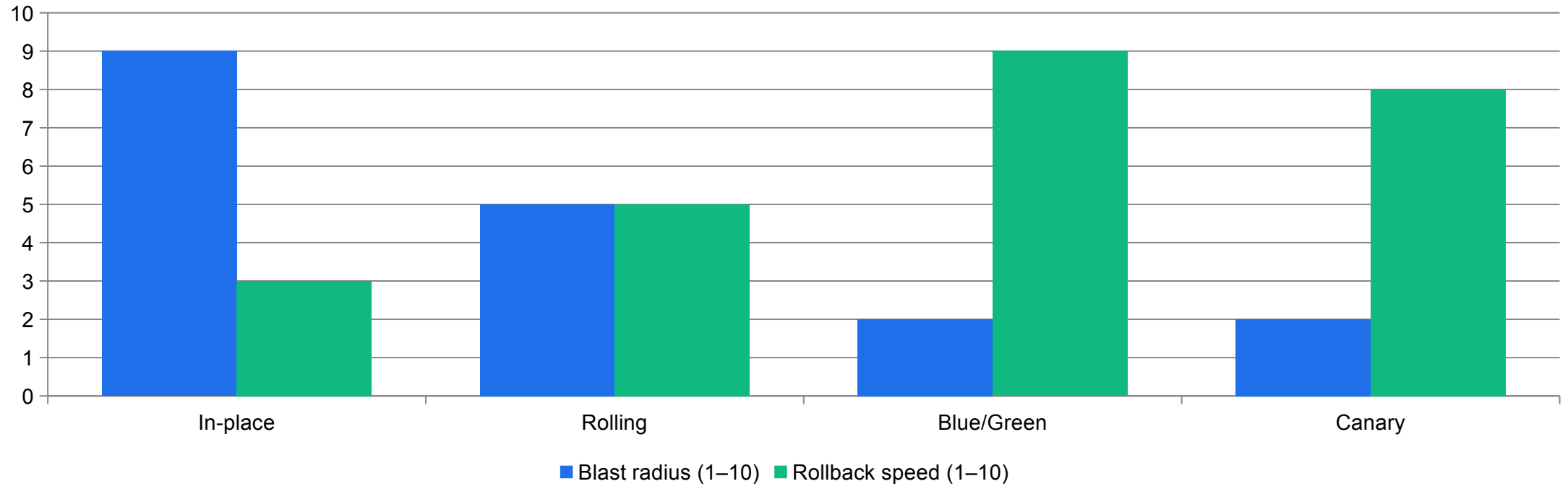
Deployment Strategies Compared

Strategy	How it works	Rollback	Best for
In-place	Update the same instances	Redeploy previous	Dev / simple apps
Rolling	Update in batches	Batch by batch	Steady, low-risk updates
Blue/Green	Shift traffic to a new fleet	Instant switch-back	Zero-downtime prod
Canary	Shift a small % first	Stop & revert	High-risk changes

© 2026 Tertiary Infotech Academy Pte Ltd

TRADE-OFFS

Deployment Strategy — Blast Radius vs Rollback Speed



WHAT THE DATA SHOWS

Blue/green and canary minimise how many users a bad release can affect and let you roll back fastest.

© 2026 Tertiary Infotech Academy Pte Ltd

SHIFT LEFT

Automated Testing in the Pipeline

1

Unit tests

Run in CodeBuild on every commit — fail fast, block the merge.

2

Integration tests

Verify services work together before promotion to staging.

3

Security scans

SAST/SCA on code and dependencies inside the build phase.

4

Quality gates

CodeGuru / linters and coverage thresholds gate the release.

© 2026 Tertiary Infotech Academy Pte Ltd

HOW CODEBUILD KNOWS WHAT TO DO

Anatomy of buildspec.yml

1

version / env

Declare the buildspec version and environment variables / secrets.

2

phases: install

Set up the runtime and install dependencies.

3

phases: pre_build

Log in to registries, run linters.

4

phases: build

Compile, run unit tests, package the app.

5

phases: post_build

Push images, generate reports.

6

artifacts

Declare the versioned output CodePipeline carries forward.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Continuous Integration — mechanics, not just merging

IN DEPTH

- Every commit to the mainline triggers an automated build and test run, so integration problems surface within minutes of the change that caused them.
- CodeBuild runs the build in an ephemeral, reproducible container defined by buildspec.yml — the same steps run for every developer.
- A red build blocks the merge; the team's working agreement is that a broken mainline is fixed before any new work continues.

WHY IT MATTERS

Small, frequent integrations keep the merge delta tiny, so defects are cheap to locate and revert — the opposite of a painful 'big-bang' integration before release.

FAILURE MODE

Long-lived feature branches that integrate weekly recreate the very merge-hell CI exists to prevent; the build stays green only because nothing is really being integrated.

TRADE-OFF / DECISION

Faster feedback needs fast builds — split slow suites into parallel CodeBuild batches or later pipeline stages so the commit-stage stays under ~10 minutes.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Continuous Delivery vs Continuous Deployment

IN DEPTH

- Continuous Delivery keeps every green build in a releasable state; a human approval action gates the final promotion to production.
- Continuous Deployment removes that gate — every change that passes the pipeline goes to production automatically.
- Both rely on the same pipeline; the only difference is whether a person presses the button.

WHY IT MATTERS

Delivery gives regulated or high-blast-radius teams a control point; Deployment gives high-trust teams maximum flow once automated tests and progressive rollout make failures cheap.

FAILURE MODE

Adopting Continuous Deployment before you have strong automated tests and fast rollback turns the pipeline into an automated way to ship bugs to customers.

TRADE-OFF / DECISION

An approval gate adds safety but also queueing delay and 'rubber-stamp' risk — reserve manual approval for genuinely irreversible or compliance-driven steps.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Source control as the pipeline trigger — GitHub via CodeConnections

IN DEPTH

- The modern AWS path connects a GitHub (or GitHub Enterprise / Bitbucket / GitLab) repository through AWS CodeConnections, so a push starts the pipeline over a managed, OAuth/app-based connection.
- CodeCommit remains an AWS-native Git option, but new work standardises on external Git providers via CodeConnections.
- The Source stage stores the commit as an artifact in an encrypted S3 bucket that later stages consume.

WHY IT MATTERS

CodeConnections removes long-lived Git credentials from the pipeline and lets teams keep their existing GitHub workflow, reviews and branch protection.

FAILURE MODE

Polling a repository instead of using the connection's webhook adds minutes of latency and can miss rapid successive commits.

TRADE-OFF / DECISION

Managed connections centralise trust in one integration — scope its permissions and monitor CodeConnections access as part of your security posture.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

buildspec.yml — teaching CodeBuild your build

IN DEPTH

- buildspec.yml declares phases (install, pre_build, build, post_build), the runtime versions, environment variables and the artifacts to export.
- Secrets are pulled at build time from Secrets Manager or SSM Parameter Store, never hard-coded in the file.
- The build produces a single versioned artifact that every downstream stage reuses — build once, deploy many times.

WHY IT MATTERS

A declarative build spec makes the build reproducible and reviewable in code review, and identical across every developer and the pipeline.

FAILURE MODE

Baking credentials or environment-specific values into buildspec.yml couples the artifact to one environment and leaks secrets into logs.

TRADE-OFF / DECISION

Rich caching (dependencies, layers) speeds builds but can hide dependency drift — pin versions and periodically build clean.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

appspec + deployment lifecycle hooks

IN DEPTH

- appspec.yml tells CodeDeploy what to copy and which lifecycle-hook scripts to run (BeforeInstall, AfterInstall, ApplicationStart, ValidateService).
- The ValidateService hook is where a health check decides whether the new version is actually serving traffic.
- For blue/green, CodeDeploy shifts traffic between target groups instead of updating files in place.

WHY IT MATTERS

Lifecycle hooks turn 'copy files' into a safe, verifiable release with an explicit go/no-go check before the deployment is declared successful.

FAILURE MODE

A ValidateService hook that always exits 0 defeats automated rollback — a broken release is reported as healthy.

TRADE-OFF / DECISION

Longer validation windows catch more problems but slow each deployment and hold the fleet in a mixed-version state.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Artifacts, versioning and traceability

IN DEPTH

- Build outputs are versioned in S3 and packages in CodeArtifact, each tied to the source commit that produced it.
- Any production release can be traced back to an exact commit, build log and test result.
- Rollback becomes 'redeploy the previous known-good artifact', not 'rebuild and hope'.

WHY IT MATTERS

Immutable, versioned artifacts make releases reproducible and auditable — essential for incident forensics and compliance evidence.

FAILURE MODE

Rebuilding from source on rollback can pull newer dependencies and reintroduce or mask the very fault you are rolling back from.

TRADE-OFF / DECISION

Retaining every artifact aids traceability but grows storage cost — apply S3 lifecycle rules to age out old, unreferenced builds.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Automated testing and quality gates in the pipeline

IN DEPTH

- Unit tests run in the commit stage; integration, security (SAST/SCA) and performance checks run in later stages before promotion.
- Quality gates — coverage thresholds, CodeGuru findings, linters — can fail the build automatically.
- Test data and environments are provisioned as code so tests are repeatable.

WHY IT MATTERS

Shifting testing left catches defects when they are cheapest to fix and keeps the mainline releasable at all times.

FAILURE MODE

Flaky tests train the team to ignore red builds; a pipeline everyone overrides provides no protection at all.

TRADE-OFF / DECISION

More gates raise confidence but lengthen lead time — order them fast-to-slow and fail fast on the cheapest checks.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Choosing a deployment strategy by blast radius

IN DEPTH

- In-place is simplest but updates live instances and risks downtime; rolling limits impact to a batch at a time.
- Blue/green stands up a parallel fleet and switches traffic — instant rollback by switching back.
- Canary shifts a small percentage first, watches metrics, then completes or aborts the rollout.

WHY IT MATTERS

Matching the strategy to the change's risk lets you trade release speed against how many users a bad release can touch.

FAILURE MODE

Blue/green without draining connections or warming caches can still cause a visible blip at cut-over.

TRADE-OFF / DECISION

Blue/green needs roughly double capacity during the shift; canary needs good metrics and automated alarms to be safe.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Pipeline structure — stages, actions and promotion

IN DEPTH

- A pipeline is stages (Source → Build → Test → Deploy-to-Staging → Approve → Deploy-to-Prod), each containing actions that can run in parallel.
- The same immutable artifact is promoted across environments; only configuration changes between them.
- Cross-account deployment uses assumed roles so the production account trusts the pipeline without sharing credentials.

WHY IT MATTERS

Modelling promotion explicitly gives a single, auditable path to production and prevents 'works in staging, different build in prod' drift.

FAILURE MODE

Rebuilding per environment breaks the guarantee that what you tested is what you shipped.

TRADE-OFF / DECISION

More environments raise confidence but add cost and lead time — keep pre-prod as production-like as budget allows.

© 2026 Tertiary Infotech Academy Pte Ltd

CHOOSING THE SOURCE PATH

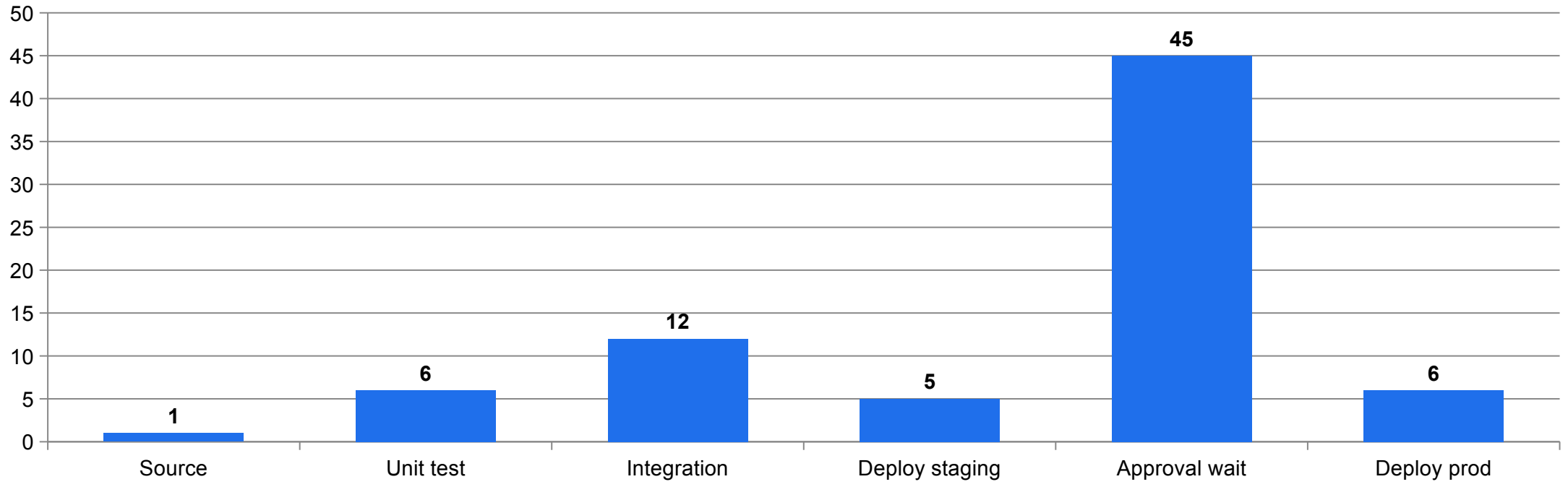
CI/CD Toolchain — AWS-native vs external

Concern	GitHub via CodeConnections	AWS CodeCommit
Where new work goes	Primary, modern path	AWS-native option; not required
Credentials	Managed connection (no static keys)	IAM / Git credentials
Team workflow	Existing GitHub PRs & branch rules	Native Git, AWS console/IAM
Trigger	Connection webhook	CloudWatch Events / EventBridge

© 2026 Tertiary Infotech Academy Pte Ltd

OPTIMISE THE BOTTLENECK

Pipeline Lead Time by Stage — Where the Minutes Go



WHAT THE DATA SHOWS

Manual approval dominates lead time here — automate the checks it waits on so approval is a formality, not a bottleneck.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT BREAKS PIPELINES

CI/CD Failure Modes & Anti-Patterns

1

Snowflake build agents

Builds pass locally, fail in CodeBuild → pin runtime-versions and build in clean containers.

2

Secrets in source/logs

Keys committed or echoed → pull from Secrets Manager/SSM at build time; mask in logs.

3

Flaky tests ignored

Red builds routinely overridden → quarantine and fix flaky tests; a gate you bypass is no gate.

4

Rebuild on promotion

Different artifact per environment → build once, promote the same artifact.

5

No automated rollback

Bad deploy needs a human at 2am → alarm-based rollback on CodeDeploy.

6

Polling instead of webhooks

Slow, missed triggers → use the CodeConnections webhook.

© 2026 Tertiary Infotech Academy Pte Ltd

PROVEN PRACTICES

Delivery Patterns Worth Adopting

1

Trunk-based development

Short-lived branches merged to trunk daily keep integration continuous.

2

Build once, deploy many

One immutable artifact promoted across environments.

3

Progressive delivery

Canary/blue-green with automated metric analysis before full rollout.

4

Pipeline as code

Define the pipeline in CloudFormation/CDK so it is versioned and reviewable.

5

Fail fast

Cheapest, fastest checks first; stop the line on first failure.

6

Immutable infrastructure

Replace, don't patch, running servers.

© 2026 Tertiary Infotech Academy Pte Ltd

Architecture Decision — Approval Gate or Not?

A — Manual approval before prod

Regulated change, irreversible action, or immature test suite. Adds a control point and audit record — at the cost of queueing delay.

B — Automated to prod

High test coverage, fast rollback and progressive rollout in place. Maximises flow — requires strong alarms and canary analysis.

C — Hybrid

Auto-deploy low-risk services; gate high-blast-radius or data-migration changes. The common enterprise middle ground.

© 2026 Tertiary Infotech Academy Pte Ltd

REASONING THROUGH THE RESPONSE

Scenario — A bad release reaches production

SITUATION

A deployment passes all pipeline stages, but error rates climb 3 minutes after it reaches production.

ANALYSIS

- The build and unit tests were green, so the fault is environment- or scale-specific — exactly what canary analysis is designed to catch.
- Because releases use immutable, versioned artifacts, the last known-good build is one redeploy away.
- A CloudWatch alarm on error rate can trigger CodeDeploy's automatic rollback without waiting for a human.

DECISION

Wire an error-rate alarm to CodeDeploy auto-rollback and move this service to a canary configuration so the next bad change is caught at 5% of traffic, not 100%.

© 2026 Tertiary Infotech Academy Pte Ltd

OPTIMISING FEEDBACK TIME

Scenario — Builds are too slow, developers merge less

SITUATION

The commit-stage build takes 25 minutes; developers batch changes to avoid the wait, so integrations get large and risky.

ANALYSIS

- Slow feedback is pushing the team back towards big-bang integration — the DORA lead-time and batch-size signals both degrade.
- Test suites can be split and run in parallel CodeBuild projects; dependency caching removes repeated downloads.
- Slow end-to-end tests can move to a later stage so the commit stage stays fast.

DECISION

Target a sub-10-minute commit stage: parallelise tests, cache dependencies, and defer slow suites downstream so small, frequent merges become painless again.

© 2026 Tertiary Infotech Academy Pte Ltd

FROM PRACTICE TO NUMBERS

Worked Analysis — Sizing the Impact of CI/CD

Before: 2 releases/month, 14-day lead time	Large batches; each release bundles ~150 changes, so failures are hard to isolate.
After: 40 releases/month, ~1-day lead time	Small batches; each release is ~7 changes, so a regression is trivially bisected.
Change-failure rate 30% → 10%	Automated gates catch defects pre-prod; progressive rollout limits blast radius.
MTTR 8h → 1h	Versioned artifacts + alarm-based rollback make recovery a redeploy, not a rebuild.

TAKEAWAY

The gain is not just speed — smaller batches make every DORA metric move together: deploy more often, recover faster, fail less.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Source and package governance

1

PURPOSE

Control how source, dependencies and container images enter the delivery system.

2

FEATURE 1

Repository events and protected branches trigger delivery without long-lived credentials.

3

FEATURE 2

CodeArtifact domains and repositories proxy, version and retain language packages.

4

FEATURE 3

ECR stores immutable image digests, supports lifecycle rules and exposes scan findings.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 1 · SERVICE DECISION

Source and package governance: selection and controls

1

CHOOSE WHEN

Use source connections for event-driven repository access, CodeArtifact for governed packages, and ECR for OCI images.

2

DEVOPS ENGINEERING

Pin versions/digests, enforce review, generate an SBOM where required and promote the same artifact between environments.

3

FAILURE MODE

Mutable tags, unpinned dependencies or broad repository tokens break provenance and repeatability.

4

EVIDENCE

Commit SHA, approval, package version, image digest, scan result and promotion record.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Build and pipeline orchestration

1

PURPOSE

Turn a source revision into tested, signed and traceable release artifacts.

2

FEATURE 1

CodePipeline sequences source, build, test, approval and deploy actions with stage transitions.

3

FEATURE 2

CodeBuild uses ephemeral environments, buildspec phases, reports, caches and exported variables.

4

FEATURE 3

Versioned/encrypted S3 artifact stores preserve hand-offs between pipeline actions.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 1 · SERVICE DECISION

Build and pipeline orchestration: selection and controls

1

CHOOSE WHEN

Use CodePipeline for managed orchestration and CodeBuild when a reproducible build/test environment is needed.

2

DEVOPS ENGINEERING

Fail fast on test/security gates, isolate service roles, encrypt artifacts and make rollback reference an immutable build.

3

FAILURE MODE

A green pipeline that does not retain test evidence or artifact identity cannot prove what was deployed.

4

EVIDENCE

Pipeline execution ID, build logs, test report, artifact checksum, approval and deployment ID.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Deployment services

1

PURPOSE

Release applications with a strategy matched to compute model, capacity and user-impact tolerance.

2

FEATURE 1

CodeDeploy coordinates in-place or blue/green deployments and lifecycle hooks.

3

FEATURE 2

Elastic Beanstalk manages application environments, versions, health and platform updates.

4

FEATURE 3

ECS services and Lambda aliases support rolling, blue/green or weighted traffic strategies.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 1 · SERVICE DECISION

Deployment services: selection and controls

1

CHOOSE WHEN

Choose the target-native mechanism, then select all-at-once, rolling, linear, canary or blue/green from the recovery objective.

2

DEVOPS ENGINEERING

Define health checks, minimum healthy capacity, alarms, hook timeouts and automatic rollback before promotion.

3

FAILURE MODE

Traffic shifting without representative health signals can promote a release that is technically alive but unusable.

4

EVIDENCE

Deployment configuration, target health, alarm state, hook logs, traffic weights and rollback event.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Serverless and container delivery

1

PURPOSE

Package and operate event-driven functions or container workloads with consistent release evidence.

2

FEATURE 1

Lambda versions and aliases give immutable function revisions and weighted routing.

3

FEATURE 2

SAM expands serverless resources and deployment preferences through CloudFormation.

4

FEATURE 3

ECS/Fargate reduces host management; EKS provides Kubernetes APIs with greater operational responsibility.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 1 · SERVICE DECISION

Serverless and container delivery: selection and controls

1

CHOOSE WHEN

Prefer Lambda for short event-driven execution, ECS/Fargate for managed container orchestration, and EKS when Kubernetes compatibility is a real requirement.

2

DEVOPS ENGINEERING

Scan images, pin digests, externalise configuration, constrain task/function roles and test startup, readiness and rollback.

3

FAILURE MODE

Selecting EKS only for portability can add control-plane, add-on and upgrade burden without business value.

4

EVIDENCE

Function version/image digest, task definition, policy diff, deployment state, health metrics and rollback proof.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: AWS CLI and SDK execution model

1

PURPOSE

Automate AWS API calls without embedding permanent credentials or hiding retry, pagination and regional behavior.

2

SERVICE BOUNDARY

AWS CLI v2 · AWS SDKs · STS · IAM Identity Center

3

PRIMARY DESIGN DECISION

Use named SSO profiles locally and a CodeBuild service role in CI; resolve account/Region at runtime and log request IDs without logging credentials.

4

FAILURE BOUNDARY

An implicit default profile can deploy to the wrong account, while unbounded retries can duplicate non-idempotent changes.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS CLI and SDK execution model: how it works

1

MECHANIC 1

The credential provider chain searches explicit configuration, environment, shared profiles, container or instance roles; production code should normally end at a workload role.

2

MECHANIC 2

A client is Region-specific and signs each request with SigV4; the Region therefore affects endpoint, resource availability and the authorization scope.

3

MECHANIC 3

SDK paginators, waiters and retry modes handle common distributed-system behavior, but applications must still set timeouts and make mutating operations idempotent.

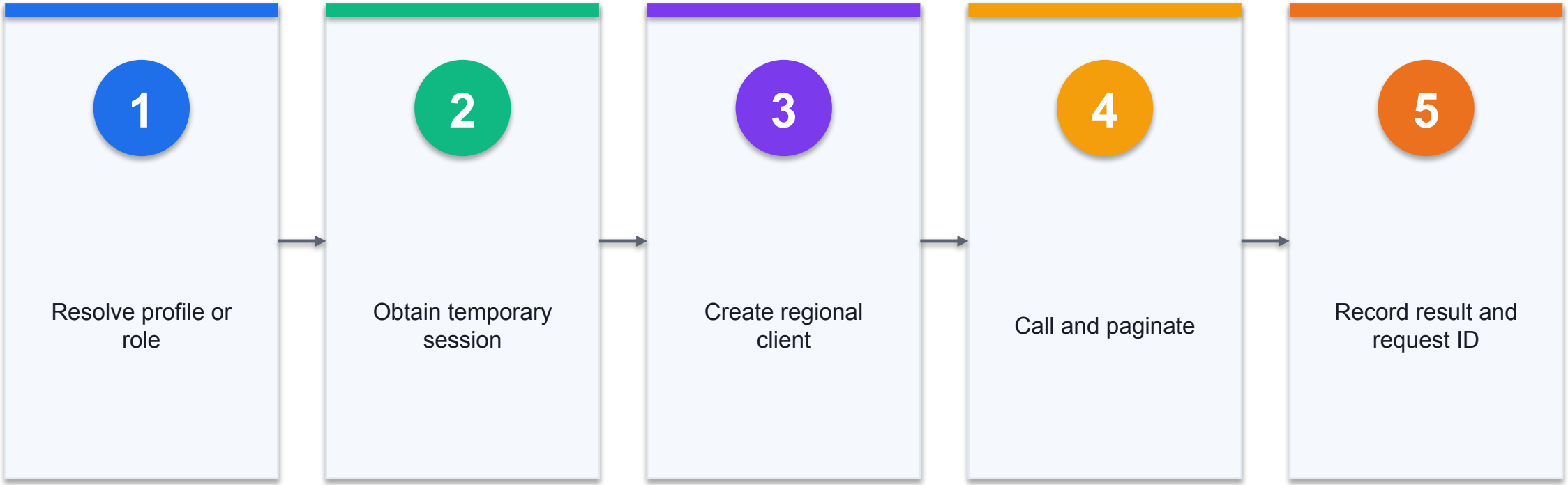
4

MECHANIC 4

CloudTrail records supported API activity, allowing a command, assumed-role session and resulting resource change to be correlated.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS CLI and SDK execution model: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS CLI and SDK execution model: configuration decisions

1

AUTHENTICATION

Use IAM Identity Center for workforce access and role credentials for workloads; reserve static keys for exceptional legacy cases.

2

RETRIES

Use standard or adaptive retry behavior with bounded attempts, jitter and service-specific handling for throttling.

3

PAGINATION

Consume every page or deliberately cap results; a first-page-only inventory creates false compliance evidence.

4

IDEMPOTENCY

Supply client tokens where supported and design repeatable state checks before retrying a write.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS CLI and SDK execution model: worked architecture decision

1

SCENARIO

A deployment utility must work from a developer laptop and CodeBuild while producing the same account and Region evidence.

2

DESIGN DECISION

Use named SSO profiles locally and a CodeBuild service role in CI; resolve account/Region at runtime and log request IDs without logging credentials.

3

FAILURE TO AVOID

An implicit default profile can deploy to the wrong account, while unbounded retries can duplicate non-idempotent changes.

4

ACCEPTANCE EVIDENCE

Caller identity, selected Region, assumed-role ARN, API request ID, retry count and final resource state.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: Package and image governance

1

PURPOSE

Create a trusted supply chain for language packages and container images, then promote immutable versions across environments.

2

SERVICE BOUNDARY

CodeArtifact · ECR · KMS · Inspector

3

PRIMARY DESIGN DECISION

Build once in the tooling account, scan the digest, grant scoped cross-account pull access and promote the same digest through environment manifests.

4

FAILURE BOUNDARY

Mutable tags can silently replace the reviewed artifact; aggressive lifecycle rules can remove the rollback image.

© 2026 Tertiary Infotech Academy Pte Ltd

Package and image governance: how it works

1

MECHANIC 1

CodeArtifact domains group repositories and can deduplicate package assets; upstream repositories proxy internal or public sources through a controlled path.

2

MECHANIC 2

Authentication tokens are temporary and should be requested at build time, never committed to package-manager configuration.

3

MECHANIC 3

ECR stores OCI images by tag and digest; deployment references should use immutable tags or, preferably, the resolved digest.

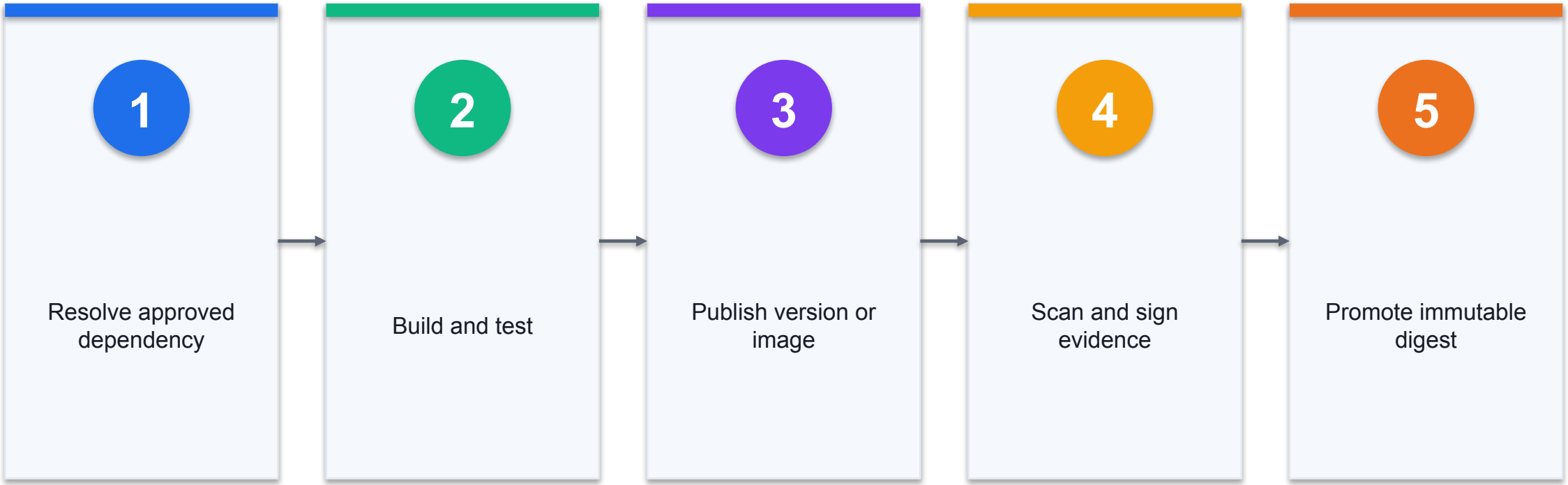
4

MECHANIC 4

Lifecycle policies control retention, while enhanced scanning can surface operating-system and language-package vulnerabilities through Inspector.

© 2026 Tertiary Infotech Academy Pte Ltd

Package and image governance: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Package and image governance: configuration decisions

1

REPOSITORY POLICY

Separate publisher and consumer permissions; restrict cross-account pulls with repository and identity policies.

2

VERSIONING

Reject overwrites where supported and give every release a unique package version or image digest.

3

SCANNING

Define severity thresholds and exception ownership; a scan with no release decision is only telemetry.

4

RETENTION

Keep releases required for rollback and audit while expiring untagged or superseded artifacts safely.

© 2026 Tertiary Infotech Academy Pte Ltd

Package and image governance: worked architecture decision

1

SCENARIO

Twenty accounts must deploy the identical reviewed container without rebuilding or granting broad registry access.

2

DESIGN DECISION

Build once in the tooling account, scan the digest, grant scoped cross-account pull access and promote the same digest through environment manifests.

3

FAILURE TO AVOID

Mutable tags can silently replace the reviewed artifact; aggressive lifecycle rules can remove the rollback image.

4

ACCEPTANCE EVIDENCE

Package version, repository policy, image digest, scan finding disposition, promotion record and successful cross-account pull.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: CodeBuild build environments and quality gates

1

PURPOSE

Turn source into reproducible artifacts while making tests, security checks and provenance part of the pipeline decision.

2

SERVICE BOUNDARY

CodeBuild · buildspec · report groups · S3 cache

3

PRIMARY DESIGN DECISION

Pin the runtime image, run deterministic installs, publish JUnit/coverage reports and create a manifest containing commit SHA and artifact checksum.

4

FAILURE BOUNDARY

A shell command followed by another successful command can hide a failed test unless exit handling is explicit.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeBuild build environments and quality gates: how it works

1

MECHANIC 1

A buildspec separates install, pre_build, build and post_build phases; finally blocks can preserve diagnostics even after a phase fails.

2

MECHANIC 2

Managed images simplify patching, while custom images provide tool control and must themselves be versioned and scanned.

3

MECHANIC 3

Privileged mode is mainly required for Docker daemon workflows and materially increases build-container authority.

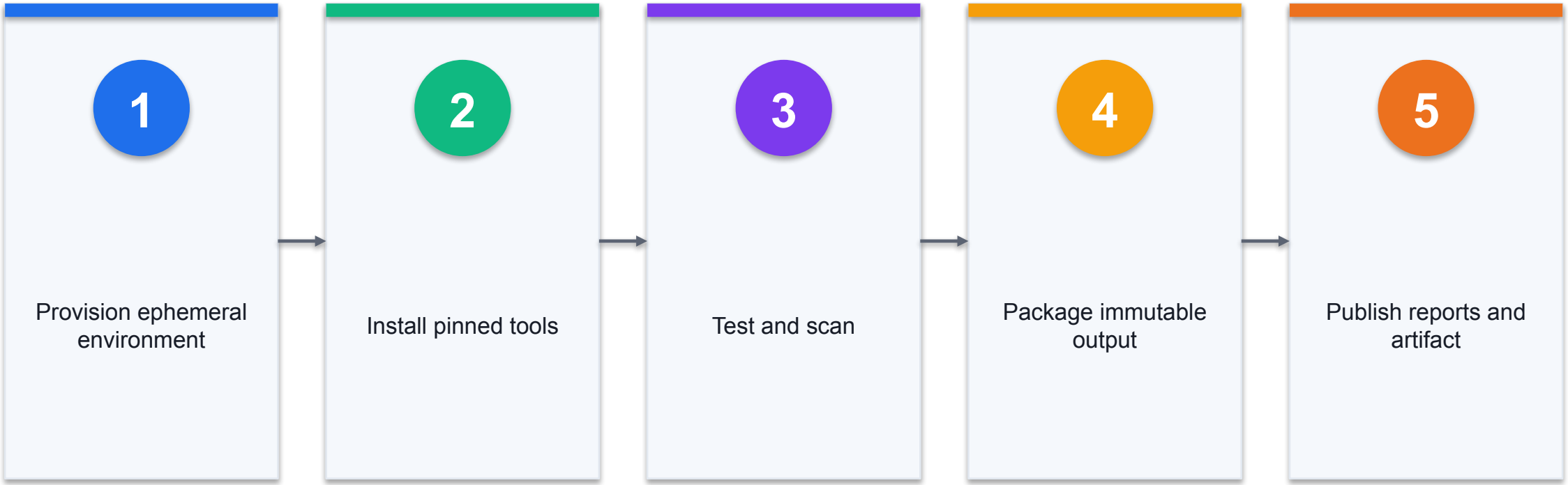
4

MECHANIC 4

Report groups retain test or coverage results; artifacts and exported variables pass controlled outputs to later pipeline actions.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeBuild build environments and quality gates: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeBuild build environments and quality gates: configuration decisions

1

COMPUTE

Select architecture, size and timeout from measured build demand; avoid oversized always-on assumptions.

2

NETWORK

Use VPC attachment only when private dependencies require it, and provide routes/endpoints for every required service.

3

CACHE

Cache dependencies for speed, but key the cache so stale libraries cannot bypass lock-file changes.

4

SECRETS

Retrieve secrets at runtime from managed stores and mask output; never place secret values in buildspec source.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeBuild build environments and quality gates: worked architecture decision

1

SCENARIO

A Python service must stop promotion if tests fail, coverage drops below policy, or the artifact cannot be traced to its source revision.

2

DESIGN DECISION

Pin the runtime image, run deterministic installs, publish JUnit/coverage reports and create a manifest containing commit SHA and artifact checksum.

3

FAILURE TO AVOID

A shell command followed by another successful command can hide a failed test unless exit handling is explicit.

4

ACCEPTANCE EVIDENCE

Build ARN, source version, image identifier, test report, coverage threshold, checksum and phase logs.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: CodePipeline orchestration and approvals

1

PURPOSE

Coordinate source, validation, approval and deployment stages while preserving artifact identity and execution context.

2

SERVICE BOUNDARY

CodePipeline · CodeConnections · CodeBuild · Lambda actions

3

PRIMARY DESIGN DECISION

Keep build resources in a tooling account, promote an immutable artifact, assume per-environment roles and require an approval owned outside the developer role.

4

FAILURE BOUNDARY

Rebuilding for production changes the artifact under review; parallel executions can race against shared CloudFormation state.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 1 · SERVICE MECHANICS

CodePipeline orchestration and approvals: how it works

1

MECHANIC 1

A pipeline execution moves named artifacts between actions through an encrypted artifact store; each action receives declared inputs and emits declared outputs.

2

MECHANIC 2

Execution modes determine how concurrent revisions behave: superseding, queueing or running in parallel must match the deployment target's safety model.

3

MECHANIC 3

Manual approvals pause execution and should present change, test, risk and rollback evidence rather than a context-free button.

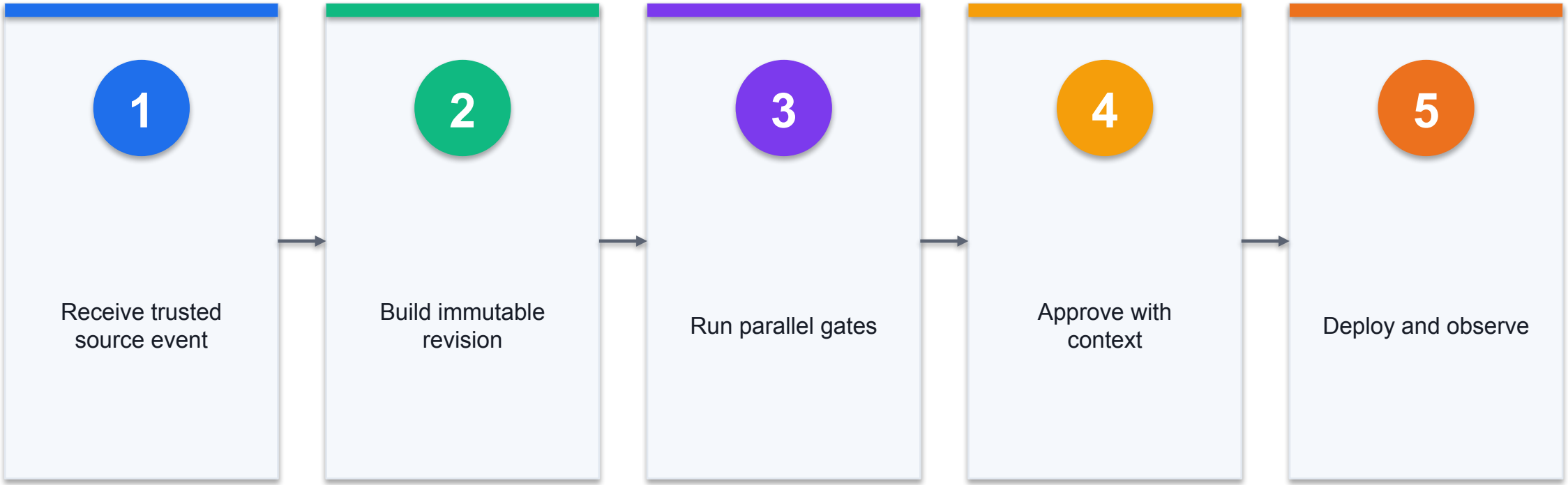
4

MECHANIC 4

Variables and namespaces pass small metadata values; release artifacts belong in governed stores, not pipeline variables.

© 2026 Tertiary Infotech Academy Pte Ltd

CodePipeline orchestration and approvals: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

CodePipeline orchestration and approvals: configuration decisions

1

TRIGGER

Filter repository events so pull requests validate while only accepted branches or tags reach deployment.

2

CONCURRENCY

Use queued execution for shared environments and parallel execution only when targets and state are isolated.

3

CROSS-ACCOUNT

Assume narrow deployment roles and use KMS/artifact policies that explicitly permit the required account path.

4

FAILURE

Stop stage transitions, retain the failed execution and automate rollback from the last-known-good artifact.

© 2026 Tertiary Infotech Academy Pte Ltd

CodePipeline orchestration and approvals: worked architecture decision

1

SCENARIO

A multi-account pipeline must deploy the same revision to test and production, with separation of duties at production approval.

2

DESIGN DECISION

Keep build resources in a tooling account, promote an immutable artifact, assume per-environment roles and require an approval owned outside the developer role.

3

FAILURE TO AVOID

Rebuilding for production changes the artifact under review; parallel executions can race against shared CloudFormation state.

4

ACCEPTANCE EVIDENCE

Pipeline execution ID, artifact digest, gate reports, approval identity/comment, assumed role and deployment result.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: CodeDeploy lifecycle, traffic shifting and rollback

1

PURPOSE

Select lifecycle hooks and deployment configurations that expose failures before they reach the full user population.

2

SERVICE BOUNDARY

CodeDeploy · EC2/On-Premises · ECS · Lambda

3

PRIMARY DESIGN DECISION

Deploy an immutable version behind an alias, shift a small weight, bind a transaction-error alarm and retain the previous alias target for rollback.

4

FAILURE BOUNDARY

Infrastructure health alone may stay green while business transactions fail; an alarm that evaluates too slowly misses the canary window.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeDeploy lifecycle, traffic shifting and rollback: how it works

1

MECHANIC 1

EC2/on-premises deployments use an AppSpec file to map files and lifecycle scripts; scripts must be idempotent because replacement or retry can rerun them.

2

MECHANIC 2

ECS blue/green deployments create a replacement task set and move listener traffic after validation through CodeDeploy-controlled hooks.

3

MECHANIC 3

Lambda deployments shift an alias between immutable versions using canary or linear increments and optional validation hooks.

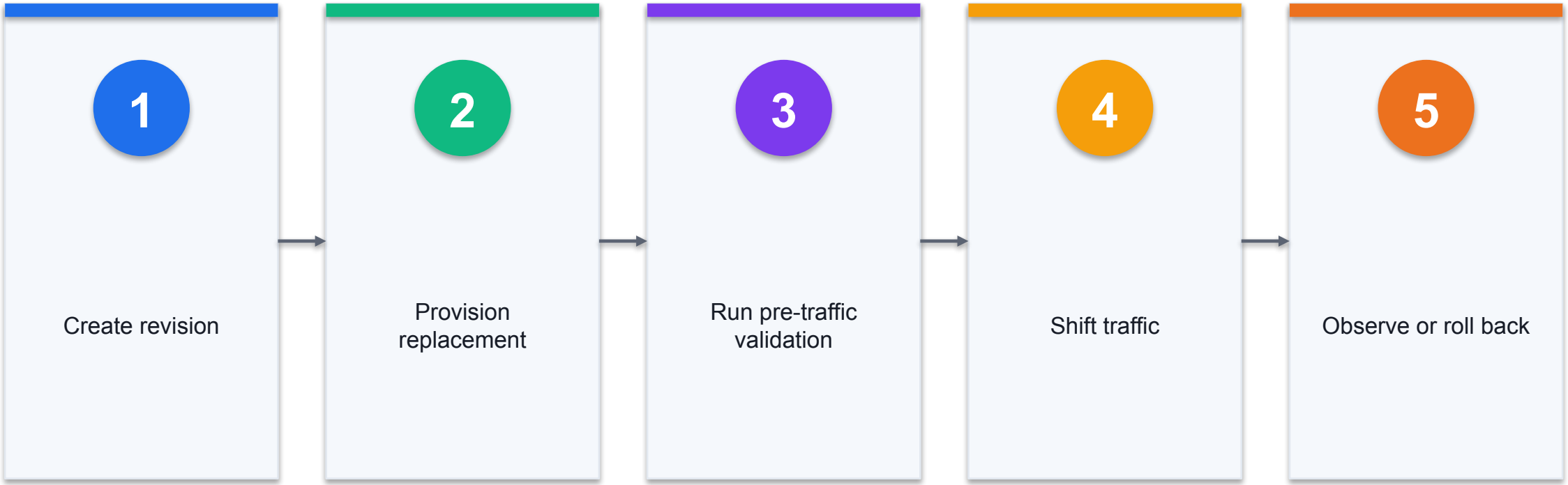
4

MECHANIC 4

Automatic rollback can react to deployment failure, stop requests or CloudWatch alarms, but only if alarms represent real release health.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeDeploy lifecycle, traffic shifting and rollback: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeDeploy lifecycle, traffic shifting and rollback: configuration decisions

1

IN-PLACE

Lower capacity cost but shares the original environment and normally has a slower recovery path.

2

BLUE/GREEN

Keeps old capacity for rapid rollback but requires duplicate capacity and compatible data changes.

3

CANARY/LINEAR

Limits early exposure and needs statistically useful traffic plus alarms that react within the interval.

4

HOOKS

Use hooks for deterministic validation, not long-running hidden configuration management.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeDeploy lifecycle, traffic shifting and rollback: worked architecture decision

1

SCENARIO

A payment API needs a small production canary and automatic restoration of the prior version when transaction errors rise.

2

DESIGN DECISION

Deploy an immutable version behind an alias, shift a small weight, bind a transaction-error alarm and retain the previous alias target for rollback.

3

FAILURE TO AVOID

Infrastructure health alone may stay green while business transactions fail; an alarm that evaluates too slowly misses the canary window.

4

ACCEPTANCE EVIDENCE

Revision/version, hook output, target health, traffic weights, alarm history, rollback event and post-rollback transaction check.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: Elastic Beanstalk managed application platforms

1

PURPOSE

Operate web applications through managed platform versions, environment configuration and deployment policies without losing architectural control.

2

SERVICE BOUNDARY

Elastic Beanstalk · Auto Scaling · ELB · RDS

3

PRIMARY DESIGN DECISION

Use a supported platform branch, separate the database lifecycle, choose immutable or rolling-with-batch deployment and test managed updates in a clone.

4

FAILURE BOUNDARY

Embedding a production database in the environment couples data lifecycle to application replacement; platform hooks can drift when undocumented.

© 2026 Tertiary Infotech Academy Pte Ltd

Elastic Beanstalk managed application platforms: how it works

1

MECHANIC 1

An application contains versions and environments; an environment creates resources such as instances, an Auto Scaling group and a load balancer from configuration options.

2

MECHANIC 2

Platform branches and versions define operating system, runtime, proxy and supported lifecycle; managed platform updates reduce but do not remove upgrade testing.

3

MECHANIC 3

Saved configurations and environment properties support repeatability, while `.ebextensions` and platform hooks apply controlled customisation.

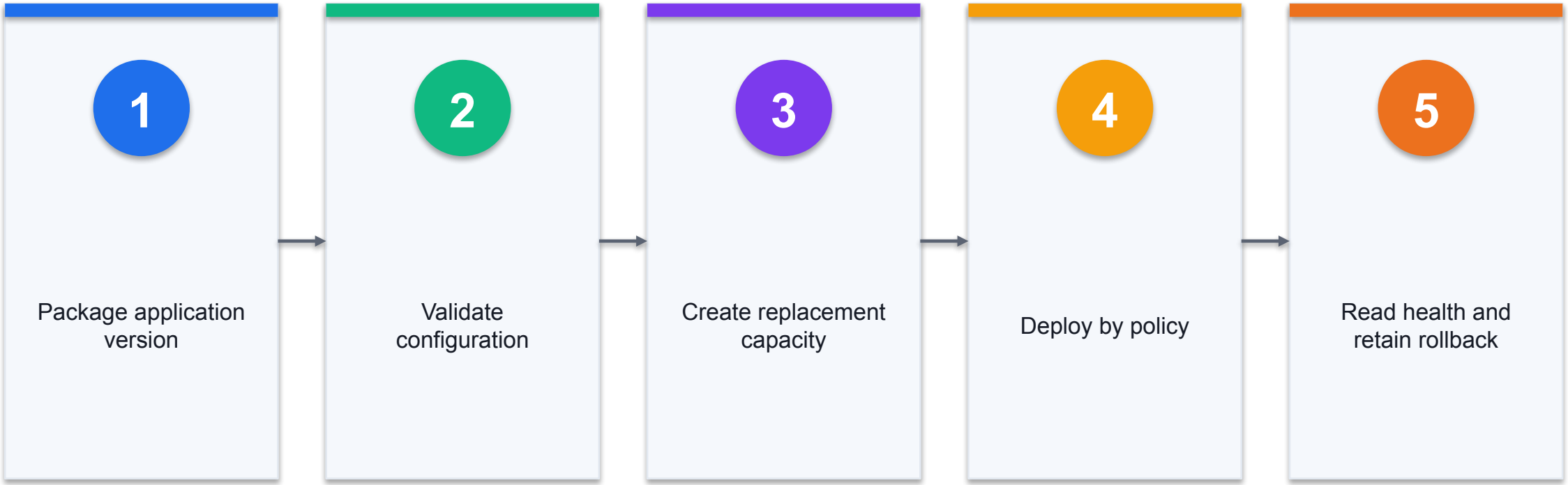
4

MECHANIC 4

Application health combines resource and request signals; enhanced health exposes instance and deployment-level status.

© 2026 Tertiary Infotech Academy Pte Ltd

Elastic Beanstalk managed application platforms: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Elastic Beanstalk managed application platforms: configuration decisions

1

ALL AT ONCE

Fast and cheap, but introduces downtime or full-fleet risk.

2

ROLLING

Updates batches while reducing available capacity during the deployment.

3

ROLLING WITH BATCH

Adds temporary capacity to preserve service level at higher short-term cost.

4

IMMUTABLE

Launches a fresh group for strong isolation and quick environment-level recovery.

© 2026 Tertiary Infotech Academy Pte Ltd

Elastic Beanstalk managed application platforms: worked architecture decision

1

SCENARIO

A legacy web service needs managed runtime patching but cannot accept reduced capacity during release.

2

DESIGN DECISION

Use a supported platform branch, separate the database lifecycle, choose immutable or rolling-with-batch deployment and test managed updates in a clone.

3

FAILURE TO AVOID

Embedding a production database in the environment couples data lifecycle to application replacement; platform hooks can drift when undocumented.

4

ACCEPTANCE EVIDENCE

Application version, configuration template, platform version, enhanced-health events, deployment policy and rollback environment.

© 2026 Tertiary Infotech Academy Pte Ltd

Continuous delivery operating model

1

OUTCOME

Turn every accepted change into a repeatable, observable release candidate.

2

ARCHITECTURE

Separate source, build, test, approval and deploy concerns; promote immutable artifacts rather than rebuilding per environment.

3

FAILURE MODE

Hidden manual steps create configuration variance and weak rollback evidence.

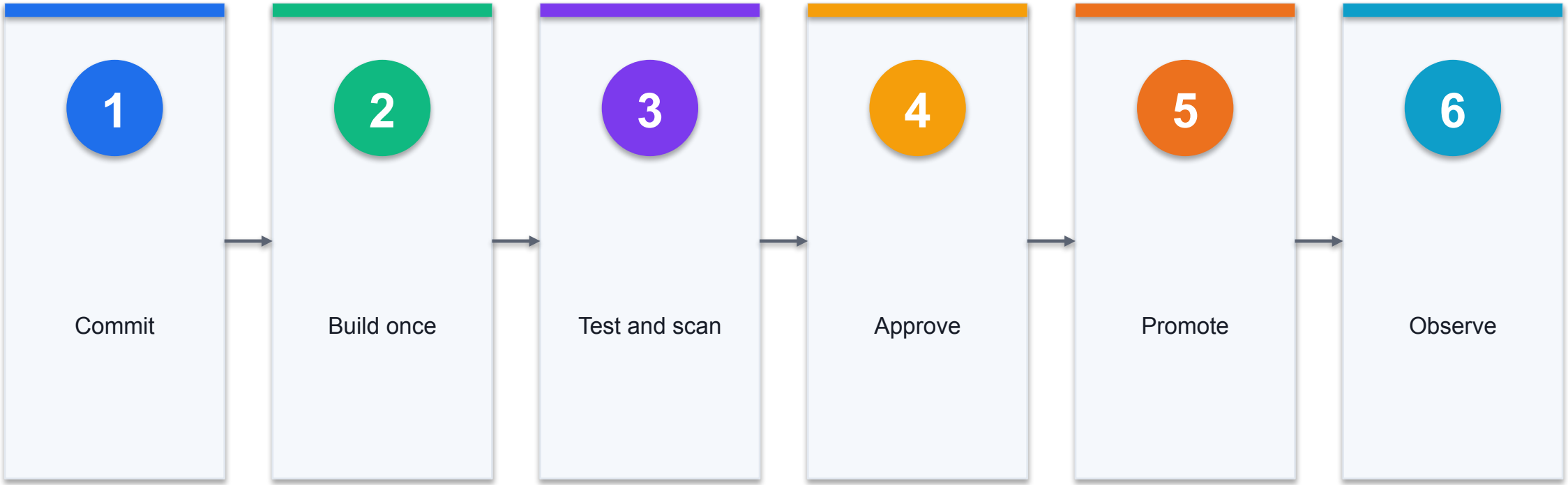
4

EVIDENCE

Commit SHA, test report, artifact digest, approval identity and deployment outcome form the release record.

© 2026 Tertiary Infotech Academy Pte Ltd

Continuous delivery operating model: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 1 · SCENARIO

Continuous delivery operating model: design decision

? SCENARIO

A regulated team needs fast releases without allowing developers to deploy directly to production.

1 DESIGN RESPONSE

Separate source, build, test, approval and deploy concerns; promote immutable artifacts rather than rebuilding per environment.

2 PRIMARY SERVICES

GitHub · CodePipeline · CodeBuild · CodeDeploy

3 FAILURE MODE

Hidden manual steps create configuration variance and weak rollback evidence.

4 PROOF

Commit SHA, test report, artifact digest, approval identity and deployment outcome form the release record.

© 2026 Tertiary Infotech Academy Pte Ltd

Continuous delivery operating model: failure diagnostic

1

RECOGNISE

Observable failure condition: Hidden manual steps create configuration variance and weak rollback evidence.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Commit through Observe; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Commit SHA, test report, artifact digest, approval identity and deployment outcome form the release record.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 1 · APPLIED SCENARIO

Continuous delivery operating model: worked decision

1

CONSTRAINT

A regulated team needs fast releases without allowing developers to deploy directly to production.

2

DECISION

Separate source, build, test, approval and deploy concerns; promote immutable artifacts rather than rebuilding per environment.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Hidden manual steps create configuration variance and weak rollback evidence.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Commit SHA, test report, artifact digest, approval identity and deployment outcome form the release record.

© 2026 Tertiary Infotech Academy Pte Ltd

Pipeline triggers and source governance

1

OUTCOME

Start the right pipeline from an authenticated source event while protecting mainline quality.

2

ARCHITECTURE

Use pull-request checks, protected branches, signed connections and event filters; avoid broad polling and shared credentials.

3

FAILURE MODE

A trigger that accepts unreviewed branches can promote untrusted code.

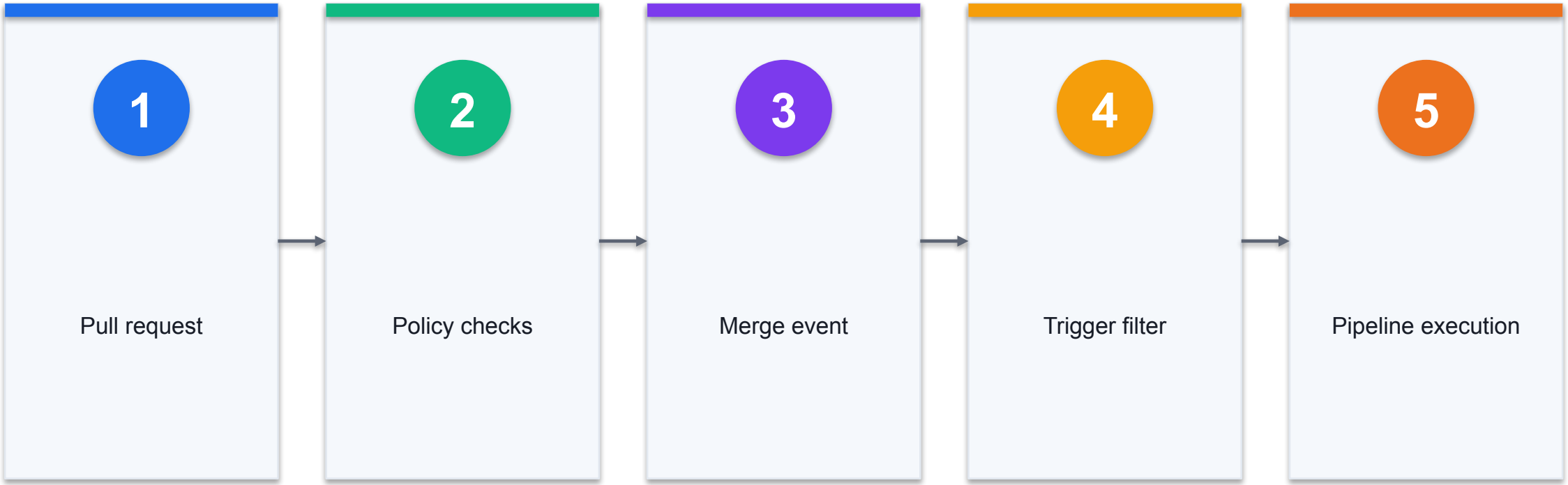
4

EVIDENCE

Repository event, branch rule, connection ARN, trigger filter and execution correlation ID.

© 2026 Tertiary Infotech Academy Pte Ltd

Pipeline triggers and source governance: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Pipeline triggers and source governance: design decision

? SCENARIO

Feature branches must run tests, but only reviewed merges to main may enter the release pipeline.

1 DESIGN RESPONSE

Use pull-request checks, protected branches, signed connections and event filters; avoid broad polling and shared credentials.

2 PRIMARY SERVICES

GitHub · EventBridge · CodeConnections

3 FAILURE MODE

A trigger that accepts unreviewed branches can promote untrusted code.

4 PROOF

Repository event, branch rule, connection ARN, trigger filter and execution correlation ID.

© 2026 Tertiary Infotech Academy Pte Ltd

Pipeline triggers and source governance: failure diagnostic

1

RECOGNISE

Observable failure condition: A trigger that accepts unreviewed branches can promote untrusted code.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Pull request through Pipeline execution; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Repository event, branch rule, connection ARN, trigger filter and execution correlation ID.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 1 · APPLIED SCENARIO

Pipeline triggers and source governance: worked decision

1

CONSTRAINT

Feature branches must run tests, but only reviewed merges to main may enter the release pipeline.

2

DECISION

Use pull-request checks, protected branches, signed connections and event filters; avoid broad polling and shared credentials.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: A trigger that accepts unreviewed branches can promote untrusted code.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Repository event, branch rule, connection ARN, trigger filter and execution correlation ID.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeBuild and automated quality gates

1

OUTCOME

Make build success mean more than compilation by enforcing test, security and quality thresholds.

2

ARCHITECTURE

Pin runtime images and dependencies, isolate privileged builds, publish reports, and fail on explicit exit codes.

3

FAILURE MODE

A green build with ignored test failures is a false control.

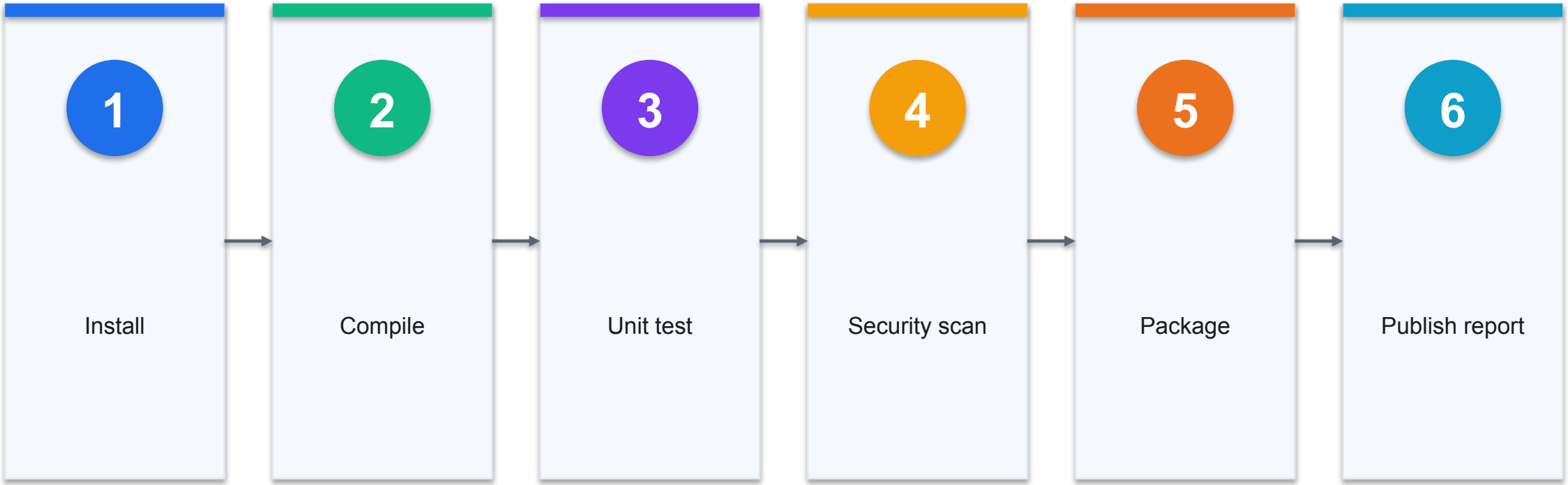
4

EVIDENCE

buildspec version, resolved image digest, report group, coverage result and exported variables.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeBuild and automated quality gates: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeBuild and automated quality gates: design decision

? SCENARIO

A Python service must fail the pipeline below 85% coverage or when a high-severity dependency finding appears.

1 DESIGN RESPONSE

Pin runtime images and dependencies, isolate privileged builds, publish reports, and fail on explicit exit codes.

2 PRIMARY SERVICES

CodeBuild · CodeGuru · test reports · coverage

3 FAILURE MODE

A green build with ignored test failures is a false control.

4 PROOF

buildspec version, resolved image digest, report group, coverage result and exported variables.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeBuild and automated quality gates: failure diagnostic

1

RECOGNISE

Observable failure condition: A green build with ignored test failures is a false control.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Install through Publish report; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with buildspec version, resolved image digest, report group, coverage result and exported variables.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 1 · APPLIED SCENARIO

CodeBuild and automated quality gates: worked decision

1

CONSTRAINT

A Python service must fail the pipeline below 85% coverage or when a high-severity dependency finding appears.

2

DECISION

Pin runtime images and dependencies, isolate privileged builds, publish reports, and fail on explicit exit codes.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: A green build with ignored test failures is a false control.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: buildspec version, resolved image digest, report group, coverage result and exported variables.

© 2026 Tertiary Infotech Academy Pte Ltd

Artifact integrity and lifecycle

1

OUTCOME

Create traceable, encrypted and retention-managed artifacts for every deployment target.

2

ARCHITECTURE

Version packages immutably, scan images, sign or checksum outputs, encrypt repositories and expire superseded artifacts deliberately.

3

FAILURE MODE

Mutable tags such as latest can deploy different bytes under the same release label.

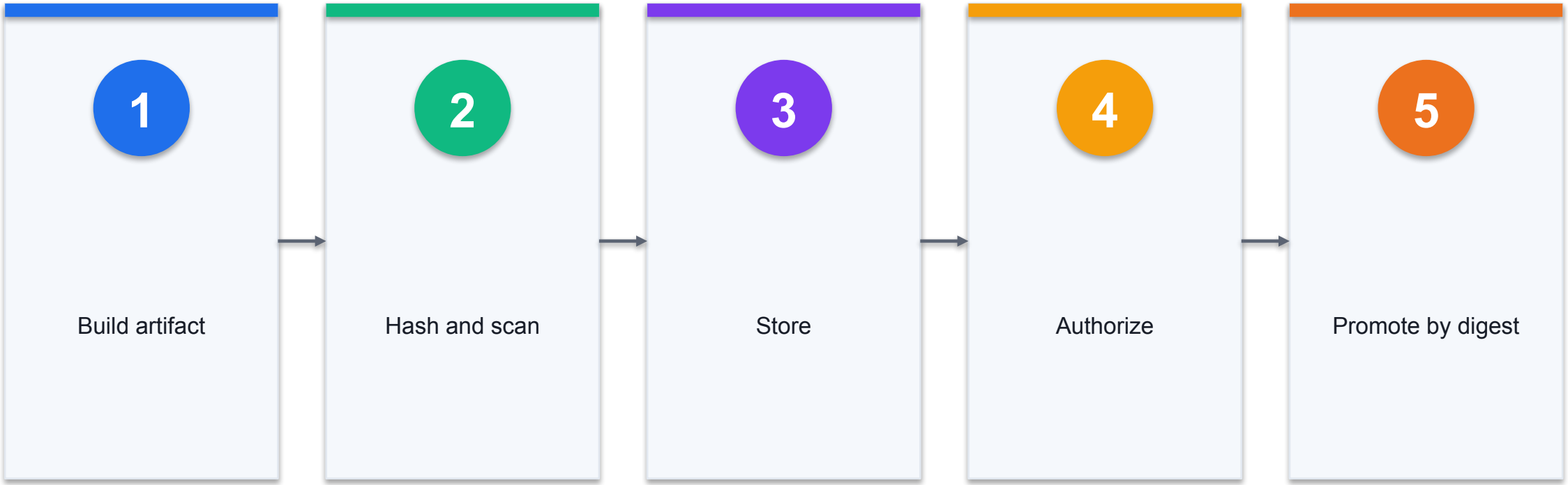
4

EVIDENCE

Package version, digest, scan status, KMS key, repository policy and retention action.

© 2026 Tertiary Infotech Academy Pte Ltd

Artifact integrity and lifecycle: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Artifact integrity and lifecycle: design decision

? SCENARIO

Teams share a container image across accounts and Regions while auditors require proof of exactly what ran.

1 DESIGN RESPONSE

Version packages immutably, scan images, sign or checksum outputs, encrypt repositories and expire superseded artifacts deliberately.

2 PRIMARY SERVICES

CodeArtifact · ECR · S3 · KMS · Image Builder

3 FAILURE MODE

Mutable tags such as latest can deploy different bytes under the same release label.

4 PROOF

Package version, digest, scan status, KMS key, repository policy and retention action.

© 2026 Tertiary Infotech Academy Pte Ltd

Artifact integrity and lifecycle: failure diagnostic

1

RECOGNISE

Observable failure condition: Mutable tags such as latest can deploy different bytes under the same release label.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Build artifact through Promote by digest; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Package version, digest, scan status, KMS key, repository policy and retention action.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 1 · APPLIED SCENARIO

Artifact integrity and lifecycle: worked decision

1

CONSTRAINT

Teams share a container image across accounts and Regions while auditors require proof of exactly what ran.

2

DECISION

Version packages immutably, scan images, sign or checksum outputs, encrypt repositories and expire superseded artifacts deliberately.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Mutable tags such as latest can deploy different bytes under the same release label.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Package version, digest, scan status, KMS key, repository policy and retention action.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeDeploy lifecycle and rollback

1

OUTCOME

Deploy safely to fleets with lifecycle hooks, health checks and a deterministic rollback path.

2

ARCHITECTURE

Match AppSpec hooks to deployment topology; use load-balancer health and alarms to halt or roll back unhealthy revisions.

3

FAILURE MODE

A successful file copy does not prove the application is healthy.

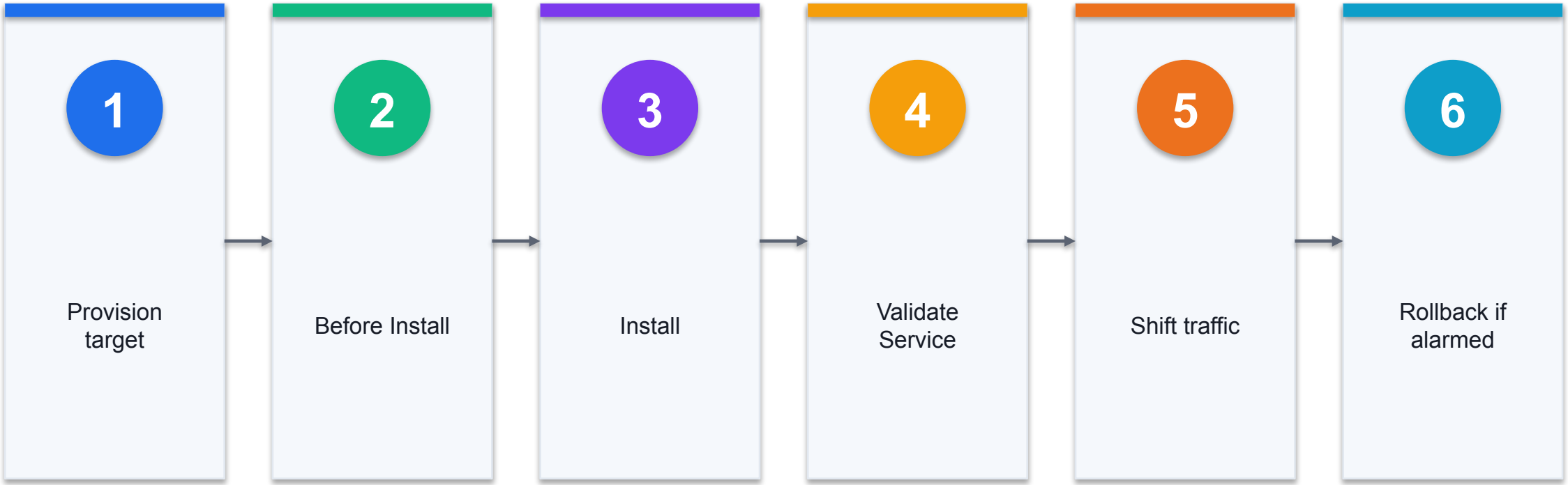
4

EVIDENCE

Revision ID, lifecycle-hook log, deployment configuration, alarm state and rollback deployment ID.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeDeploy lifecycle and rollback: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 1 · SCENARIO

CodeDeploy lifecycle and rollback: design decision

? SCENARIO

An in-place EC2 deployment leaves half the fleet serving incompatible API versions.

1 DESIGN RESPONSE

Match AppSpec hooks to deployment topology; use load-balancer health and alarms to halt or roll back unhealthy revisions.

2 PRIMARY SERVICES

CodeDeploy · EC2 · Auto Scaling · ALB

3 FAILURE MODE

A successful file copy does not prove the application is healthy.

4 PROOF

Revision ID, lifecycle-hook log, deployment configuration, alarm state and rollback deployment ID.

© 2026 Tertiary Infotech Academy Pte Ltd

CodeDeploy lifecycle and rollback: failure diagnostic

1

RECOGNISE

Observable failure condition: A successful file copy does not prove the application is healthy.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Provision target through Rollback if alarmed; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Revision ID, lifecycle-hook log, deployment configuration, alarm state and rollback deployment ID.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 1 · APPLIED SCENARIO

CodeDeploy lifecycle and rollback: worked decision

1

CONSTRAINT

An in-place EC2 deployment leaves half the fleet serving incompatible API versions.

2

DECISION

Match AppSpec hooks to deployment topology; use load-balancer health and alarms to halt or roll back unhealthy revisions.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: A successful file copy does not prove the application is healthy.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Revision ID, lifecycle-hook log, deployment configuration, alarm state and rollback deployment ID.

© 2026 Tertiary Infotech Academy Pte Ltd

Container and serverless deployment strategies

1

OUTCOME

Select canary, linear or all-at-once traffic shifting based on blast radius and recovery needs.

2

ARCHITECTURE

Pair weighted traffic with pre/post hooks, alarms and versioned task definitions or Lambda aliases.

3

FAILURE MODE

Traffic shifts without representative health signals can amplify failure gradually but silently.

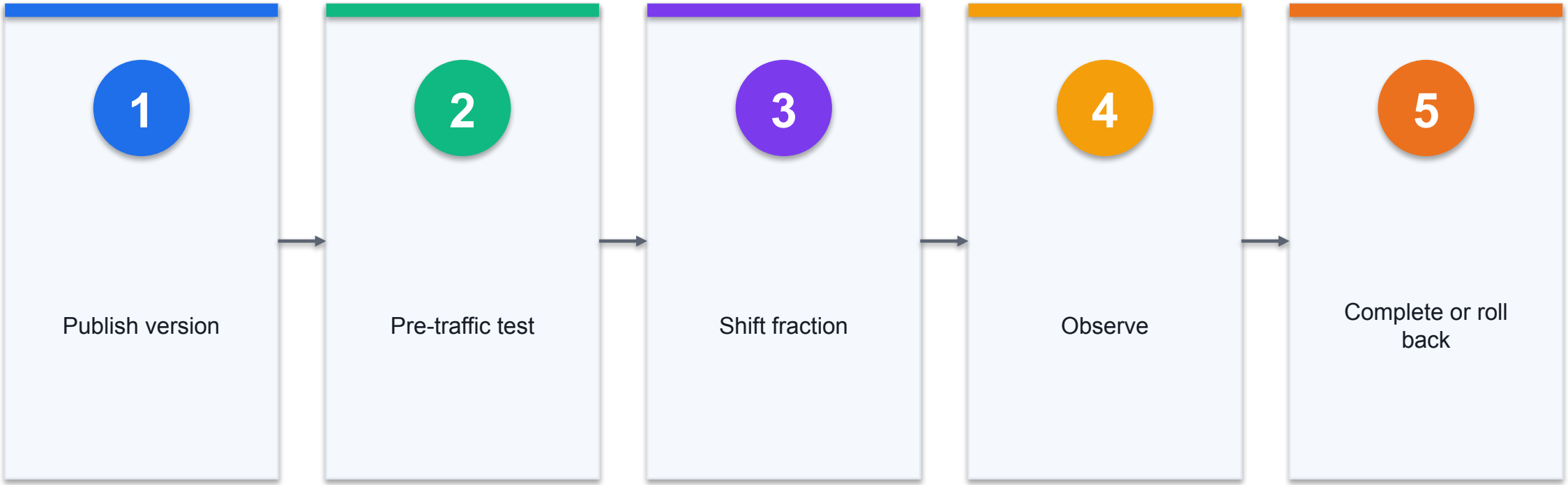
4

EVIDENCE

Task-definition revision or Lambda version, traffic weights, hook results, alarms and rollback state.

© 2026 Tertiary Infotech Academy Pte Ltd

Container and serverless deployment strategies: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Container and serverless deployment strategies: design decision

? SCENARIO

A customer API requires zero-downtime releases and automatic rollback within five minutes.

1 DESIGN RESPONSE

Pair weighted traffic with pre/post hooks, alarms and versioned task definitions or Lambda aliases.

2 PRIMARY SERVICES

ECS · Lambda · CodeDeploy · SAM

3 FAILURE MODE

Traffic shifts without representative health signals can amplify failure gradually but silently.

4 PROOF

Task-definition revision or Lambda version, traffic weights, hook results, alarms and rollback state.

© 2026 Tertiary Infotech Academy Pte Ltd

Container and serverless deployment strategies: failure diagnostic

1

RECOGNISE

Observable failure condition: Traffic shifts without representative health signals can amplify failure gradually but silently.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Publish version through Complete or roll back; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Task-definition revision or Lambda version, traffic weights, hook results, alarms and rollback state.

© 2026 Tertiary Infotech Academy Pte Ltd

Container and serverless deployment strategies: worked decision

1

CONSTRAINT

A customer API requires zero-downtime releases and automatic rollback within five minutes.

2

DECISION

Pair weighted traffic with pre/post hooks, alarms and versioned task definitions or Lambda aliases.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Traffic shifts without representative health signals can amplify failure gradually but silently.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Task-definition revision or Lambda version, traffic weights, hook results, alarms and rollback state.

© 2026 Tertiary Infotech Academy Pte Ltd

Multi-account and multi-Region delivery

1

OUTCOME

Promote one release across governed accounts and Regions without duplicating build logic.

2

ARCHITECTURE

Use cross-account roles, Region-specific artifact stores and keys, ordered waves, approval boundaries and central evidence.

3

FAILURE MODE

Missing KMS or trust-policy permissions often appear only at the first cross-account action.

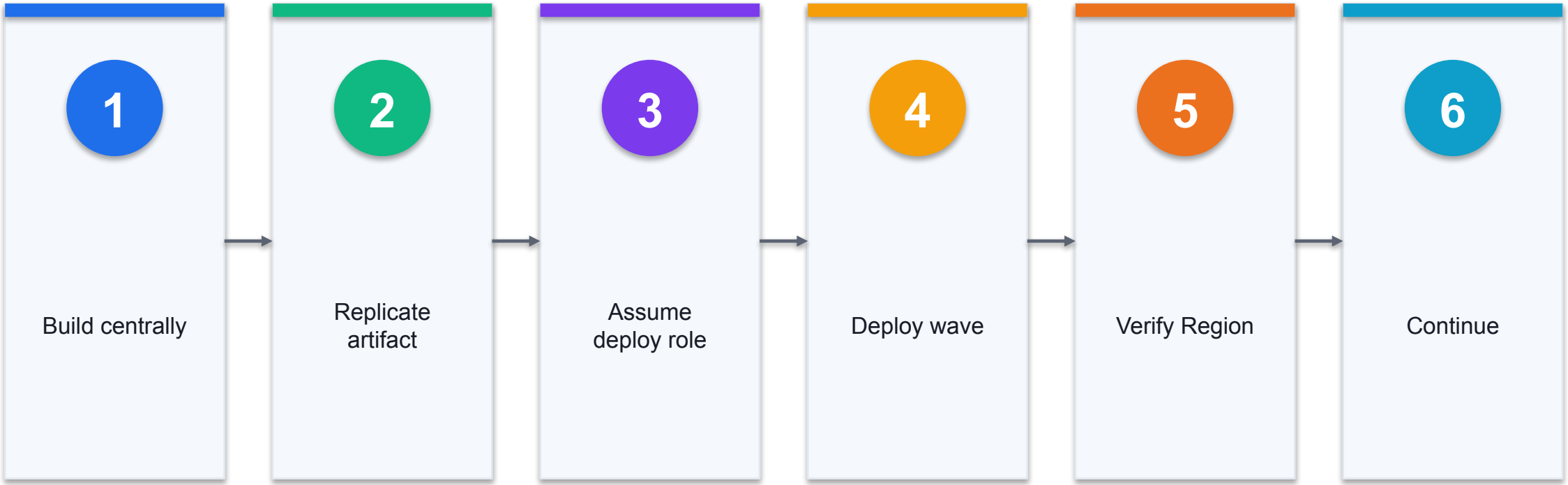
4

EVIDENCE

Role assumption, regional artifact copy, StackSet operation, wave status and approval record.

© 2026 Tertiary Infotech Academy Pte Ltd

Multi-account and multi-Region delivery: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 1 · SCENARIO

Multi-account and multi-Region delivery: design decision

? SCENARIO

A platform team releases to development, staging and two production Regions from one governed pipeline.

1 DESIGN RESPONSE

Use cross-account roles, Region-specific artifact stores and keys, ordered waves, approval boundaries and central evidence.

2 PRIMARY SERVICES

CodePipeline · CloudFormation StackSets · KMS · IAM

3 FAILURE MODE

Missing KMS or trust-policy permissions often appear only at the first cross-account action.

4 PROOF

Role assumption, regional artifact copy, StackSet operation, wave status and approval record.

© 2026 Tertiary Infotech Academy Pte Ltd

Multi-account and multi-Region delivery: failure diagnostic

1

RECOGNISE

Observable failure condition: Missing KMS or trust-policy permissions often appear only at the first cross-account action.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Build centrally through Continue; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Role assumption, regional artifact copy, StackSet operation, wave status and approval record.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 1 · APPLIED SCENARIO

Multi-account and multi-Region delivery: worked decision

1

CONSTRAINT

A platform team releases to development, staging and two production Regions from one governed pipeline.

2

DECISION

Use cross-account roles, Region-specific artifact stores and keys, ordered waves, approval boundaries and central evidence.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Missing KMS or trust-policy permissions often appear only at the first cross-account action.

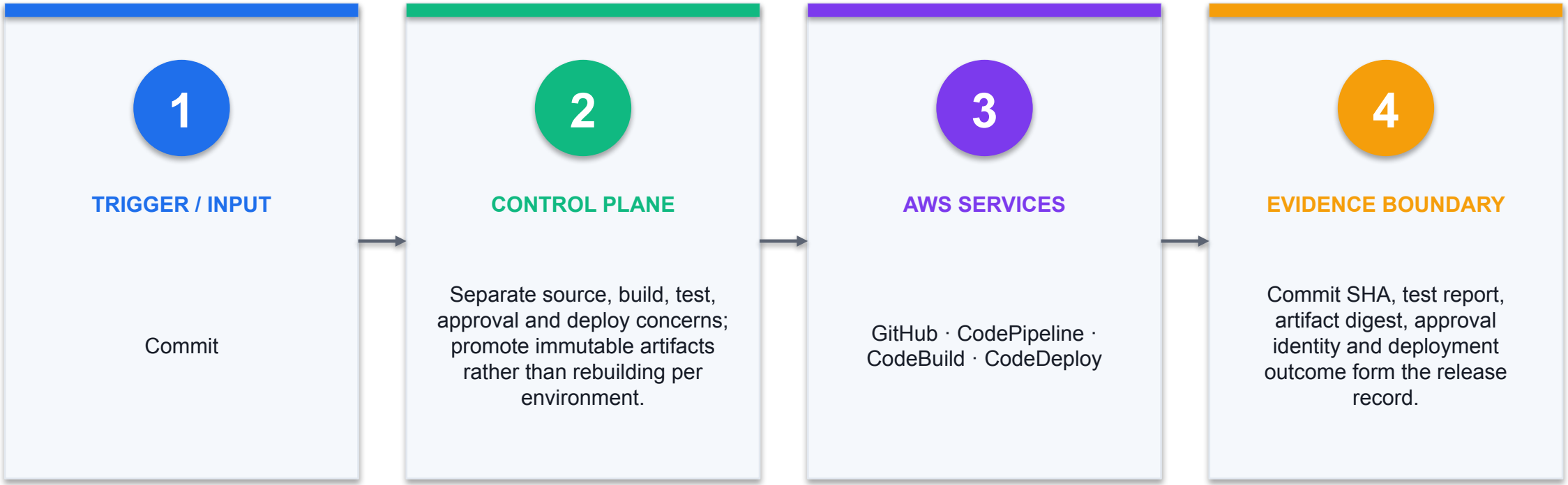
4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Role assumption, regional artifact copy, StackSet operation, wave status and approval record.

© 2026 Tertiary Infotech Academy Pte Ltd

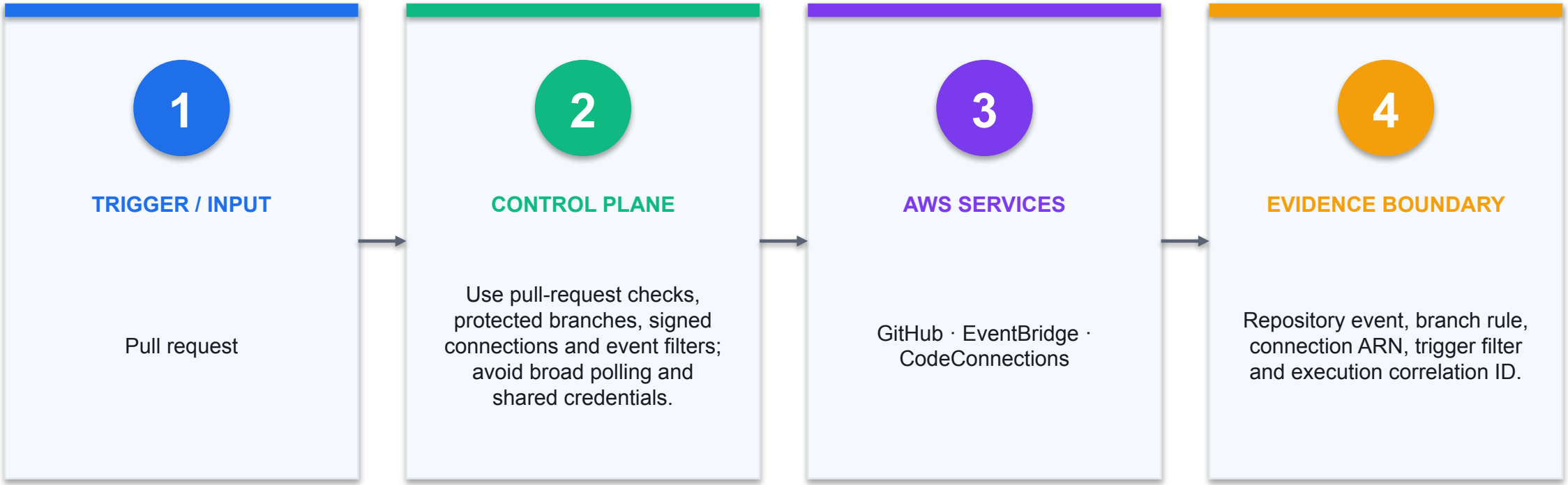
Continuous delivery operating model: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

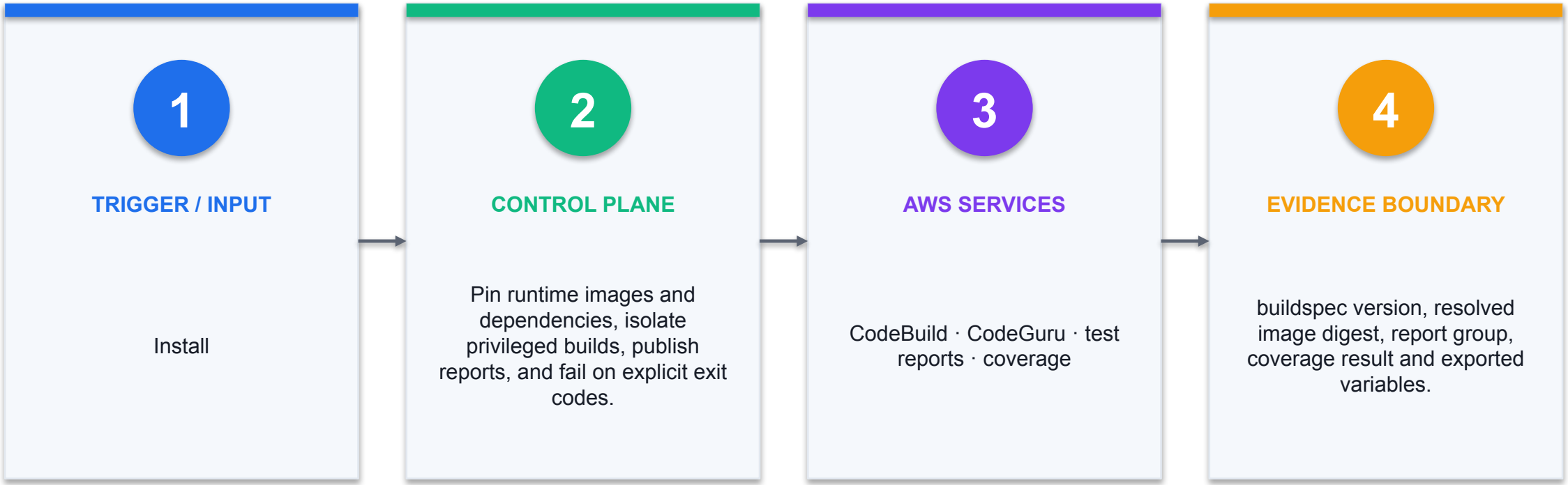
Pipeline triggers and source governance: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

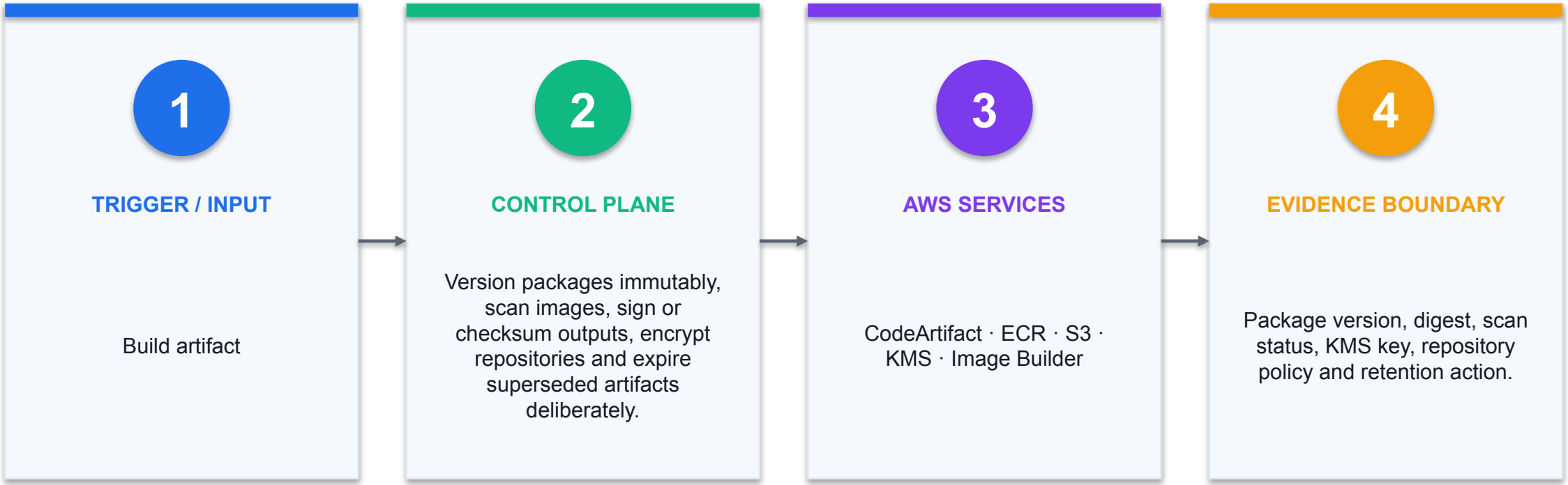
CodeBuild and automated quality gates: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

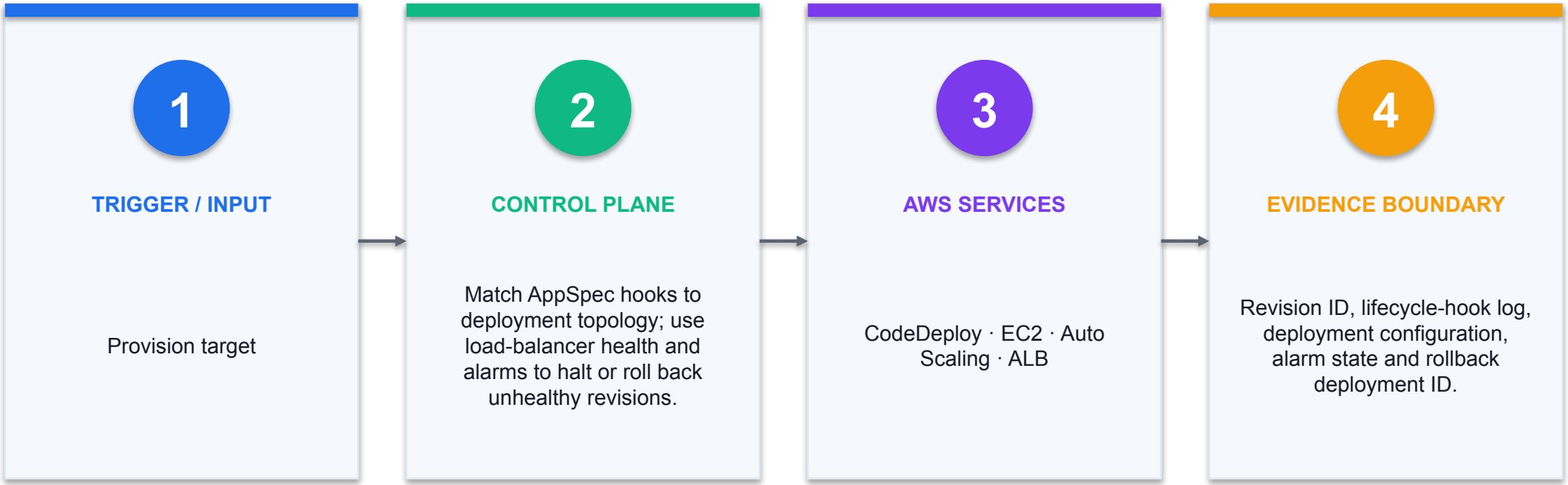
Artifact integrity and lifecycle: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

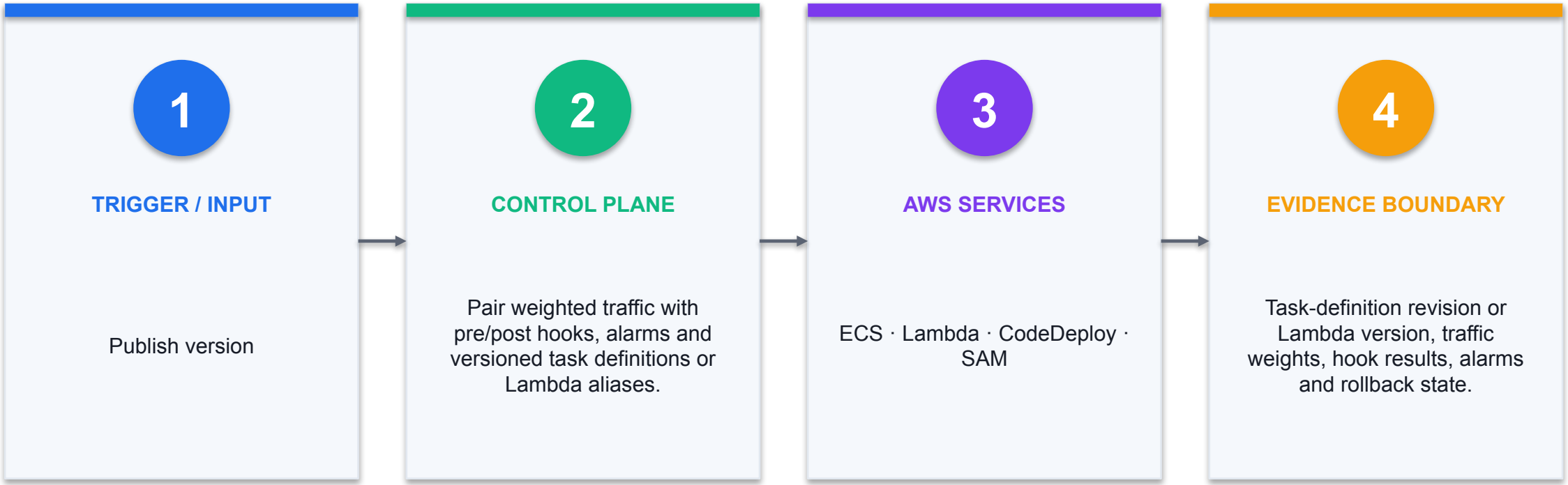
CodeDeploy lifecycle and rollback: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

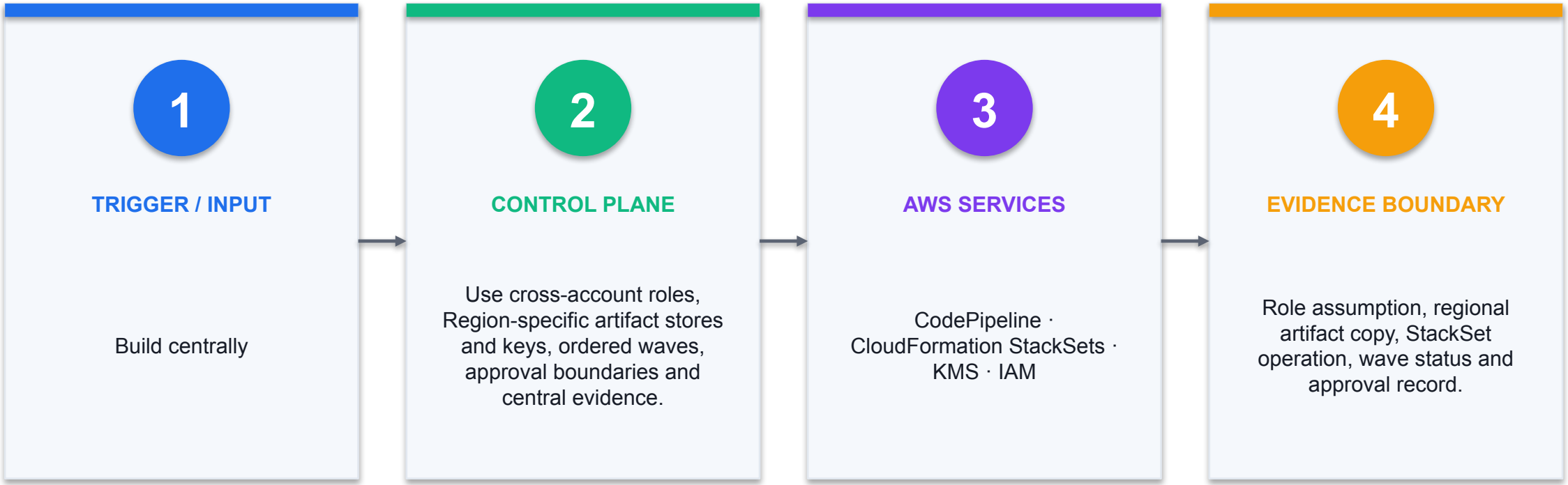
Container and serverless deployment strategies: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Multi-account and multi-Region delivery: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

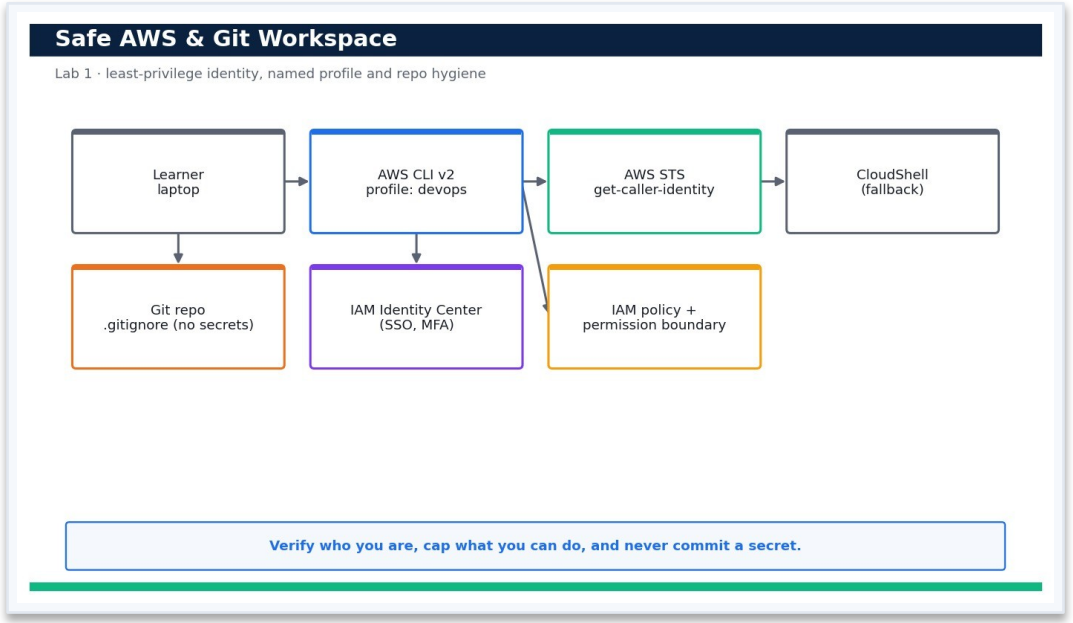
© 2026 Tertiary Infotech Academy Pte Ltd

Set Up a Safe AWS & Git Working Environment and Verify Identity

LAB 1

Day 1 · LO1 · K1, A1

Before automating anything you need a workspace you can trust.



Reference architecture

YOU'LL BUILD

A named, MFA-aware, least-privilege 'devops' CLI profile plus a Git repo with a secrets-safe.gitignore, verified with an AWS caller-identity check and a clean secret scan.

Tools: AWS IAM, AWS CLI v2, IAM Identity Center, CloudShell, Git

OUTCOME & EVIDENCE

Outcome: Stand up a least-privilege AWS CLI/Git workspace and confirm your identity.

Evidence: Screenshot of showing your own Account and ARN.

© 2026 Tertiary Infotech Academy Pte Ltd

Establish a Source-Control Workflow and Branching for the Sample App

LAB 2

Day 1 · LO1 · K1, A1

Good delivery starts with disciplined source control.

YOU'LL BUILD

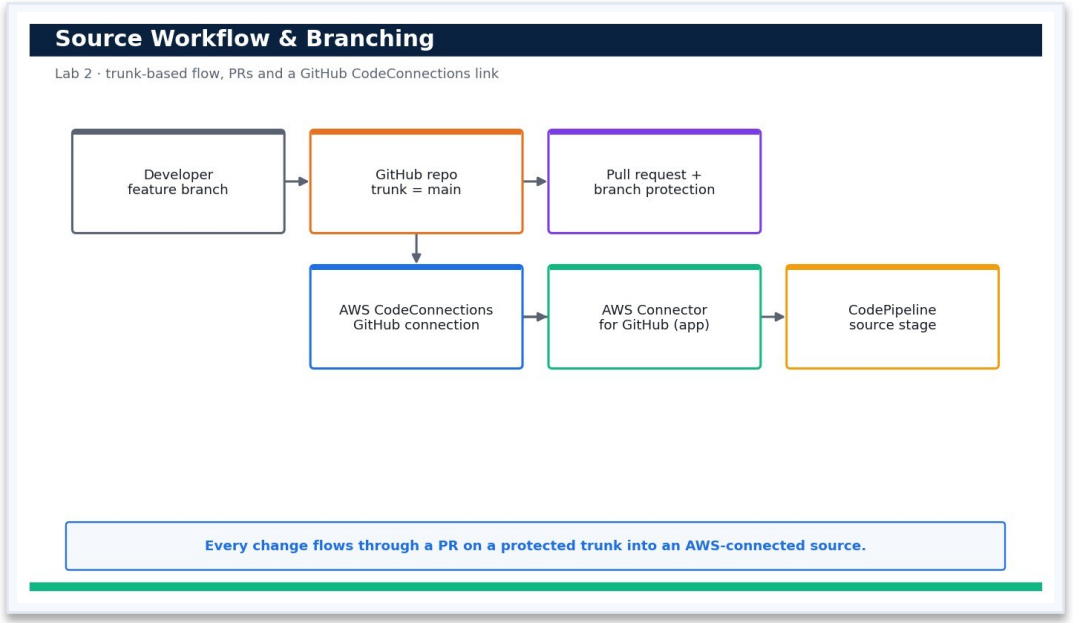
A protected trunk-based GitHub repo with an open/merged PR, plus an AVAILABLE AWS CodeConnections connection to GitHub ready to drive CodeBuild and CodePipeline.

Tools: GitHub, AWS CodeConnections, Git, AWS CodePipeline (source stage)

OUTCOME & EVIDENCE

Outcome: Adopt a trunk-based Git workflow and connect the repository to AWS with CodeConnections.

Evidence: The GitHub PR page for feature/bump-version showing the diff and an approving review.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · DAY 1 · LO1 · HANDS-ON

Run Automated Tests on Every Commit with AWS CodeBuild

LAB 3

Day 1 · LO1 · K1, A1

Automated tests are the gate that keeps broken code out of production.

YOU'LL BUILD

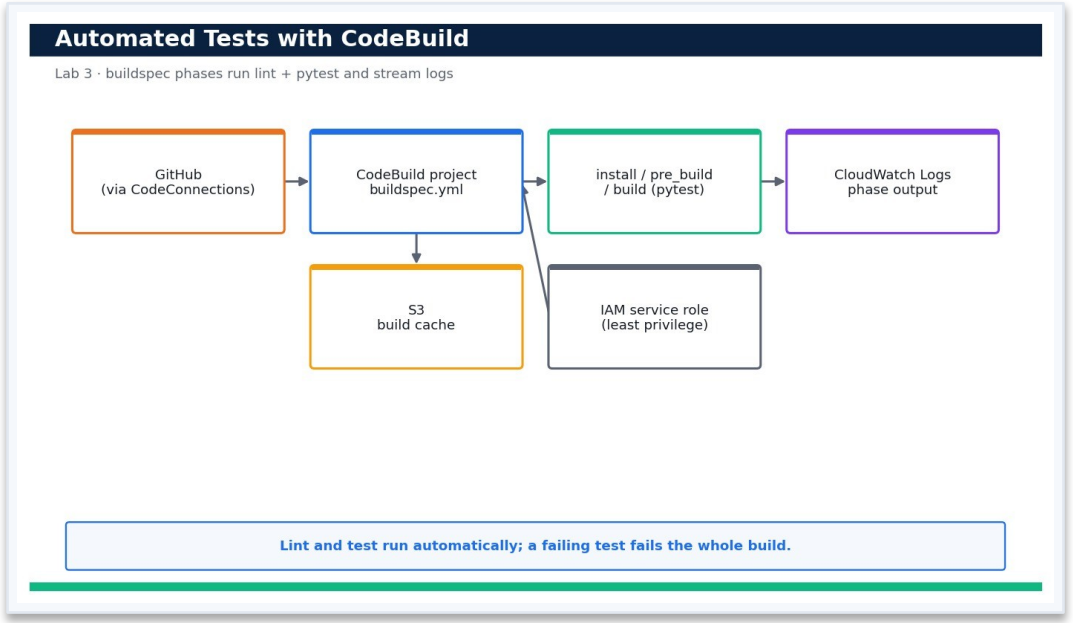
A CodeBuild project that runs the linter and, streams phase logs to CloudWatch, and fails loudly when a test is broken.

Tools: AWS CodeBuild, Amazon S3, AWS IAM, Amazon CloudWatch Logs

OUTCOME & EVIDENCE

Outcome: Author a buildspec and a CodeBuild project that automatically lints and tests the app.

Evidence: The CodeBuild build page showing buildStatus SUCCEEDED with INSTALL/PRE_BUILD/BUILD green.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Produce and Store Versioned Build Artifacts with a Buildspec

LAB 4

Day 1 · LO1 · K1, A1

A pipeline is only trustworthy if you can identify and reproduce exactly what it built.

YOU'LL BUILD

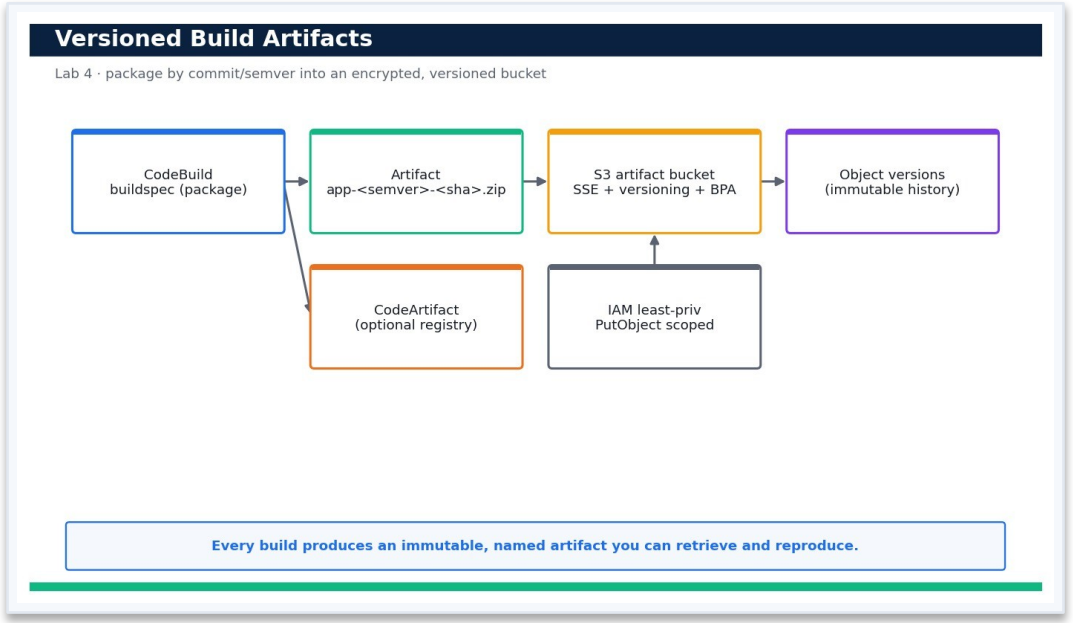
An encrypted, version-enabled S3 artifact bucket holding immutable, commit/semver-named build artifacts that you can list, retrieve and reproduce.

Tools: AWS CodeBuild, Amazon S3, AWS CodeArtifact, AWS IAM

OUTCOME & EVIDENCE

Outcome: Extend the buildspec to package versioned artifacts and store them in an encrypted, versioned bucket.

Evidence: output showing Status: Enabled.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Assemble a CodePipeline with Manual Approval and a Deployment Strategy

LAB 5

Day 1 · LO1 · K1, A1

Now you connect everything into one governed release path.

YOU'LL BUILD

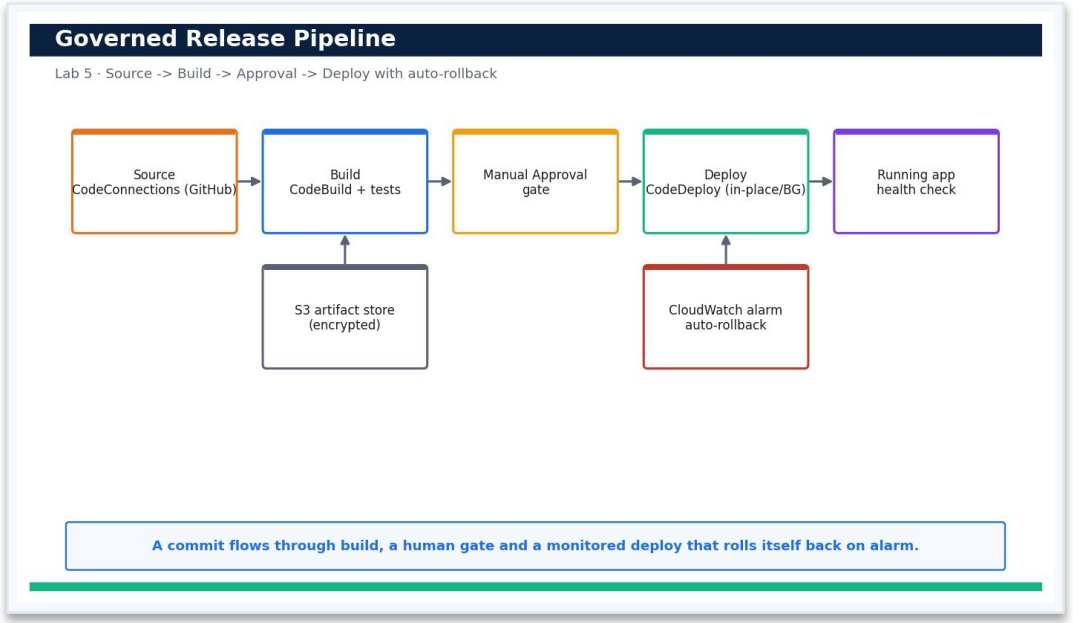
A four-stage CodePipeline (Source -> Build -> Approve -> Deploy) with a manual approval gate, a chosen deployment strategy and alarm-based automatic rollback.

Tools: AWS CodePipeline, CodeBuild, CodeDeploy, CodeConnections, Amazon S3 + 2 more

OUTCOME & EVIDENCE

Outcome: Wire an end-to-end pipeline with a manual approval gate, a deployment strategy and auto-rollback.

Evidence: The CodePipeline view showing Source, Build, Approve and Deploy, paused at the Approve gate.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN RECAP

Recap — SDLC Automation

1

Lab 1

Stand up a least-privilege AWS CLI/Git workspace and confirm your identity.

2

Lab 2

Adopt a trunk-based Git workflow and connect the repository to AWS with CodeConnections.

3

Lab 3

Author a buildspec and a CodeBuild project that automatically lints and tests the app.

4

Lab 4

Extend the buildspec to package versioned artifacts and store them in an encrypted, versioned bucket.

5

Lab 5

Wire an end-to-end pipeline with a manual approval gate, a deployment strategy and auto-rollback.

6

Key principle

Developers push often; CodeBuild runs the build and tests on every commit so bugs are found early, not at release.

© 2026 Tertiary Infotech Academy Pte Ltd

02

DOMAIN 02

Configuration Management and IaC

Exam weighting 17% · Infrastructure as Code · reusable templates · multi-account & multi-Region

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 02 · WEIGHTING 17%

Key Concepts — Configuration Management and IaC

1

Infrastructure as Code

Declare infrastructure in version-controlled templates so environments are repeatable, reviewable and disposable.

2

CloudFormation

Declarative JSON/YAML stacks with parameters, mappings, outputs, change sets, drift detection and automatic rollback.

3

Reusable components

Nested stacks and modules parameterise a pattern once and reuse it across teams and Regions.

4

CDK vs CloudFormation vs Terraform

CDK generates CloudFormation from real code; Terraform is multi-cloud — all describe desired state.

5

Configuration management

Systems Manager (State Manager, Automation, Parameter Store) keeps running instances in their desired configuration.

6

Multi-account & multi-Region

Organizations, StackSets and Control Tower create, onboard and secure accounts at scale.

© 2026 Tertiary Infotech Academy Pte Ltd

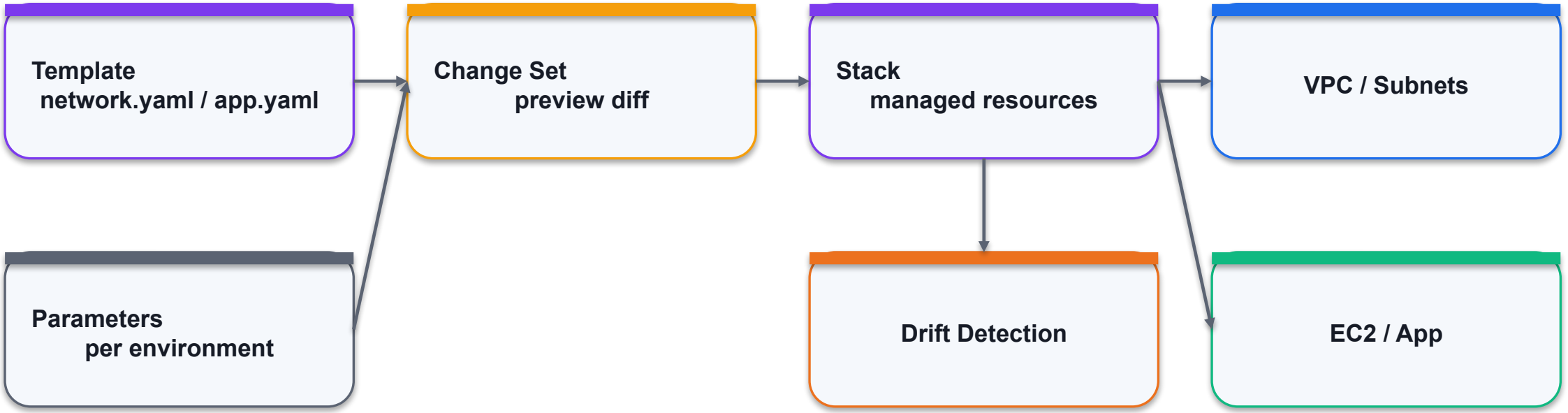
DECLARE DESIRED STATE

IaC Options on AWS

Tool	Language	Scope	When to use
CloudFormation	YAML / JSON	AWS-native declarative	Standard AWS IaC, StackSets
AWS CDK	Python / TS / Java	Code → CloudFormation	Reusable, testable constructs
Terraform	HCL	Multi-cloud	Mixed-cloud estates
SAM	YAML	Serverless-focused	Lambda / API / event apps

© 2026 Tertiary Infotech Academy Pte Ltd

CloudFormation — Reusable Stacks



- One parameterised template deploys identical dev, test and prod environments; change sets preview, drift detection audits.

© 2026 Tertiary Infotech Academy Pte Ltd

MULTI-ACCOUNT / MULTI-REGION

Configuration Management & Governance at Scale

1

Nested stacks / modules

Package a pattern once and reuse it across teams.

2

Systems Manager

State Manager, Automation and Parameter Store keep instances in desired state.

3

StackSets

Deploy the same stack across many accounts and Regions.

4

Organizations & Control Tower

Create, onboard and secure accounts with a landing zone.

© 2026 Tertiary Infotech Academy Pte Ltd

TEMPLATE STRUCTURE

Anatomy of a CloudFormation Template

1

Parameters

Inputs that make the template reusable across environments.

2

Mappings

Static lookup tables (e.g. AMI by Region).

3

Conditions

Create resources only when a condition is met.

4

Resources

The AWS resources to provision (the only required section).

5

Outputs

Values to export and import into other stacks.

6

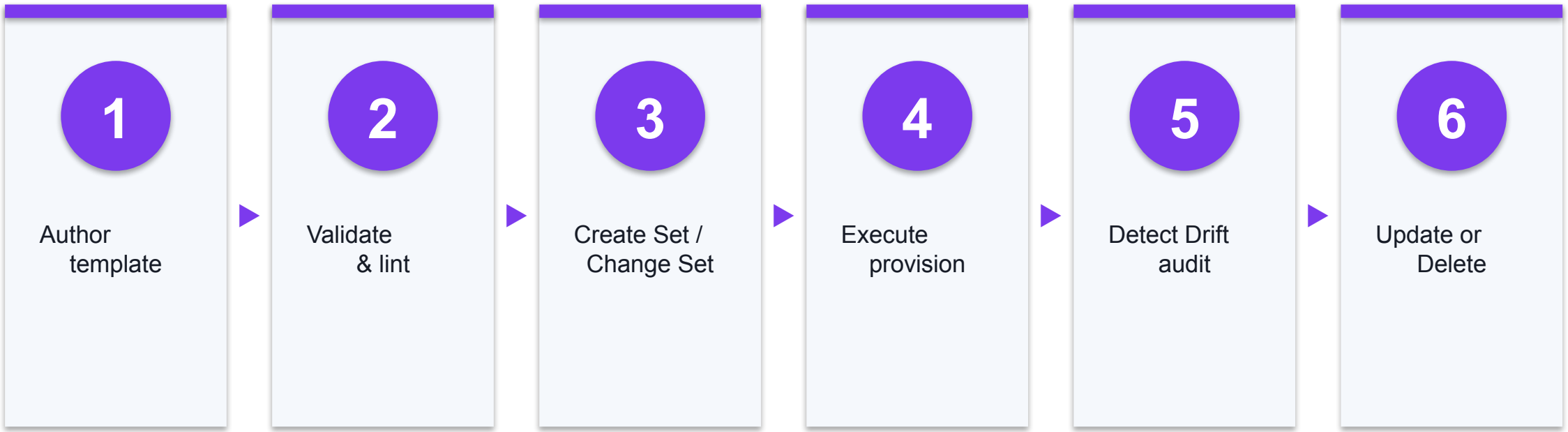
Metadata

Extra structured information about the template.

© 2026 Tertiary Infotech Academy Pte Ltd

SAFE CHANGE MANAGEMENT

The Stack Lifecycle



© 2026 Tertiary Infotech Academy Pte Ltd

Parameter Store vs Secrets Manager

Store	Best for	Rotation	Cost
SSM Parameter Store	Config values & non-secret parameters	Manual	Free (standard)
Secrets Manager	Passwords, API keys, DB credentials	Automatic	Per-secret

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Infrastructure as Code — declarative desired state

IN DEPTH

- You describe the target state of infrastructure in version-controlled templates; the service computes and applies the difference.
- The same template deploys identical dev, test and prod environments — parameters, not copy-paste, capture the differences.
- Infrastructure changes go through code review and the pipeline, just like application code.

WHY IT MATTERS

Repeatable, reviewable, disposable environments eliminate configuration drift and the 'it works on that server' class of failure.

FAILURE MODE

Mixing manual console changes with IaC creates drift the template no longer describes — the next deploy can revert or clash with them.

TRADE-OFF / DECISION

Declarative IaC is safe and auditable but less flexible than imperative scripts — use change sets and modules to regain expressiveness.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

CloudFormation stacks, change sets and rollback

IN DEPTH

- A stack is a unit of resources created, updated and deleted together from one template.
- A change set previews exactly what an update will add, modify or replace before you execute it.
- On failure, CloudFormation automatically rolls the stack back to its last known-good state.

WHY IT MATTERS

Preview-then-apply plus automatic rollback make infrastructure changes as safe and reversible as a code deployment.

FAILURE MODE

A resource that must be replaced (not updated in place) can cause downtime or data loss if you don't read the change set's Replacement column.

TRADE-OFF / DECISION

Automatic rollback protects you but can mask the real error — always read the stack Events for the first failing resource.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Template anatomy — parameters, mappings, conditions, outputs

IN DEPTH

- Parameters make a template reusable across environments; Mappings hold static lookups such as AMI-by-Region.
- Conditions create resources only when appropriate (e.g. a NAT gateway only in prod); Outputs export values for other stacks.
- Only the Resources section is mandatory — everything else exists to make templates reusable and composable.

WHY IT MATTERS

A well-structured template is a reusable product, not a one-off script — one artifact serves many teams and Regions.

FAILURE MODE

Hard-coded AMI IDs, account numbers or Region strings make a template that only works in the account it was written in.

TRADE-OFF / DECISION

Heavy parameterisation increases flexibility but reduces readability — prefer sensible defaults and SSM public parameters.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Reuse — nested stacks, modules and StackSets

IN DEPTH

- Nested stacks and CloudFormation modules package a pattern once and reuse it across many stacks.
- StackSets deploy the same stack across many accounts and Regions from a single operation.
- Cross-stack references and exports let a network stack publish IDs that app stacks import.

WHY IT MATTERS

Reuse turns a proven design into an organisational standard, so every team gets the same secure, compliant baseline.

FAILURE MODE

Over-nesting creates tightly coupled stacks where one change forces wide, slow updates and hard-to-debug failures.

TRADE-OFF / DECISION

Central StackSets enforce consistency but need governance so account teams aren't blocked waiting on the platform team.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

CDK, SAM and Terraform — when to reach for each

IN DEPTH

- The AWS CDK lets you express infrastructure in a general-purpose language and synthesises CloudFormation.
- AWS SAM is a CloudFormation extension optimised for serverless apps.
- Terraform describes desired state in HCL across many clouds and providers.

WHY IT MATTERS

Choosing the right tool matches the team's skills and estate — real code for testable constructs, SAM for serverless, Terraform for multi-cloud.

FAILURE MODE

Adopting CDK without CloudFormation literacy hides what is really deployed and makes drift and failures harder to reason about.

TRADE-OFF / DECISION

Higher-level abstractions boost productivity but add a layer to debug when the generated template misbehaves.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Configuration management with Systems Manager

IN DEPTH

- State Manager keeps instances in a defined configuration; Automation documents run repeatable operational runbooks.
- Parameter Store holds configuration and (with Secrets Manager) secrets, versioned and access-controlled.
- Patch Manager and Inventory keep fleets patched and auditable without SSH access.

WHY IT MATTERS

Config management closes the gap between 'provisioned' and 'correctly configured and maintained over time'.

FAILURE MODE

Treating servers as pets you SSH into and hand-tune reintroduces drift that no template can track.

TRADE-OFF / DECISION

Agent-based management adds an agent to run and secure, but removes bastion hosts and standing SSH access.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Drift detection and remediation

IN DEPTH

- Drift detection compares live resources against the template and reports what changed out-of-band.
- Config rules can watch for the same drift continuously and trigger remediation.
- The fix is to update the template to match intent or revert the resource to the template.

WHY IT MATTERS

Detecting drift early prevents the slow divergence that eventually makes a stack un-updatable.

FAILURE MODE

Ignoring drift until an urgent change is needed means your 'safe' update now collides with undocumented manual edits.

TRADE-OFF / DECISION

Strict no-drift policies improve auditability but can slow emergency fixes — provide a documented break-glass path.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Multi-account governance — Organizations, Control Tower, SCPs

IN DEPTH

- AWS Organizations groups accounts; Service Control Policies set the maximum permissions any account can grant.
- Control Tower stands up a landing zone with guardrails, a shared logging account and account factory.
- New accounts are vended pre-configured to the security and network baseline.

WHY IT MATTERS

Account-level isolation plus org-wide guardrails give strong blast-radius control and a consistent, auditable baseline.

FAILURE MODE

A single shared account for everything makes least-privilege impossible and one mistake blast-radius huge.

TRADE-OFF / DECISION

More accounts improve isolation but add networking and operational overhead — automate account vending to keep it manageable.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Secrets & configuration — Parameter Store vs Secrets Manager

IN DEPTH

- Parameter Store holds configuration values (and SecureString secrets) with a generous free tier.
- Secrets Manager adds built-in, scheduled rotation and tight integration with RDS and other services.
- Both keep sensitive values out of templates, code and logs, retrieved at deploy/run time by IAM-scoped principals.

WHY IT MATTERS

Externalising configuration and secrets lets one immutable artifact run safely in every environment.

FAILURE MODE

Baking a database password into a template or AMI leaks it into every copy and every log line.

TRADE-OFF / DECISION

Automatic rotation is safest but must be tested — a rotation that apps don't reload can cause outages.

© 2026 Tertiary Infotech Academy Pte Ltd

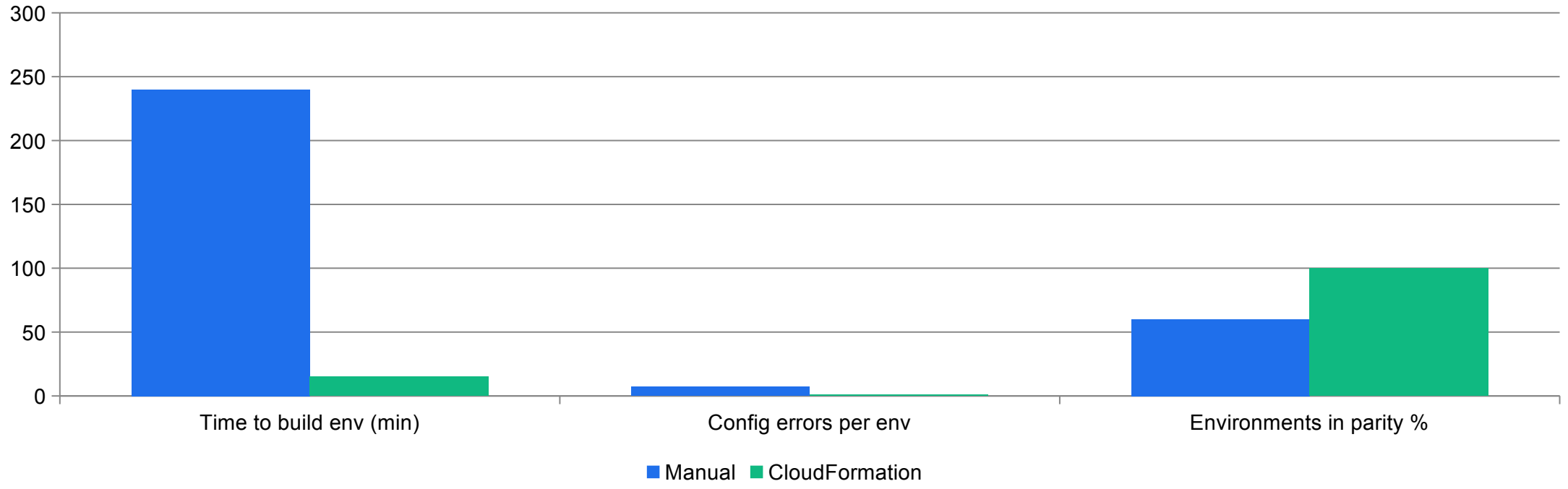
Change Set Replacement — Read Before You Execute

Change type	CloudFormation action	Operational risk
Update in place	Modify the existing resource	Low — no replacement
Replacement	Create new, delete old	High — new ID, possible data loss
Add / remove	Create or delete a resource	Medium — check dependencies
No interruption	Property change, resource kept	Low

© 2026 Tertiary Infotech Academy Pte Ltd

REPEATABILITY PAYS OFF

Environment Provisioning — Manual vs IaC



WHAT THE DATA SHOWS

IaC collapses environment build time and errors and — most importantly — makes every environment identical, so tests actually predict production.

© 2026 Tertiary Infotech Academy Pte Ltd

WHERE IaC GOES WRONG

IaC Failure Modes & Anti-Patterns

1

Out-of-band console edits

Drift the template can't see → detect drift; make console read-only in prod.

2

Hard-coded IDs/Regions

Template only works in one account → parameters + SSM public parameters.

3

Giant monolith stack

Slow, risky updates → split by lifecycle and blast radius.

4

No change-set review

Surprise replacements → always preview before execute.

5

Secrets in templates

Leaked credentials → Secrets Manager / SSM SecureString.

6

Copy-paste per environment

Divergent envs → one template, many parameter sets.

© 2026 Tertiary Infotech Academy Pte Ltd

PROVEN PRACTICES

IaC Patterns Worth Adopting

1

Layered stacks

Separate network, platform and app stacks by change rate.

2

Landing zone

Control Tower baseline + guardrails for every account.

3

Immutable AMIs

Bake with a pipeline (e.g. EC2 Image Builder); replace, don't patch.

4

Public SSM parameters

Reference latest AMIs without hard-coding IDs.

5

Policy as code

Guardrails via SCPs and Config rules, versioned like IaC.

6

Pipeline-deployed stacks

No manual stack updates in production.

© 2026 Tertiary Infotech Academy Pte Ltd

STACK DECOMPOSITION

Architecture Decision — One Big Stack or Many?

A Single stack

Small app, one team, one lifecycle. Simple to reason about — but every change risks the whole system and updates get slow as it grows.

B Layered stacks

Separate network / platform / app by change rate and owner. Smaller blast radius and faster updates — needs disciplined exports/imports.

C StackSets across accounts

Same baseline in many accounts/Regions. Enforces consistency at scale — requires central governance and account-team coordination.

© 2026 Tertiary Infotech Academy Pte Ltd

THE COST OF DRIFT

Scenario — A 'quick' console fix breaks the next deploy

SITUATION

An engineer widens a security group in the console during an incident. Two weeks later, a routine stack update fails or silently reverts the fix.

ANALYSIS

- The live resource no longer matches the template, so CloudFormation's view of desired state is wrong.
- Drift detection would have flagged this the same day; Config rules could have alarmed on the open port immediately.
- The durable fix is to encode the change in the template and re-apply, not to keep patching by hand.

DECISION

Run drift detection on a schedule, alarm on security-group drift via Config, and require that emergency console changes are back-ported into the template within the incident's follow-up.

© 2026 Tertiary Infotech Academy Pte Ltd

SCALING GOVERNANCE

Scenario — Standing up a compliant new account fast

SITUATION

A new business unit needs an AWS account that already meets the company's security, logging and network standards on day one.

ANALYSIS

- Hand-building the account is slow and error-prone and guarantees it drifts from the standard.
- Control Tower's account factory vends accounts pre-wired to guardrails and centralised logging.
- StackSets push the standard network and security stacks into the new account automatically.

DECISION

Vend the account through Control Tower and apply the baseline via StackSets, so 'compliant by default' is the path of least resistance rather than a later audit finding.

© 2026 Tertiary Infotech Academy Pte Ltd

ONE ARTIFACT, MANY ENVIRONMENTS

Worked Analysis — Parameterising One Template for Three Environments

Instance size by env	dev t3.micro, staging t3.small, prod m5.large — a single InstanceType parameter.
High availability by env	A Condition creates the Multi-AZ standby and NAT redundancy only when EnvName = prod.
AMI without hard-coding	An SSM public parameter resolves the latest patched Amazon Linux AMI at deploy time.
Secrets by env	Each environment references its own Secrets Manager secret; none appear in the template.

TAKEAWAY

The differences between environments become a short parameter file — the template itself is identical, so staging genuinely predicts production.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Infrastructure as code engines

1

PURPOSE

Model infrastructure as reviewable desired state and apply it through controlled change sets.

2

FEATURE 1

CloudFormation manages dependency order, rollback, drift detection, stack policies and change sets.

3

FEATURE 2

CDK synthesises familiar programming constructs into CloudFormation templates.

4

FEATURE 3

SAM adds serverless shorthand while retaining CloudFormation deployment semantics.

© 2026 Tertiary Infotech Academy Pte Ltd

Infrastructure as code engines: selection and controls

1

CHOOSE WHEN

Use CloudFormation as the deployment contract; choose CDK for reusable programmatic constructs and SAM for serverless-focused applications.

2

DEVOPS ENGINEERING

Lint, synthesize, test, create a change set, review replacement impact, deploy and detect drift.

3

FAILURE MODE

Generated IaC does not remove the need to review IAM expansion, data replacement and destructive removal policies.

4

EVIDENCE

Template digest, test output, change set, approval, stack event, drift status and rollback result.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Multi-account provisioning

1

PURPOSE

Apply repeatable baselines across accounts and Regions while separating governance from workload delivery.

2

FEATURE 1

Organizations groups accounts in OUs and applies policies at hierarchical boundaries.

3

FEATURE 2

StackSets deploys CloudFormation stacks across target accounts/Regions with concurrency and failure controls.

4

FEATURE 3

Control Tower adds landing-zone guardrails, account vending and drift visibility.

© 2026 Tertiary Infotech Academy Pte Ltd

Multi-account provisioning: selection and controls

1

CHOOSE WHEN

Use StackSets for repeatable resources and Control Tower when managed landing-zone governance and account lifecycle are needed.

2

DEVOPS ENGINEERING

Pilot in one OU, bound concurrency/failure tolerance, preserve delegated administration and verify every target instance.

3

FAILURE MODE

A broad organization rollout can multiply one template or policy defect across the estate.

4

EVIDENCE

OU target list, stack-instance status, failed-account list, policy version and drift record.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Fleet configuration and patching

1

PURPOSE

Operate heterogeneous compute without inbound administrative ports or manual host-by-host changes.

2

FEATURE 1

Managed nodes use the SSM Agent and an instance/hybrid activation identity.

3

FEATURE 2

State Manager enforces associations; Patch Manager evaluates baselines and compliance.

4

FEATURE 3

Automation runbooks compose AWS API actions, commands, approvals, branching and rollback.

© 2026 Tertiary Infotech Academy Pte Ltd

Fleet configuration and patching: selection and controls

1

CHOOSE WHEN

Use State Manager for desired configuration, Patch Manager for patch compliance and Automation for multi-step operational workflows.

2

DEVOPS ENGINEERING

Target by tags/resource groups, use MaxConcurrency/MaxErrors, canary first and keep parameters out of scripts.

3

FAILURE MODE

A command that targets all instances at once can turn routine remediation into a fleet-wide outage.

4

EVIDENCE

Target query, association/runbook version, command output, compliance state, error threshold and rollback.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Workflow and event automation

1

PURPOSE

Decouple operational events from durable, observable automation workflows.

2

FEATURE 1

EventBridge rules and event buses route matched events between producers and targets.

3

FEATURE 2

Step Functions models state, retries, catches, parallelism, waits and human approval.

4

FEATURE 3

SQS buffers work and provides visibility timeouts, redrive and dead-letter queues.

© 2026 Tertiary Infotech Academy Pte Ltd

Workflow and event automation: selection and controls

1

CHOOSE WHEN

Use EventBridge for routing, Step Functions for stateful orchestration and SQS when buffering/back-pressure are required.

2

DEVOPS ENGINEERING

Design idempotent tasks, bounded retries with jitter/backoff, explicit timeouts and dead-letter handling.

3

FAILURE MODE

Unlimited retry or non-idempotent recovery can amplify failure and duplicate side effects.

4

EVIDENCE

Event ID, rule match, execution history, retry count, DLQ depth and final state.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: CloudFormation template and stack lifecycle

1

PURPOSE

Represent infrastructure as desired state, preview resource impact and preserve data through controlled lifecycle policies.

2

SERVICE BOUNDARY

CloudFormation · change sets · drift detection · hooks

3

PRIMARY DESIGN DECISION

Generate a change set, inspect replacement flags, snapshot or migrate data explicitly, apply retention policies and rehearse the recovery path.

4

FAILURE BOUNDARY

Treating every stack update as in-place can delete or orphan stateful resources; drift can make the preview incomplete.

© 2026 Tertiary Infotech Academy Pte Ltd

CloudFormation template and stack lifecycle: how it works

1

MECHANIC 1

Parameters provide deployment-time values, mappings hold known lookup data and conditions control resource inclusion without duplicating templates.

2

MECHANIC 2

References and `Fn::GetAtt` build dependency edges; explicit `DependsOn` is reserved for dependencies CloudFormation cannot infer.

3

MECHANIC 3

Update and replacement behavior is resource-property specific, so a change set must be read for replacement, deletion and import consequences.

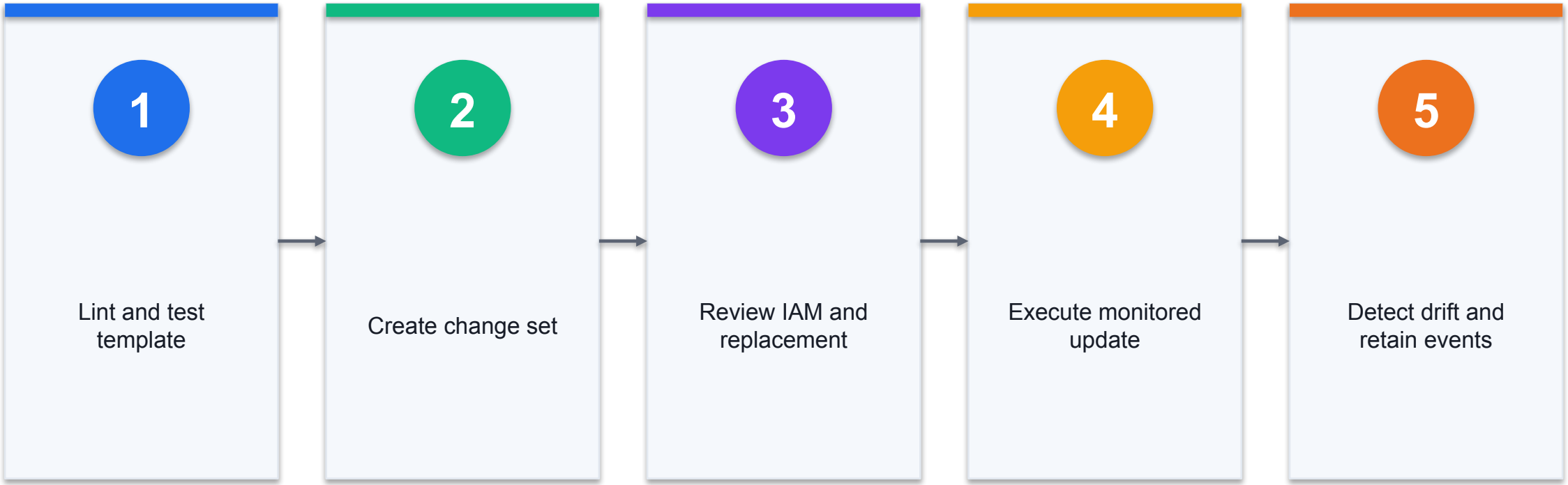
4

MECHANIC 4

DeletionPolicy and UpdateReplacePolicy govern retained data; stack policies and hooks protect or validate sensitive changes.

© 2026 Tertiary Infotech Academy Pte Ltd

CloudFormation template and stack lifecycle: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

CloudFormation template and stack lifecycle: configuration decisions

1

NESTED STACKS

Compose separately owned templates but manage update order and exported dependencies deliberately.

2

MODULES

Package reusable resource patterns while retaining native CloudFormation semantics.

3

IMPORTS

Bring existing resources under management only after identifiers and property compatibility are verified.

4

ROLLBACK

Use stack events and continue-update-rollback carefully; never delete diagnostic evidence first.

© 2026 Tertiary Infotech Academy Pte Ltd

CloudFormation template and stack lifecycle: worked architecture decision

1

SCENARIO

A database encryption change may replace a resource holding production data and must be proven safe before execution.

2

DESIGN DECISION

Generate a change set, inspect replacement flags, snapshot or migrate data explicitly, apply retention policies and rehearse the recovery path.

3

FAILURE TO AVOID

Treating every stack update as in-place can delete or orphan stateful resources; drift can make the preview incomplete.

4

ACCEPTANCE EVIDENCE

Template digest, validation output, change set, policy review, stack events, retained snapshot and drift status.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: AWS CDK and SAM synthesis boundaries

1

PURPOSE

Use higher-level authoring tools without skipping review of the CloudFormation contract they generate.

2

SERVICE BOUNDARY

AWS CDK · constructs · SAM · CloudFormation

3

PRIMARY DESIGN DECISION

Add policy assertions, narrow grants, parameterise environment behavior and require retention in production before publishing the construct version.

4

FAILURE BOUNDARY

High-level constructs can hide resources, IAM expansion and destructive defaults; a successful synth is not a security review.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS CDK and SAM synthesis boundaries: how it works

1

MECHANIC 1

CDK constructs range from low-level resource mappings to opinionated patterns; all synthesize into CloudFormation resources and IAM statements.

2

MECHANIC 2

Context values, feature flags and asset hashes affect synthesis, so toolchain versions and context files must be controlled.

3

MECHANIC 3

CDK bootstrap resources support asset publishing and deployment roles; modern bootstrap trust must be scoped to intended accounts.

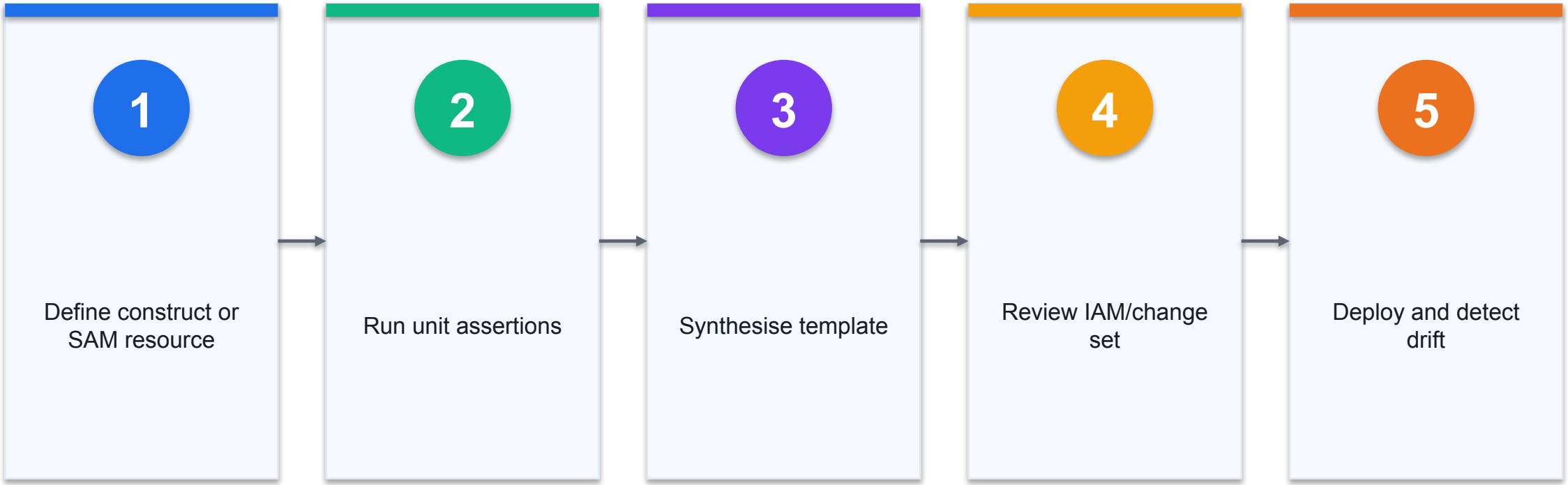
4

MECHANIC 4

SAM transforms concise serverless resources into CloudFormation and can attach deployment preferences, API events and policies.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS CDK and SAM synthesis boundaries: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS CDK and SAM synthesis boundaries: configuration decisions

1

L1/L2/L3

Choose low-level control, intent-based resource abstractions or complete patterns according to ownership and audit needs.

2

ASSETS

Pin dependencies and publish deterministic assets; do not let a local path silently change production content.

3

POLICY GENERATION

Inspect generated IAM permissions and replace broad convenience grants with resource/action constraints.

4

REMOVAL

Set retention or destruction behavior explicitly for every stateful resource and environment.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS CDK and SAM synthesis boundaries: worked architecture decision

1

SCENARIO

A reusable construct speeds delivery but currently creates wildcard permissions and destroys a bucket when a test stack is removed.

2

DESIGN DECISION

Add policy assertions, narrow grants, parameterise environment behavior and require retention in production before publishing the construct version.

3

FAILURE TO AVOID

High-level constructs can hide resources, IAM expansion and destructive defaults; a successful synth is not a security review.

4

ACCEPTANCE EVIDENCE

Toolchain lock, synth output, assertion tests, IAM diff, asset digest, change set and stack state.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: Runtime configuration and safe feature release

1

PURPOSE

Separate deployable code from environment configuration and change high-risk values through validation and progressive rollout.

2

SERVICE BOUNDARY

AppConfig · Parameter Store · Secrets Manager · Lambda extensions

3

PRIMARY DESIGN DECISION

Store the flag in AppConfig, validate its schema, deploy gradually, bind a latency alarm and use the local agent for cached retrieval.

4

FAILURE BOUNDARY

Reading a parameter on every request can create throttling and latency; logging resolved values can expose secrets.

© 2026 Tertiary Infotech Academy Pte Ltd

Runtime configuration and safe feature release: how it works

1

MECHANIC 1

Parameter Store supports hierarchical values, versions and SecureString encryption; it is suited to configuration that does not require managed rotation.

2

MECHANIC 2

Secrets Manager manages secret versions and staging labels and can coordinate rotation through a Lambda function.

3

MECHANIC 3

AppConfig profiles can source hosted or external configuration, validate content and deploy through linear, canary or exponential strategies.

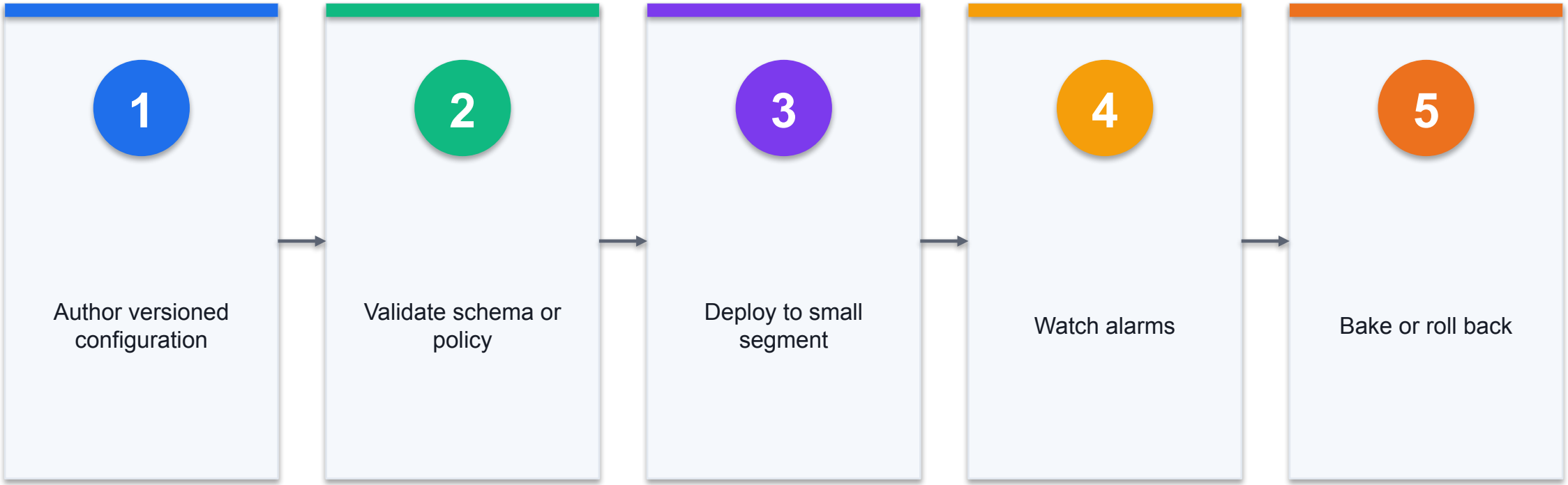
4

MECHANIC 4

The AppConfig agent or Lambda extension caches configuration locally, reducing API calls and providing controlled retrieval behavior.

© 2026 Tertiary Infotech Academy Pte Ltd

Runtime configuration and safe feature release: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Runtime configuration and safe feature release: configuration decisions

1

PARAMETER

Use for non-secret or encrypted configuration with hierarchy and straightforward versioning.

2

SECRET

Use when credential rotation, secret staging and secret-specific access are required.

3

APPCONFIG

Use when validation, progressive rollout and automatic rollback are required independently of code deployment.

4

CACHING

Define refresh and stale-value behavior so a temporary control-plane issue does not overload or stop the application.

© 2026 Tertiary Infotech Academy Pte Ltd

Runtime configuration and safe feature release: worked architecture decision

1

SCENARIO

A feature flag can increase database load and must be reversed automatically if latency crosses the release threshold.

2

DESIGN DECISION

Store the flag in AppConfig, validate its schema, deploy gradually, bind a latency alarm and use the local agent for cached retrieval.

3

FAILURE TO AVOID

Reading a parameter on every request can create throttling and latency; logging resolved values can expose secrets.

4

ACCEPTANCE EVIDENCE

Configuration version, validator result, deployment strategy, retrieval version, alarm state and rollback event.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: Step Functions orchestration semantics

1

PURPOSE

Model stateful workflows with explicit data flow, timeout, retry and compensation behavior instead of hiding orchestration in code.

2

SERVICE BOUNDARY

Step Functions Standard · Express · service integrations

3

PRIMARY DESIGN DECISION

Persist an idempotency key, retry only transient errors, catch business rejection separately and compensate completed steps before terminal failure.

4

FAILURE BOUNDARY

A catch-all retry can repeat permanent errors or duplicate side effects; missing timeouts leave executions indefinitely open.

© 2026 Tertiary Infotech Academy Pte Ltd

Step Functions orchestration semantics: how it works

1

MECHANIC 1

InputPath, Parameters, ResultSelector, ResultPath and OutputPath shape JSON between states; uncontrolled payload growth can breach limits and obscure contracts.

2

MECHANIC 2

Retry policies match error names and apply interval, backoff and maximum attempts; Catch transitions to recovery or terminal states.

3

MECHANIC 3

Standard workflows provide durable, exactly-once workflow execution semantics for long-running auditable flows; Express targets high-rate short workflows with different delivery semantics.

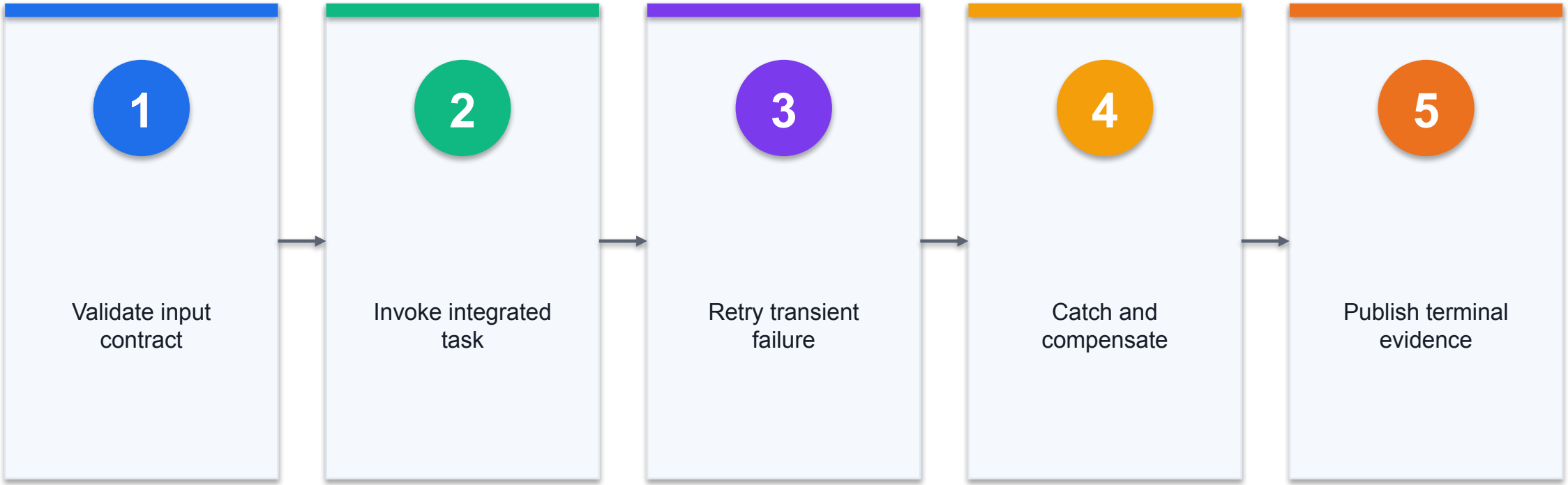
4

MECHANIC 4

Optimized service integrations can call AWS APIs, run jobs or wait for callback task tokens without custom polling Lambda functions.

© 2026 Tertiary Infotech Academy Pte Ltd

Step Functions orchestration semantics: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Step Functions orchestration semantics: configuration decisions

1

STANDARD

Choose for durable history, long waits, human approval or non-idempotent workflow coordination.

2

EXPRESS

Choose for high-volume, short, idempotent processing with logging configured for required evidence.

3

MAP

Use inline or distributed map based on item count, concurrency, payload source and isolation requirements.

4

CALLBACK

Use task tokens for external approval or asynchronous work and set a heartbeat/timeout to avoid stuck executions.

© 2026 Tertiary Infotech Academy Pte Ltd

Step Functions orchestration semantics: worked architecture decision

1

SCENARIO

An order workflow calls an unreliable partner and must not charge twice when a response is lost.

2

DESIGN DECISION

Persist an idempotency key, retry only transient errors, catch business rejection separately and compensate completed steps before terminal failure.

3

FAILURE TO AVOID

A catch-all retry can repeat permanent errors or duplicate side effects; missing timeouts leave executions indefinitely open.

4

ACCEPTANCE EVIDENCE

Execution ARN, input version, state transitions, retry count, idempotency record, compensation result and terminal status.

© 2026 Tertiary Infotech Academy Pte Ltd

CloudFormation lifecycle

1

OUTCOME

Treat infrastructure changes as reviewed, previewable and recoverable releases.

2

ARCHITECTURE

Validate templates, create change sets, inspect replacements, execute with a service role and capture stack events.

3

FAILURE MODE

Unexpected replacement of a stateful resource can turn a routine change into data loss.

4

EVIDENCE

Template digest, parameters, change-set diff, execution role and stack event timeline.

© 2026 Tertiary Infotech Academy Pte Ltd

CloudFormation lifecycle: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

CloudFormation lifecycle: design decision

?

SCENARIO

A database parameter update may replace a resource; the team needs proof before execution.

1

DESIGN RESPONSE

Validate templates, create change sets, inspect replacements, execute with a service role and capture stack events.

2

PRIMARY SERVICES

CloudFormation · Change Sets · CloudTrail

3

FAILURE MODE

Unexpected replacement of a stateful resource can turn a routine change into data loss.

4

PROOF

Template digest, parameters, change-set diff, execution role and stack event timeline.

© 2026 Tertiary Infotech Academy Pte Ltd

CloudFormation lifecycle: failure diagnostic

1

RECOGNISE

Observable failure condition: Unexpected replacement of a stateful resource can turn a routine change into data loss.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Author through Observe events; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Template digest, parameters, change-set diff, execution role and stack event timeline.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 2 · APPLIED SCENARIO

CloudFormation lifecycle: worked decision

1

CONSTRAINT

A database parameter update may replace a resource; the team needs proof before execution.

2

DECISION

Validate templates, create change sets, inspect replacements, execute with a service role and capture stack events.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Unexpected replacement of a stateful resource can turn a routine change into data loss.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Template digest, parameters, change-set diff, execution role and stack event timeline.

© 2026 Tertiary Infotech Academy Pte Ltd

Reusable infrastructure components

1

OUTCOME

Create governed building blocks that teams can reuse without copying templates.

2

ARCHITECTURE

Define stable interfaces, semantic versions, ownership, safe defaults and deprecation paths for components.

3

FAILURE MODE

Copy-and-edit templates fork controls and make security fixes inconsistent.

4

EVIDENCE

Component version, input contract, output contract, policy checks and consuming stacks.

© 2026 Tertiary Infotech Academy Pte Ltd

Reusable infrastructure components: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Reusable infrastructure components: design decision

? SCENARIO

Twenty application teams need the same encrypted logging bucket with centrally managed controls.

1 DESIGN RESPONSE

Define stable interfaces, semantic versions, ownership, safe defaults and deprecation paths for components.

2 PRIMARY SERVICES

Nested stacks · Modules · CDK constructs · Service Catalog

3 FAILURE MODE

Copy-and-edit templates fork controls and make security fixes inconsistent.

4 PROOF

Component version, input contract, output contract, policy checks and consuming stacks.

© 2026 Tertiary Infotech Academy Pte Ltd

Reusable infrastructure components: failure diagnostic

1

RECOGNISE

Observable failure condition: Copy-and-edit templates fork controls and make security fixes inconsistent.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Define interface through Deprecate safely; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Component version, input contract, output contract, policy checks and consuming stacks.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 2 · APPLIED SCENARIO

Reusable infrastructure components: worked decision

1

CONSTRAINT

Twenty application teams need the same encrypted logging bucket with centrally managed controls.

2

DECISION

Define stable interfaces, semantic versions, ownership, safe defaults and deprecation paths for components.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Copy-and-edit templates fork controls and make security fixes inconsistent.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Component version, input contract, output contract, policy checks and consuming stacks.

© 2026 Tertiary Infotech Academy Pte Ltd

Parameters, mappings and dynamic references

1

OUTCOME

Separate environment configuration and secrets from reusable infrastructure code.

2

ARCHITECTURE

Use parameters for deployment choices, mappings for curated constants and dynamic references for secret values.

3

FAILURE MODE

Embedding secrets in templates, outputs or pipeline logs creates durable exposure.

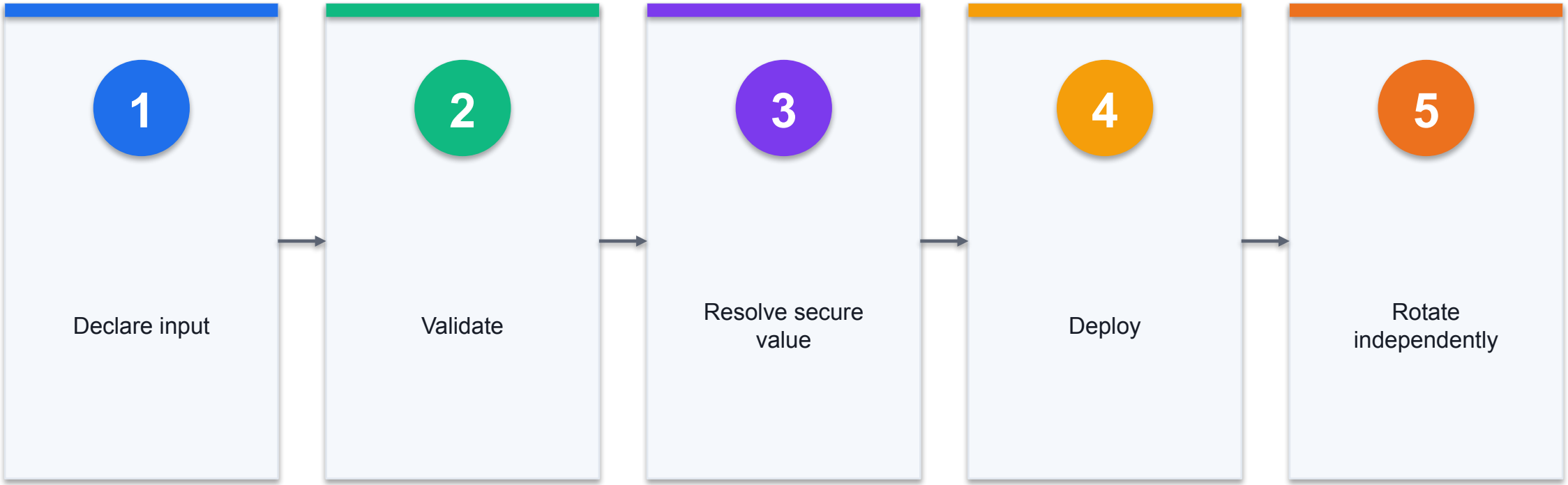
4

EVIDENCE

Parameter type, allowed values, secret ARN/version, access policy and redacted execution logs.

© 2026 Tertiary Infotech Academy Pte Ltd

Parameters, mappings and dynamic references: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Parameters, mappings and dynamic references: design decision

? SCENARIO

One template serves three environments while database credentials rotate every 30 days.

1 DESIGN RESPONSE

Use parameters for deployment choices, mappings for curated constants and dynamic references for secret values.

2 PRIMARY SERVICES

CloudFormation · Parameter Store · Secrets Manager

3 FAILURE MODE

Embedding secrets in templates, outputs or pipeline logs creates durable exposure.

4 PROOF

Parameter type, allowed values, secret ARN/version, access policy and redacted execution logs.

© 2026 Tertiary Infotech Academy Pte Ltd

Parameters, mappings and dynamic references: failure diagnostic

1

RECOGNISE

Observable failure condition: Embedding secrets in templates, outputs or pipeline logs creates durable exposure.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Declare input through Rotate independently; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Parameter type, allowed values, secret ARN/version, access policy and redacted execution logs.

© 2026 Tertiary Infotech Academy Pte Ltd

Parameters, mappings and dynamic references: worked decision

1

CONSTRAINT

One template serves three environments while database credentials rotate every 30 days.

2

DECISION

Use parameters for deployment choices, mappings for curated constants and dynamic references for secret values.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Embedding secrets in templates, outputs or pipeline logs creates durable exposure.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Parameter type, allowed values, secret ARN/version, access policy and redacted execution logs.

© 2026 Tertiary Infotech Academy Pte Ltd

Drift, change control and rollback

1

OUTCOME

Detect unmanaged changes and return resources to an approved state without masking the cause.

2

ARCHITECTURE

Schedule drift detection, triage intent, import legitimate changes into code or remediate unauthorized variance.

3

FAILURE MODE

Blindly reapplying a stack can overwrite an emergency repair before it is understood.

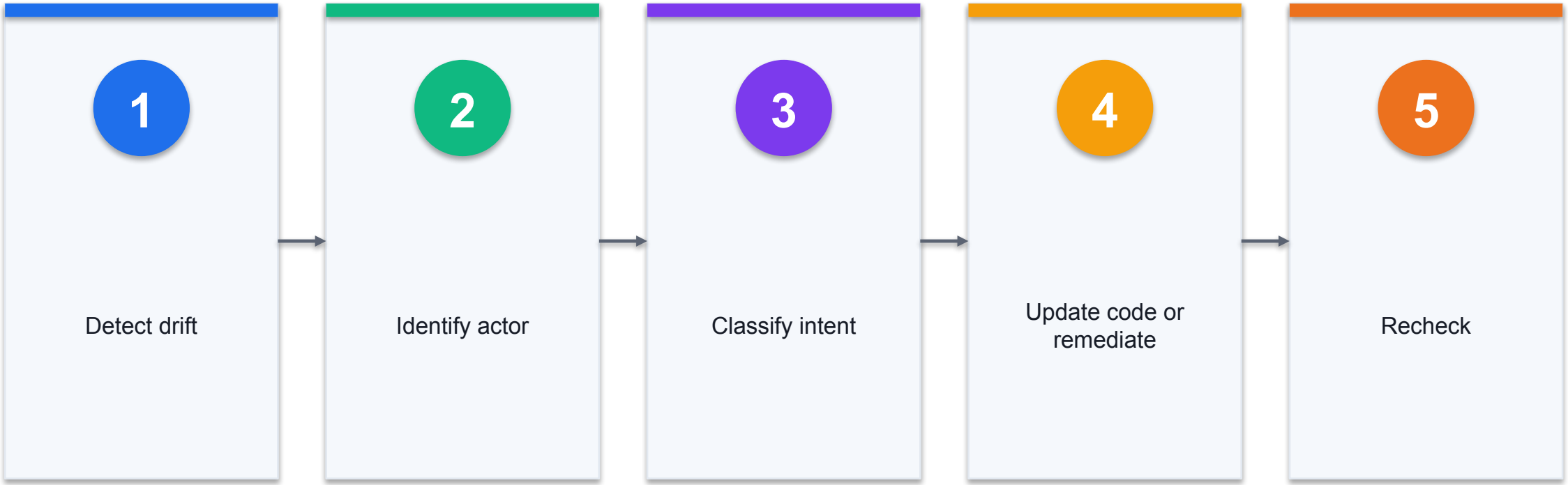
4

EVIDENCE

Drift result, actor from CloudTrail, approved exception, change set and post-remediation state.

© 2026 Tertiary Infotech Academy Pte Ltd

Drift, change control and rollback: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 2 · SCENARIO

Drift, change control and rollback: design decision

? SCENARIO

An operator changes a security group during an incident and forgets to restore it.

1 DESIGN RESPONSE

Schedule drift detection, triage intent, import legitimate changes into code or remediate unauthorized variance.

2 PRIMARY SERVICES

CloudFormation Drift · Config · Change Sets

3 FAILURE MODE

Blindly reapplying a stack can overwrite an emergency repair before it is understood.

4 PROOF

Drift result, actor from CloudTrail, approved exception, change set and post-remediation state.

© 2026 Tertiary Infotech Academy Pte Ltd

Drift, change control and rollback: failure diagnostic

1

RECOGNISE

Observable failure condition: Blindly reapplying a stack can overwrite an emergency repair before it is understood.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Detect drift through Recheck; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Drift result, actor from CloudTrail, approved exception, change set and post-remediation state.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 2 · APPLIED SCENARIO

Drift, change control and rollback: worked decision

1

CONSTRAINT

An operator changes a security group during an incident and forgets to restore it.

2

DECISION

Schedule drift detection, triage intent, import legitimate changes into code or remediate unauthorized variance.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Blindly reapplying a stack can overwrite an emergency repair before it is understood.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Drift result, actor from CloudTrail, approved exception, change set and post-remediation state.

© 2026 Tertiary Infotech Academy Pte Ltd

StackSets and account vending

1

OUTCOME

Deploy mandatory baselines consistently as accounts and Regions are created.

2

ARCHITECTURE

Use organizational units, delegated administration, automatic deployment and controlled failure tolerance.

3

FAILURE MODE

An aggressive concurrent rollout can spread a faulty baseline across the organization.

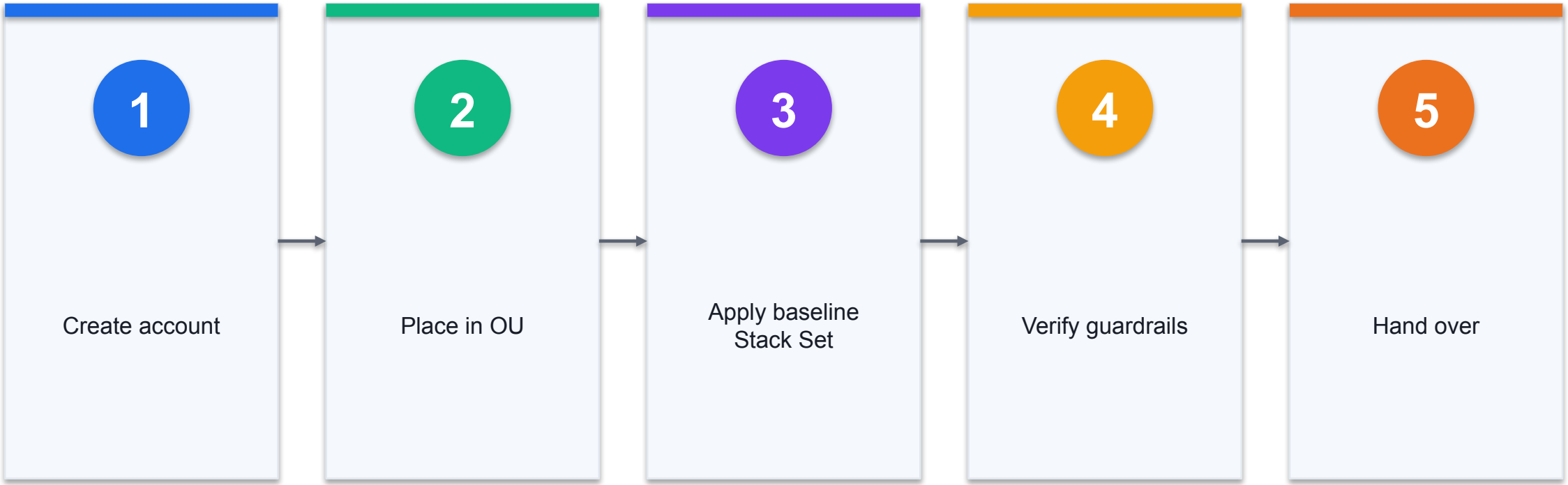
4

EVIDENCE

OU target, permission model, operation preferences, per-account status and drift result.

© 2026 Tertiary Infotech Academy Pte Ltd

StackSets and account vending: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

StackSets and account vending: design decision

? SCENARIO

A new production account must receive logging, security roles and Config before application access.

1 DESIGN RESPONSE

Use organizational units, delegated administration, automatic deployment and controlled failure tolerance.

2 PRIMARY SERVICES

Organizations · StackSets · Control Tower · Account Factory

3 FAILURE MODE

An aggressive concurrent rollout can spread a faulty baseline across the organization.

4 PROOF

OU target, permission model, operation preferences, per-account status and drift result.

© 2026 Tertiary Infotech Academy Pte Ltd

StackSets and account vending: failure diagnostic

1

RECOGNISE

Observable failure condition: An aggressive concurrent rollout can spread a faulty baseline across the organization.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Create account through Hand over; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with OU target, permission model, operation preferences, per-account status and drift result.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 2 · APPLIED SCENARIO

StackSets and account vending: worked decision

1

CONSTRAINT

A new production account must receive logging, security roles and Config before application access.

2

DECISION

Use organizational units, delegated administration, automatic deployment and controlled failure tolerance.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: An aggressive concurrent rollout can spread a faulty baseline across the organization.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: OU target, permission model, operation preferences, per-account status and drift result.

© 2026 Tertiary Infotech Academy Pte Ltd

Systems Manager fleet automation

1

OUTCOME

Operate hybrid and EC2 fleets without inbound administrative ports or unmanaged scripts.

2

ARCHITECTURE

Use tags, documents, maintenance windows, rate controls, concurrency limits and command output encryption.

3

FAILURE MODE

A fleet-wide command without error thresholds can create a synchronized outage.

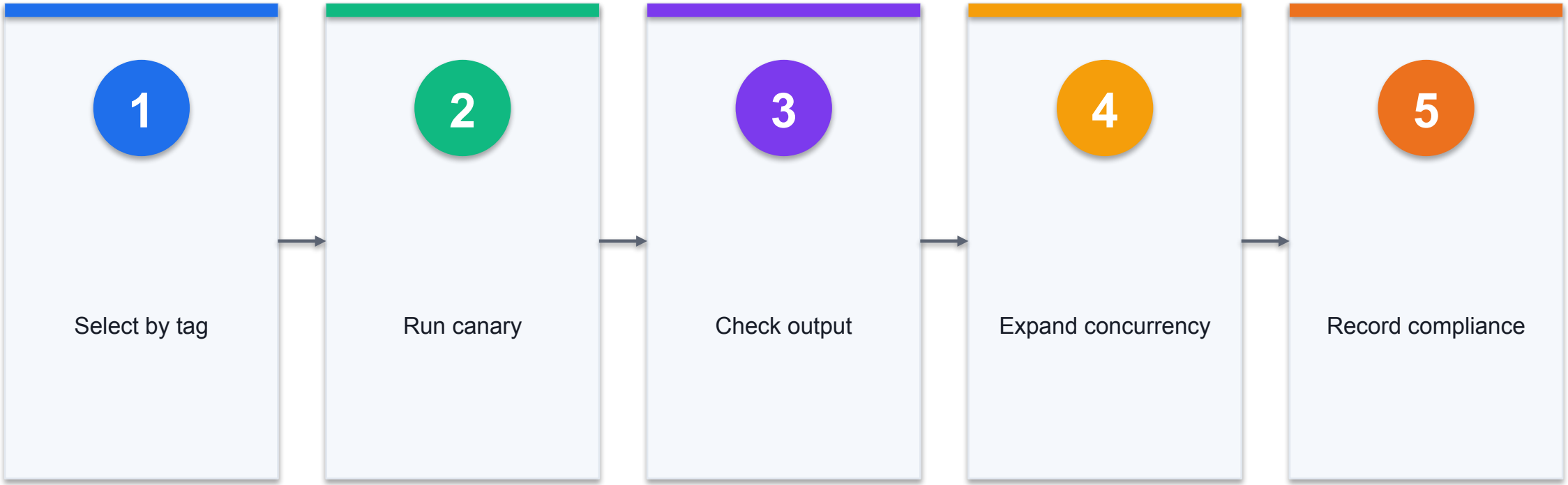
4

EVIDENCE

Target query, document version, invocation output, compliance result and exception record.

© 2026 Tertiary Infotech Academy Pte Ltd

Systems Manager fleet automation: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 2 · SCENARIO

Systems Manager fleet automation: design decision

SCENARIO

A monthly patch cycle must update hundreds of instances while limiting simultaneous disruption.

1 DESIGN RESPONSE

Use tags, documents, maintenance windows, rate controls, concurrency limits and command output encryption.

2 PRIMARY SERVICES

Systems Manager · Run Command · Automation · Patch Manager

3 FAILURE MODE

A fleet-wide command without error thresholds can create a synchronized outage.

4 PROOF

Target query, document version, invocation output, compliance result and exception record.

© 2026 Tertiary Infotech Academy Pte Ltd

Systems Manager fleet automation: failure diagnostic

1

RECOGNISE

Observable failure condition: A fleet-wide command without error thresholds can create a synchronized outage.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Select by tag through Record compliance; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Target query, document version, invocation output, compliance result and exception record.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 2 · APPLIED SCENARIO

Systems Manager fleet automation: worked decision

1

CONSTRAINT

A monthly patch cycle must update hundreds of instances while limiting simultaneous disruption.

2

DECISION

Use tags, documents, maintenance windows, rate controls, concurrency limits and command output encryption.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: A fleet-wide command without error thresholds can create a synchronized outage.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Target query, document version, invocation output, compliance result and exception record.

© 2026 Tertiary Infotech Academy Pte Ltd

CDK, SAM and workflow orchestration

1

OUTCOME

Choose the right abstraction while preserving reviewable infrastructure output and safe runtime change.

2

ARCHITECTURE

Synthesize templates, test constructs, separate build-time from runtime configuration, and orchestrate retries explicitly.

3

FAILURE MODE

High-level abstractions can hide IAM expansion or replacement behavior unless synthesized output is reviewed.

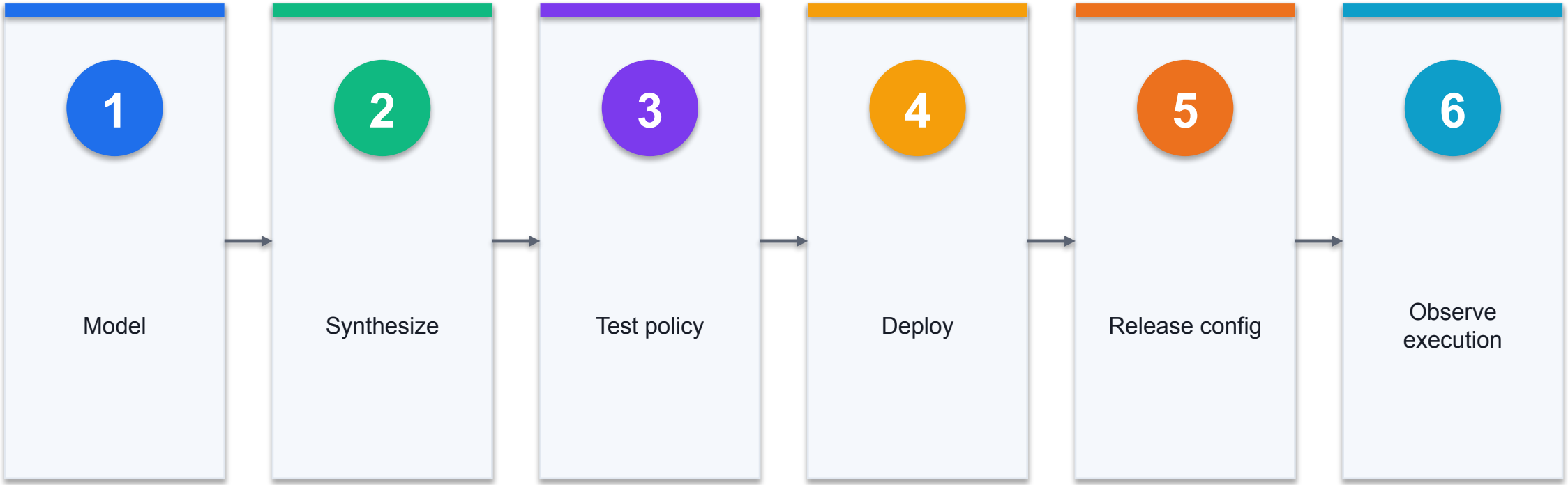
4

EVIDENCE

Synthesized template, assertions, state-machine history, configuration version and deployment strategy.

© 2026 Tertiary Infotech Academy Pte Ltd

CDK, SAM and workflow orchestration: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

CDK, SAM and workflow orchestration: design decision

? SCENARIO

A serverless workflow needs bounded retries and a feature flag that can be rolled back independently.

1 DESIGN RESPONSE

Synthesize templates, test constructs, separate build-time from runtime configuration, and orchestrate retries explicitly.

2 PRIMARY SERVICES

CDK · SAM · Step Functions · AppConfig

3 FAILURE MODE

High-level abstractions can hide IAM expansion or replacement behavior unless synthesized output is reviewed.

4 PROOF

Synthesized template, assertions, state-machine history, configuration version and deployment strategy.

© 2026 Tertiary Infotech Academy Pte Ltd

CDK, SAM and workflow orchestration: failure diagnostic

1

RECOGNISE

Observable failure condition: High-level abstractions can hide IAM expansion or replacement behavior unless synthesized output is reviewed.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Model through Observe execution; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Synthesized template, assertions, state-machine history, configuration version and deployment strategy.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 2 · APPLIED SCENARIO

CDK, SAM and workflow orchestration: worked decision

1

CONSTRAINT

A serverless workflow needs bounded retries and a feature flag that can be rolled back independently.

2

DECISION

Synthesize templates, test constructs, separate build-time from runtime configuration, and orchestrate retries explicitly.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: High-level abstractions can hide IAM expansion or replacement behavior unless synthesized output is reviewed.

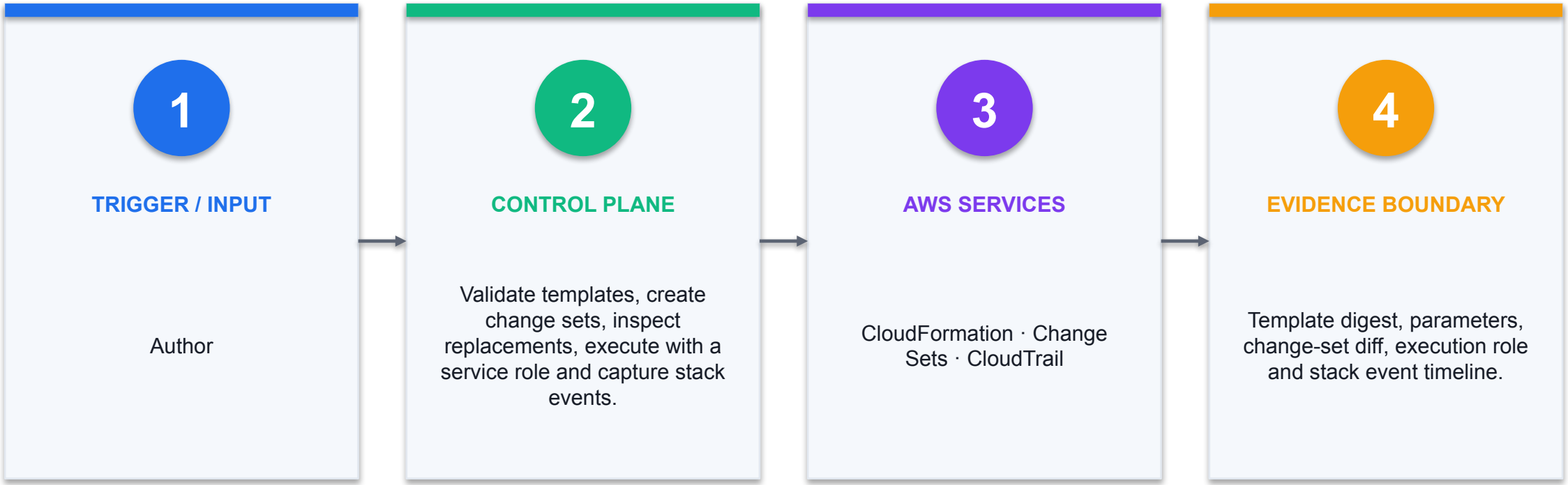
4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Synthesized template, assertions, state-machine history, configuration version and deployment strategy.

© 2026 Tertiary Infotech Academy Pte Ltd

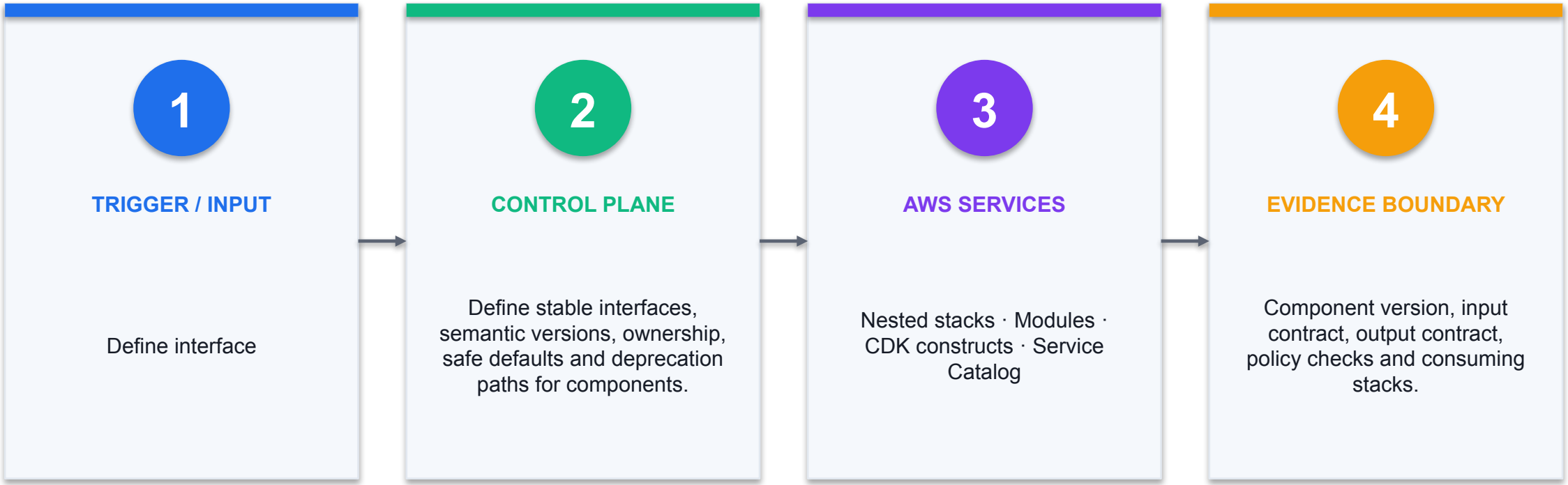
CloudFormation lifecycle: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

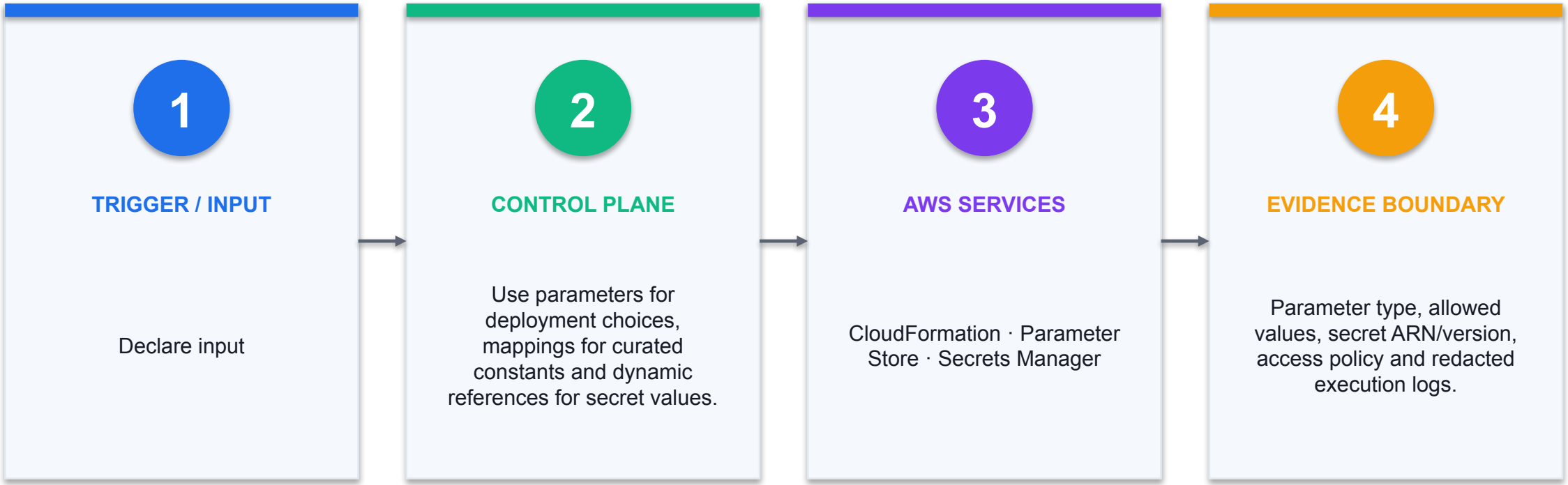
Reusable infrastructure components: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

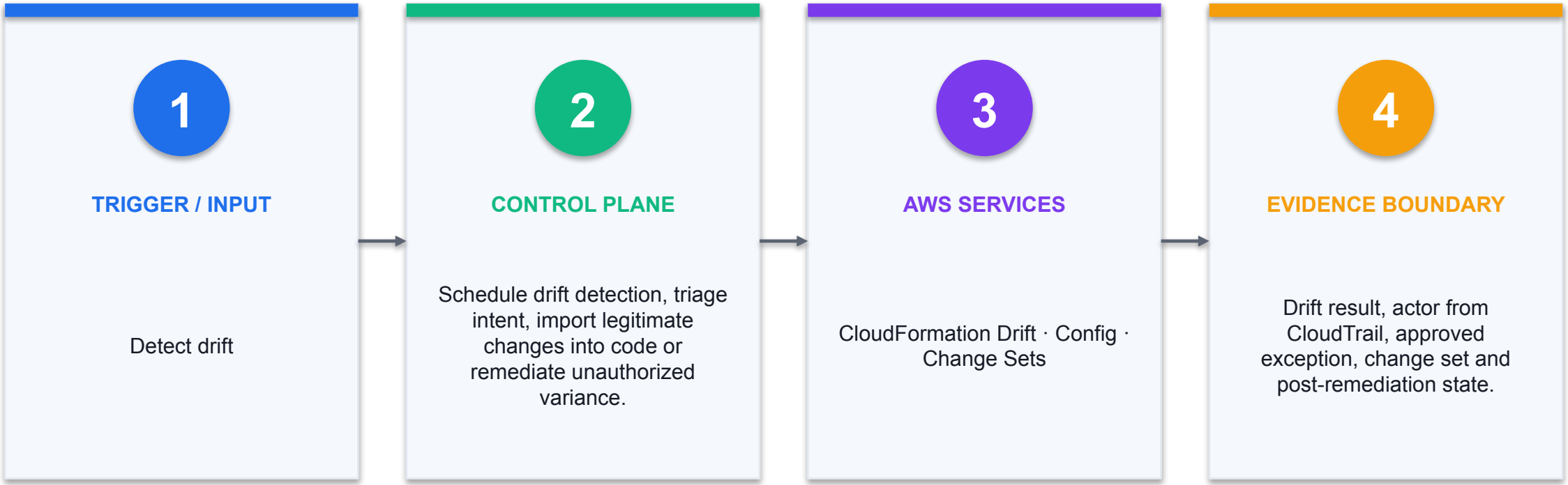
Parameters, mappings and dynamic references: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

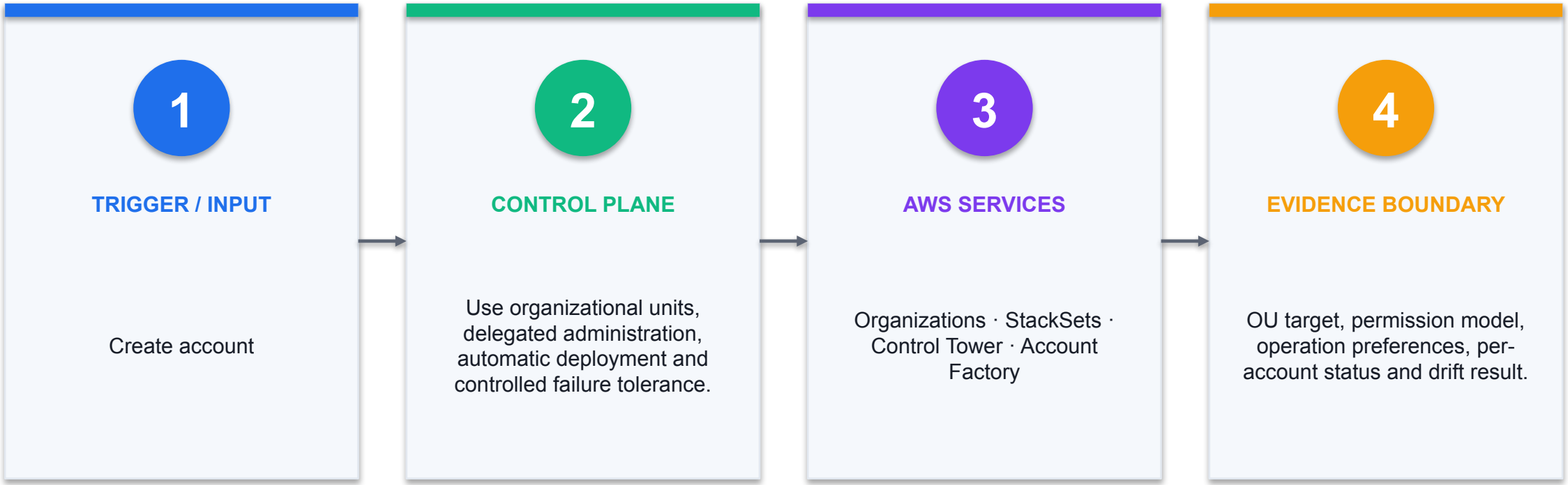
Drift, change control and rollback: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

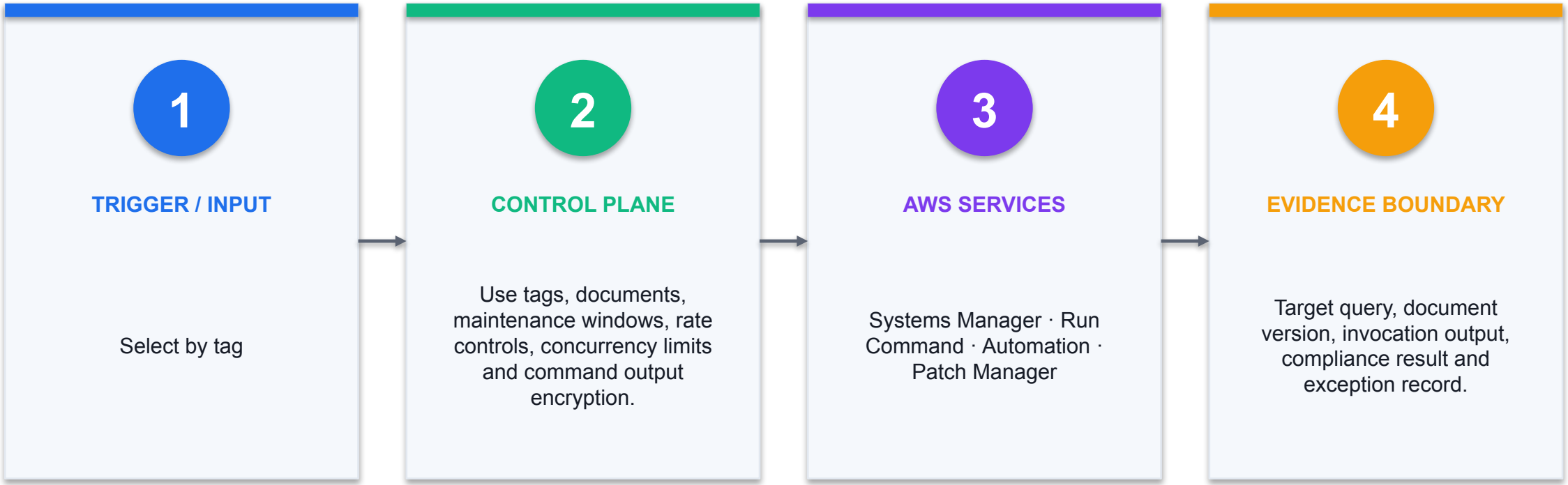
StackSets and account vending: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

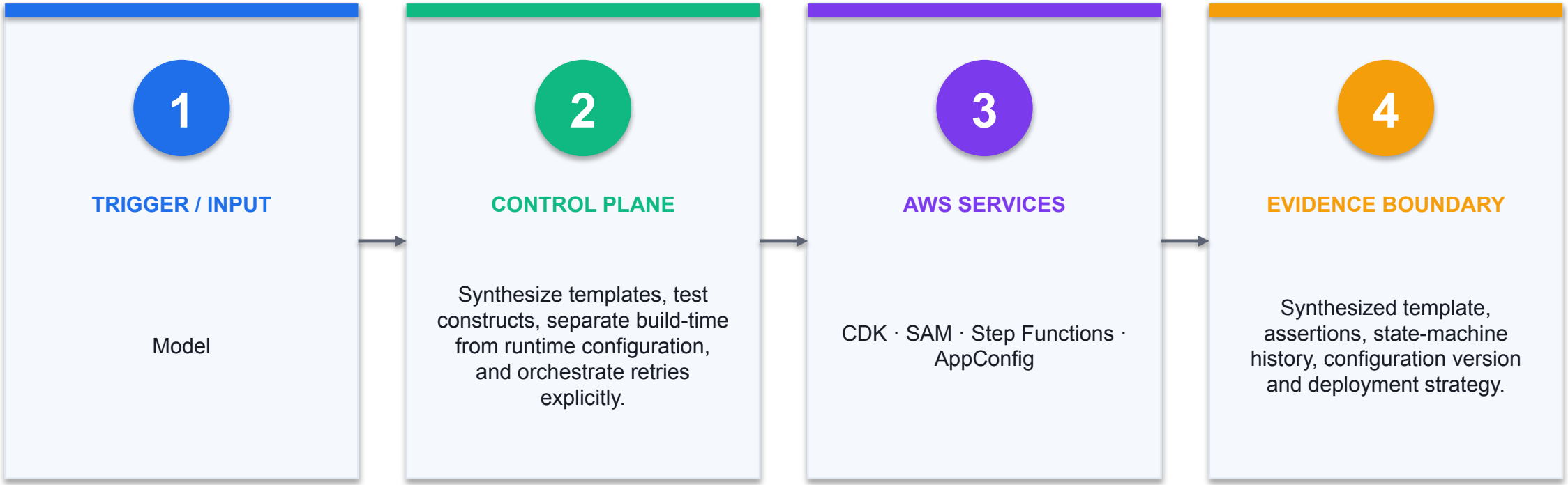
Systems Manager fleet automation: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

CDK, SAM and workflow orchestration: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Deploy a Reusable VPC Network Baseline with CloudFormation

LAB 6

Day 2 · LO2 · K3, A3

Your team keeps building VPCs by hand and no two look alike.

YOU'LL BUILD

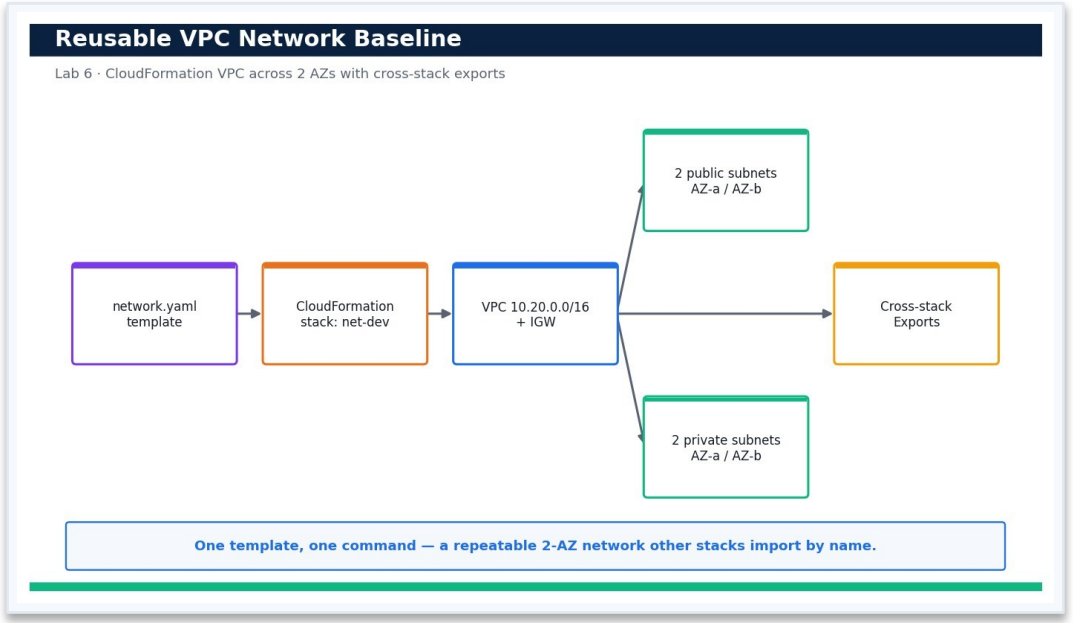
A version-controlled network.yaml that deploys a 2-AZ VPC (2 public + 2 private subnets, IGW, route tables) and publishes VpcId, subnet IDs and the VPC CIDR as reusable cross-stack Exports.

Tools: AWS CloudFormation, Amazon VPC, Amazon EC2

OUTCOME & EVIDENCE

Outcome: Author, validate and deploy a reusable VPC network baseline with CloudFormation and publish cross-stack exports (LO2, K3, A3).

Evidence: describe-stacks output showing net-dev at CREATE_COMPLETE with the Outputs tab populated.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Deploy a Parameterized Application Stack that Imports Network Exports

LAB 7

Day 2 · LO2 · K3, A3

With the network baseline from Lab 6 in place, you now deploy the application tier on top of it — without copy-pasting a single subnet or VPC ID.

YOU'LL BUILD

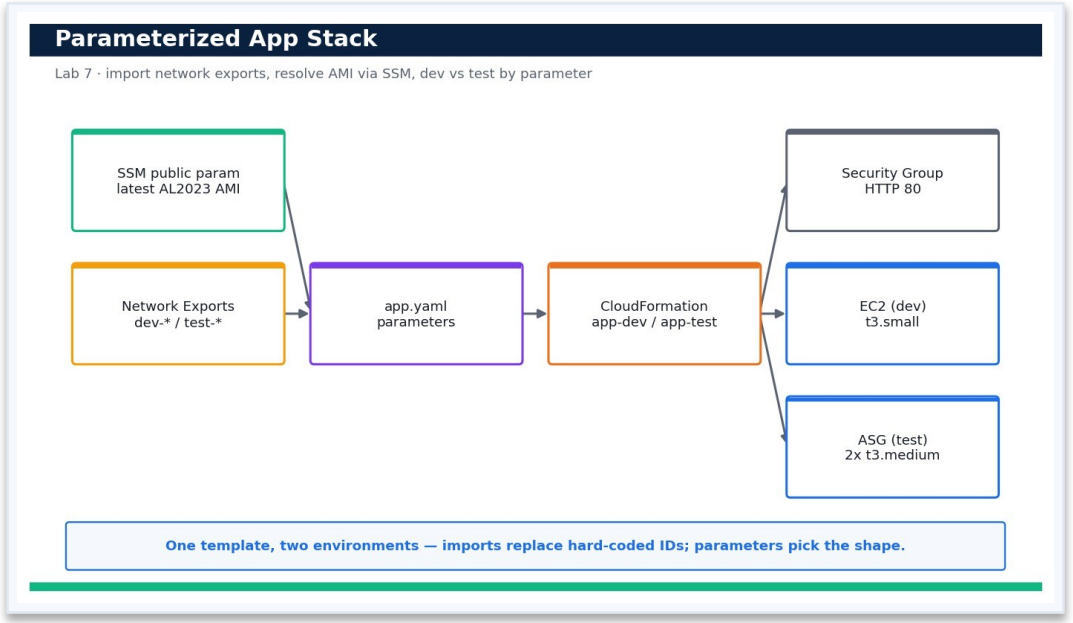
A parameterized app.yaml that imports the network exports, resolves the latest AMI via SSM.

Tools: AWS CloudFormation, Amazon EC2, AWS Systems Manager (SSM public parameters)

OUTCOME & EVIDENCE

Outcome: Deploy a parameterized application stack that consumes cross-stack exports and reuse it for dev and test via parameter files (LO2, K3, A3).

Evidence: describe-stacks output showing app-dev and app-test both at a *_COMPLETE status.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Preview Changes with Change Sets and Detect & Reconcile Drift

LAB 8

Day 2 · LO2 · K3, A3

Production changes are risky when you cannot see them coming, and infrastructure silently drifts when people click in the console.

YOU'LL BUILD

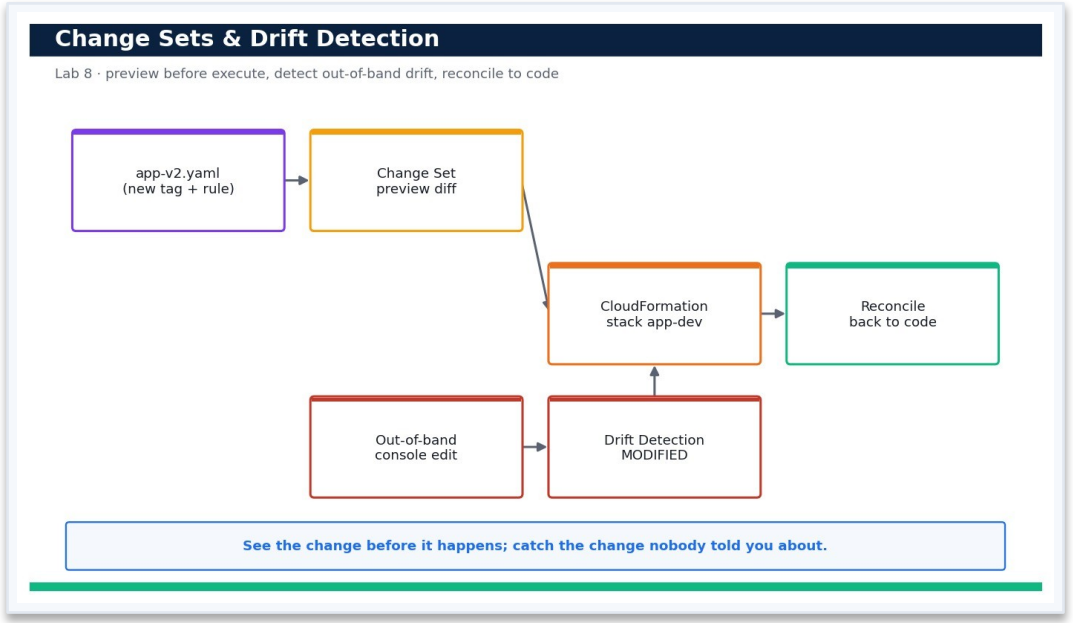
A repeatable workflow: create-change-set.sh renders a reviewable preview of a stack change, and drift-check.sh detects and reports out-of-band drift so you can reconcile infrastructure back to code.

Tools: AWS CloudFormation

OUTCOME & EVIDENCE

Outcome: Preview a stack modification safely with a change set before executing, then detect out-of-band drift and reconcile it (LO2, K3, A3).

Evidence: describe-change-set output showing the previewed changes with Replacement=False (no destructive changes).



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Externalize Configuration with SSM Parameter Store

LAB 9

Day 2 · LO2 · K3, A3

Hard-coding configuration and credentials into code and templates is a recipe for leaks and rebuilds.

YOU'LL BUILD

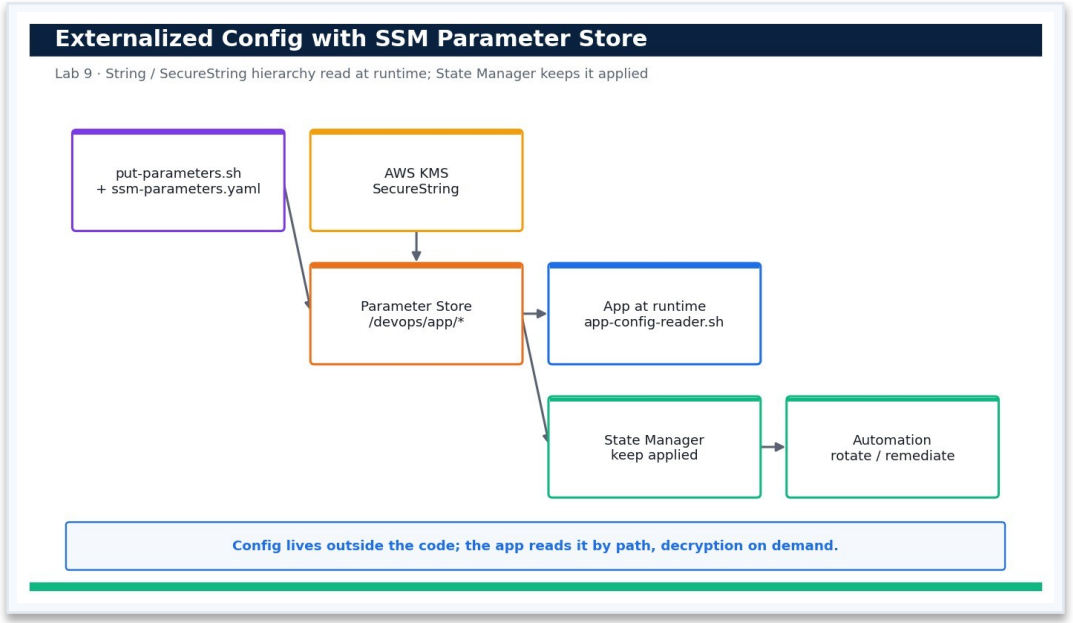
A /devops/app/* parameter hierarchy in SSM (String, StringList and KMS-encrypted SecureString) plus an app-config-reader.sh that pulls the whole configuration at runtime with decryption.

Tools: AWS Systems Manager (Parameter Store, State Manager, Automation), AWS KMS, AWS CloudFormation

OUTCOME & EVIDENCE

Outcome: Externalize application configuration into SSM Parameter Store (String and SecureString), read it at runtime, and compare Parameter Store.

Evidence: get-parameters-by-path output showing the /devops/app hierarchy with String, StringList and SecureString types.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 10 · DAY 2 · LO2 · HANDS-ON

Multi-Account, Multi-Region Guardrails with CloudFormation StackSets

LAB 10

Day 2 · LO2 · K3, A3

Scaling infrastructure to many accounts by hand does not work.

YOU'LL BUILD

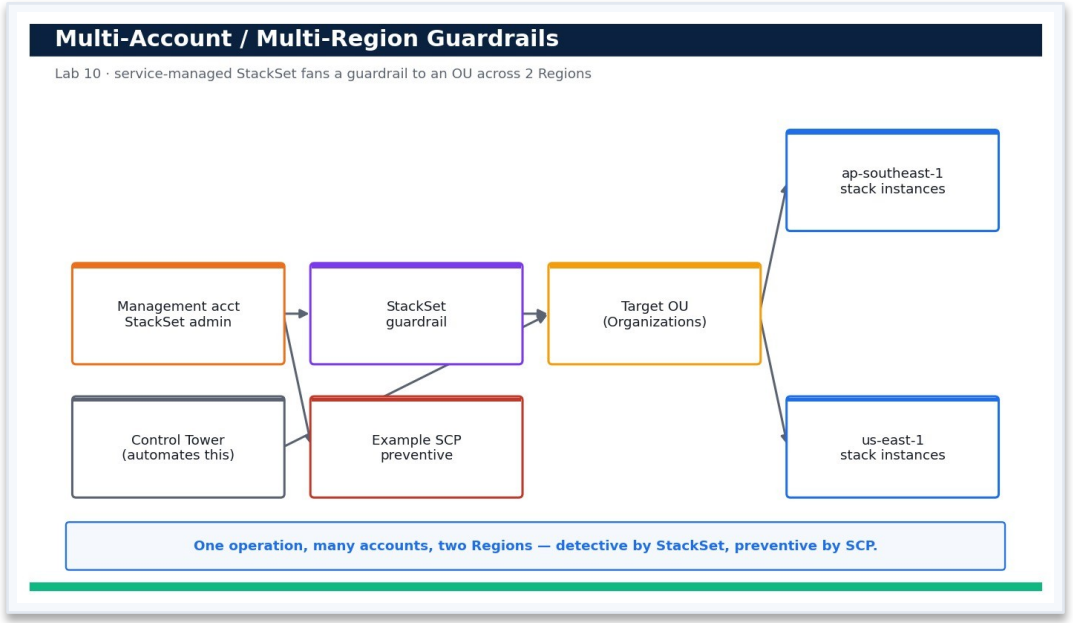
A StackSet (service-managed) that fans a guardrail stack out to an OU across two Regions in one operation, plus an example SCP that enforces the same guardrail preventively.

Tools: AWS CloudFormation StackSets, AWS Organizations, AWS Control Tower (concept)

OUTCOME & EVIDENCE

Outcome: Design and deploy a multi-account, multi-Region guardrail using CloudFormation StackSets.

Evidence: list-stack-instances output showing the guardrail CURRENT in each account across both Regions.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN RECAP

Recap — Configuration Management and IaC

1

Lab 6

Author, validate and deploy a reusable VPC network baseline with CloudFormation and publish cross-stack exports (LO2, K3, A3).

2

Lab 7

Deploy a parameterized application stack that consumes cross-stack exports.

3

Lab 8

Preview a stack modification safely with a change set before executing, then detect out-of-band drift.

4

Lab 9

Externalize application configuration into SSM Parameter Store (String and SecureString), read it at runtime, and compare Parameter Store.

5

Lab 10

Design and deploy a multi-account, multi-Region guardrail using CloudFormation StackSets.

6

Key principle

Declare infrastructure in version-controlled templates so environments are repeatable, reviewable and disposable.

© 2026 Tertiary Infotech Academy Pte Ltd

03

DOMAIN 03

Resilient Cloud Solutions

Exam weighting 15% · High availability · scalability · automated recovery (RTO/RPO)

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 03 · WEIGHTING 15%

Key Concepts — Resilient Cloud Solutions

1

High availability

Spread compute across multiple Availability Zones behind an Elastic Load Balancer so one AZ failure is survivable.

2

Elasticity & Auto Scaling

Auto Scaling groups add and remove capacity on demand using target-tracking and step scaling policies.

3

Decoupling

SQS, SNS and EventBridge decouple producers from consumers so components fail and scale independently.

4

RTO & RPO

Recovery Time / Point Objectives drive the DR pattern: backup & restore, pilot light, warm standby or multi-site active/active.

5

Data durability

Multi-AZ RDS, DynamoDB global tables, S3 cross-Region replication and automated snapshots protect state.

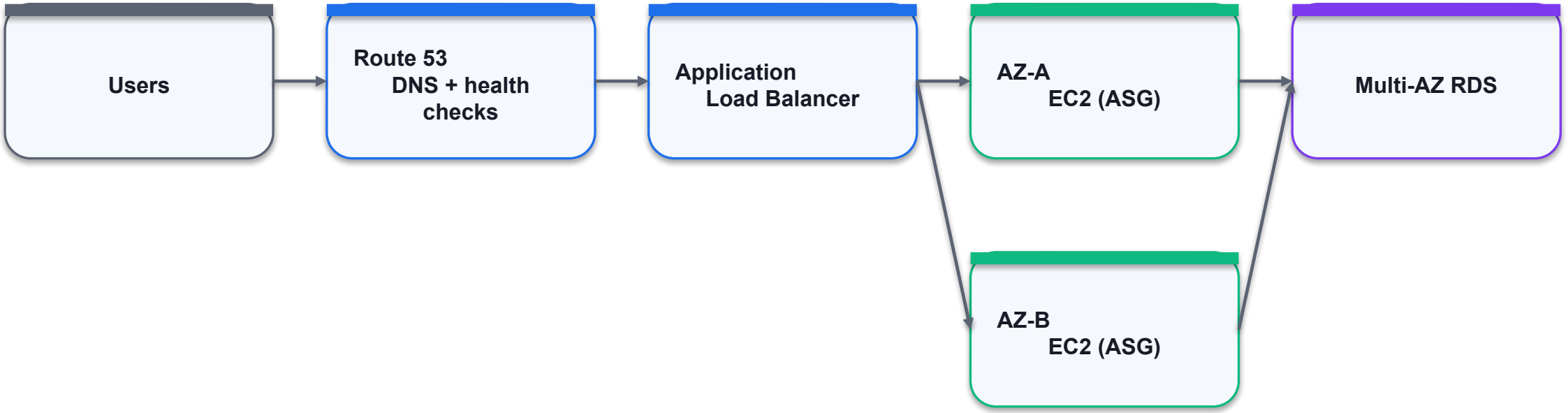
6

Automated recovery

EC2 auto-recovery, ASG health checks and Route 53 health-check failover restore service without human action.

© 2026 Tertiary Infotech Academy Pte Ltd

Highly Available, Auto-Scaling Architecture



- Traffic spreads across two AZs behind the ALB; the Auto Scaling group self-heals and Multi-AZ RDS protects data.

© 2026 Tertiary Infotech Academy Pte Ltd

MEETING RTO / RPO

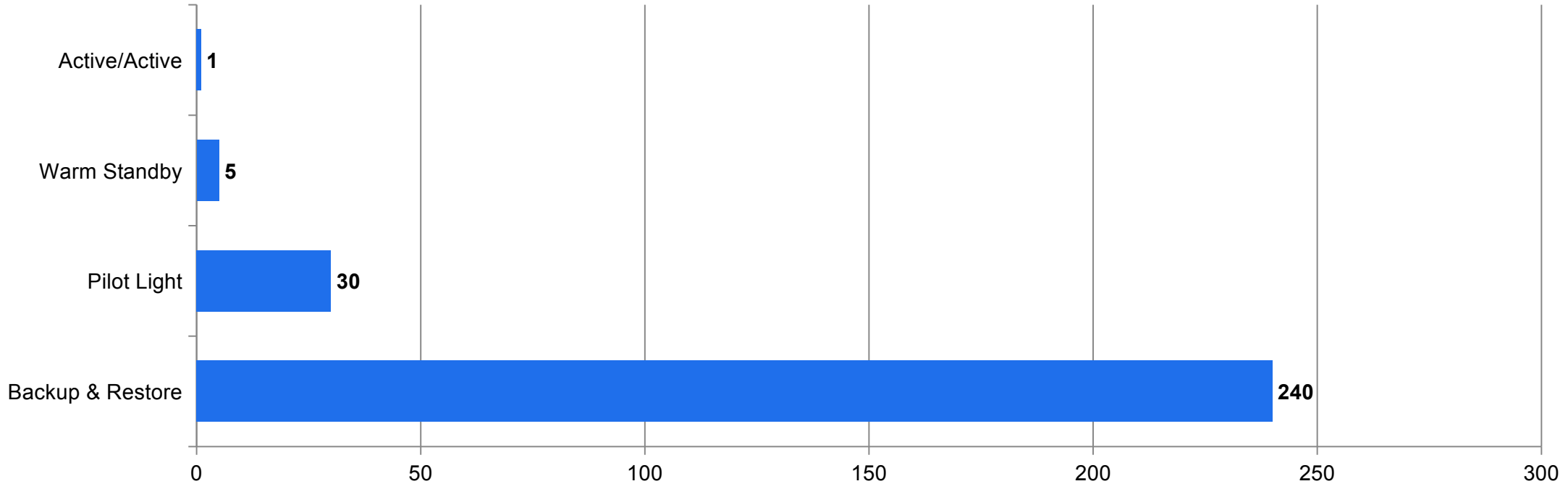
Disaster Recovery Strategies

Pattern	RTO	RPO	Cost
Backup & Restore	Hours	Hours	$
Pilot Light	10s of min	Minutes	$$
Warm Standby	Minutes	Seconds	$$$
Multi-site Active/Active	Near-zero	Near-zero	$$$$

© 2026 Tertiary Infotech Academy Pte Ltd

LOWER IS BETTER

DR Patterns — Approximate Recovery Time (RTO)



WHAT THE DATA SHOWS

As you move from backup/restore to active/active, RTO falls from hours to near-zero — at rising cost.

© 2026 Tertiary Infotech Academy Pte Ltd

DESIGN PRINCIPLES

Building for Resilience

1

Multi-AZ by default

Never depend on a single Availability Zone.

2

Elastic Load Balancing

Distribute traffic and route around unhealthy targets.

3

Auto Scaling

Match capacity to demand; replace failed instances automatically.

4

Decouple with queues

SQS / SNS / EventBridge let components fail independently.

© 2026 Tertiary Infotech Academy Pte Ltd

Auto Scaling Policy Types

Policy	How it decides	Use when
Target tracking	Keep a metric at a target (e.g. 50% CPU)	Default, simplest
Step scaling	Add/remove capacity by alarm breach size	Fine-grained control
Scheduled	Scale at known times	Predictable traffic patterns
Predictive	ML forecasts demand ahead of time	Recurring cyclical load

© 2026 Tertiary Infotech Academy Pte Ltd

Elastic Load Balancer Types

Type	Layer	Best for
Application LB (ALB)	HTTP/HTTPS (L7)	Web apps, path/host routing
Network LB (NLB)	TCP/UDP (L4)	Ultra-low latency, static IP
Gateway LB (GWLB)	L3 gateway	Inline security appliances

© 2026 Tertiary Infotech Academy Pte Ltd

DURABILITY & RECOVERY

Protecting Your Data

1

Multi-AZ RDS

Synchronous standby with automatic failover.

2

DynamoDB global tables

Multi-Region, active-active replication.

3

S3 cross-Region replication

Durable object copies in another Region.

4

Automated snapshots

Point-in-time recovery for RDS/EBS.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

High availability across Availability Zones

IN DEPTH

- Availability Zones are physically isolated data centres within a Region with independent power, cooling and networking.
- Spreading compute across at least two AZs behind an Elastic Load Balancer lets the system survive a full AZ failure.
- Stateless application tiers make AZ-level redundancy trivial — any instance can serve any request.

WHY IT MATTERS

Multi-AZ is the cheapest, highest-leverage resilience decision — most real outages are single-AZ or single-instance, both survivable.

FAILURE MODE

A single-AZ deployment, or a stateful session pinned to one instance, turns a routine hardware failure into a customer-visible outage.

TRADE-OFF / DECISION

Cross-AZ traffic incurs latency and data-transfer cost — a fair price for surviving an AZ loss.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Elasticity and Auto Scaling

IN DEPTH

- Auto Scaling groups add and remove instances to match demand and replace unhealthy ones automatically.
- Target tracking holds a metric (e.g. 50% CPU) at target; step and scheduled policies handle spikes and known patterns; predictive scaling forecasts demand.
- Health checks (EC2 and ELB) let the group cull and replace bad instances without human action.

WHY IT MATTERS

Elasticity matches cost to demand and provides self-healing — a terminated instance is silently replaced.

FAILURE MODE

Scaling on the wrong metric (CPU for an I/O-bound app) leaves you slow under load and over-provisioned when idle.

TRADE-OFF / DECISION

Aggressive scaling reacts fast but can thrash — tune cooldowns and warm-up so the group settles.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Load balancer types and when to use them

IN DEPTH

- Application Load Balancer routes HTTP/HTTPS with host/path rules and works well with containers and Lambda targets.
- Network Load Balancer handles TCP/UDP at ultra-low latency with static IPs.
- Gateway Load Balancer inserts inline security appliances transparently.

WHY IT MATTERS

Choosing the right layer-7 or layer-4 balancer shapes routing, TLS termination, latency and cost.

FAILURE MODE

Forcing an L7 ALB where you need raw TCP throughput or a static IP adds latency and misses the requirement.

TRADE-OFF / DECISION

ALB gives rich routing at higher per-request processing; NLB gives raw speed with fewer features.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Decoupling with queues, topics and events

IN DEPTH

- SQS buffers work between producers and consumers so a slow or failed consumer doesn't drop requests.
- SNS fans a message out to many subscribers; EventBridge routes events by content to many targets.
- Decoupled components fail and scale independently and absorb spikes as backlog rather than errors.

WHY IT MATTERS

Loose coupling converts synchronous, cascading failures into asynchronous backlogs the system can recover from.

FAILURE MODE

A synchronous call chain means one slow downstream service ties up threads and cascades into a full outage.

TRADE-OFF / DECISION

Asynchronous designs add eventual-consistency and idempotency concerns — consumers must handle duplicates and out-of-order delivery.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

RTO and RPO drive the DR pattern

IN DEPTH

- Recovery Time Objective is how quickly you must restore service; Recovery Point Objective is how much data you can afford to lose.
- Backup & restore, pilot light, warm standby and multi-site active/active trade rising cost for falling RTO/RPO.
- The right pattern is a business decision priced against the cost of downtime.

WHY IT MATTERS

Naming RTO/RPO turns 'make it resilient' into a concrete, testable and budgetable design target.

FAILURE MODE

Buying active/active for a back-office app wastes money; choosing backup-and-restore for a payment system risks the business.

TRADE-OFF / DECISION

Lower RTO/RPO means more running (and paid-for) standby capacity — match spend to the revenue at risk.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Data durability and multi-Region protection

IN DEPTH

- Multi-AZ RDS keeps a synchronous standby with automatic failover; read replicas add scale and cross-Region copies.
- DynamoDB global tables replicate active-active across Regions; S3 offers cross-Region replication and eleven-nines durability.
- Automated snapshots and point-in-time recovery protect against data corruption and mistakes.

WHY IT MATTERS

Compute is replaceable; data is not — durable, recoverable state is the foundation everything else rebuilds on.

FAILURE MODE

A single-AZ database or un-tested backups mean the app tier can be perfectly redundant yet still lose the business's data.

TRADE-OFF / DECISION

Synchronous replication protects data but adds write latency; cross-Region copies add cost and lag.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Automated recovery and self-healing

IN DEPTH

- ASG health checks replace failed instances; EC2 auto-recovery restores impaired instances.
- Route 53 health checks fail traffic over to a healthy Region or endpoint.
- The goal is that routine failures are handled by the system, not by a pager.

WHY IT MATTERS

Automating recovery shrinks MTTR toward zero for the common failures and frees humans for the novel ones.

FAILURE MODE

Manual failover runbooks that are never rehearsed fail exactly when you need them, under incident pressure.

TRADE-OFF / DECISION

Automated failover must guard against flapping and split-brain — tune thresholds and use health checks carefully.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Designing for graceful degradation

IN DEPTH

- Under stress a resilient system sheds load, serves cached or default responses and protects its core function.
- Timeouts, retries with backoff and jitter, and circuit breakers stop failures from amplifying.
- Bulkheads isolate resources so one hot tenant or feature can't starve the rest.

WHY IT MATTERS

Graceful degradation keeps the most important user journeys working even when parts of the system are unhealthy.

FAILURE MODE

Unbounded retries during an incident create a retry storm that turns a brownout into a full outage.

TRADE-OFF / DECISION

Serving degraded responses reduces functionality temporarily to preserve availability — usually the right call.

© 2026 Tertiary Infotech Academy Pte Ltd

PRICING RESILIENCE

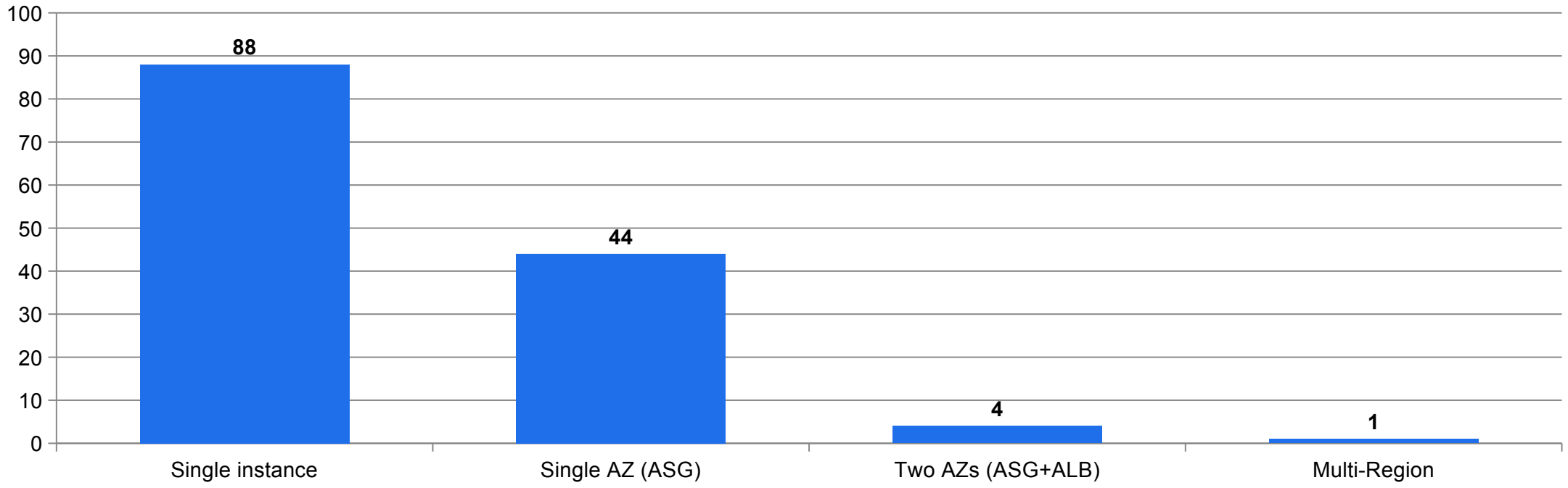
Disaster Recovery Patterns — Cost vs Recovery

Pattern	RTO	RPO	Relative cost
Backup & Restore	Hours	Hours	$
Pilot Light	Tens of minutes	Minutes	$$
Warm Standby	Minutes	Seconds	$$$
Multi-site Active/Active	Near-zero	Near-zero	$$$$

© 2026 Tertiary Infotech Academy Pte Ltd

WHY MULTI-AZ

Availability Math — Adding a Second AZ



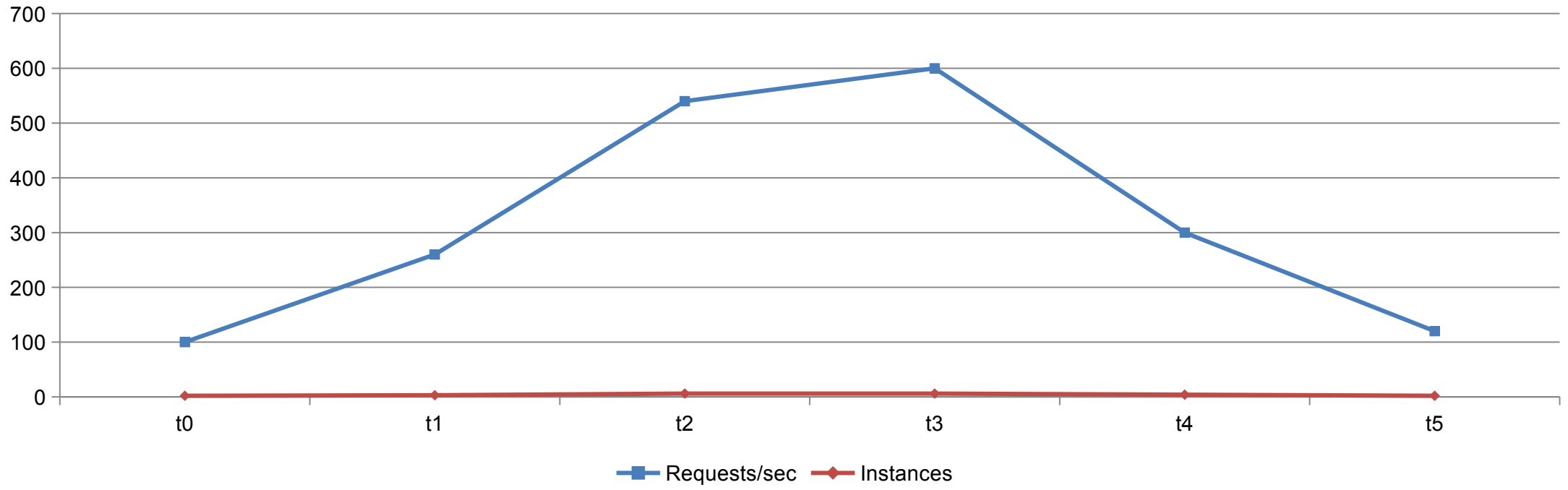
WHAT THE DATA SHOWS

Most availability gain comes from the first jump to multi-AZ; multi-Region adds resilience against rare Region events at much higher cost.

© 2026 Tertiary Infotech Academy Pte Ltd

ELASTICITY IN ACTION

Auto Scaling Under a Load Spike



WHAT THE DATA SHOWS

Target tracking follows demand up and back down, so capacity tracks load — you pay for peak only while the peak lasts.

© 2026 Tertiary Infotech Academy Pte Ltd

HOW 'HA' SYSTEMS STILL FALL OVER

Resilience Failure Modes & Anti-Patterns

1

Single-AZ database

App tier redundant, data tier isn't → Multi-AZ RDS / global tables.

2

Sticky in-memory sessions

One instance loss logs everyone out → externalise session state.

3

Retry storms

Unbounded retries amplify an outage → backoff + jitter + circuit breakers.

4

Untested failover

Runbook fails under pressure → game-days and automated failover.

5

Scaling on wrong metric

Slow under load, wasteful when idle → scale on the true constraint.

6

No health-check depth

LB routes to a process that's up but broken → deep health checks.

© 2026 Tertiary Infotech Academy Pte Ltd

PROVEN PRACTICES

Resilience Patterns Worth Adopting

1. **Multi-AZ by default**
   Never depend on a single Availability Zone.

2. **Stateless tiers**
   Push state to RDS/DynamoDB/ElastiCache so any instance is disposable.

3. **Backoff with jitter**
   Spread retries to avoid synchronised load.

4. **Circuit breaker**
   Stop calling a failing dependency; fail fast and recover.

5. **Health-check failover**
   Route 53 / ELB route around unhealthy targets.

6. **Game-days**
   Rehearse failure so recovery is routine, not heroic.

© 2026 Tertiary Infotech Academy Pte Ltd

MATCH SPEND TO RISK

Architecture Decision — Which DR Pattern?

A Backup & Restore

Non-critical or easily rebuilt workloads. Cheapest — but recovery takes hours and loses recent data.

B Pilot Light / Warm Standby

Important workloads needing minutes-scale recovery. A scaled-down copy runs ready to promote — moderate standing cost.

C Multi-site Active/Active

Revenue-critical systems that can't be down. Near-zero RTO/RPO — at roughly double the running cost and real complexity.

© 2026 Tertiary Infotech Academy Pte Ltd

Scenario — Surviving an Availability Zone outage

SITUATION

An AZ becomes impaired. Half the fleet is there, and the primary database node runs in it.

ANALYSIS

- Because compute is stateless behind an ALB across two AZs, the load balancer simply stops routing to the failed AZ.
- The Auto Scaling group launches replacement instances in the healthy AZ to restore capacity.
- Multi-AZ RDS fails over to the synchronous standby in the other AZ automatically.

DECISION

No human action is required for the common case — the design absorbs an AZ loss. Confirm capacity headroom so the healthy AZ can carry full load during the event.

© 2026 Tertiary Infotech Academy Pte Ltd

ELASTICITY UNDER PRESSURE

Scenario — A Black-Friday traffic spike

SITUATION

Traffic will rise 6x for a few hours during a sale, then return to normal.

ANALYSIS

- Fixed capacity sized for the peak wastes money the other 360 days; sized for normal, it falls over during the sale.
- Scheduled scaling pre-warms capacity just before the known event; target tracking then follows real demand.
- Decoupling checkout with SQS lets bursts queue instead of failing when a downstream service saturates.

DECISION

Combine scheduled pre-scaling for the known start time with target-tracking for the actual curve, and buffer spiky writes through a queue so load is absorbed as backlog.

© 2026 Tertiary Infotech Academy Pte Ltd

FROM SLA TO ARCHITECTURE

Worked Analysis — Reading an RTO/RPO Requirement

'Restore within 15 minutes' (RTO)	Rules out backup & restore (hours). Warm standby or pilot light needed.
'Lose at most a few seconds' (RPO)	Requires synchronous or near-real-time replication — Multi-AZ RDS or global tables.
'Survive a Region event'	Forces a cross-Region strategy, not just multi-AZ.
Budget is limited	Pilot light: minimal always-on footprint, scaled up on failover — a pragmatic middle.

TAKEAWAY

Each phrase in the requirement eliminates patterns until one fits — resilience design is requirement-driven, not taste-driven.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Compute and elasticity

1

PURPOSE

Match execution model and scaling behaviour to availability, latency, control and operational-effort requirements.

2

FEATURE 1

EC2 exposes instance families, purchase models, launch templates and full operating-system control.

3

FEATURE 2

Auto Scaling replaces unhealthy capacity and changes desired capacity from metrics or schedules.

4

FEATURE 3

ALB routes HTTP by host/path; NLB provides high-performance Layer 4 routing and static IP options.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 3 · SERVICE DECISION

Compute and elasticity: selection and controls

1

CHOOSE WHEN

Use EC2 for host-level control, Lambda for event-driven short execution, and Fargate when containers are wanted without host management.

2

DEVOPS ENGINEERING

Spread across AZs, use health checks and warm-up, scale on a demand-correlated signal and test replacement capacity.

3

FAILURE MODE

CPU-only scaling can miss queue depth, concurrency, connection or downstream bottlenecks.

4

EVIDENCE

Launch-template version, healthy-host count, scaling activity, target latency, saturation and replacement test.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Storage services

1

PURPOSE

Select durable object, block or shared-file storage with lifecycle and recovery controls matched to the workload.

2

FEATURE 1

S3 provides regional object durability, versioning, lifecycle tiers, replication and event notifications.

3

FEATURE 2

EBS is AZ-scoped block storage with volume types, snapshots and encryption.

4

FEATURE 3

EFS is elastic regional NFS; FSx supplies managed file systems such as Windows, Lustre, NetApp ONTAP and OpenZFS.

© 2026 Tertiary Infotech Academy Pte Ltd

Storage services: selection and controls

1

CHOOSE WHEN

Choose from access protocol, latency/throughput, sharing scope, AZ/Region boundary, retention and recovery objectives.

2

DEVOPS ENGINEERING

Encrypt, version, block public access, automate backup/copy, test restore and separate backup administration.

3

FAILURE MODE

Replication improves availability but can replicate corruption or deletion; it is not automatically a backup.

4

EVIDENCE

Storage class/volume type, encryption key, version/snapshot, replication status, restore test and recovery age.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Databases and data durability

1

PURPOSE

Provide transactional or key-value persistence with availability, scaling and recovery appropriate to data access patterns.

2

FEATURE 1

RDS Multi-AZ provides synchronous standby/failover; read replicas primarily scale reads.

3

FEATURE 2

Aurora separates compute/storage, supports reader endpoints and Global Database options.

4

FEATURE 3

DynamoDB offers partitioned key-value/document storage, on-demand/provisioned capacity, PITR, Streams and global tables.

© 2026 Tertiary Infotech Academy Pte Ltd

Databases and data durability: selection and controls

1

CHOOSE WHEN

Choose relational services for joins/transactions and DynamoDB for known key-access patterns at elastic scale; cache only where stale-data semantics are acceptable.

2

DEVOPS ENGINEERING

Enable backups/PITR, test failover/restore, monitor connections/lag/throttling and version schema changes.

3

FAILURE MODE

Confusing a read replica with a synchronous standby can produce an incorrect recovery design.

4

EVIDENCE

Backup/PITR state, replication lag, failover time, throttles, query latency and restore validation.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Networking and global delivery

1

PURPOSE

Create isolated network boundaries, predictable routing and resilient regional/global traffic paths.

2

FEATURE 1

VPC subnets are AZ-scoped; route tables, gateways, security groups and NACLs shape connectivity.

3

FEATURE 2

Transit Gateway provides hub-and-spoke routing across VPCs and on-premises connections.

4

FEATURE 3

Route 53 uses DNS policies/health checks; CloudFront caches HTTP content; Global Accelerator uses anycast static IPs over the AWS network.

© 2026 Tertiary Infotech Academy Pte Ltd

Networking and global delivery: selection and controls

1

CHOOSE WHEN

Choose DNS routing for endpoint selection, CloudFront for cacheable edge delivery and Global Accelerator for fast failover/static global entry points.

2

DEVOPS ENGINEERING

Design CIDRs and routes centrally, use private endpoints where appropriate, test health-check perspective and log network decisions.

3

FAILURE MODE

A healthy load balancer check may not prove dependencies, data freshness or the user transaction is healthy.

4

EVIDENCE

Route table, security-group diff, flow log, endpoint policy, DNS answer, edge metric and failover measurement.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: EC2, Auto Scaling and Elastic Load Balancing

1

PURPOSE

Build replaceable compute fleets whose capacity, routing and health behavior reflect the actual workload bottleneck.

2

SERVICE BOUNDARY

EC2 · launch templates · Auto Scaling · ALB/NLB/GWLB

3

PRIMARY DESIGN DECISION

Scale from backlog per instance, cap maximum capacity from database throughput, warm instances before service and test Spot interruption handling.

4

FAILURE BOUNDARY

CPU target tracking never reacts to the true bottleneck; immediate termination can abandon in-flight messages.

© 2026 Tertiary Infotech Academy Pte Ltd

EC2, Auto Scaling and Elastic Load Balancing: how it works

1

MECHANIC 1

Launch templates version AMI, instance type, IAM profile, network and user data; Auto Scaling groups should reference a controlled version rather than an accidental latest value.

2

MECHANIC 2

Target-tracking, step, predictive and scheduled scaling solve different demand patterns and must account for warm-up and downstream capacity.

3

MECHANIC 3

ALB routes HTTP/HTTPS using host, path and headers; NLB handles Layer 4 flows and static addresses; GWLB inserts compatible network appliances.

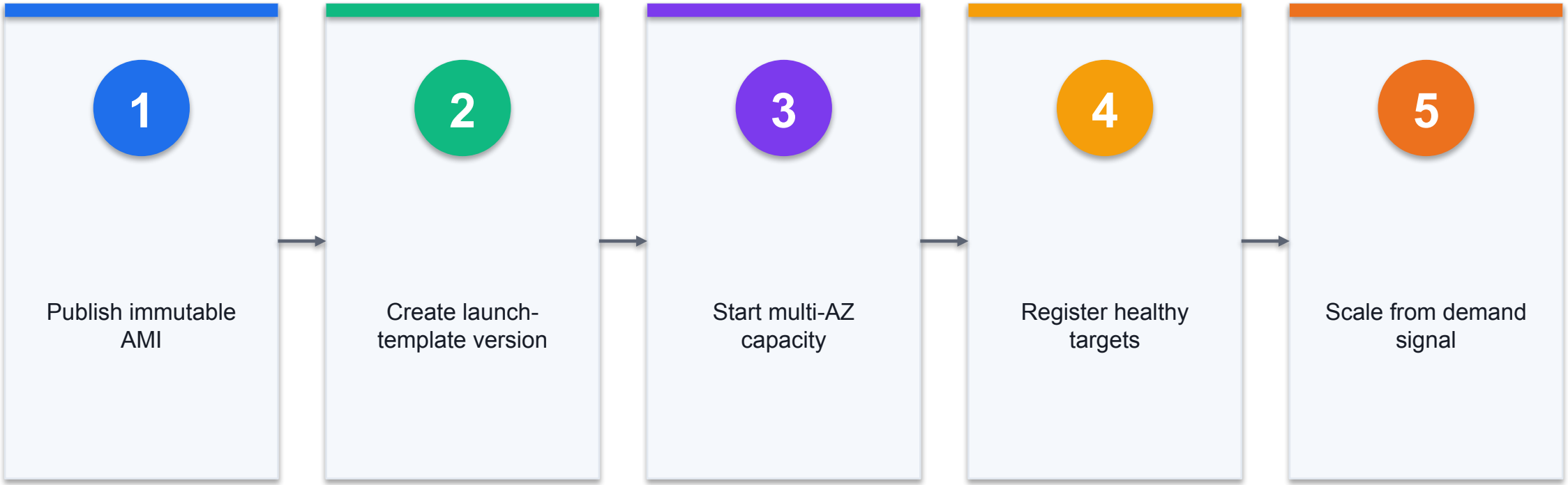
4

MECHANIC 4

EC2, load-balancer and custom health checks serve different purposes; replacement should use the signal that best represents serviceability.

© 2026 Tertiary Infotech Academy Pte Ltd

EC2, Auto Scaling and Elastic Load Balancing: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

EC2, Auto Scaling and Elastic Load Balancing: configuration decisions

1

PURCHASE

Combine On-Demand, Spot and commitments from interruption tolerance and baseline demand, not from unit price alone.

2

SCALING METRIC

Use request count, queue depth or concurrency when CPU does not track the true saturation point.

3

REFRESH

Use instance refresh with checkpoints and minimum healthy percentage for controlled fleet replacement.

4

CONNECTION HANDLING

Set deregistration delay and application shutdown so in-flight work completes during replacement.

© 2026 Tertiary Infotech Academy Pte Ltd

EC2, Auto Scaling and Elastic Load Balancing: worked architecture decision

1

SCENARIO

A queue worker fleet has low CPU but an increasing backlog and must recover without overwhelming the database.

2

DESIGN DECISION

Scale from backlog per instance, cap maximum capacity from database throughput, warm instances before service and test Spot interruption handling.

3

FAILURE TO AVOID

CPU target tracking never reacts to the true bottleneck; immediate termination can abandon in-flight messages.

4

ACCEPTANCE EVIDENCE

Launch-template version, healthy targets, scaling activity, backlog age, database saturation and graceful termination evidence.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: S3, EBS and shared file storage

1

PURPOSE

Choose object, block or file semantics from access pattern, failure boundary, performance and recovery requirements.

2

SERVICE BOUNDARY

S3 · EBS · EFS · FSx · AWS Backup

3

PRIMARY DESIGN DECISION

Use regional EFS with mount targets across application AZs, encrypted transit/storage and independent AWS Backup policy with restore testing.

4

FAILURE BOUNDARY

An EBS volume cannot attach across AZs as shared NFS; replication alone may reproduce deletion or corruption.

© 2026 Tertiary Infotech Academy Pte Ltd

S3, EBS and shared file storage: how it works

1

MECHANIC 1

S3 stores objects in regional buckets with versioning, lifecycle, replication, event notification and multiple storage classes; consistency does not remove application concurrency concerns.

2

MECHANIC 2

EBS is AZ-scoped block storage; gp3 separates baseline size from provisioned IOPS/throughput, while io2 targets sustained high IOPS and durability requirements.

3

MECHANIC 3

EFS provides regional NFS with mount targets and performance/throughput modes; access points isolate application paths and POSIX identities.

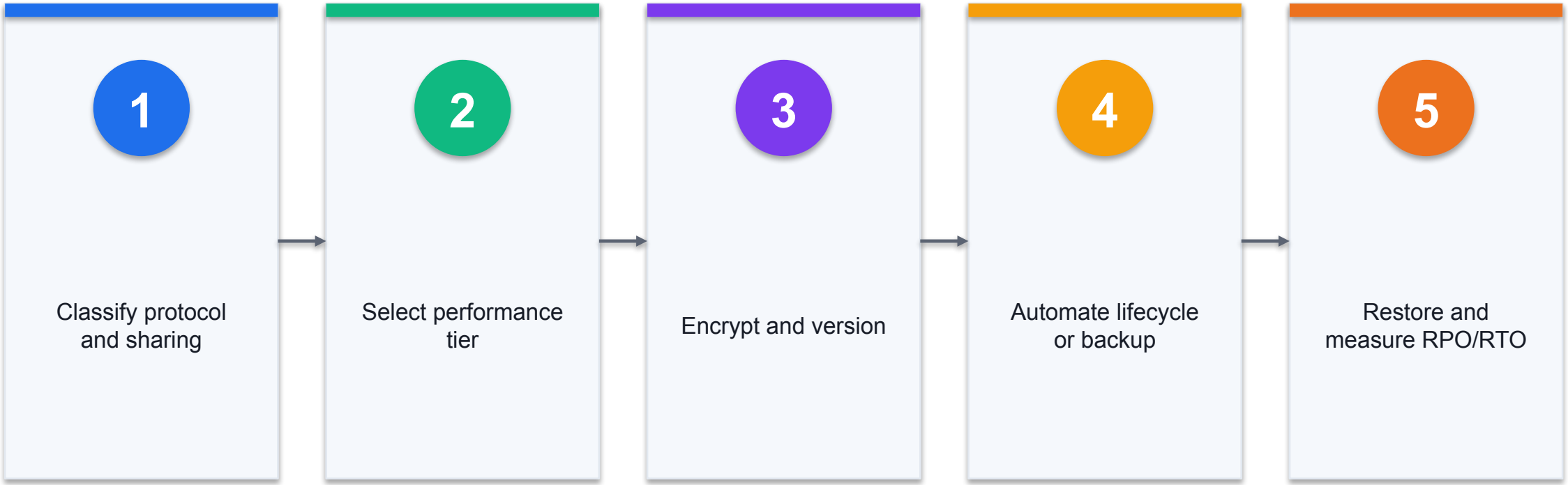
4

MECHANIC 4

FSx families expose workload-specific file protocols and performance, while AWS Backup centralizes policy, vault and copy workflows across supported services.

© 2026 Tertiary Infotech Academy Pte Ltd

S3, EBS and shared file storage: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

S3, EBS and shared file storage: configuration decisions

1

S3 PROTECTION

Enable Block Public Access, least-privilege bucket policy, versioning and Object Lock where retention requires immutability.

2

EBS RECOVERY

Use application-consistent snapshots where necessary and copy/encrypt across account or Region for isolation.

3

EFS DESIGN

Create mount targets in required AZs, secure NFS traffic and choose lifecycle policies from measured access.

4

BACKUP

Separate backup administration and vault permissions so workload compromise cannot delete every recovery point.

© 2026 Tertiary Infotech Academy Pte Ltd

S3, EBS and shared file storage: worked architecture decision

1

SCENARIO

A shared application volume must survive one AZ failure and retain recoverable versions against accidental deletion.

2

DESIGN DECISION

Use regional EFS with mount targets across application AZs, encrypted transit/storage and independent AWS Backup policy with restore testing.

3

FAILURE TO AVOID

An EBS volume cannot attach across AZs as shared NFS; replication alone may reproduce deletion or corruption.

4

ACCEPTANCE EVIDENCE

Storage configuration, encryption key, version/snapshot, backup job, vault policy, restore checksum and recovery timing.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: RDS, Aurora and ElastiCache data tiers

1

PURPOSE

Provide transactional persistence and caching while distinguishing availability, read scaling, connection management and disaster recovery.

2

SERVICE BOUNDARY

RDS · Aurora · RDS Proxy · ElastiCache

3

PRIMARY DESIGN DECISION

Keep writes on the relational writer, add RDS Proxy for pooled connections, direct safe reads to replicas and cache only explicitly stale-tolerant data.

4

FAILURE BOUNDARY

Caching transaction state or treating replica reads as immediately current creates correctness defects hidden by low latency.

© 2026 Tertiary Infotech Academy Pte Ltd

RDS, Aurora and ElastiCache data tiers: how it works

1

MECHANIC 1

RDS Multi-AZ deployments provide synchronous standby behavior for availability; read replicas are asynchronous and primarily serve read scaling or DR patterns.

2

MECHANIC 2

Aurora separates distributed storage from compute, exposes writer and reader endpoints and can add Global Database for cross-Region read and recovery designs.

3

MECHANIC 3

RDS Proxy pools connections and can protect databases from serverless connection storms while integrating with IAM or Secrets Manager authentication.

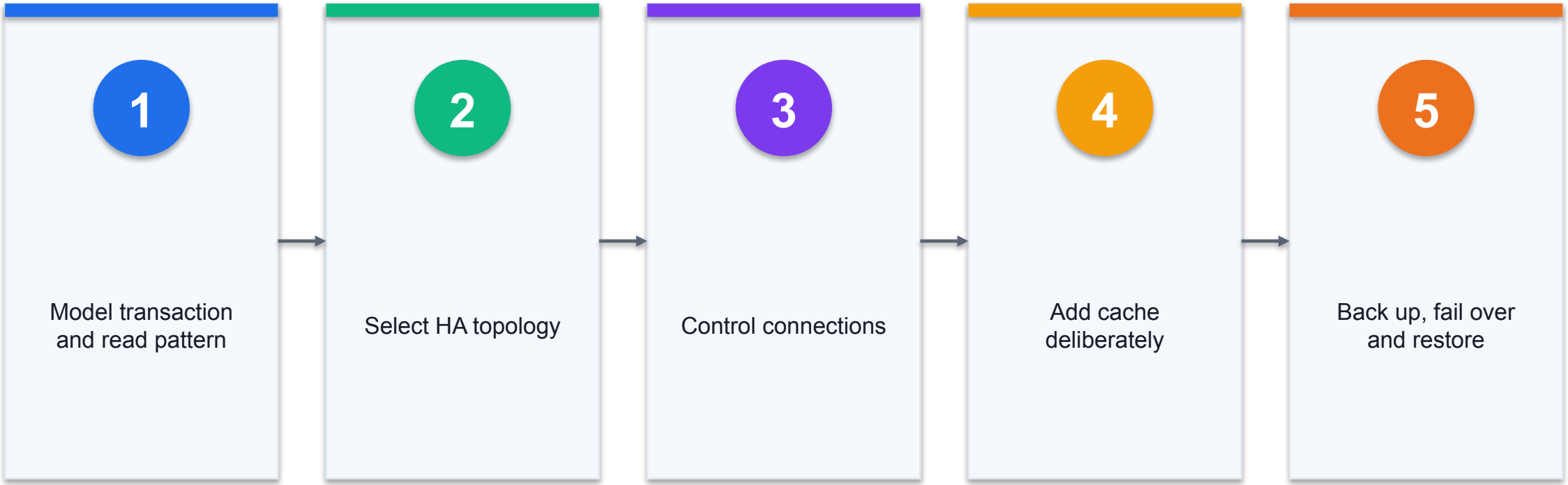
4

MECHANIC 4

ElastiCache for Redis/Valkey or Memcached changes latency and load but introduces eviction, expiry and stale-data semantics that the application must own.

© 2026 Tertiary Infotech Academy Pte Ltd

RDS, Aurora and ElastiCache data tiers: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

RDS, Aurora and ElastiCache data tiers: configuration decisions

1

STANDBY VS REPLICA

Use Multi-AZ for service continuity and replicas for read capacity; do not claim a replica is synchronous unless the selected architecture provides it.

2

CREDENTIALS

Store and rotate database secrets or use IAM authentication where supported; keep credentials out of application packages.

3

CACHE POLICY

Choose cache-aside, write-through or other behavior with explicit TTL, invalidation and failure fallback.

4

SCHEMA CHANGE

Use backward-compatible expansion and contraction so application and database versions can overlap during deployment.

© 2026 Tertiary Infotech Academy Pte Ltd

RDS, Aurora and ElastiCache data tiers: worked architecture decision

1

SCENARIO

A Lambda workload causes connection spikes and read latency during release, but writes must remain strongly transactional.

2

DESIGN DECISION

Keep writes on the relational writer, add RDS Proxy for pooled connections, direct safe reads to replicas and cache only explicitly stale-tolerant data.

3

FAILURE TO AVOID

Caching transaction state or treating replica reads as immediately current creates correctness defects hidden by low latency.

4

ACCEPTANCE EVIDENCE

Connection count, replica lag, failover event, cache hit/eviction, query latency, backup status and restore validation.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: DynamoDB keys, capacity and change streams

1

PURPOSE

Design key-value workloads from access patterns, partition distribution and consistency requirements before writing application code.

2

SERVICE BOUNDARY

DynamoDB · Streams · DAX · global tables

3

PRIMARY DESIGN DECISION

Increase partition-key cardinality with a controlled shard suffix, retain tenant/time queryability through an index or aggregation path and add jittered retries.

4

FAILURE BOUNDARY

Raising total capacity alone does not redistribute a single hot logical key; blind retries can amplify throttling.

© 2026 Tertiary Infotech Academy Pte Ltd

DynamoDB keys, capacity and change streams: how it works

1

MECHANIC 1

Partition and optional sort keys define item placement and query order; scans read broadly and are not a substitute for a missing access pattern.

2

MECHANIC 2

Global secondary indexes add alternate partition/sort keys and their own capacity behavior; local secondary indexes share the base partition key and must be created with the table.

3

MECHANIC 3

On-demand capacity absorbs variable traffic, while provisioned capacity and auto scaling fit predictable demand; both can throttle hot partitions.

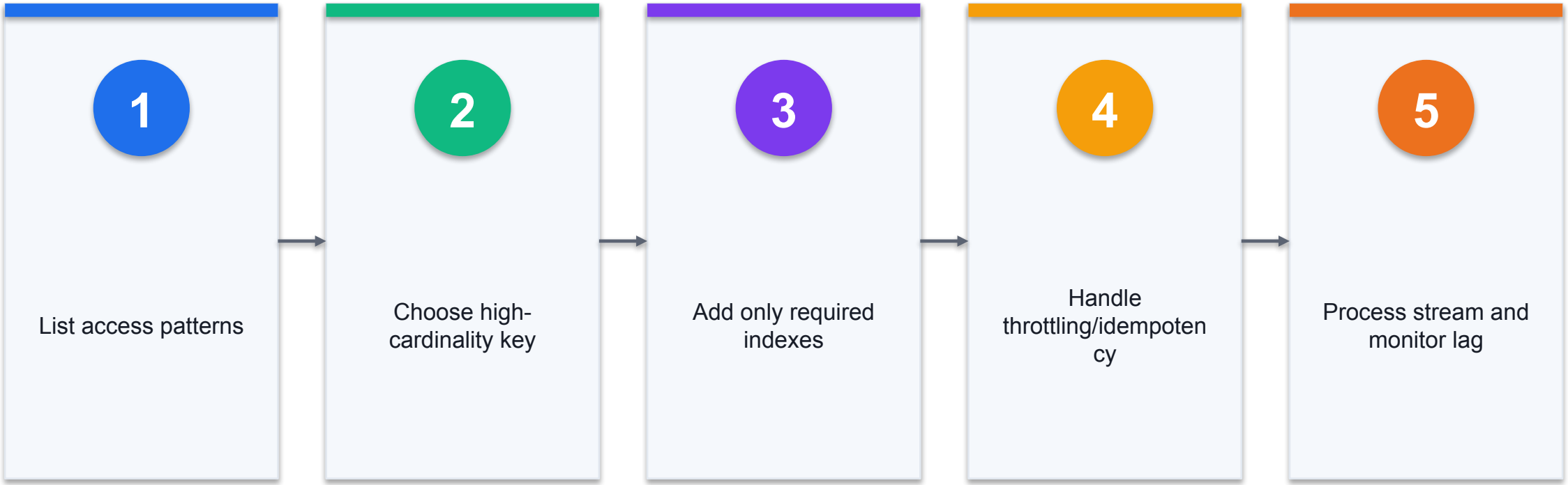
4

MECHANIC 4

Streams capture item-level changes for time-bounded processing; TTL expiration, transactions, conditional writes and global tables solve different lifecycle and concurrency problems.

© 2026 Tertiary Infotech Academy Pte Ltd

DynamoDB keys, capacity and change streams: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

DynamoDB keys, capacity and change streams: configuration decisions

1

CONSISTENCY

Strongly consistent reads are available only on supported base-table/index paths and cost more read capacity; global replication remains asynchronous.

2

CONCURRENCY

Use condition expressions or transactions to prevent lost updates and duplicate state transitions.

3

LARGE OBJECTS

Store large payloads in S3 and keep an object reference plus integrity metadata in the item.

4

HOT KEYS

Write-shard only after understanding the required read aggregation and operational complexity.

© 2026 Tertiary Infotech Academy Pte Ltd

DynamoDB keys, capacity and change streams: worked architecture decision

1

SCENARIO

An event-ingestion table throttles one tenant even though overall capacity is below the configured total.

2

DESIGN DECISION

Increase partition-key cardinality with a controlled shard suffix, retain tenant/time queryability through an index or aggregation path and add jittered retries.

3

FAILURE TO AVOID

Raising total capacity alone does not redistribute a single hot logical key; blind retries can amplify throttling.

4

ACCEPTANCE EVIDENCE

Consumed capacity by operation, throttled requests, key distribution sample, index metrics, stream iterator age and conditional-write failures.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: VPC, DNS and global content delivery

1

PURPOSE

Control application reachability and global user paths across routing, security, name resolution, caching and failover layers.

2

SERVICE BOUNDARY

VPC · PrivateLink · Route 53 · CloudFront · Global Accelerator

3

PRIMARY DESIGN DECISION

Use a private API endpoint with interface endpoints or controlled network connectivity, private DNS and layered endpoint/resource policies.

4

FAILURE BOUNDARY

A private subnet label provides no isolation by itself; missing routes or broad endpoint policies can defeat the intended boundary.

© 2026 Tertiary Infotech Academy Pte Ltd

VPC, DNS and global content delivery: how it works

1

MECHANIC 1

Subnets are AZ-scoped, route tables choose next hops, security groups are stateful and NACLs are stateless; each layer answers a different connectivity question.

2

MECHANIC 2

Gateway and interface VPC endpoints keep supported service traffic on AWS networks and can apply endpoint policies as an additional authorization boundary.

3

MECHANIC 3

Route 53 policies choose DNS answers from health, latency, geography, weights or failover; TTL controls how quickly resolvers can observe changes.

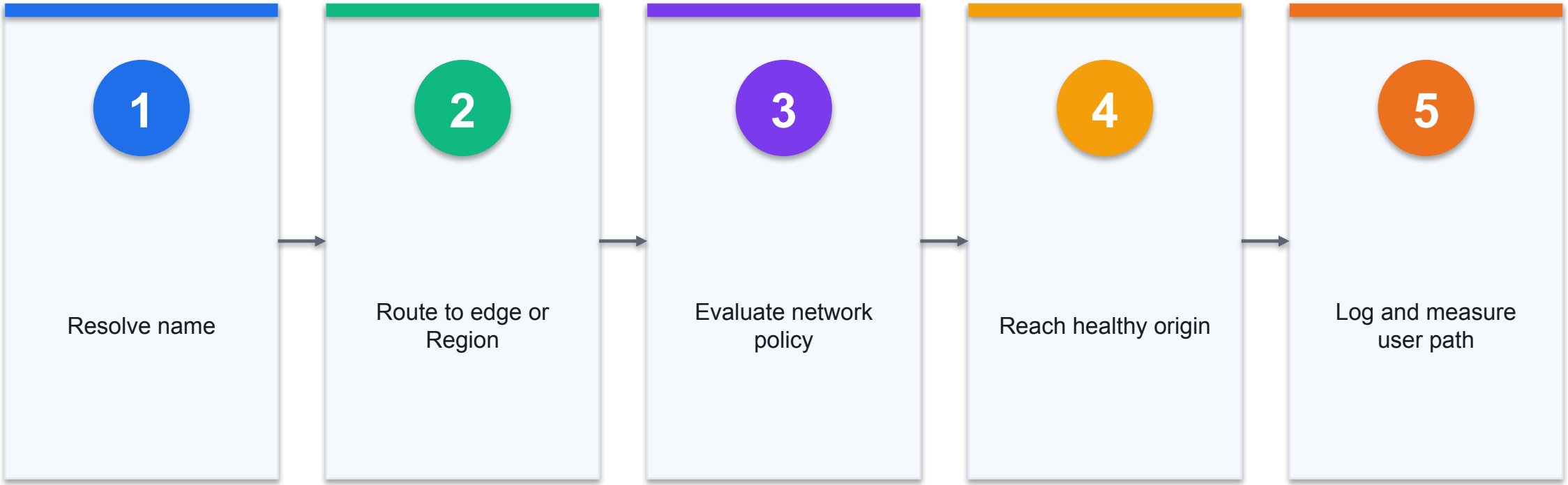
4

MECHANIC 4

CloudFront caches HTTP content at the edge using cache and origin-request policies, while Global Accelerator provides static anycast entry addresses and fast endpoint routing.

© 2026 Tertiary Infotech Academy Pte Ltd

VPC, DNS and global content delivery: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

VPC, DNS and global content delivery: configuration decisions

1

PRIVATE ACCESS

Use endpoints and private DNS for service APIs where egress control matters; verify endpoint and resource policies together.

2

CACHING

Build cache keys only from values that change the response and protect private content with signed access patterns.

3

FAILOVER

Health checks must represent the user transaction and account for DNS caching or connection persistence.

4

TROUBLESHOOTING

Check DNS, routes, security groups, NACL return paths, endpoint policy, target health and application logs in order.

© 2026 Tertiary Infotech Academy Pte Ltd

VPC, DNS and global content delivery: worked architecture decision

1

SCENARIO

A private API must be consumed by workloads in several VPCs and expose no public internet path.

2

DESIGN DECISION

Use a private API endpoint with interface endpoints or controlled network connectivity, private DNS and layered endpoint/resource policies.

3

FAILURE TO AVOID

A private subnet label provides no isolation by itself; missing routes or broad endpoint policies can defeat the intended boundary.

4

ACCEPTANCE EVIDENCE

DNS response, route path, endpoint ID/policy, flow log, target health, access log and reachability analysis.

© 2026 Tertiary Infotech Academy Pte Ltd

Availability requirements and failure modes

1

OUTCOME

Translate business impact into explicit availability, recovery and dependency requirements.

2

ARCHITECTURE

Model component failure domains, remove single points of failure and test degraded modes before selecting services.

3

FAILURE MODE

Adding replicas does not help when they share the same failure domain or control-plane dependency.

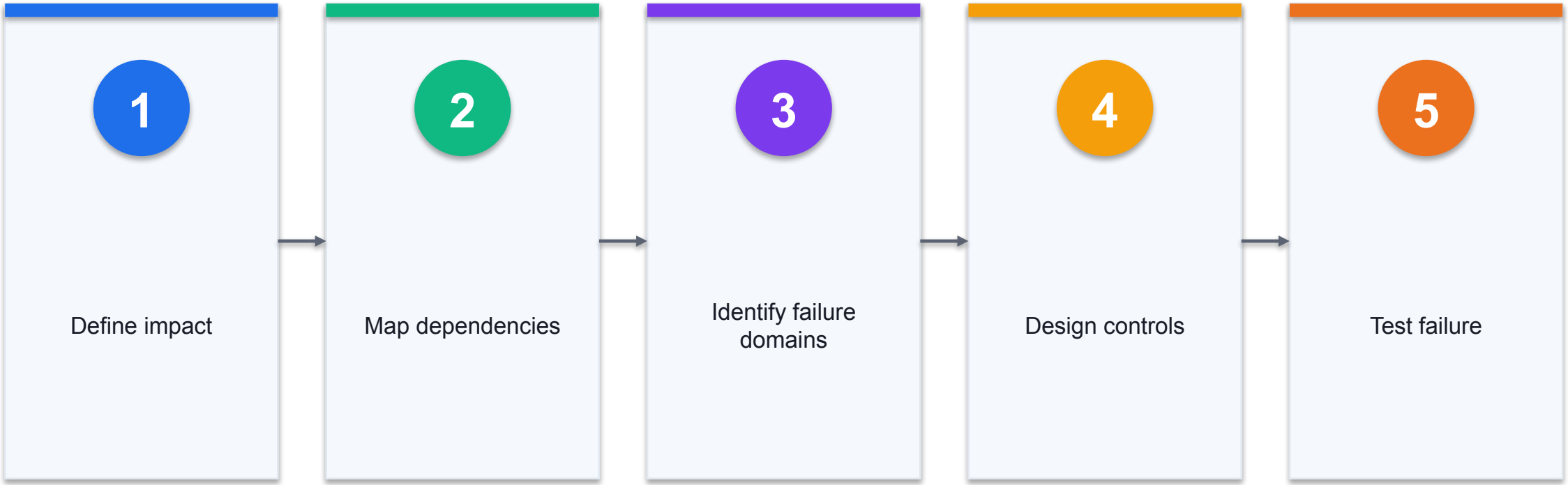
4

EVIDENCE

Availability target, dependency map, failure-mode analysis, resilience recommendation and test result.

© 2026 Tertiary Infotech Academy Pte Ltd

Availability requirements and failure modes: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Availability requirements and failure modes: design decision

? SCENARIO

An order service must remain available through an Availability Zone outage.

1 DESIGN RESPONSE

Model component failure domains, remove single points of failure and test degraded modes before selecting services.

2 PRIMARY SERVICES

Multi-AZ · AWS Resilience Hub · Well-Architected Tool

3 FAILURE MODE

Adding replicas does not help when they share the same failure domain or control-plane dependency.

4 PROOF

Availability target, dependency map, failure-mode analysis, resilience recommendation and test result.

© 2026 Tertiary Infotech Academy Pte Ltd

Availability requirements and failure modes: failure diagnostic

1

RECOGNISE

Observable failure condition: Adding replicas does not help when they share the same failure domain or control-plane dependency.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Define impact through Test failure; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Availability target, dependency map, failure-mode analysis, resilience recommendation and test result.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 3 · APPLIED SCENARIO

Availability requirements and failure modes: worked decision

1

CONSTRAINT

An order service must remain available through an Availability Zone outage.

2

DECISION

Model component failure domains, remove single points of failure and test degraded modes before selecting services.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Adding replicas does not help when they share the same failure domain or control-plane dependency.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Availability target, dependency map, failure-mode analysis, resilience recommendation and test result.

© 2026 Tertiary Infotech Academy Pte Ltd

Elastic compute and load distribution

1

OUTCOME

Match scaling signals, capacity buffers and load-balancer health to workload behavior.

2

ARCHITECTURE

Use target tracking where possible, warm-up periods, lifecycle hooks, multiple AZs and meaningful health checks.

3

FAILURE MODE

CPU is a poor scaling signal for queue-driven or latency-bound workloads.

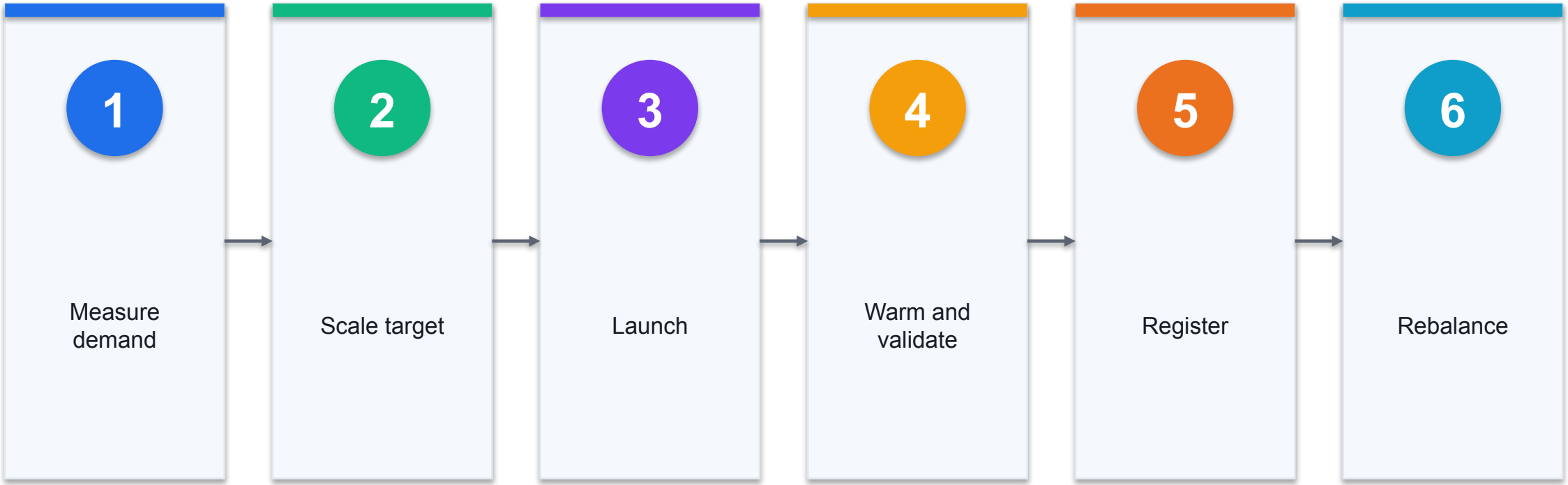
4

EVIDENCE

Scaling policy, metric math, warm-up, lifecycle state, target health and capacity event.

© 2026 Tertiary Infotech Academy Pte Ltd

Elastic compute and load distribution: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 3 · SCENARIO

Elastic compute and load distribution: design decision

?

SCENARIO

Traffic spikes arrive in ten minutes, but new instances need eight minutes to become healthy.

1

DESIGN RESPONSE

Use target tracking where possible, warm-up periods, lifecycle hooks, multiple AZs and meaningful health checks.

2

PRIMARY SERVICES

Auto Scaling · ALB · NLB · EC2 Fleet

3

FAILURE MODE

CPU is a poor scaling signal for queue-driven or latency-bound workloads.

4

PROOF

Scaling policy, metric math, warm-up, lifecycle state, target health and capacity event.

© 2026 Tertiary Infotech Academy Pte Ltd

Elastic compute and load distribution: failure diagnostic

1

RECOGNISE

Observable failure condition: CPU is a poor scaling signal for queue-driven or latency-bound workloads.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Measure demand through Rebalance; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Scaling policy, metric math, warm-up, lifecycle state, target health and capacity event.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 3 · APPLIED SCENARIO

Elastic compute and load distribution: worked decision

1

CONSTRAINT

Traffic spikes arrive in ten minutes, but new instances need eight minutes to become healthy.

2

DECISION

Use target tracking where possible, warm-up periods, lifecycle hooks, multiple AZs and meaningful health checks.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: CPU is a poor scaling signal for queue-driven or latency-bound workloads.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Scaling policy, metric math, warm-up, lifecycle state, target health and capacity event.

© 2026 Tertiary Infotech Academy Pte Ltd

DNS and multi-Region traffic control

1

OUTCOME

Route users according to health, latency, geography and recovery objectives.

2

ARCHITECTURE

Combine endpoint health with failover or latency policies; understand DNS caching and use anycast when convergence must be faster.

3

FAILURE MODE

A low TTL cannot force clients or resolvers to refresh immediately.

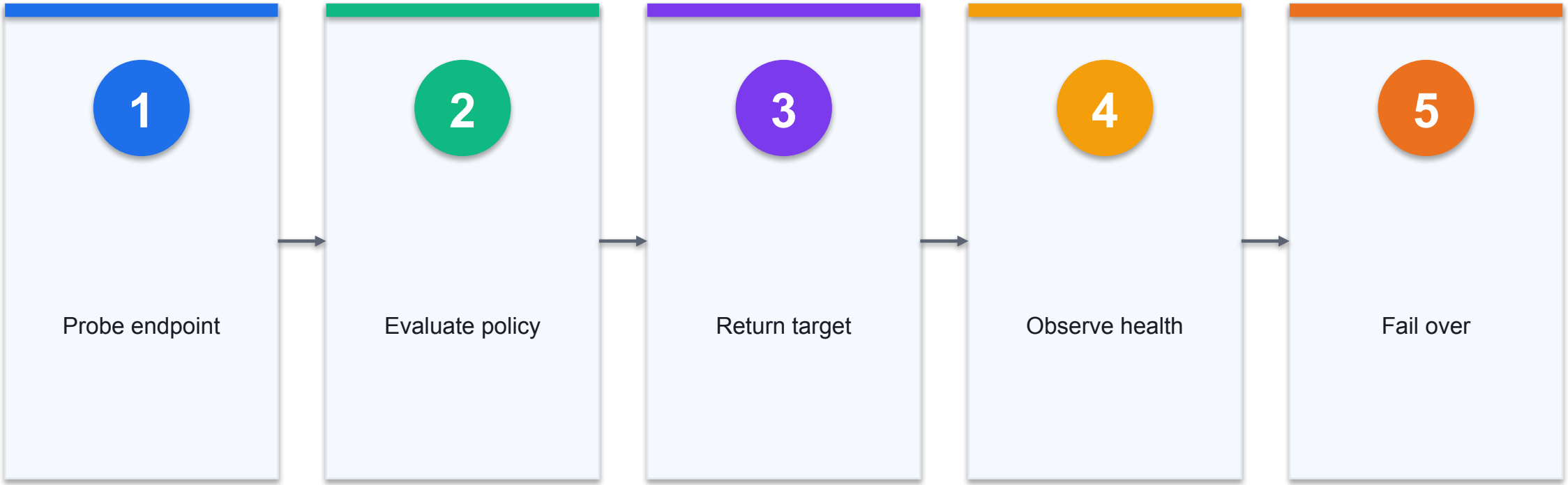
4

EVIDENCE

Routing policy, health-check state, TTL, endpoint weight and observed failover time.

© 2026 Tertiary Infotech Academy Pte Ltd

DNS and multi-Region traffic control: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 3 · SCENARIO

DNS and multi-Region traffic control: design decision

? SCENARIO

A global API needs regional failover within one minute with stable client endpoints.

1 DESIGN RESPONSE

Combine endpoint health with failover or latency policies; understand DNS caching and use anycast when convergence must be faster.

2 PRIMARY SERVICES

Route 53 · Global Accelerator · CloudFront

3 FAILURE MODE

A low TTL cannot force clients or resolvers to refresh immediately.

4 PROOF

Routing policy, health-check state, TTL, endpoint weight and observed failover time.

© 2026 Tertiary Infotech Academy Pte Ltd

DNS and multi-Region traffic control: failure diagnostic

1

RECOGNISE

Observable failure condition: A low TTL cannot force clients or resolvers to refresh immediately.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Probe endpoint through Fail over; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Routing policy, health-check state, TTL, endpoint weight and observed failover time.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 3 · APPLIED SCENARIO

DNS and multi-Region traffic control: worked decision

1

CONSTRAINT

A global API needs regional failover within one minute with stable client endpoints.

2

DECISION

Combine endpoint health with failover or latency policies; understand DNS caching and use anycast when convergence must be faster.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: A low TTL cannot force clients or resolvers to refresh immediately.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Routing policy, health-check state, TTL, endpoint weight and observed failover time.

© 2026 Tertiary Infotech Academy Pte Ltd

Resilient data stores

1

OUTCOME

Select replication and failover mechanisms that meet consistency, RPO and RTO requirements.

2

ARCHITECTURE

Distinguish read scaling from high availability, test failover, plan write ownership and monitor replication lag.

3

FAILURE MODE

A read replica is not automatically a synchronous standby or a complete DR strategy.

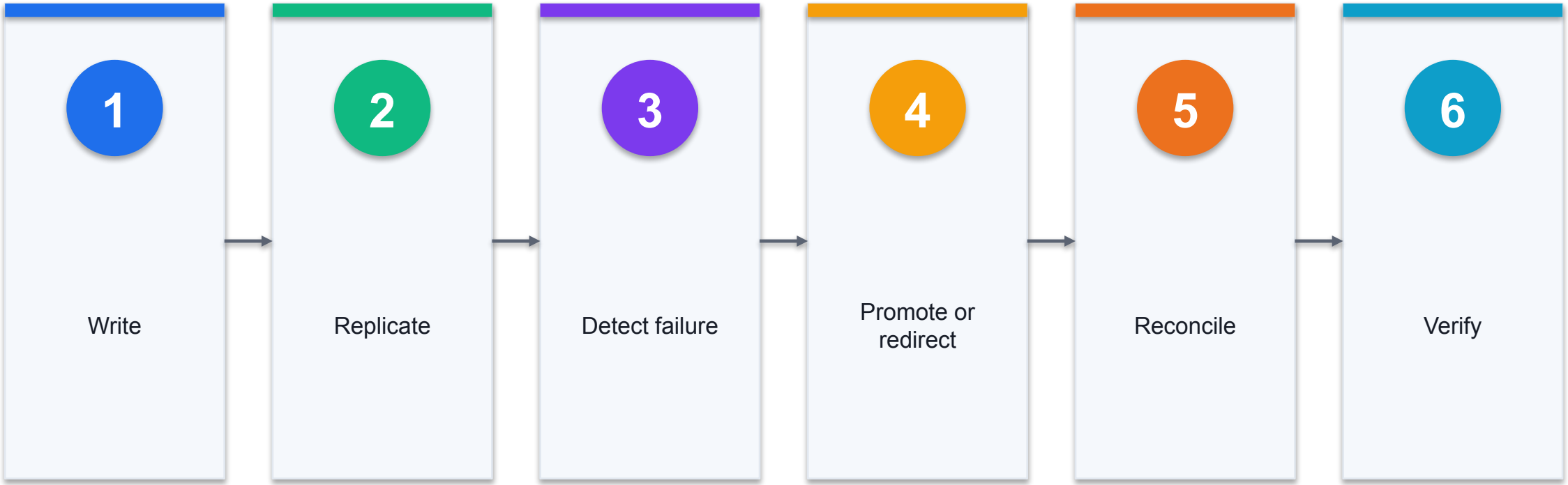
4

EVIDENCE

Topology, replication mode, lag metric, backup status, failover test and recovery evidence.

© 2026 Tertiary Infotech Academy Pte Ltd

Resilient data stores: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Resilient data stores: design decision

? SCENARIO

A payment datastore needs near-zero data loss and a tested cross-Region recovery path.

1 DESIGN RESPONSE

Distinguish read scaling from high availability, test failover, plan write ownership and monitor replication lag.

2 PRIMARY SERVICES

RDS Multi-AZ · Aurora Global Database · DynamoDB Global Tables

3 FAILURE MODE

A read replica is not automatically a synchronous standby or a complete DR strategy.

4 PROOF

Topology, replication mode, lag metric, backup status, failover test and recovery evidence.

© 2026 Tertiary Infotech Academy Pte Ltd

Resilient data stores: failure diagnostic

1

RECOGNISE

Observable failure condition: A read replica is not automatically a synchronous standby or a complete DR strategy.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Write through Verify; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Topology, replication mode, lag metric, backup status, failover test and recovery evidence.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 3 · APPLIED SCENARIO

Resilient data stores: worked decision

1

CONSTRAINT

A payment datastore needs near-zero data loss and a tested cross-Region recovery path.

2

DECISION

Distinguish read scaling from high availability, test failover, plan write ownership and monitor replication lag.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: A read replica is not automatically a synchronous standby or a complete DR strategy.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Topology, replication mode, lag metric, backup status, failover test and recovery evidence.

© 2026 Tertiary Infotech Academy Pte Ltd

Serverless resilience

1

OUTCOME

Control concurrency, retries, idempotency and failure destinations in event-driven workloads.

2

ARCHITECTURE

Use queues for buffering, partial batch response, reserved concurrency, DLQs and idempotency keys.

3

FAILURE MODE

Unbounded retries can multiply load and duplicate side effects during partial failure.

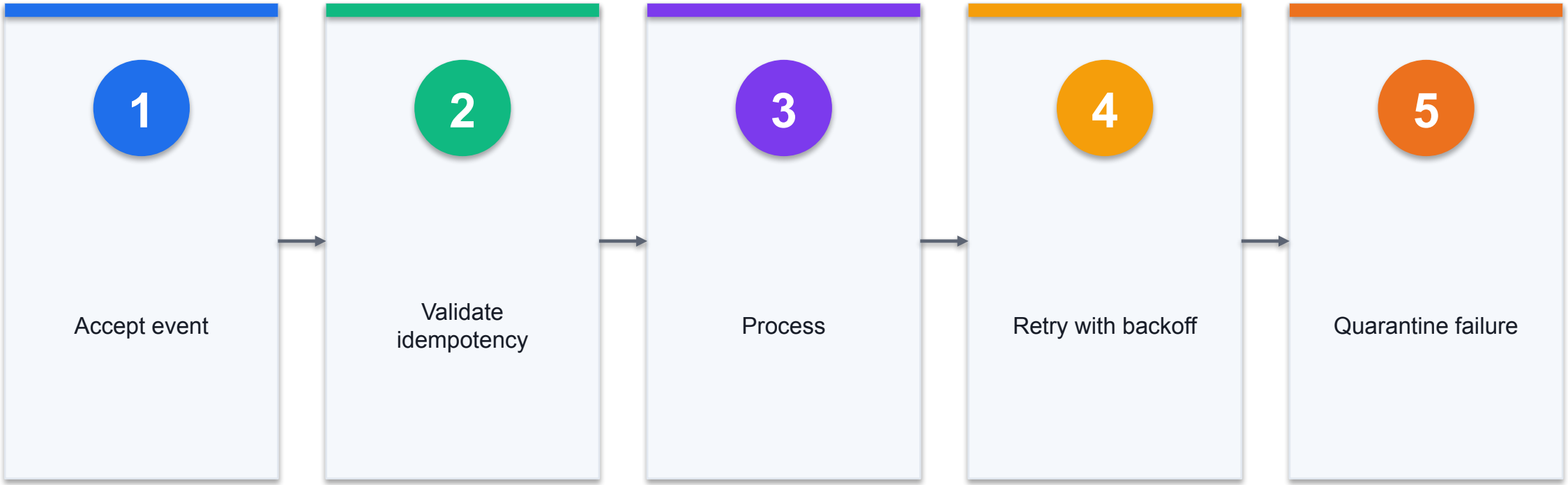
4

EVIDENCE

Request ID, retry count, idempotency record, queue age, DLQ message and concurrency metric.

© 2026 Tertiary Infotech Academy Pte Ltd

Serverless resilience: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Serverless resilience: design decision

? SCENARIO

A batch consumer occasionally times out after charging a card but before acknowledging the message.

1 DESIGN RESPONSE

Use queues for buffering, partial batch response, reserved concurrency, DLQs and idempotency keys.

2 PRIMARY SERVICES

Lambda · API Gateway · SQS · Step Functions

3 FAILURE MODE

Unbounded retries can multiply load and duplicate side effects during partial failure.

4 PROOF

Request ID, retry count, idempotency record, queue age, DLQ message and concurrency metric.

© 2026 Tertiary Infotech Academy Pte Ltd

Serverless resilience: failure diagnostic

1

RECOGNISE

Observable failure condition: Unbounded retries can multiply load and duplicate side effects during partial failure.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Accept event through Quarantine failure; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Request ID, retry count, idempotency record, queue age, DLQ message and concurrency metric.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 3 · APPLIED SCENARIO

Serverless resilience: worked decision

1

CONSTRAINT

A batch consumer occasionally times out after charging a card but before acknowledging the message.

2

DECISION

Use queues for buffering, partial batch response, reserved concurrency, DLQs and idempotency keys.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Unbounded retries can multiply load and duplicate side effects during partial failure.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Request ID, retry count, idempotency record, queue age, DLQ message and concurrency metric.

© 2026 Tertiary Infotech Academy Pte Ltd

Container workload resilience

1

OUTCOME

Keep container services available through node, task and deployment failures.

2

ARCHITECTURE

Spread tasks, set disruption budgets, use readiness checks, pin image digests and scale both service and capacity.

3

FAILURE MODE

Scaling tasks without underlying capacity leaves workloads pending.

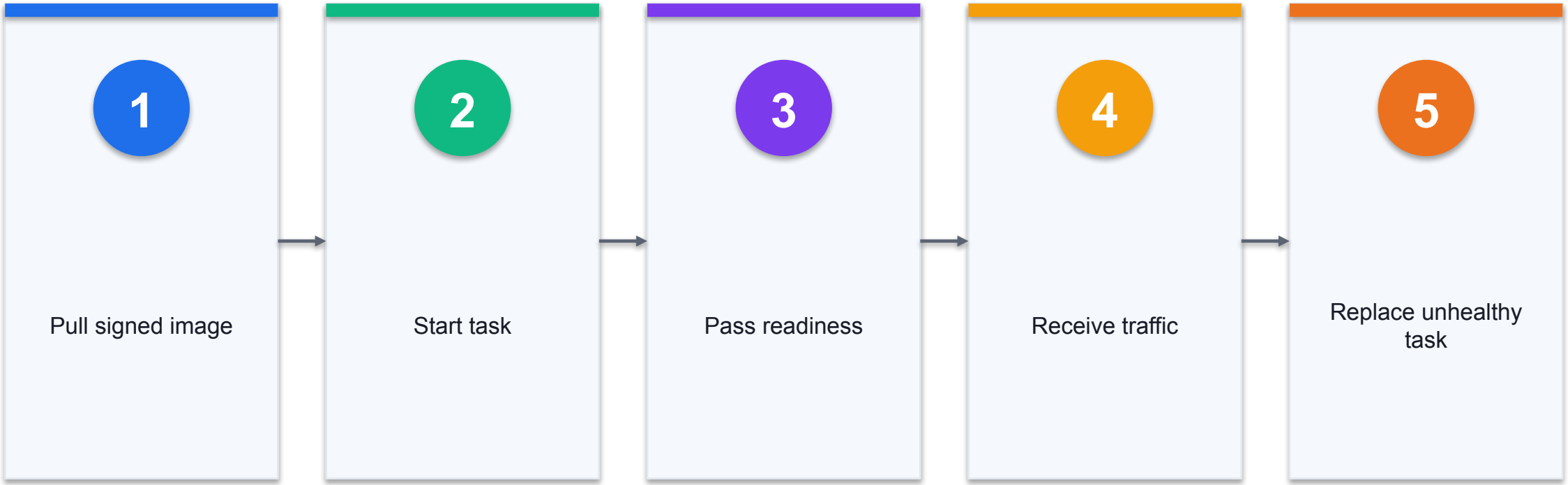
4

EVIDENCE

Task placement, readiness state, image digest, service events, capacity-provider metric and deployment circuit breaker.

© 2026 Tertiary Infotech Academy Pte Ltd

Container workload resilience: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Container workload resilience: design decision

? SCENARIO

An ECS service needs blue/green deployment and automatic rollback when error rate rises.

1 DESIGN RESPONSE

Spread tasks, set disruption budgets, use readiness checks, pin image digests and scale both service and capacity.

2 PRIMARY SERVICES

ECS · EKS · ECR · Application Auto Scaling

3 FAILURE MODE

Scaling tasks without underlying capacity leaves workloads pending.

4 PROOF

Task placement, readiness state, image digest, service events, capacity-provider metric and deployment circuit breaker.

© 2026 Tertiary Infotech Academy Pte Ltd

Container workload resilience: failure diagnostic

1

RECOGNISE

Observable failure condition: Scaling tasks without underlying capacity leaves workloads pending.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Pull signed image through Replace unhealthy task; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Task placement, readiness state, image digest, service events, capacity-provider metric and deployment circuit breaker.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 3 · APPLIED SCENARIO

Container workload resilience: worked decision

1

CONSTRAINT

An ECS service needs blue/green deployment and automatic rollback when error rate rises.

2

DECISION

Spread tasks, set disruption budgets, use readiness checks, pin image digests and scale both service and capacity.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Scaling tasks without underlying capacity leaves workloads pending.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Task placement, readiness state, image digest, service events, capacity-provider metric and deployment circuit breaker.

© 2026 Tertiary Infotech Academy Pte Ltd

Disaster recovery and automated recovery

1

OUTCOME

Choose and test backup/restore, pilot-light, warm-standby or multi-site strategies against business objectives.

2

ARCHITECTURE

Map each workload to RTO/RPO, automate backups and copies, document dependencies and conduct game days.

3

FAILURE MODE

A backup job marked successful does not prove the application can be restored coherently.

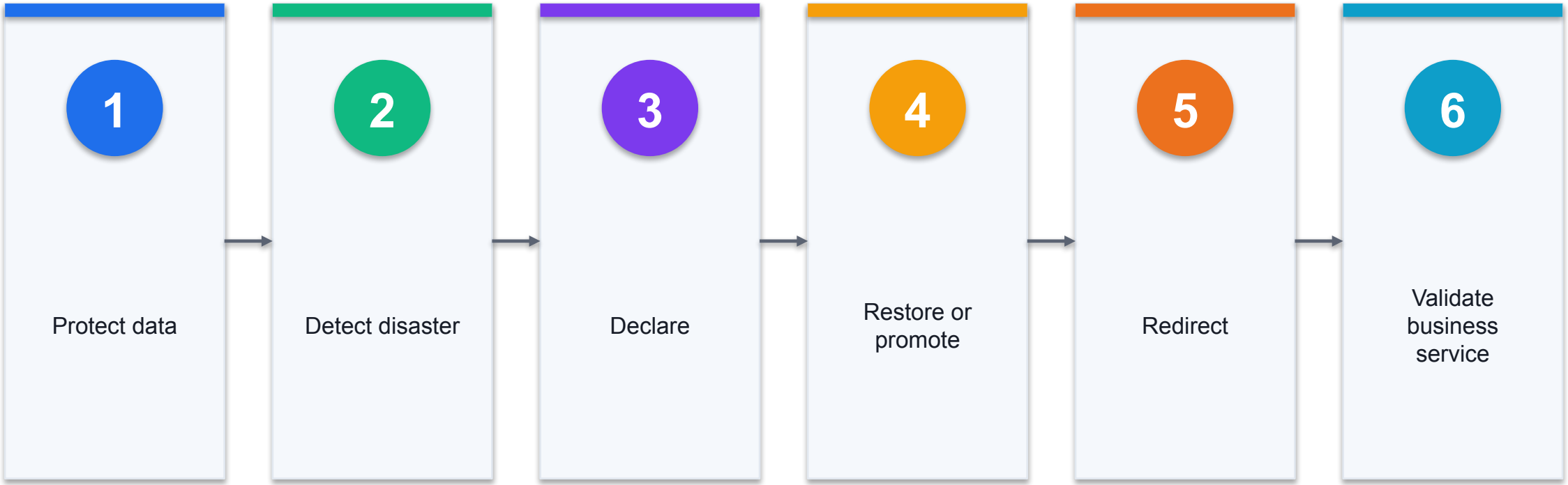
4

EVIDENCE

Recovery-point age, restore-test outcome, runbook duration, dependency validation and actual RTO/RPO.

© 2026 Tertiary Infotech Academy Pte Ltd

Disaster recovery and automated recovery: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Disaster recovery and automated recovery: design decision

? SCENARIO

A regional disruption requires the customer portal to recover within 60 minutes with at most 15 minutes of data loss.

1 DESIGN RESPONSE

Map each workload to RTO/RPO, automate backups and copies, document dependencies and conduct game days.

2 PRIMARY SERVICES

AWS Backup · Elastic Disaster Recovery · Route 53

3 FAILURE MODE

A backup job marked successful does not prove the application can be restored coherently.

4 PROOF

Recovery-point age, restore-test outcome, runbook duration, dependency validation and actual RTO/RPO.

© 2026 Tertiary Infotech Academy Pte Ltd

Disaster recovery and automated recovery: failure diagnostic

1

RECOGNISE

Observable failure condition: A backup job marked successful does not prove the application can be restored coherently.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Protect data through Validate business service; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Recovery-point age, restore-test outcome, runbook duration, dependency validation and actual RTO/RPO.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 3 · APPLIED SCENARIO

Disaster recovery and automated recovery: worked decision

1

CONSTRAINT

A regional disruption requires the customer portal to recover within 60 minutes with at most 15 minutes of data loss.

2

DECISION

Map each workload to RTO/RPO, automate backups and copies, document dependencies and conduct game days.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: A backup job marked successful does not prove the application can be restored coherently.

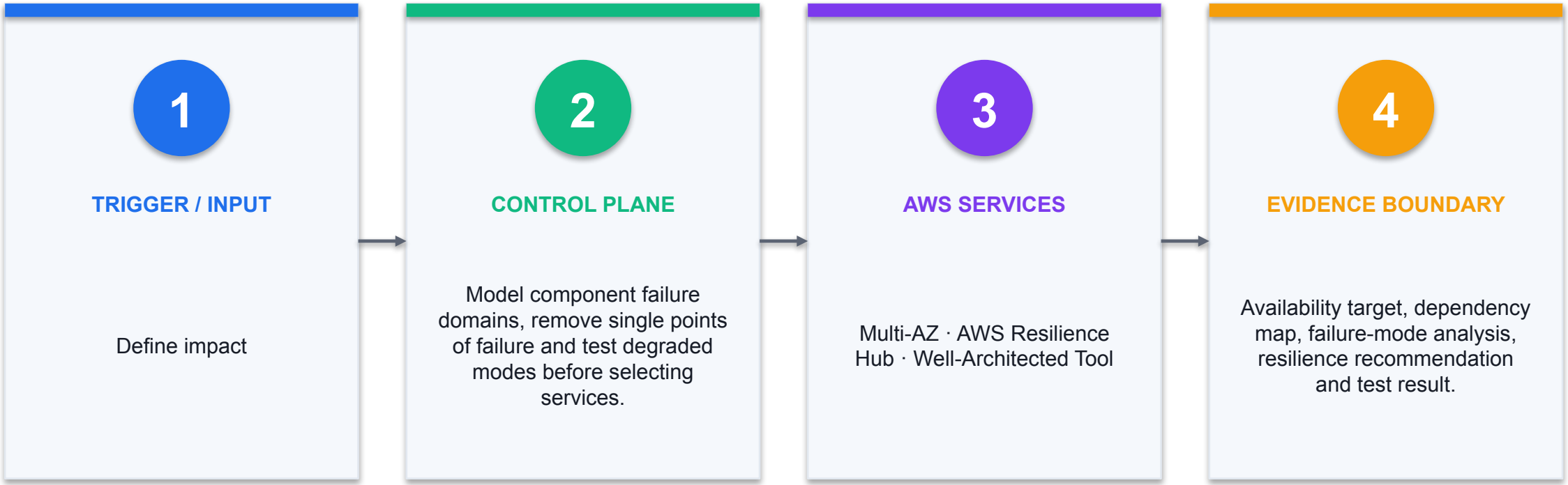
4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Recovery-point age, restore-test outcome, runbook duration, dependency validation and actual RTO/RPO.

© 2026 Tertiary Infotech Academy Pte Ltd

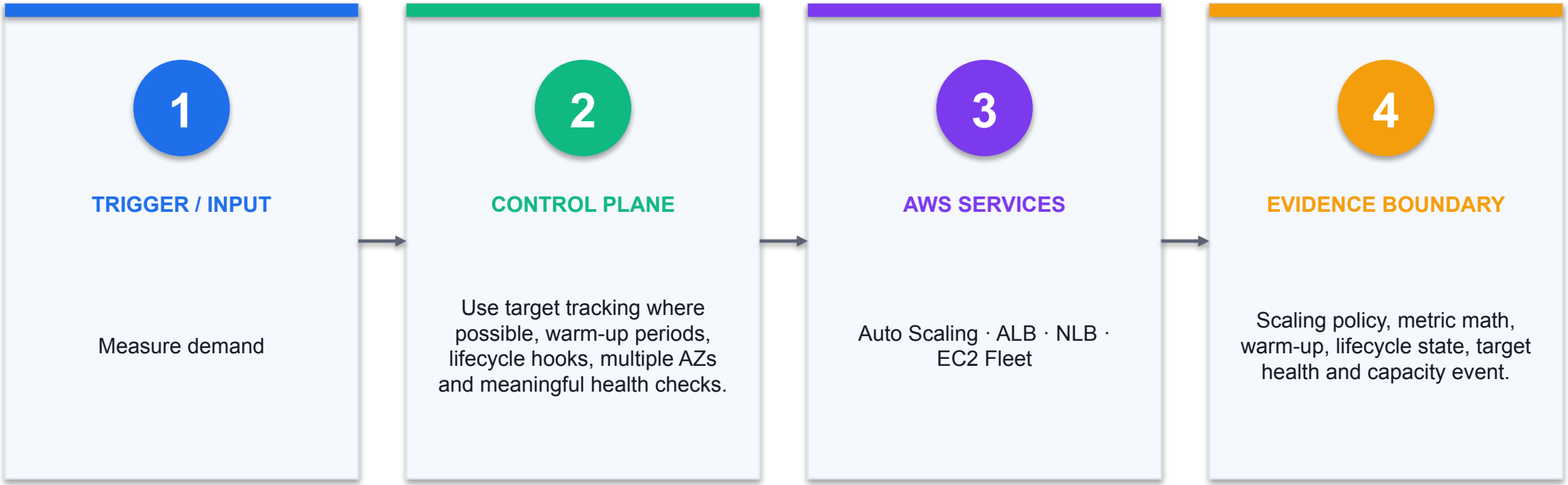
Availability requirements and failure modes: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

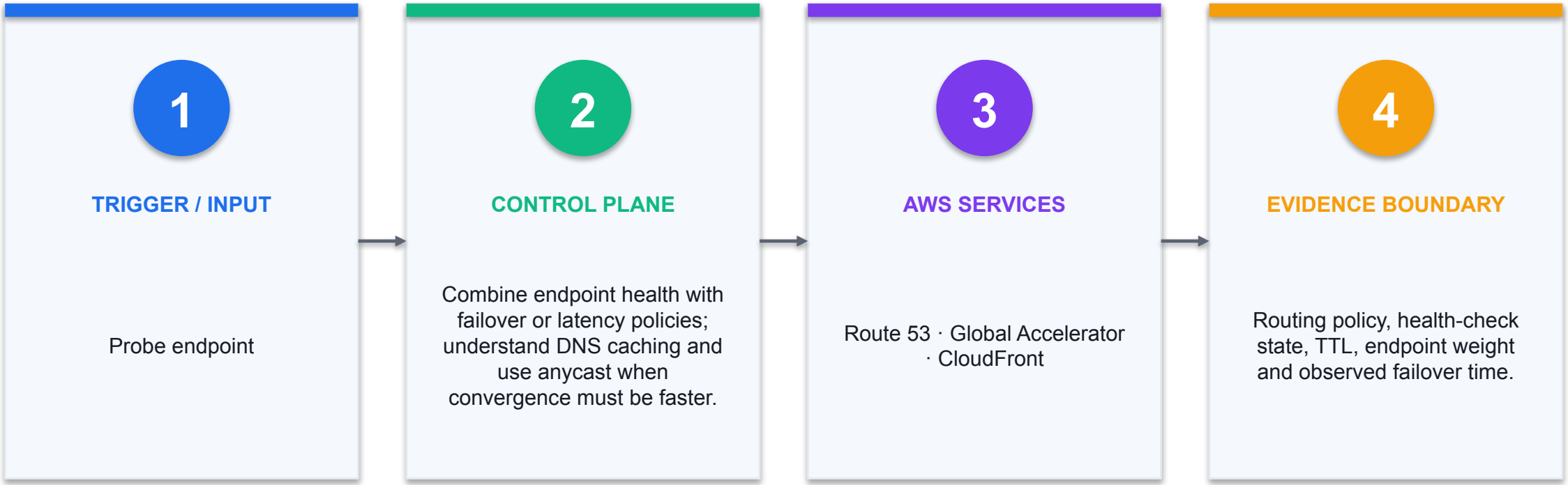
Elastic compute and load distribution: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

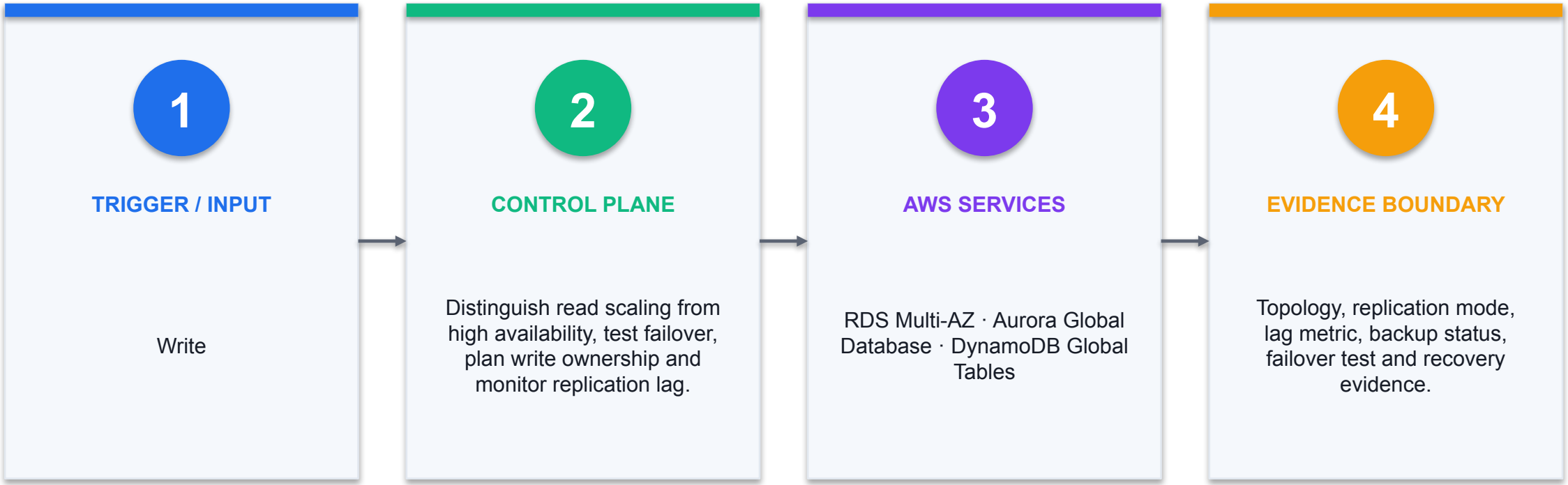
DNS and multi-Region traffic control: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

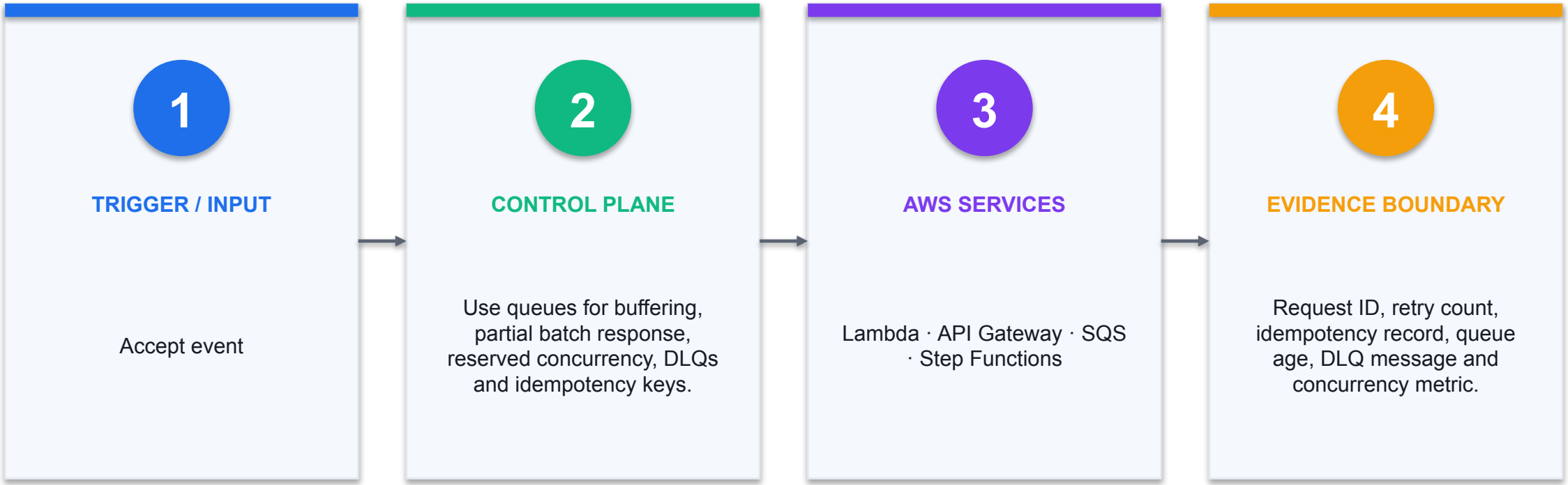
Resilient data stores: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

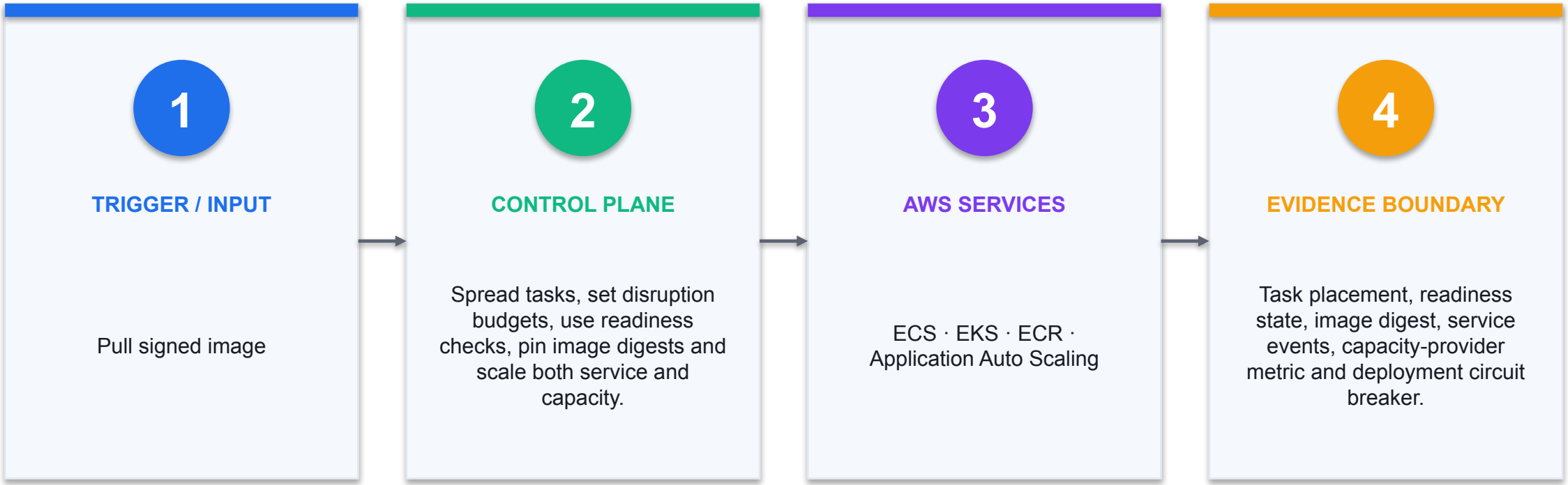
Serverless resilience: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

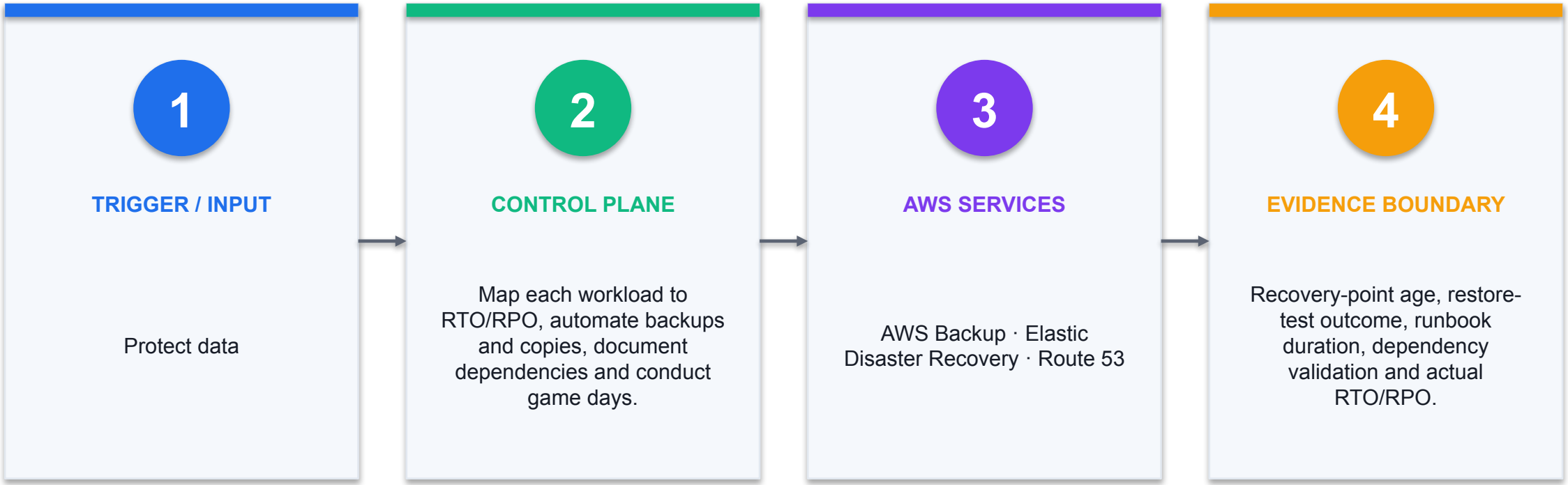
Container workload resilience: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Disaster recovery and automated recovery: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Deploy a Multi-AZ Application Load Balancer with an Auto Scaling Group

LAB 11

Day 3 · LO3 · K2, K4, A2

A single instance in a single Availability Zone is a single point of failure.

YOU'LL BUILD

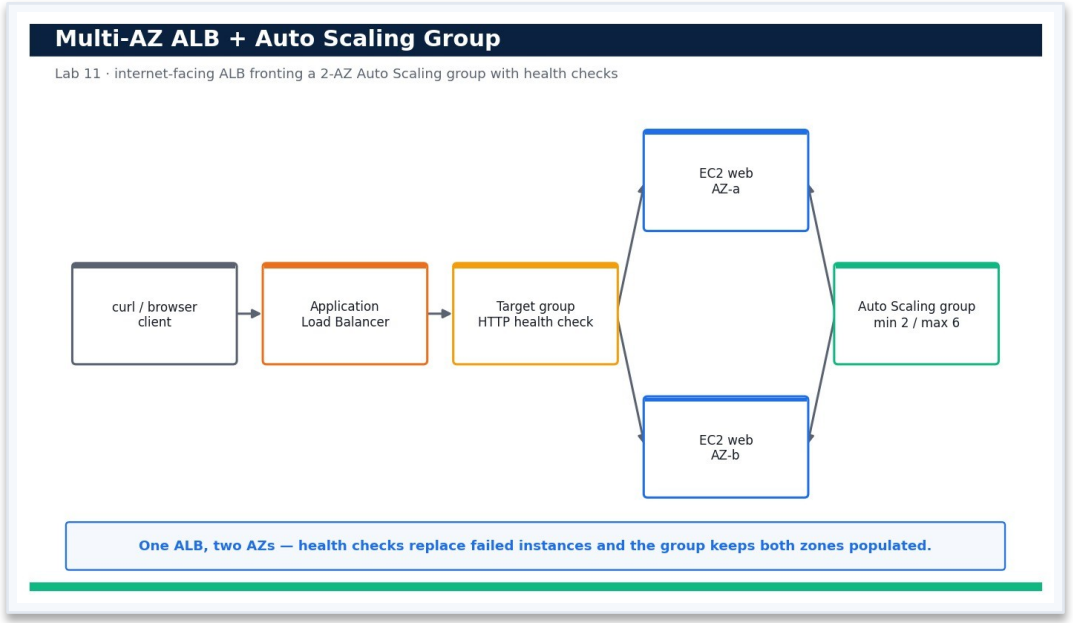
A running multi-AZ stack — an ALB, a target group.

Tools: Amazon EC2, Elastic Load Balancing (ELBv2 / ALB), AWS Auto Scaling, AWS CloudFormation

OUTCOME & EVIDENCE

Outcome: Deploy a highly available multi-AZ Application Load Balancer fronting an Auto Scaling group.

Evidence: describe-stacks output showing stack ha-web at CREATE_COMPLETE with AlbDnsName and AsgName in the Outputs.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Attach Target-Tracking Auto Scaling and Validate Scale-Out Under Load

LAB 12

Day 3 · LO3 · K2, K4, A2

A static two-instance web tier survives an AZ failure but not a traffic spike.

YOU'LL BUILD

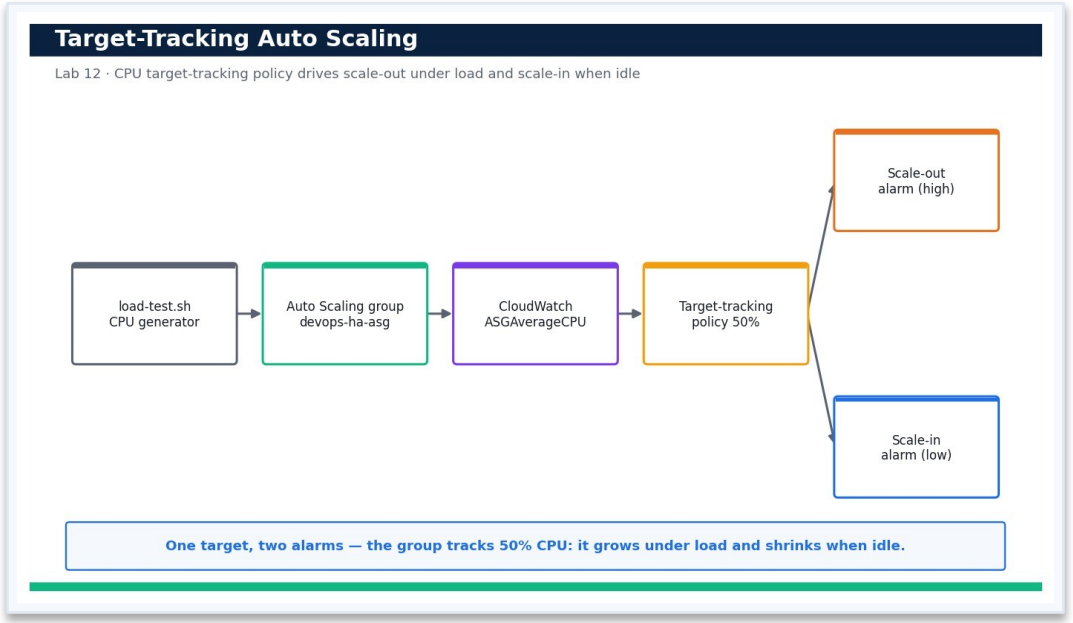
A target-tracking scaling policy (50% average CPU) on devops-ha-asg with its two auto-created CloudWatch alarms, plus a repeatable load test that drives observable scale-out and scale-in.

Tools: AWS Auto Scaling, Amazon CloudWatch, Amazon EC2

OUTCOME & EVIDENCE

Outcome: Attach a target-tracking scaling policy to an Auto Scaling group.

Evidence: describe-policies output showing cpu-target-tracking-50 attached with TargetValue 50.0.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Select a DR Strategy by RTO/RPO and Implement a Pilot-Light Skeleton

LAB 13

Day 3 · LO3 · K2, K4, A2

Not every workload needs active-active — and not every business can accept a day-long restore.

YOU'LL BUILD

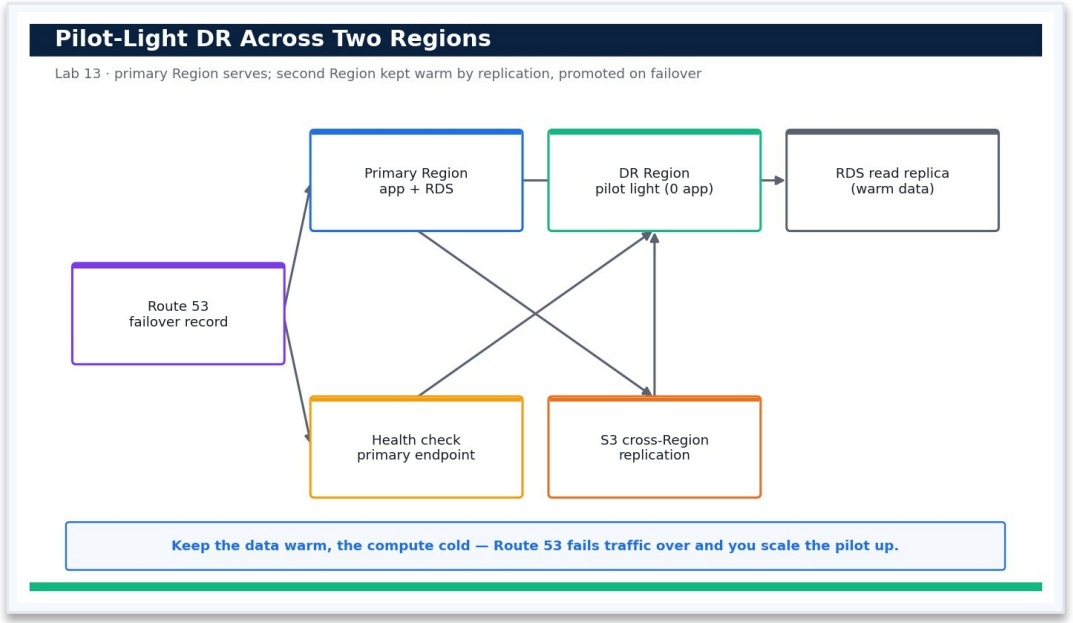
A completed DR selection worksheet plus a deployed pilot-light skeleton in a second Region (minimal VPC, a scaled-to-zero app tier.

Tools: Amazon Route 53, Amazon RDS, AWS CloudFormation, Amazon S3 (Cross-Region Replication)

OUTCOME & EVIDENCE

Outcome: Compare the four AWS disaster-recovery strategies against RTO/RPO targets, select an appropriate pattern.

Evidence: The completed dr-plan.md worksheet showing the workload RTO/RPO and the justified strategy choice.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Automate Backup, Restore and Data Protection with AWS Backup

LAB 14

Day 3 · LO3 · K2, K4, A2

Resilience is not only about staying up — it is about getting your data back.

YOU'LL BUILD

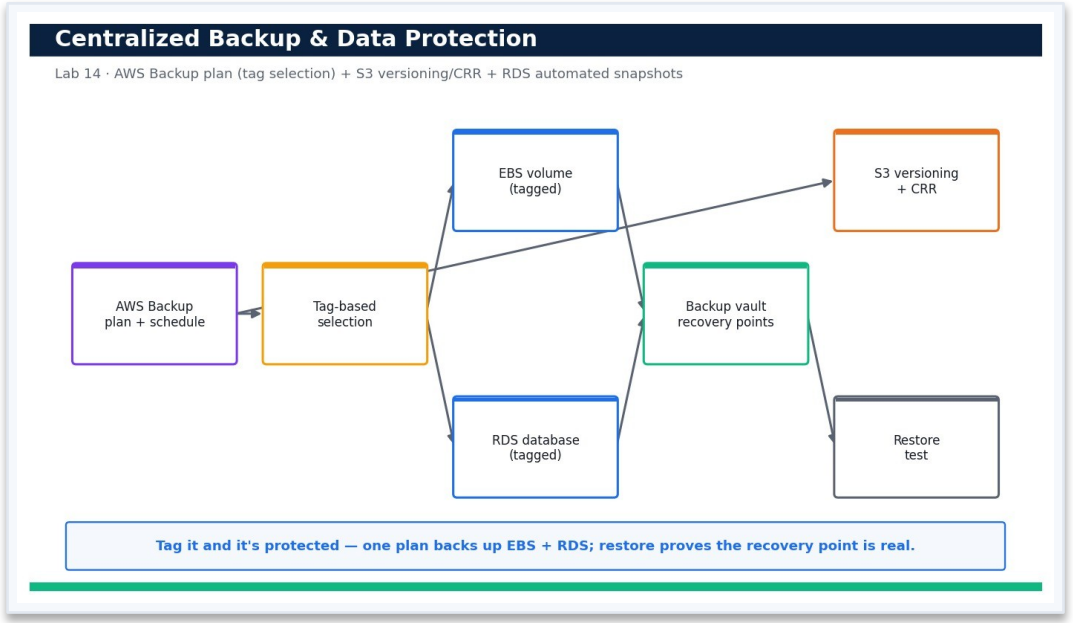
An AWS Backup plan with a tag-based selection protecting EBS and RDS, a verified restore from a recovery point, plus S3 versioning + cross-Region replication and RDS automated snapshots enabled.

Tools: AWS Backup, Amazon RDS, Amazon S3, Amazon EBS

OUTCOME & EVIDENCE

Outcome: Automate centralized backups with AWS Backup (plan + tag-based selection), test a restore.

Evidence: get-backup-plan output showing devops-daily-plan with its daily rule and 35-day lifecycle.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Inject Failure and Verify Self-Healing with AWS FIS

LAB 15

Day 3 · LO3 · K2, K4, A2

A resilient design is only proven when you break it on purpose.

YOU'LL BUILD

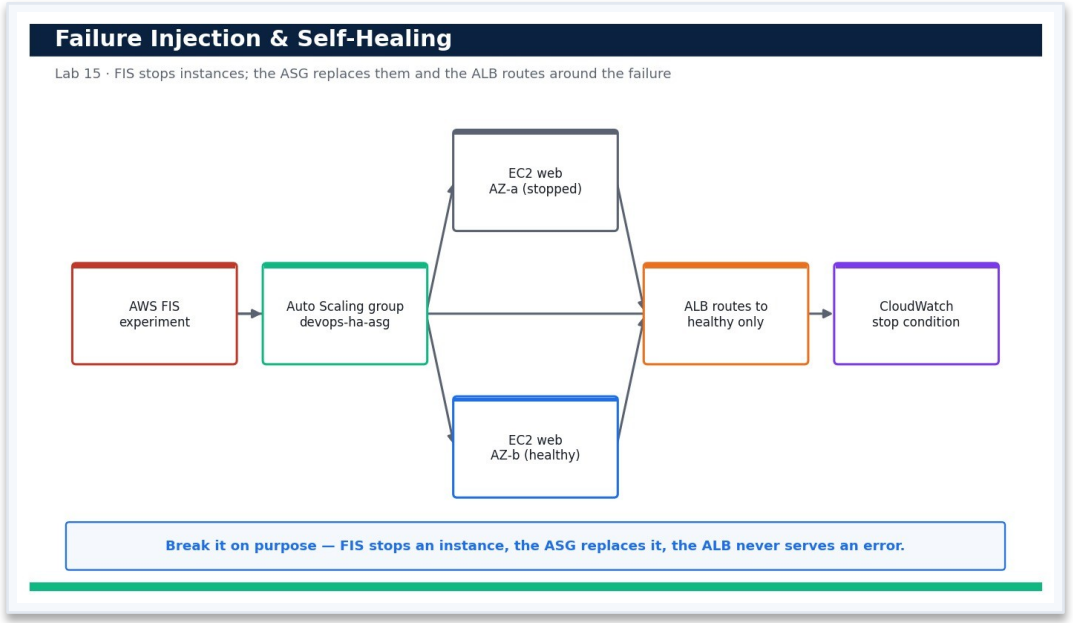
A verified self-healing web tier: a manual instance termination is auto-replaced by the Auto Scaling group, and an AWS FIS experiment stops instances while the ALB continues to serve traffic.

Tools: AWS Fault Injection Service (FIS), AWS Auto Scaling, Amazon EC2, Amazon CloudWatch

OUTCOME & EVIDENCE

Outcome: Prove the multi-AZ tier self-heals by terminating an instance.

Evidence: describe-scaling-activities output showing a successful 'Launching a new EC2 instance' after the manual termination.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN RECAP

Recap — Resilient Cloud Solutions

1

Lab 11

Deploy a highly available multi-AZ Application Load Balancer fronting an Auto Scaling group.

2

Lab 12

Attach a target-tracking scaling policy to an Auto Scaling group.

3

Lab 13

Compare the four AWS disaster-recovery strategies against RTO/RPO targets, select an appropriate pattern.

4

Lab 14

Automate centralized backups with AWS Backup (plan + tag-based selection), test a restore.

5

Lab 15

Prove the multi-AZ tier self-heals by terminating an instance.

6

Key principle

Spread compute across multiple Availability Zones behind an Elastic Load Balancer so one AZ failure is survivable.

© 2026 Tertiary Infotech Academy Pte Ltd

04

DOMAIN 04

Monitoring and Logging

Exam weighting 15% · Metrics · logs · dashboards · alarms · automated event management

© 2026 Tertiary Infotech Academy Pte Ltd

Monitoring and Logging



- Resilience and observability working together to keep services healthy

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 04 · WEIGHTING 15%

Key Concepts — Monitoring and Logging

1

CloudWatch metrics

Namespaces, dimensions and custom metrics measure system and application health over time.

2

CloudWatch Logs

Log groups, streams, subscription filters and metric filters turn raw logs into searchable, alertable signals.

3

Dashboards

A single pane of glass correlates metrics, logs and alarms for fast troubleshooting.

4

Alarms & composite alarms

Thresholds and anomaly detection notify SNS or trigger auto-remediation before users are affected.

5

Tracing & insights

X-Ray traces distributed requests; CloudWatch Logs Insights and Container/Lambda Insights find the root cause.

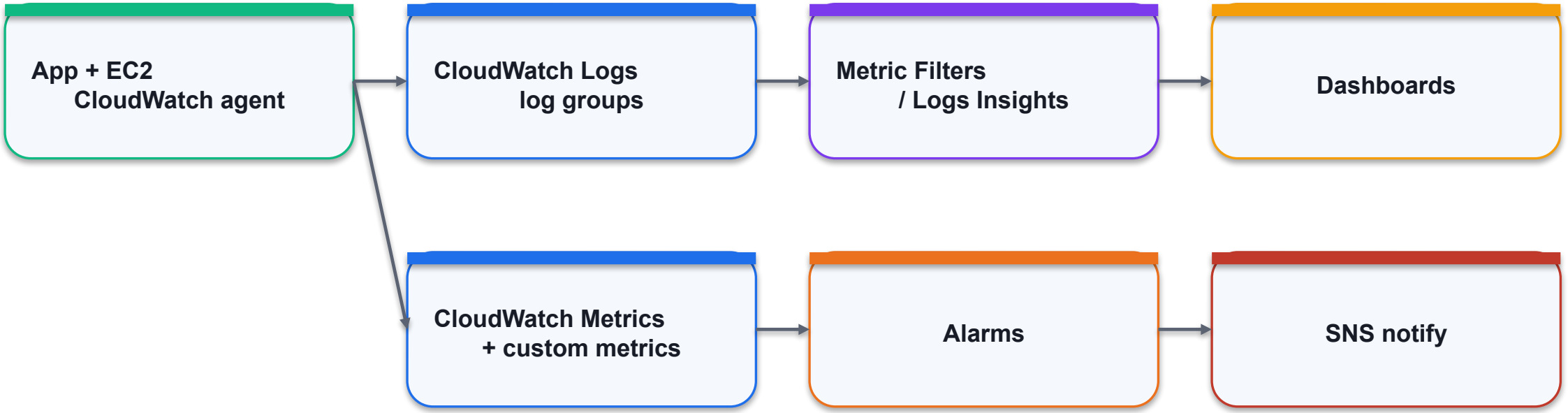
6

Aggregation at scale

Cross-account observability, the CloudWatch agent and OpenTelemetry centralise telemetry from many sources.

© 2026 Tertiary Infotech Academy Pte Ltd

Monitoring & Logging Pipeline



- Logs and metrics feed dashboards and alarms; alarms notify SNS and trigger the incident response in Domain 05.

© 2026 Tertiary Infotech Academy Pte Ltd

KNOW YOUR SIGNALS

The Three Pillars of Observability

1

Metrics

Numeric time-series — CPU, latency, error rate, request count.

2

Logs

Event records — application, system and access logs in CloudWatch Logs.

3

Traces

Request paths across services with AWS X-Ray.

4

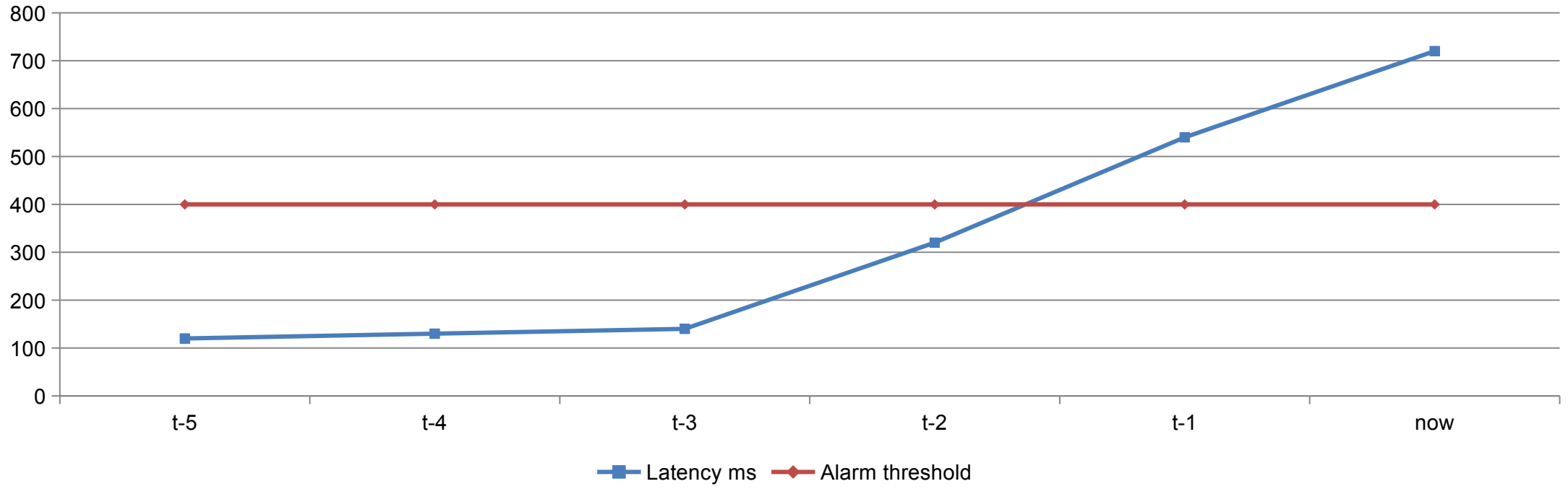
Dashboards & alarms

Correlate all three; alarm on anomalies before users notice.

© 2026 Tertiary Infotech Academy Pte Ltd

READING THE SIGNALS

Alarm Thresholds — Healthy vs Incident



WHAT THE DATA SHOWS

When p90 latency crosses the alarm threshold, CloudWatch fires and the automated response kicks in.

© 2026 Tertiary Infotech Academy Pte Ltd

THE METRICS MODEL

Anatomy of CloudWatch

1

Namespaces

Group metrics by service (AWS/EC2, custom app namespaces).

2

Dimensions

Name/value pairs that identify a metric (InstanceId, etc.).

3

Statistics & periods

Aggregate over time — Average, Sum, p90, p99.

4

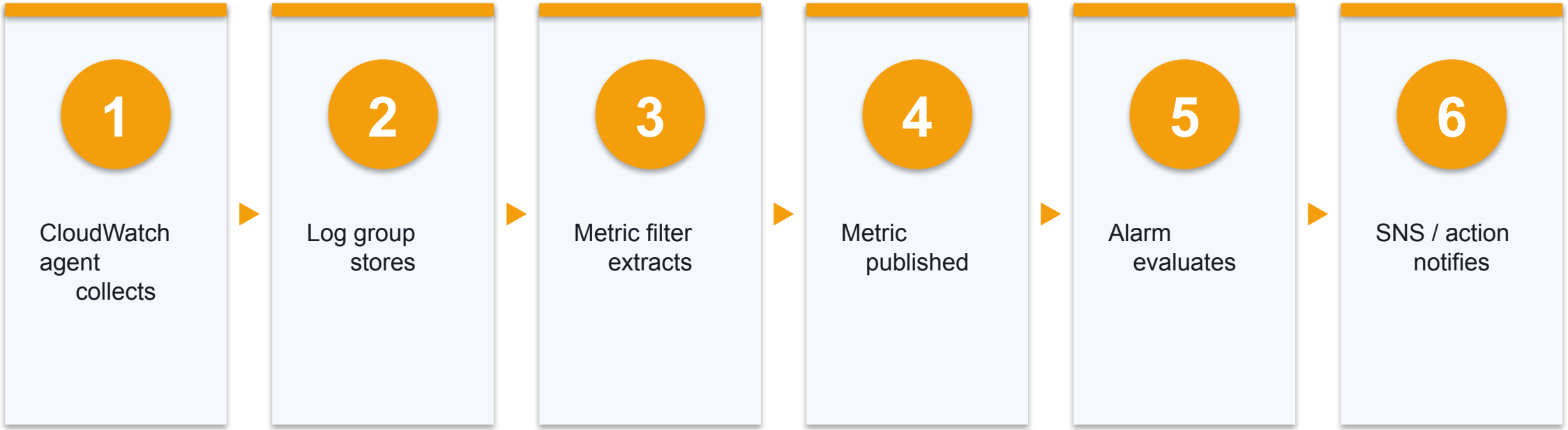
Alarms & composite

Trigger on thresholds or anomaly detection; combine with AND/OR.

© 2026 Tertiary Infotech Academy Pte Ltd

TURNING LOGS INTO SIGNALS

The Logs-to-Action Pipeline



© 2026 Tertiary Infotech Academy Pte Ltd

CloudWatch Insights Tools

Tool	What it shows
Logs Insights	Query and analyse log data with a purpose-built language
Container Insights	Metrics & logs for ECS/EKS/Fargate
Lambda Insights	Performance & cold-start metrics for functions
AWS X-Ray	Distributed traces across microservices

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

The three pillars — metrics, logs, traces

IN DEPTH

- Metrics are cheap numeric time-series for trends and alarms; logs are detailed event records for context; traces follow one request across services.
- CloudWatch provides metrics and logs; X-Ray provides distributed traces.
- You alarm on metrics, investigate with logs, and localise with traces.

WHY IT MATTERS

Each pillar answers a different question — 'is it broken?', 'why?', and 'where?' — and you need all three to troubleshoot fast.

FAILURE MODE

Logging everything but graphing nothing leaves you searching text during an outage instead of seeing the problem at a glance.

TRADE-OFF / DECISION

High-cardinality metrics and verbose logs cost money and can slow queries — sample and aggregate deliberately.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

CloudWatch metrics model — namespaces, dimensions, statistics

IN DEPTH

- Metrics live in namespaces (AWS/EC2, custom app namespaces) and are identified by dimensions such as InstanceId.
- Statistics (Average, Sum, p90, p99) and periods aggregate raw data points over time.
- Custom and high-resolution metrics capture application-level signals AWS can't see.

WHY IT MATTERS

Understanding dimensions and percentiles is what separates 'average looks fine' from seeing the p99 pain real users feel.

FAILURE MODE

Alarming on Average hides tail latency — the average is healthy while your slowest 1% of users are timing out.

TRADE-OFF / DECISION

Percentiles and high-resolution metrics cost more but reveal problems averages mask.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Turning logs into signals

IN DEPTH

- CloudWatch Logs stores log groups and streams; metric filters extract numbers (e.g. ERROR count) from log text.
- Subscription filters stream logs in real time to Lambda, OpenSearch or Kinesis for processing.
- Logs Insights queries large volumes with a purpose-built query language.

WHY IT MATTERS

A metric filter converts unsearchable text into an alarmable metric, so logs drive automated response, not just post-hoc reading.

FAILURE MODE

Relying on humans to eyeball logs means problems are found only after users complain.

TRADE-OFF / DECISION

Extracting many metrics from logs adds cost and complexity — extract the few that drive alarms and dashboards.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Alarms, composite alarms and anomaly detection

IN DEPTH

- Alarms evaluate a metric against a threshold over N periods and change state (OK/ALARM/INSUFFICIENT_DATA).
- Composite alarms combine several alarms with AND/OR to cut noise and express real conditions.
- Anomaly detection bands adapt to normal daily/weekly patterns instead of a fixed line.

WHY IT MATTERS

Well-designed alarms notify on real, actionable conditions and feed the automated incident response in Domain 5.

FAILURE MODE

Too many noisy single-metric alarms cause alert fatigue; responders learn to ignore the pager.

TRADE-OFF / DECISION

Anomaly detection reduces false alarms but needs a learning period and can miss slow, gradual regressions.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Dashboards and the single pane of glass

IN DEPTH

- Dashboards correlate metrics, logs and alarm state so responders see the whole system in one view.
- Service and cross-account dashboards aggregate telemetry from many sources.
- The best dashboards are organised around user journeys and the golden signals, not raw resource lists.

WHY IT MATTERS

During an incident a good dashboard turns minutes of hunting into seconds of seeing — it is a force multiplier for MTTR.

FAILURE MODE

A wall of 200 unlabelled graphs is as useless as none — no one can spot the anomaly under pressure.

TRADE-OFF / DECISION

Curated dashboards need maintenance as the system changes — stale dashboards mislead.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Distributed tracing with X-Ray

IN DEPTH

- X-Ray captures a trace per request and a service map of dependencies and latencies.
- Traces reveal which downstream call in a chain is slow or failing.
- Sampling keeps overhead and cost bounded on high-traffic services.

WHY IT MATTERS

In microservices, a symptom (slow checkout) and its cause (a slow third-party call three hops away) are far apart — tracing connects them.

FAILURE MODE

Without tracing, teams blame each other's services because no one can see the end-to-end request path.

TRADE-OFF / DECISION

Full tracing is expensive at scale — sampling trades completeness for affordable, representative visibility.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Observability at scale — agents, OpenTelemetry, cross-account

IN DEPTH

- The CloudWatch agent and the AWS Distro for OpenTelemetry collect host and application telemetry.
- Container Insights and Lambda Insights add curated metrics for ECS/EKS and functions.
- Cross-account observability centralises telemetry from many accounts into a monitoring account.

WHY IT MATTERS

A consistent collection standard across a large estate makes telemetry comparable and centrally queryable.

FAILURE MODE

Every team inventing its own metrics and log formats makes cross-service correlation impossible.

TRADE-OFF / DECISION

Centralisation eases correlation but concentrates cost and access — govern the monitoring account carefully.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

The golden signals and SLOs

IN DEPTH

- Latency, traffic, errors and saturation are the golden signals that describe almost any service's health.
- Service Level Objectives set target thresholds (e.g. 99.9% of requests < 300ms) and an error budget.
- Alarms and dashboards are built to protect the SLO, not to watch every raw resource metric.

WHY IT MATTERS

Tying monitoring to SLOs focuses attention on what users actually experience and gives an objective 'are we healthy?' answer.

FAILURE MODE

Alerting on CPU while the SLO quietly burns means you page on non-issues and miss the user-facing failure.

TRADE-OFF / DECISION

Strict SLOs drive reliability work but can slow feature delivery — the error budget is the negotiated balance.

© 2026 Tertiary Infotech Academy Pte Ltd

DEEPER TROUBLESHOOTING

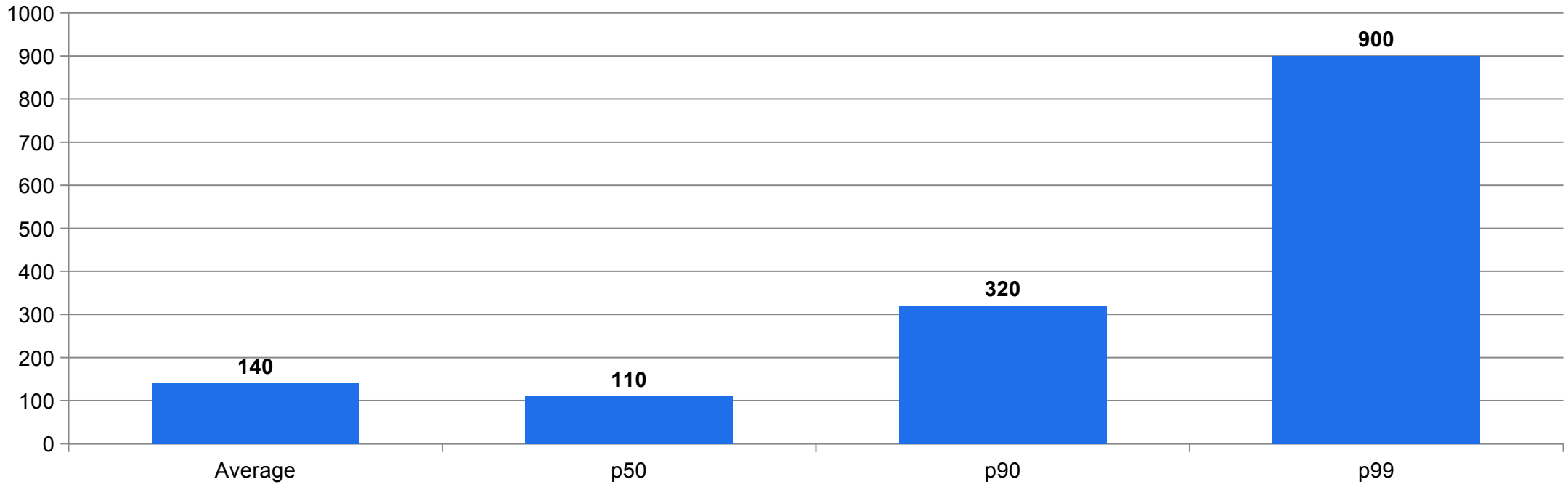
CloudWatch Insight Tools — Pick the Right Lens

Tool	What it shows	Reach for it when
Logs Insights	Ad-hoc queries over log data	You need the 'why' from raw events
Container Insights	Curated ECS/EKS/Fargate metrics	A container platform is slow
Lambda Insights	Function perf & cold starts	Serverless latency/cost issues
X-Ray	End-to-end request traces	Latency spans many services

© 2026 Tertiary Infotech Academy Pte Ltd

WHY p99 MATTERS

Average Hides the Tail — Latency Percentiles



WHAT THE DATA SHOWS

The average looks healthy at 140ms while the p99 user waits nearly a second — always alarm on the percentile your users feel.

© 2026 Tertiary Infotech Academy Pte Ltd

BLIND SPOTS THAT HURT

Monitoring Failure Modes & Anti-Patterns

1

Alarm on average

Tail latency hidden → alarm on p90/p99.

2

Alert fatigue

Noisy alarms ignored → composite alarms, SLO-based alerting.

3

Logs but no metrics

Manual searching in incidents → metric filters + dashboards.

4

No traces in microservices

Cross-team blame → adopt X-Ray/OTel.

5

Vanity dashboards

200 graphs, no signal → golden-signals layout.

6

Unmonitored dependencies

Third-party outage invisible → synthetic canaries.

© 2026 Tertiary Infotech Academy Pte Ltd

PROVEN PRACTICES

Observability Patterns Worth Adopting

1. **Golden-signals dashboards**
   Latency, traffic, errors, saturation per service.
2. **SLOs + error budgets**
   Objective health and a delivery/reliability balance.
3. **Structured logging**
   JSON logs with correlation IDs for Logs Insights.
4. **Synthetic canaries**
   Probe key journeys before real users hit failures.
5. **Metric filters → alarms**
   Turn log events into automated response.
6. **Centralised observability**
   Cross-account monitoring account.

© 2026 Tertiary Infotech Academy Pte Ltd

SIGNAL vs NOISE

Architecture Decision — What Should Page a Human?

A — Page immediately

SLO-burning, user-facing symptoms (error rate, latency breach). Actionable now — keep these few and trustworthy.

B — Ticket / review later

Trends and capacity signals (rising saturation). Important but not urgent — dashboards and weekly review.

C — Auto-remediate, don't page

Known, safe fixes (restart, scale, isolate). Let EventBridge + Lambda handle it and notify, not wake, the team.

© 2026 Tertiary Infotech Academy Pte Ltd

TRUST THE RIGHT SIGNAL

Scenario — 'Everything is green' but users complain

SITUATION

Dashboards show CPU and memory healthy, yet support reports slow checkouts.

ANALYSIS

- Resource metrics are averages and miss the user-facing symptom; the SLO is about request latency, not CPU.
- A p99 latency metric and a Logs Insights query on the checkout path would reveal the slow requests.
- An X-Ray service map would localise the slow downstream call.

DECISION

Re-orient alerting around the golden signals and the SLO (p99 latency, error rate), and add tracing so the next 'green but slow' is visible and localisable in seconds.

© 2026 Tertiary Infotech Academy Pte Ltd

REDUCING NOISE

Scenario — Taming an alert storm

SITUATION

A single database blip fires 40 alarms across dependent services and the on-call mutes the channel.

ANALYSIS

- Many single-metric alarms describe the same root cause, creating noise and fatigue.
- A composite alarm can express 'DB unhealthy AND app errors rising' and suppress the downstream chatter.
- SLO-based alerting pages on user impact, not on every correlated resource wobble.

DECISION

Consolidate with composite alarms and alert on SLO burn, so one meaningful page fires instead of forty — restoring trust in the pager.

© 2026 Tertiary Infotech Academy Pte Ltd

THE LOGS-TO-ACTION PATH

Worked Analysis — From Log Line to Alarm

Raw log: 'ERROR checkout timeout'	Unstructured text — invisible to metrics and alarms as-is.
Metric filter extracts AppErrorCount	A numeric metric now exists in a custom namespace.
Alarm: Sum > 5 in 1 minute	The metric becomes an actionable state change.
Alarm → SNS/EventBridge	Notifies responders and can trigger auto-remediation (Domain 5).

TAKEAWAY

Observability is a pipeline: capture → extract → alarm → act. Each stage turns raw data into faster recovery.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Metrics, logs and alarms

1

PURPOSE

Turn workload and deployment signals into actionable views, alarms and release decisions.

2

FEATURE 1

Metrics are time-series values with dimensions; high-cardinality dimensions can increase cost and fragmentation.

3

FEATURE 2

Logs supports structured ingestion, retention, subscriptions and Logs Insights queries.

4

FEATURE 3

Metric math, anomaly detection and composite alarms reduce noisy single-signal decisions.

© 2026 Tertiary Infotech Academy Pte Ltd

Metrics, logs and alarms: selection and controls

1

CHOOSE WHEN

Use metrics for numeric trends/alarms, logs for event detail and composite/SLO signals for user impact.

2

DEVOPS ENGINEERING

Standardise dimensions, correlate release IDs, set retention, treat missing data deliberately and test alarm actions.

3

FAILURE MODE

A low-level alarm without an owner, action or user-impact relationship creates noise rather than observability.

4

EVIDENCE

Dashboard, query, alarm history, notification, release annotation and operator response.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Tracing and service health

1

PURPOSE

Correlate a user request across services and identify latency, faults, errors and dependency bottlenecks.

2

FEATURE 1

Trace segments/subsegments represent service work and downstream calls.

3

FEATURE 2

Sampling limits cost while preserving representative traces; annotations support indexed filtering.

4

FEATURE 3

Application Signals adds service, operation and SLO-oriented telemetry for supported workloads.

© 2026 Tertiary Infotech Academy Pte Ltd

Tracing and service health: selection and controls

1

CHOOSE WHEN

Use distributed tracing when logs alone cannot explain cross-service latency or failure propagation.

2

DEVOPS ENGINEERING

Propagate trace context, avoid sensitive annotations, correlate trace/release IDs and define service-level objectives.

3

FAILURE MODE

Unsampled rare failures or missing context propagation create misleading service maps.

4

EVIDENCE

Trace ID, service map, fault/latency segment, correlated log, SLO burn and release identifier.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Telemetry analytics

1

PURPOSE

Search, retain and visualise operational telemetry using tools matched to log, metric and dashboard workloads.

2

FEATURE 1

OpenSearch indexes/searches documents and can visualise log/event data.

3

FEATURE 2

Managed Prometheus provides PromQL-compatible metric ingestion/query for container ecosystems.

4

FEATURE 3

Managed Grafana federates dashboards across approved data sources.

© 2026 Tertiary Infotech Academy Pte Ltd

Telemetry analytics: selection and controls

1

CHOOSE WHEN

Use native CloudWatch first for AWS-integrated operations; add OpenSearch or Prometheus/Grafana when search/query or ecosystem requirements justify them.

2

DEVOPS ENGINEERING

Control tenancy, encryption, retention, shard/cardinality growth and dashboard access through federation.

3

FAILURE MODE

Unbounded log indexing or metric labels can create cost and performance failures.

4

EVIDENCE

Index policy, ingestion lag, query latency, series count, dashboard ownership and access log.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Streaming and data engineering signals

1

PURPOSE

Move high-volume operational events into durable streams, delivery destinations and queryable evidence stores.

2

FEATURE 1

Kinesis Data Streams partitions records by key and exposes shard/consumer lag and retention controls.

3

FEATURE 2

Firehose buffers, optionally transforms and delivers data to destinations such as S3/OpenSearch.

4

FEATURE 3

Glue Catalog describes datasets; Athena queries data in S3 and benefits from partitioning and columnar formats.

© 2026 Tertiary Infotech Academy Pte Ltd

Streaming and data engineering signals: selection and controls

1

CHOOSE WHEN

Use Streams for custom low-latency consumers/replay, Firehose for managed delivery, and S3/Glue/Athena for economical historical analysis.

2

DEVOPS ENGINEERING

Choose partition keys carefully, monitor iterator age, design replay/idempotency, encrypt records and control schema evolution.

3

FAILURE MODE

A hot partition throttles one shard even when the stream has unused capacity elsewhere.

4

EVIDENCE

Put-record result, shard distribution, iterator age, delivery error, catalog version, query and retention policy.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: CloudWatch metrics, logs and alarm engineering

1

PURPOSE

Create actionable observability from service, application and business signals instead of accumulating unowned telemetry.

2

SERVICE BOUNDARY

CloudWatch Metrics · Logs · alarms · Logs Insights

3

PRIMARY DESIGN DECISION

Emit transaction success/failure and latency, annotate releases, alarm on error-rate/SLO burn and use a composite signal for rollback.

4

FAILURE BOUNDARY

CPU is not a user outcome; unbounded tenant IDs as dimensions can create excessive custom time series.

© 2026 Tertiary Infotech Academy Pte Ltd

CloudWatch metrics, logs and alarm engineering: how it works

1

MECHANIC 1

Metrics combine namespace, name, dimensions, timestamp and value; every new dimension combination creates a distinct time series with cost and cardinality implications.

2

MECHANIC 2

Embedded Metric Format extracts metrics from structured logs, allowing a single event to retain detail and contribute numeric signals.

3

MECHANIC 3

Log groups define retention and encryption; subscription filters stream matching events to supported destinations for central analysis.

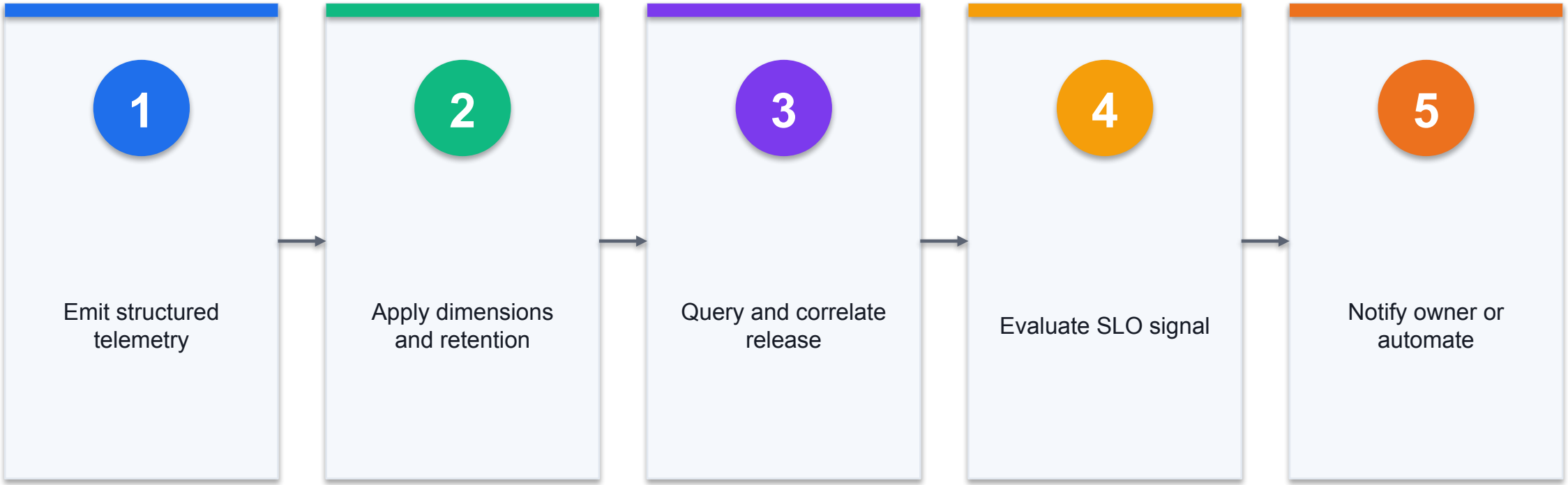
4

MECHANIC 4

Metric math, anomaly detection and composite alarms combine signals, while missing-data treatment changes whether absence is healthy, breaching or unknown.

© 2026 Tertiary Infotech Academy Pte Ltd

CloudWatch metrics, logs and alarm engineering: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

CloudWatch metrics, logs and alarm engineering: configuration decisions

1

SIGNAL

Prefer request, error, duration and saturation indicators plus business outcomes over infrastructure-only dashboards.

2

ALARM

Define owner, severity, action, evaluation window and recovery behavior; test alarm transitions deliberately.

3

LOGS

Use structured fields and correlation IDs, redact secrets and set retention from operational/legal need.

4

COST

Control custom metric cardinality, log volume, ingestion class and query scan range.

© 2026 Tertiary Infotech Academy Pte Ltd

CloudWatch metrics, logs and alarm engineering: worked architecture decision

1

SCENARIO

A deployment increases checkout failures without raising CPU, so the existing fleet alarm remains green.

2

DESIGN DECISION

Emit transaction success/failure and latency, annotate releases, alarm on error-rate/SLO burn and use a composite signal for rollback.

3

FAILURE TO AVOID

CPU is not a user outcome; unbounded tenant IDs as dimensions can create excessive custom time series.

4

ACCEPTANCE EVIDENCE

Metric schema, sample structured log, dashboard, query, alarm history, release annotation and operator action.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: Distributed tracing with X-Ray and OpenTelemetry

1

PURPOSE

Follow requests across service boundaries and connect latency or faults to dependency, release and log evidence.

2

SERVICE BOUNDARY

AWS X-Ray · ADOT · Application Signals · ServiceLens

3

PRIMARY DESIGN DECISION

Propagate trace context, instrument SDK/dependency calls, retain message correlation and compare queue delay with processing duration.

4

FAILURE BOUNDARY

A trace covering only the first Lambda misattributes queue time and hides downstream throttling.

© 2026 Tertiary Infotech Academy Pte Ltd

Distributed tracing with X-Ray and OpenTelemetry: how it works

1

MECHANIC 1

Trace context propagates a trace identifier and sampling decision; each service contributes spans or X-Ray segments/subsegments describing timed work.

2

MECHANIC 2

Sampling reduces telemetry volume and must preserve enough rare or high-value failures for diagnosis; head and tail sampling have different visibility trade-offs.

3

MECHANIC 3

Annotations are indexed fields for trace filtering, while metadata stores unindexed context; neither should contain secrets or regulated payloads.

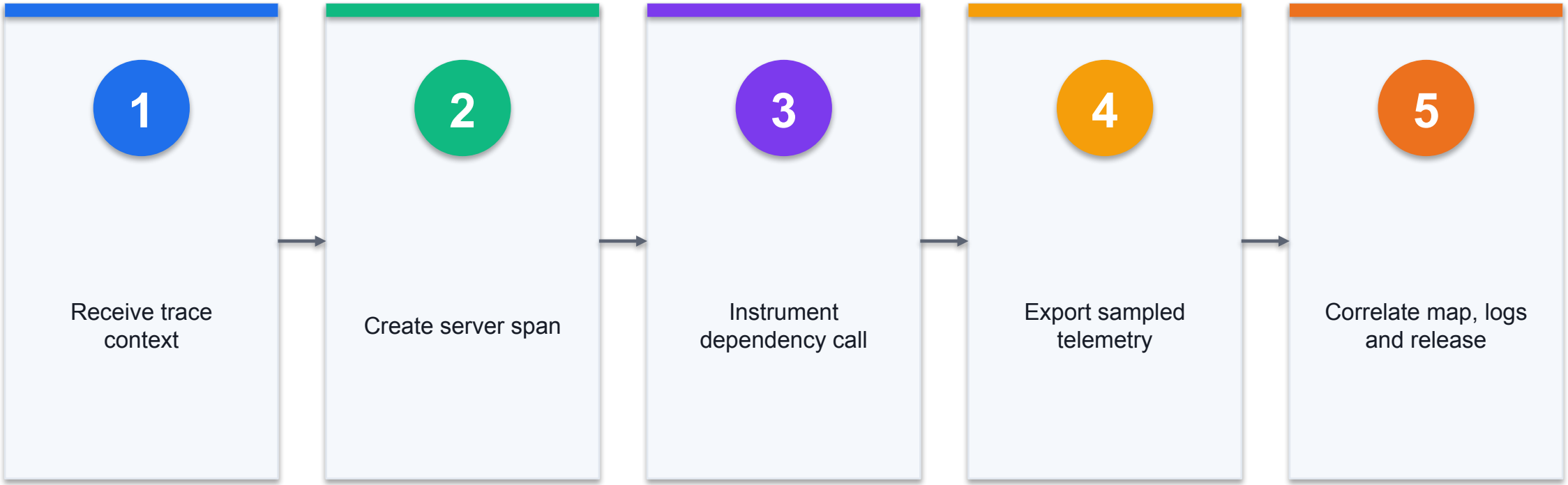
4

MECHANIC 4

AWS Distro for OpenTelemetry collects standard traces and metrics and can export to supported AWS or third-party backends.

© 2026 Tertiary Infotech Academy Pte Ltd

Distributed tracing with X-Ray and OpenTelemetry: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Distributed tracing with X-Ray and OpenTelemetry: configuration decisions

1

PROPAGATION

Instrument every protocol boundary or explicitly bridge trace headers; one missing hop fragments the service map.

2

SAMPLING

Set rules by service/path/value and monitor cost; preserve critical errors through appropriate sampling strategy.

3

ASYNC WORK

Link producer and consumer context without pretending queued time is synchronous processing.

4

PRIVACY

Record identifiers needed for correlation but exclude credentials, tokens and sensitive payloads.

© 2026 Tertiary Infotech Academy Pte Ltd

Distributed tracing with X-Ray and OpenTelemetry: worked architecture decision

1

SCENARIO

A serverless request is fast at the API edge but intermittently slow after two asynchronous downstream steps.

2

DESIGN DECISION

Propagate trace context, instrument SDK/dependency calls, retain message correlation and compare queue delay with processing duration.

3

FAILURE TO AVOID

A trace covering only the first Lambda misattributes queue time and hides downstream throttling.

4

ACCEPTANCE EVIDENCE

Trace ID, sampled spans, service map, fault/latency segment, correlated log/request ID and deployment version.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: Kinesis, Firehose and queryable event lakes

1

PURPOSE

Ingest, replay, deliver and query high-volume operational events while controlling partition balance, lag and schema evolution.

2

SERVICE BOUNDARY

Kinesis Data Streams · Firehose · Glue · Athena

3

PRIMARY DESIGN DECISION

Redesign the partition key with controlled distribution, preserve customer queryability in record fields and scale consumers from measured lag.

4

FAILURE BOUNDARY

Adding shards does not fix an unchanged hot key; retrying failed records without checkpoints can duplicate downstream work.

© 2026 Tertiary Infotech Academy Pte Ltd

Kinesis, Firehose and queryable event lakes: how it works

1

MECHANIC 1

Data Streams assigns records to shards from the partition key; consumer lag and hot shards are therefore functions of key distribution and per-shard throughput.

2

MECHANIC 2

Enhanced fan-out gives registered consumers dedicated read throughput, while shared polling consumers compete within shard limits.

3

MECHANIC 3

Firehose buffers by size/time, can transform records and delivers to destinations with backup/error prefixes for failed data.

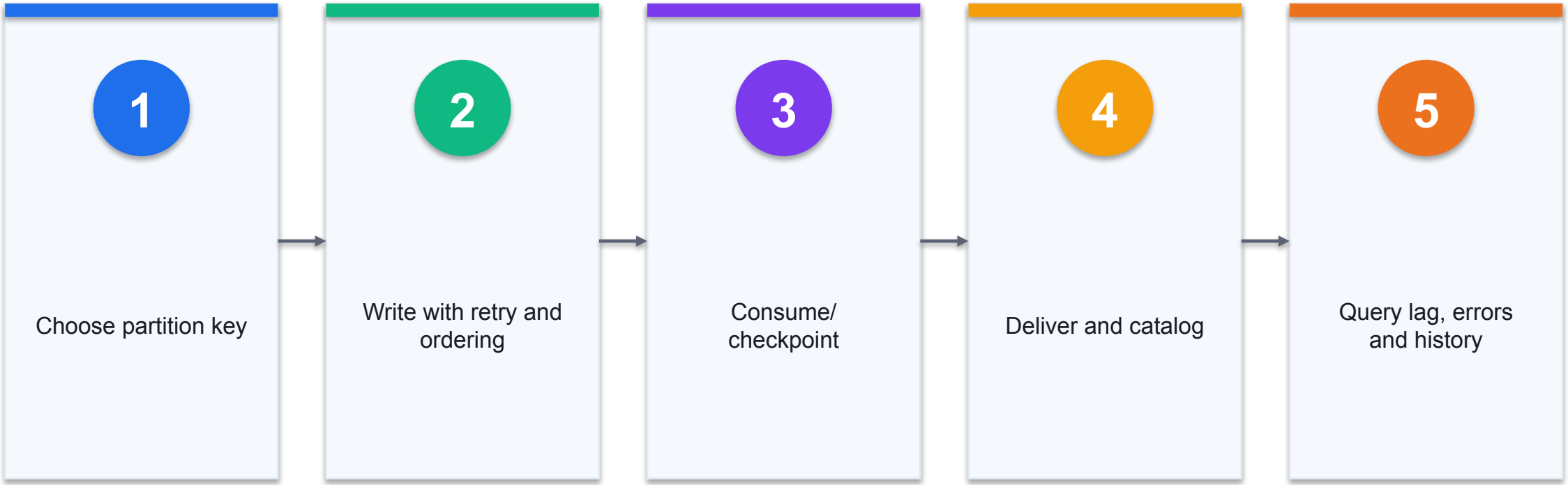
4

MECHANIC 4

S3 partitioning, Glue catalog metadata and columnar formats such as Parquet reduce Athena scan volume and make historical evidence economical.

© 2026 Tertiary Infotech Academy Pte Ltd

Kinesis, Firehose and queryable event lakes: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Kinesis, Firehose and queryable event lakes: configuration decisions

1

STREAMS

Choose when custom consumers, replay, ordering within a shard or low-latency processing are required.

2

FIREHOSE

Choose for managed buffered delivery where application-controlled replay/consumer logic is not required.

3

SCHEMA

Version records compatibly and quarantine malformed events instead of blocking an entire consumer shard.

4

SECURITY

Encrypt in transit/at rest, scope producer/consumer roles and protect destination prefixes/catalog access.

© 2026 Tertiary Infotech Academy Pte Ltd

Kinesis, Firehose and queryable event lakes: worked architecture decision

1

SCENARIO

One customer ID dominates the stream and causes high iterator age while other shards remain idle.

2

DESIGN DECISION

Redesign the partition key with controlled distribution, preserve customer queryability in record fields and scale consumers from measured lag.

3

FAILURE TO AVOID

Adding shards does not fix an unchanged hot key; retrying failed records without checkpoints can duplicate downstream work.

4

ACCEPTANCE EVIDENCE

Put-record responses, per-shard throughput, iterator age, checkpoint, delivery errors, catalog version and Athena query result.

© 2026 Tertiary Infotech Academy Pte Ltd

Metrics, dimensions and service objectives

1

OUTCOME

Measure user outcomes and resource behavior with metrics that support decisions.

2

ARCHITECTURE

Define SLIs and thresholds, use dimensions carefully, aggregate intentionally and avoid high-cardinality custom metrics.

3

FAILURE MODE

Averages can hide tail latency and a small population of severe failures.

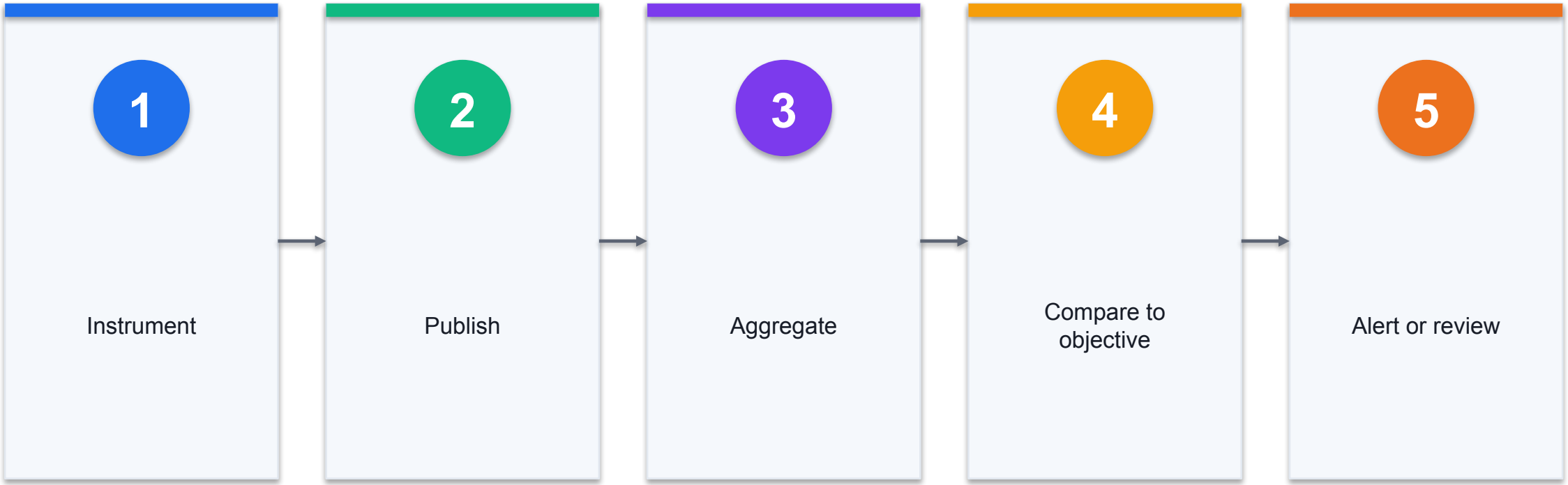
4

EVIDENCE

Metric namespace, dimensions, statistic, period, percentile, SLO and alarm history.

© 2026 Tertiary Infotech Academy Pte Ltd

Metrics, dimensions and service objectives: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Metrics, dimensions and service objectives: design decision

? SCENARIO

An API meets average-latency targets but its slowest five percent of requests breach the user promise.

1 DESIGN RESPONSE

Define SLIs and thresholds, use dimensions carefully, aggregate intentionally and avoid high-cardinality custom metrics.

2 PRIMARY SERVICES

CloudWatch Metrics · Metric Math · Application Signals

3 FAILURE MODE

Averages can hide tail latency and a small population of severe failures.

4 PROOF

Metric namespace, dimensions, statistic, period, percentile, SLO and alarm history.

© 2026 Tertiary Infotech Academy Pte Ltd

Metrics, dimensions and service objectives: failure diagnostic

1

RECOGNISE

Observable failure condition: Averages can hide tail latency and a small population of severe failures.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Instrument through Alert or review; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Metric namespace, dimensions, statistic, period, percentile, SLO and alarm history.

© 2026 Tertiary Infotech Academy Pte Ltd

Metrics, dimensions and service objectives: worked decision

1

CONSTRAINT

An API meets average-latency targets but its slowest five percent of requests breach the user promise.

2

DECISION

Define SLIs and thresholds, use dimensions carefully, aggregate intentionally and avoid high-cardinality custom metrics.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Averages can hide tail latency and a small population of severe failures.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Metric namespace, dimensions, statistic, period, percentile, SLO and alarm history.

© 2026 Tertiary Infotech Academy Pte Ltd

Centralized log architecture

1

OUTCOME

Collect searchable logs across accounts and Regions while meeting retention and access requirements.

2

ARCHITECTURE

Standardize structure, encrypt log groups, centralize subscriptions, partition archives and prevent recursive delivery.

3

FAILURE MODE

Centralization without account/Region context makes incident correlation harder.

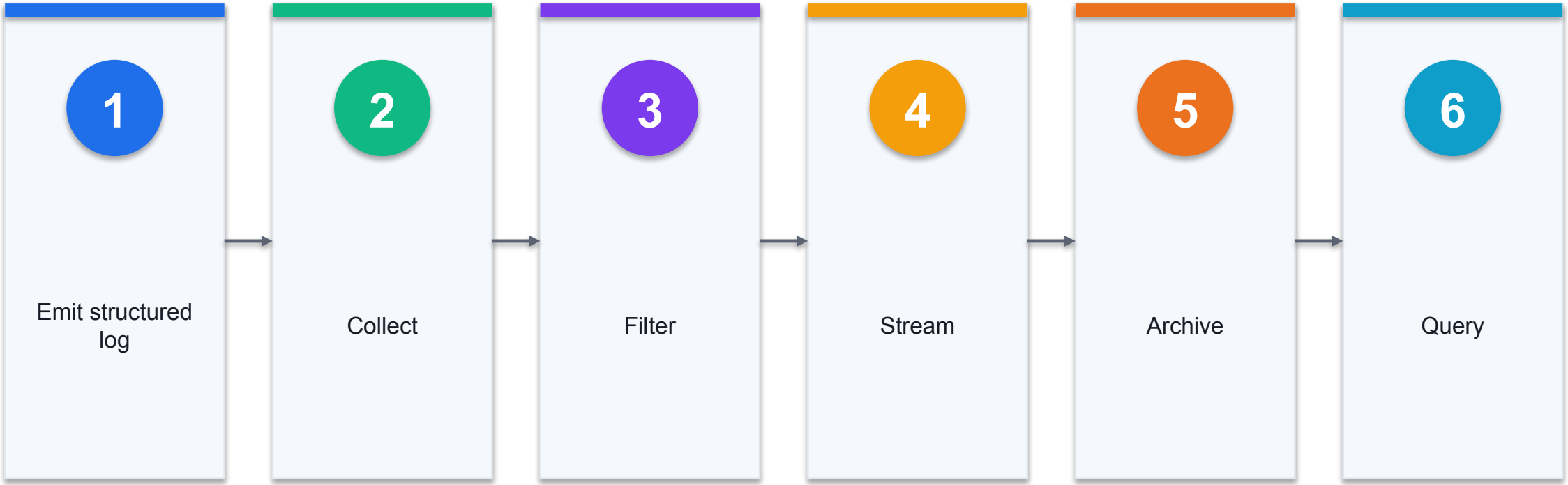
4

EVIDENCE

Log-group policy, subscription filter, destination policy, archive partition and retention setting.

© 2026 Tertiary Infotech Academy Pte Ltd

Centralized log architecture: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Centralized log architecture: design decision

? SCENARIO

Security operations needs a central account for application and audit logs from every workload account.

1 DESIGN RESPONSE

Standardize structure, encrypt log groups, centralize subscriptions, partition archives and prevent recursive delivery.

2 PRIMARY SERVICES

CloudWatch Logs · Kinesis Data Firehose · S3 · OpenSearch

3 FAILURE MODE

Centralization without account/Region context makes incident correlation harder.

4 PROOF

Log-group policy, subscription filter, destination policy, archive partition and retention setting.

© 2026 Tertiary Infotech Academy Pte Ltd

Centralized log architecture: failure diagnostic

1

RECOGNISE

Observable failure condition: Centralization without account/Region context makes incident correlation harder.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Emit structured log through Query; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Log-group policy, subscription filter, destination policy, archive partition and retention setting.

© 2026 Tertiary Infotech Academy Pte Ltd

Centralized log architecture: worked decision

1

CONSTRAINT

Security operations needs a central account for application and audit logs from every workload account.

2

DECISION

Standardize structure, encrypt log groups, centralize subscriptions, partition archives and prevent recursive delivery.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Centralization without account/Region context makes incident correlation harder.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Log-group policy, subscription filter, destination policy, archive partition and retention setting.

© 2026 Tertiary Infotech Academy Pte Ltd

Agents and custom telemetry

1

OUTCOME

Capture operating-system, application and business signals not emitted by managed services.

2

ARCHITECTURE

Deploy versioned agent configuration, use least privilege, buffer safely and validate time synchronization.

3

FAILURE MODE

A broken agent rollout can remove visibility exactly when a change is being evaluated.

4

EVIDENCE

Agent config version, collection status, dropped records, IAM policy and sample telemetry.

© 2026 Tertiary Infotech Academy Pte Ltd

Agents and custom telemetry: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Agents and custom telemetry: design decision

? SCENARIO

A hybrid fleet must publish memory utilization, application latency and deployment version consistently.

1 DESIGN RESPONSE

Deploy versioned agent configuration, use least privilege, buffer safely and validate time synchronization.

2 PRIMARY SERVICES

CloudWatch Agent · OpenTelemetry Collector · Embedded Metric Format

3 FAILURE MODE

A broken agent rollout can remove visibility exactly when a change is being evaluated.

4 PROOF

Agent config version, collection status, dropped records, IAM policy and sample telemetry.

© 2026 Tertiary Infotech Academy Pte Ltd

Agents and custom telemetry: failure diagnostic

1

RECOGNISE

Observable failure condition: A broken agent rollout can remove visibility exactly when a change is being evaluated.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Configure receiver through Verify; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Agent config version, collection status, dropped records, IAM policy and sample telemetry.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 4 · APPLIED SCENARIO

Agents and custom telemetry: worked decision

1

CONSTRAINT

A hybrid fleet must publish memory utilization, application latency and deployment version consistently.

2

DECISION

Deploy versioned agent configuration, use least privilege, buffer safely and validate time synchronization.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: A broken agent rollout can remove visibility exactly when a change is being evaluated.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Agent config version, collection status, dropped records, IAM policy and sample telemetry.

© 2026 Tertiary Infotech Academy Pte Ltd

Alarms, anomaly detection and composite signals

1

OUTCOME

Trigger action only when evidence is meaningful enough to justify interruption or automation.

2

ARCHITECTURE

Tune evaluation periods and missing-data behavior; combine symptoms to reduce noise and attach runbook context.

3

FAILURE MODE

One noisy metric can cause alert fatigue or repeated automated recovery.

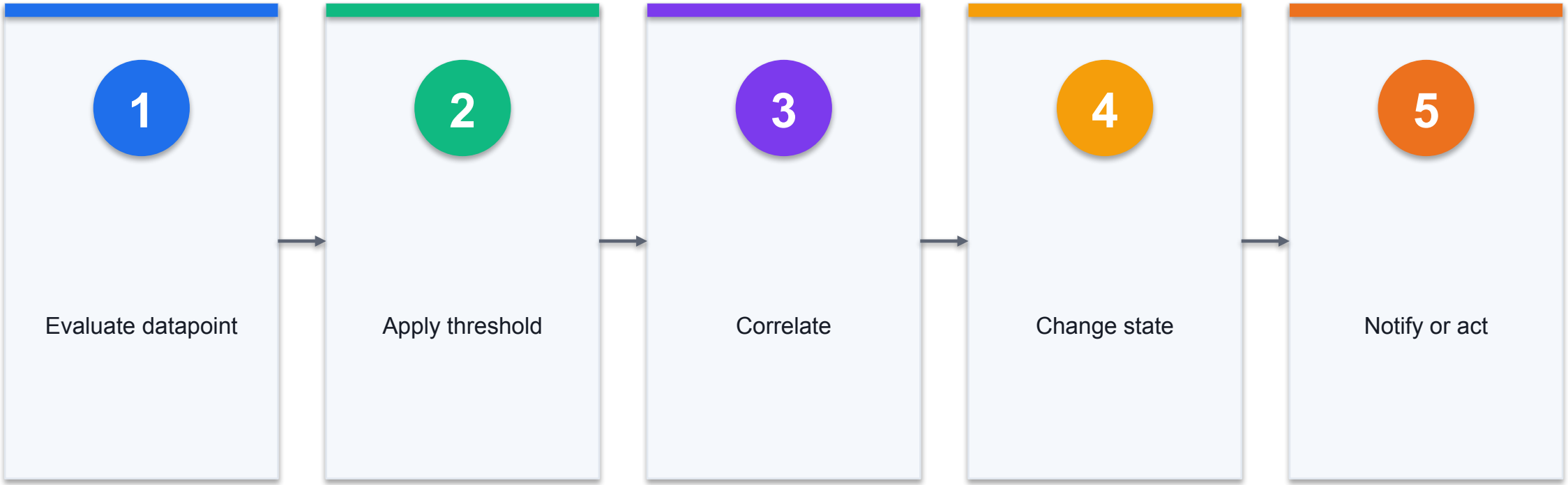
4

EVIDENCE

Alarm configuration, state transition, contributing alarms, notification payload and operator action.

© 2026 Tertiary Infotech Academy Pte Ltd

Alarms, anomaly detection and composite signals: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Alarms, anomaly detection and composite signals: design decision

? SCENARIO

Operations wants a page only when latency is high and the error rate indicates customer impact.

1 DESIGN RESPONSE

Tune evaluation periods and missing-data behavior; combine symptoms to reduce noise and attach runbook context.

2 PRIMARY SERVICES

CloudWatch Alarms · Composite Alarms · SNS

3 FAILURE MODE

One noisy metric can cause alert fatigue or repeated automated recovery.

4 PROOF

Alarm configuration, state transition, contributing alarms, notification payload and operator action.

© 2026 Tertiary Infotech Academy Pte Ltd

Alarms, anomaly detection and composite signals: failure diagnostic

1

RECOGNISE

Observable failure condition: One noisy metric can cause alert fatigue or repeated automated recovery.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Evaluate datapoint through Notify or act; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Alarm configuration, state transition, contributing alarms, notification payload and operator action.

© 2026 Tertiary Infotech Academy Pte Ltd

Alarms, anomaly detection and composite signals: worked decision

1

CONSTRAINT

Operations wants a page only when latency is high and the error rate indicates customer impact.

2

DECISION

Tune evaluation periods and missing-data behavior; combine symptoms to reduce noise and attach runbook context.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: One noisy metric can cause alert fatigue or repeated automated recovery.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Alarm configuration, state transition, contributing alarms, notification payload and operator action.

© 2026 Tertiary Infotech Academy Pte Ltd

Distributed tracing and service maps

1

OUTCOME

Trace a request across services to locate latency, faults and dependency behavior.

2

ARCHITECTURE

Propagate trace context, sample intentionally, protect sensitive attributes and correlate traces with logs and metrics.

3

FAILURE MODE

Missing context propagation creates disconnected traces and incorrect service maps.

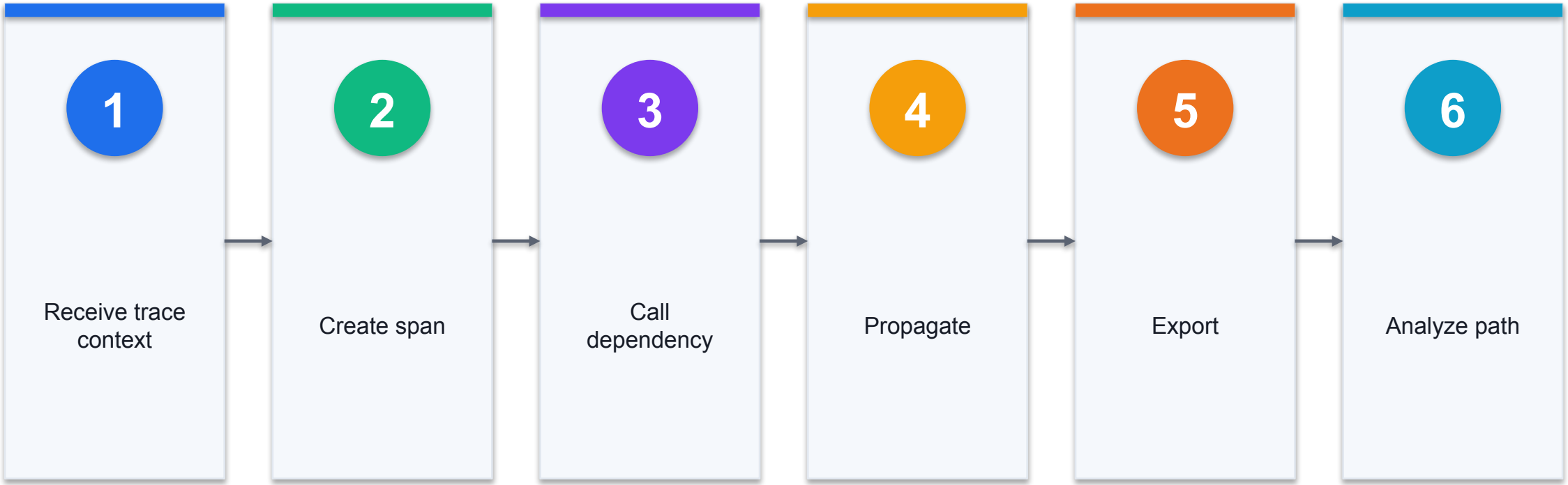
4

EVIDENCE

Trace ID, segment/subsegment, annotation, sampling rule, fault edge and correlated log event.

© 2026 Tertiary Infotech Academy Pte Ltd

Distributed tracing and service maps: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Distributed tracing and service maps: design decision

? SCENARIO

A request is slow only when it crosses an asynchronous worker and a third-party dependency.

1 DESIGN RESPONSE

Propagate trace context, sample intentionally, protect sensitive attributes and correlate traces with logs and metrics.

2 PRIMARY SERVICES

AWS X-Ray · ADOT · CloudWatch ServiceLens

3 FAILURE MODE

Missing context propagation creates disconnected traces and incorrect service maps.

4 PROOF

Trace ID, segment/subsegment, annotation, sampling rule, fault edge and correlated log event.

© 2026 Tertiary Infotech Academy Pte Ltd

Distributed tracing and service maps: failure diagnostic

1

RECOGNISE

Observable failure condition: Missing context propagation creates disconnected traces and incorrect service maps.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Receive trace context through Analyze path; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Trace ID, segment/subsegment, annotation, sampling rule, fault edge and correlated log event.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 4 · APPLIED SCENARIO

Distributed tracing and service maps: worked decision

1

CONSTRAINT

A request is slow only when it crosses an asynchronous worker and a third-party dependency.

2

DECISION

Propagate trace context, sample intentionally, protect sensitive attributes and correlate traces with logs and metrics.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Missing context propagation creates disconnected traces and incorrect service maps.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Trace ID, segment/subsegment, annotation, sampling rule, fault edge and correlated log event.

© 2026 Tertiary Infotech Academy Pte Ltd

Operational analysis and review

1

OUTCOME

Turn telemetry into repeatable system reviews and prioritized modifications.

2

ARCHITECTURE

Build hypothesis-driven queries, compare deploy markers, keep dashboards audience-specific and record decisions.

3

FAILURE MODE

A dashboard with many graphs but no decision threshold becomes decorative monitoring.

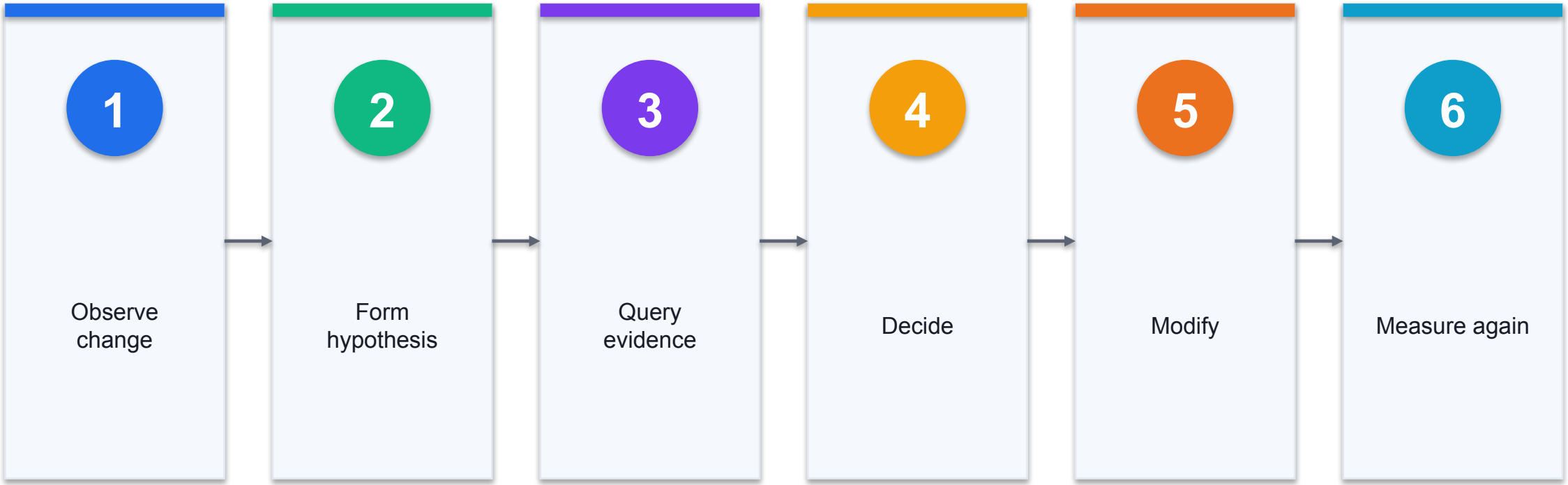
4

EVIDENCE

Saved query, dashboard version, deployment annotation, review finding, owner and verified improvement.

© 2026 Tertiary Infotech Academy Pte Ltd

Operational analysis and review: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 4 · SCENARIO

Operational analysis and review: design decision

? SCENARIO

A weekly architecture review must explain rising throttles and recommend a measurable corrective action.

1 DESIGN RESPONSE

Build hypothesis-driven queries, compare deploy markers, keep dashboards audience-specific and record decisions.

2 PRIMARY SERVICES

Logs Insights · Dashboards · Contributor Insights · Metrics Streams

3 FAILURE MODE

A dashboard with many graphs but no decision threshold becomes decorative monitoring.

4 PROOF

Saved query, dashboard version, deployment annotation, review finding, owner and verified improvement.

© 2026 Tertiary Infotech Academy Pte Ltd

Operational analysis and review: failure diagnostic

1

RECOGNISE

Observable failure condition: A dashboard with many graphs but no decision threshold becomes decorative monitoring.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Observe change through Measure again; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Saved query, dashboard version, deployment annotation, review finding, owner and verified improvement.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 4 · APPLIED SCENARIO

Operational analysis and review: worked decision

1

CONSTRAINT

A weekly architecture review must explain rising throttles and recommend a measurable corrective action.

2

DECISION

Build hypothesis-driven queries, compare deploy markers, keep dashboards audience-specific and record decisions.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: A dashboard with many graphs but no decision threshold becomes decorative monitoring.

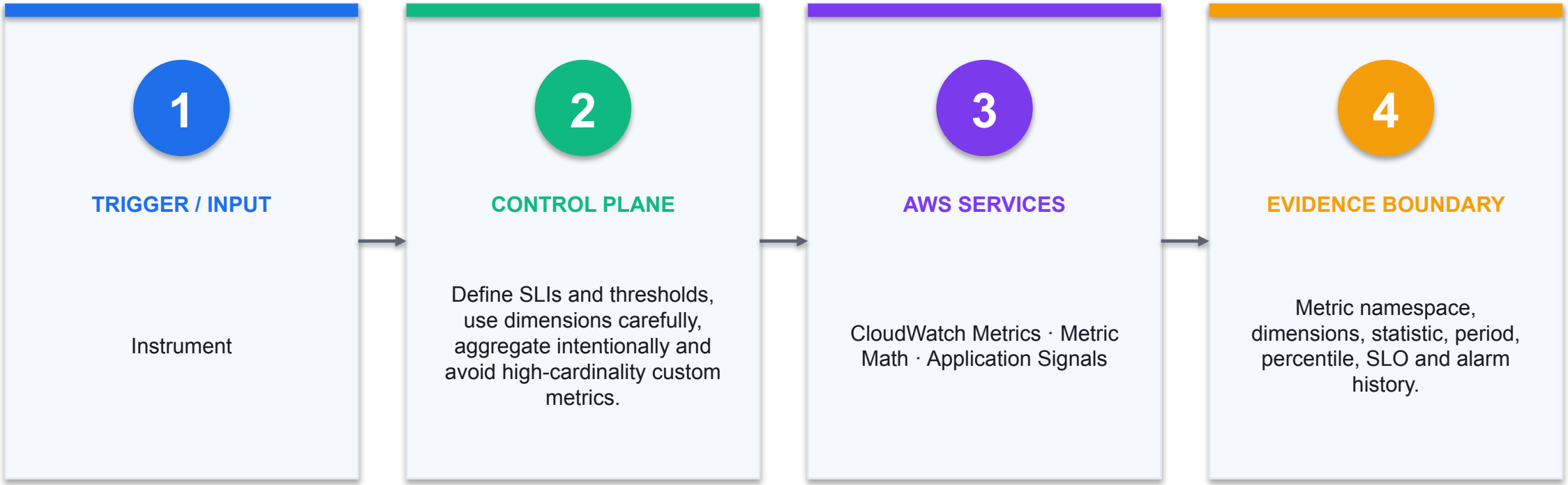
4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Saved query, dashboard version, deployment annotation, review finding, owner and verified improvement.

© 2026 Tertiary Infotech Academy Pte Ltd

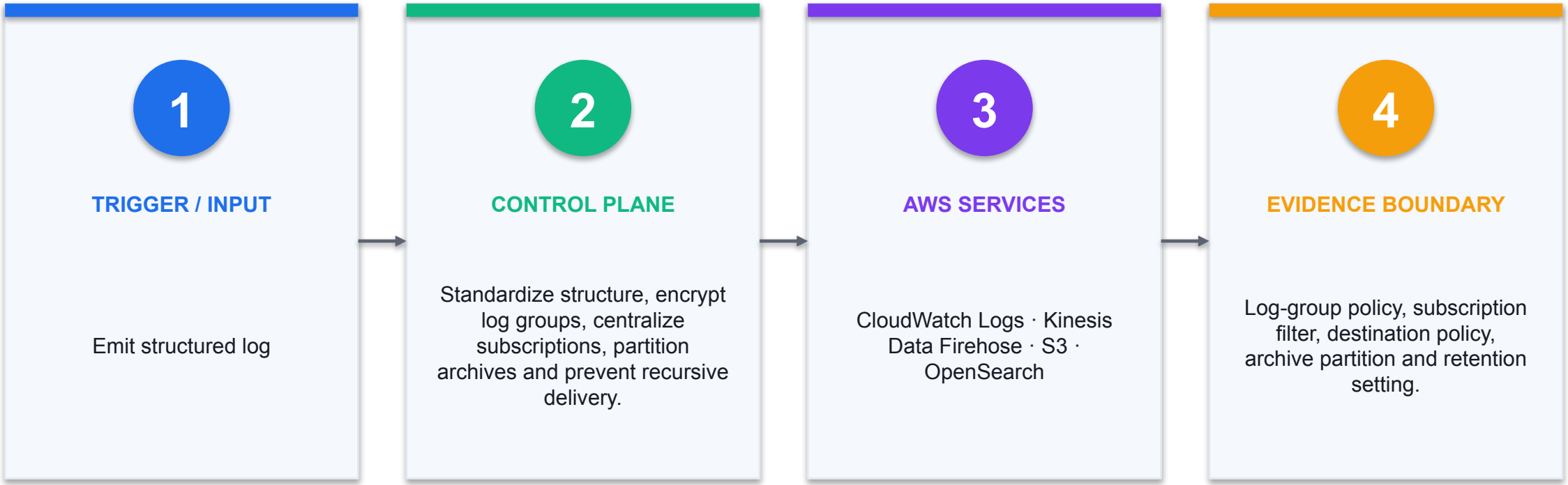
Metrics to service objectives: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

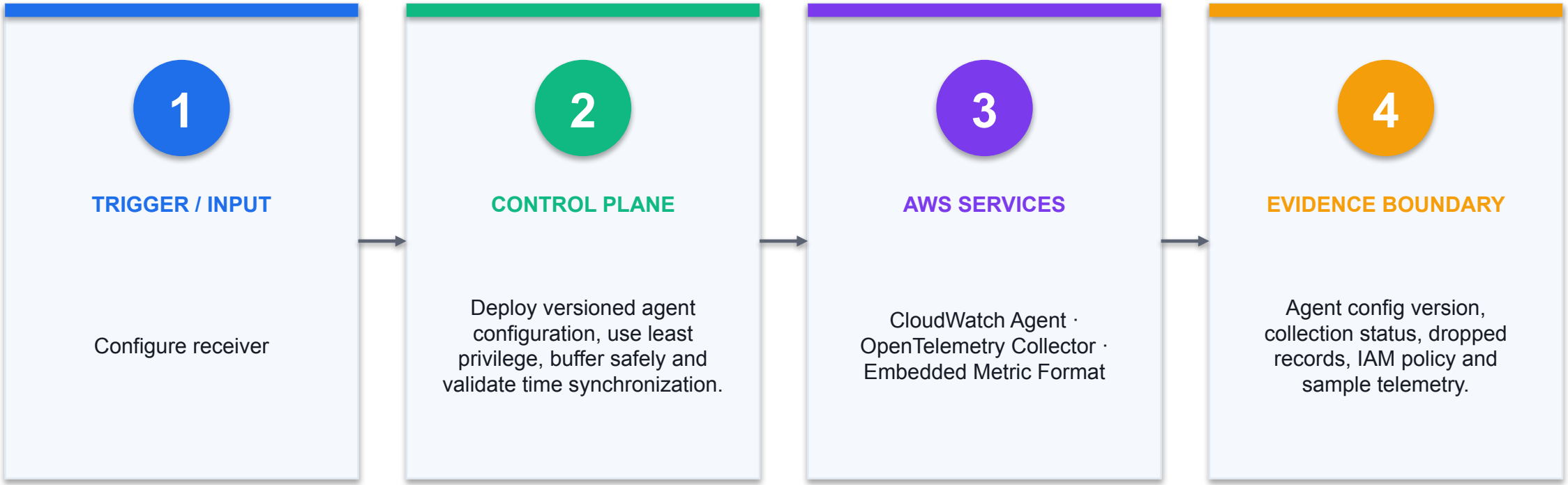
Centralized log architecture: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

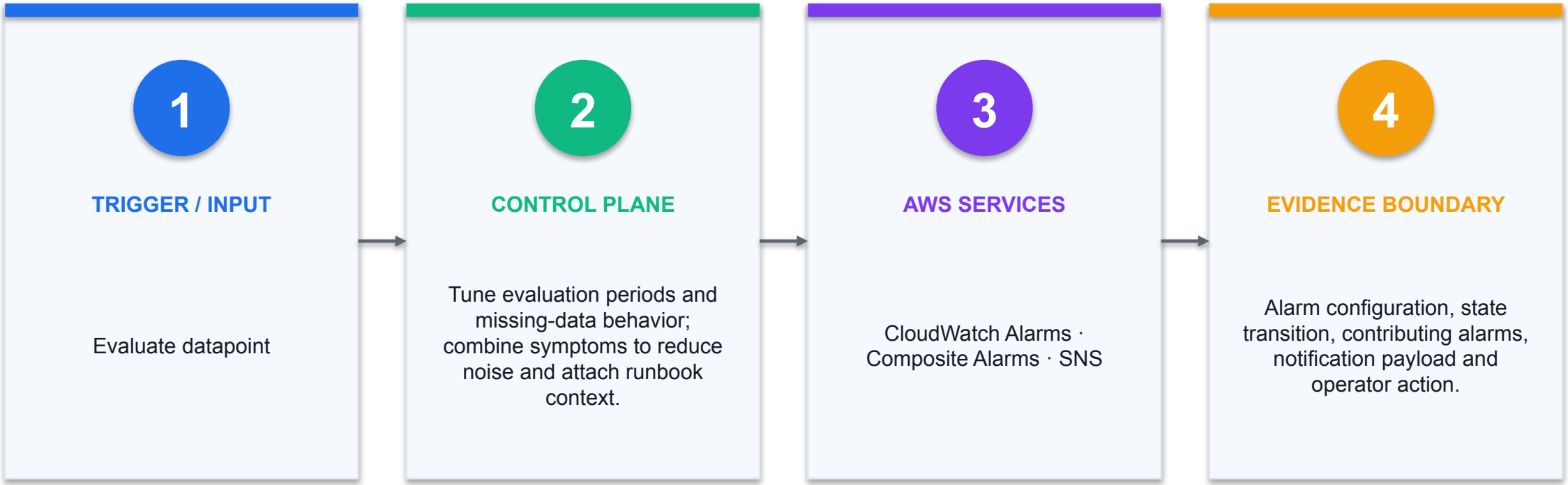
Agents and custom telemetry: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

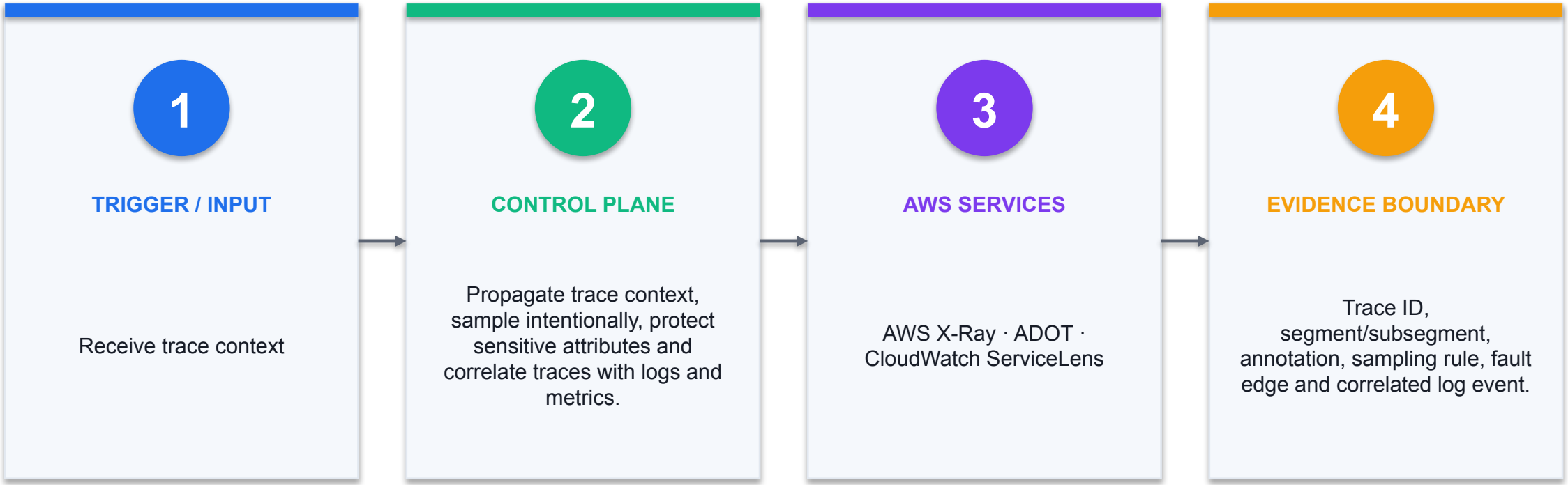
Alarms, anomaly detection and composite signals: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

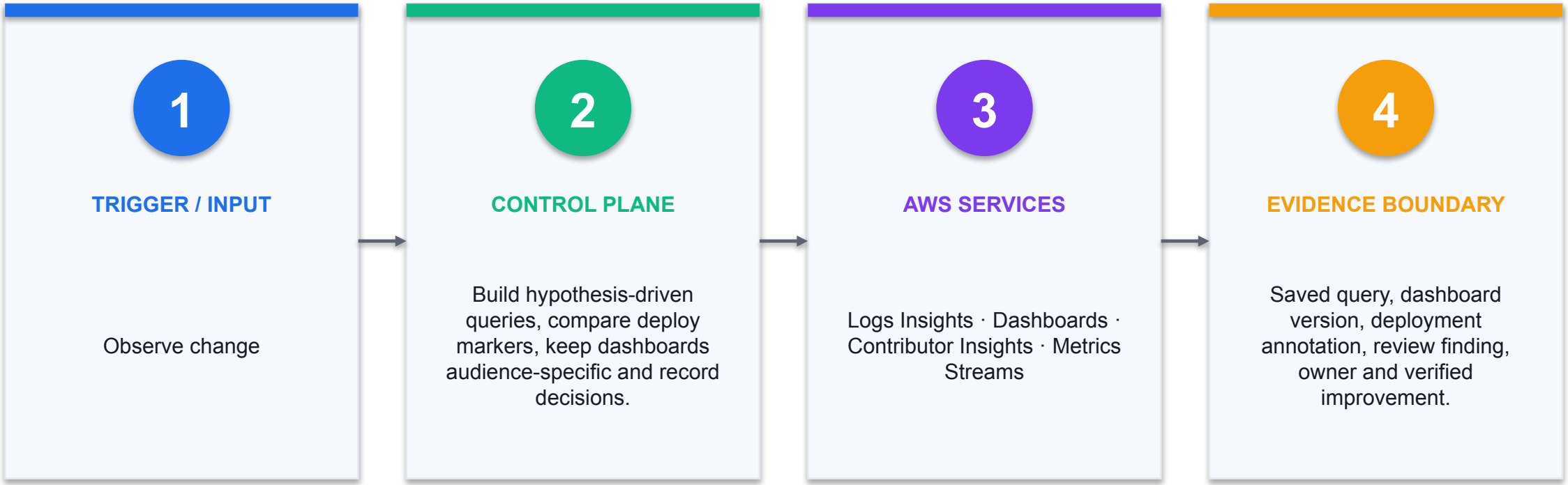
Distributed tracing and service maps: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Operational analysis and review: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Collect Custom Metrics with the CloudWatch Agent and Build an Operations Dashboard

LAB 16

Day 4 · LO4 · K5, A6

Your team runs a web app on EC2 but only has the default hypervisor CPU metric — you are blind to memory pressure, disk usage and application latency.

YOU'LL BUILD

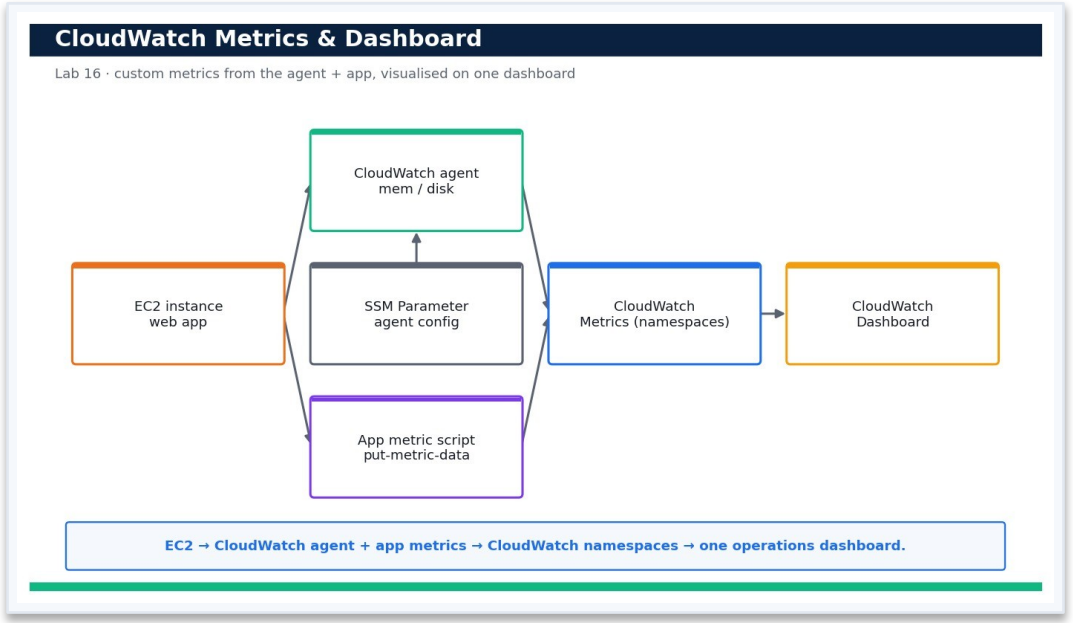
A running CloudWatch agent emitting memory/disk custom metrics, a script publishing app latency/request/error metrics, and a CloudWatch dashboard visualising CPU, p90 latency, errors and request count.

Tools: Amazon CloudWatch, CloudWatch agent, Amazon EC2, AWS Systems Manager (SSM Parameter Store), IAM

OUTCOME & EVIDENCE

Outcome: Instrument an EC2 workload with memory/disk.

Evidence: Output of showing the agent running.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Centralise Logs in CloudWatch Logs and Analyse Them with Logs Insights

LAB 17

Day 4 · LO4 · K5, A6

When an incident hits, grepping across scattered instances wastes precious minutes.

YOU'LL BUILD

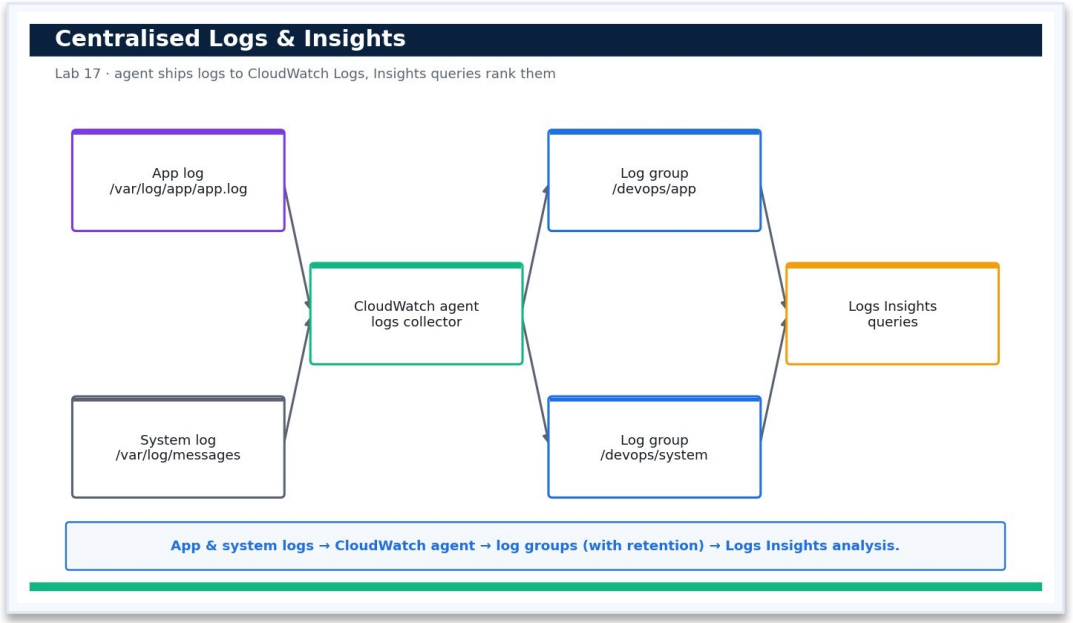
Centralised app and system log groups (with retention) fed by the CloudWatch agent, plus a set of saved Logs Insights queries that surface the slowest and most error-prone requests.

Tools: Amazon CloudWatch Logs, CloudWatch Logs Insights, CloudWatch agent, Amazon EC2, IAM

OUTCOME & EVIDENCE

Outcome: Ship application and system logs into CloudWatch Logs.

Evidence: output showing /devops/app and /devops/system with retentionInDays 14.



Reference architecture

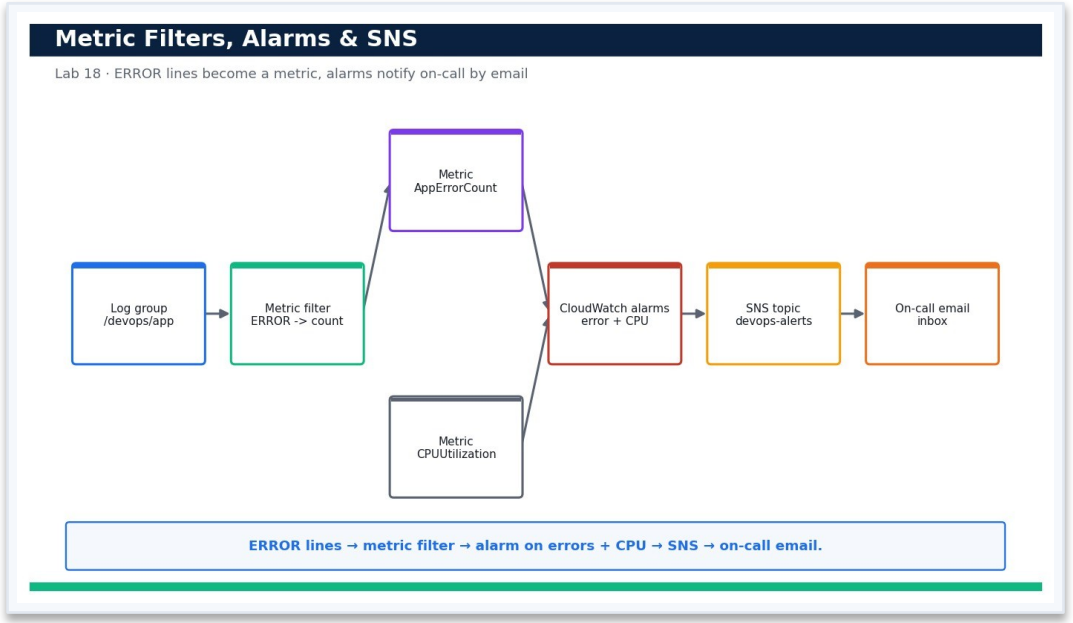
© 2026 Tertiary Infotech Academy Pte Ltd

Turn Logs into Signals: Metric Filters, CloudWatch Alarms and SNS Notifications

LAB 18

Day 4 · LO4 · K5, A6

Logs are only useful if they page someone when things go wrong.



Reference architecture

YOU'LL BUILD

A metric filter converting ERROR log lines into a CloudWatch metric, two alarms (errors and high CPU) wired to an SNS topic.

Tools: Amazon CloudWatch (metric filters + alarms), Amazon CloudWatch Logs, Amazon SNS, AWS CloudFormation, IAM

OUTCOME & EVIDENCE

Outcome: Create a metric filter that counts ERROR log lines, alarm on that metric and on high CPU.

Evidence: output showing app-error-count attached to /devops/app.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN RECAP

Recap — Monitoring and Logging

1

Lab 16

Instrument an EC2 workload with memory/disk.

2

Lab 17

Ship application and system logs into CloudWatch Logs.

3

Lab 18

Create a metric filter that counts ERROR log lines, alarm on that metric and on high CPU.

4

Key principle

Namespaces, dimensions and custom metrics measure system and application health over time.

© 2026 Tertiary Infotech Academy Pte Ltd

05

DOMAIN 05

Incident and Event Response

Exam weighting 14% · Event sources · automated response · configuration change · troubleshooting

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Incident and Event Response

1

Event-driven operations

EventBridge routes events from AWS services and applications to targets that act automatically.

2

Notify & act

SNS notifies people; Lambda, Systems Manager Automation and Step Functions take the remediating action.

3

Auto-remediation

AWS Config rules and EventBridge patterns detect drift and fix it — tag, isolate, restart or roll back.

4

Runbooks & playbooks

Systems Manager documents codify the response so incidents are handled consistently and fast.

5

Deployment failure response

CloudWatch alarms roll back CodeDeploy; pipeline stages stop and alert on failure.

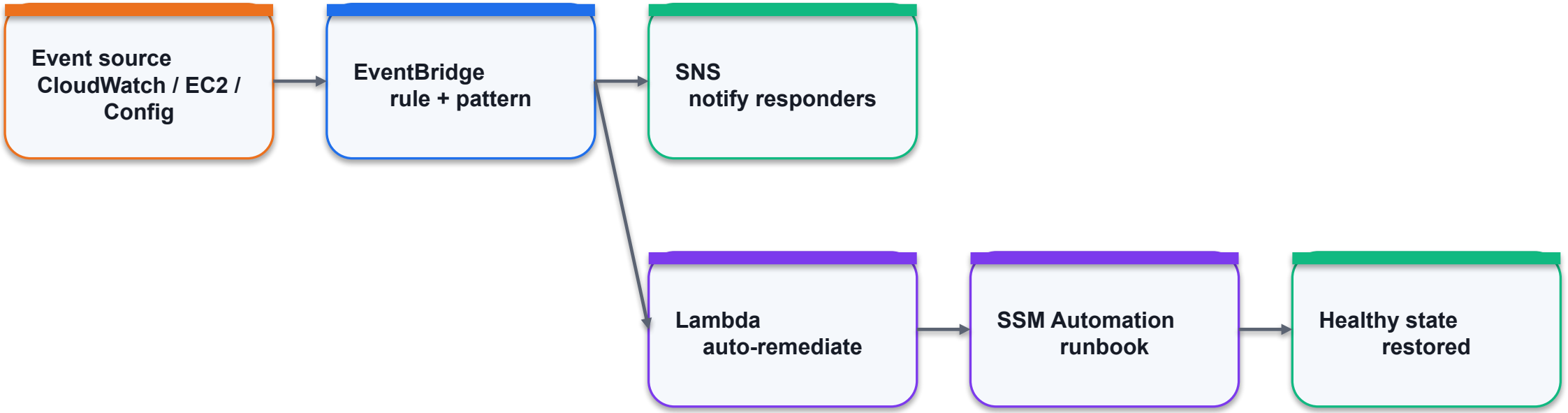
6

Troubleshooting

Correlate metrics, logs, traces and CloudTrail events to find and fix system and application failures.

© 2026 Tertiary Infotech Academy Pte Ltd

Event-Driven Incident Response



- EventBridge routes the event to notify people and trigger a Lambda/SSM runbook that fixes the issue automatically.

© 2026 Tertiary Infotech Academy Pte Ltd

THE RESPONSE LOOP

From Detection to Remediation

1

Detect

CloudWatch alarms, Config rules and service events signal a problem.

2

Route

EventBridge matches the event pattern and fans out to targets.

3

Notify

SNS alerts on-call responders by email / chat / SMS.

4

Remediate

Lambda or SSM Automation runs the runbook — restart, isolate, roll back.

5

Roll back

CloudWatch alarms auto-roll-back a bad CodeDeploy deployment.

6

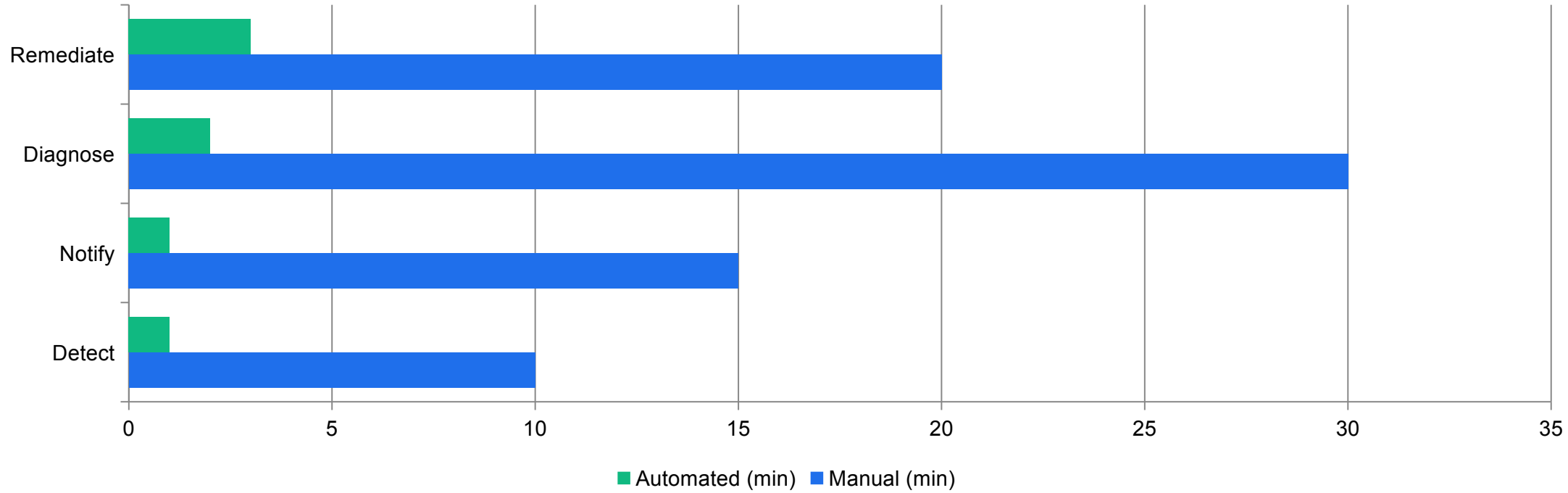
Learn

Blameless review; codify the fix into the runbook.

© 2026 Tertiary Infotech Academy Pte Ltd

WHY AUTOMATE RESPONSE

Automation Cuts Mean Time To Recover



WHAT THE DATA SHOWS

Event-driven automation collapses MTTR from over an hour to a few minutes across every phase.

© 2026 Tertiary Infotech Academy Pte Ltd

THE EVENT ROUTER

Amazon EventBridge Concepts

1

Event bus

Receives events from AWS services, SaaS and your apps.

2

Rules & patterns

Match events by source, type and detail.

3

Targets

Route matched events to Lambda, SNS, SQS, Step Functions…

4

Scheduler

Run tasks on a cron/rate schedule at scale.

© 2026 Tertiary Infotech Academy Pte Ltd

CODIFYING THE RESPONSE

Runbooks vs Playbooks

Automate the known

- Runbook
  - Step-by-step operational procedure
  - Often codified as SSM Automation documents
  - Automates a known, repeatable fix

Guide the unknown

- Playbook
  - Broader incident-response guidance
  - Roles, comms and decision points
  - Used for novel or major incidents

© 2026 Tertiary Infotech Academy Pte Ltd

FAIL SAFELY

Responding to Deployment Failures

1

Alarm-based rollback

CloudWatch alarms auto-roll-back a bad CodeDeploy deployment.

2

Pipeline stop

A failed stage halts CodePipeline and alerts the team.

3

Circuit breaker

ECS deployment circuit breaker reverts failed rollouts.

4

Blue/green switch-back

Instantly shift traffic back to the last good fleet.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Event-driven operations with EventBridge

IN DEPTH

- EventBridge receives events from AWS services, SaaS partners and your applications on event buses.
- Rules match events by source, detail-type and content pattern and route them to targets.
- Targets include Lambda, SNS, SQS, Step Functions and Systems Manager Automation.

WHY IT MATTERS

Events let operations react the instant something changes, instead of polling or waiting for a human to notice.

FAILURE MODE

Over-broad event patterns match far more than intended and trigger remediation on benign events.

TRADE-OFF / DECISION

Fine-grained rules are precise but numerous to manage; broad rules are simple but risk false triggers.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Notify people, trigger machines

IN DEPTH

- SNS notifies humans by email, SMS or chat; it also fans out to multiple subscribers.
- Lambda, Systems Manager Automation and Step Functions take the actual remediating action.
- The two paths run in parallel: tell the responders and start the fix simultaneously.

WHY IT MATTERS

Separating 'notify' from 'act' means the automated fix starts immediately while humans stay informed and can intervene.

FAILURE MODE

Notifying without automating leaves recovery gated on human reaction time at 3am.

TRADE-OFF / DECISION

Automating actions demands tight IAM scoping and validation so a bad event can't trigger a harmful action.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Auto-remediation with Config and Automation

IN DEPTH

- AWS Config detects non-compliant or drifted resources and can invoke an SSM Automation remediation.
- Common auto-fixes: re-enable encryption, remove a public S3 setting, isolate an instance, re-tag resources.
- Every remediation is logged for audit and can require approval for higher-risk actions.

WHY IT MATTERS

Continuous detection plus automated remediation keeps the environment compliant without waiting for a human review cycle.

FAILURE MODE

Auto-remediating without guardrails can fight a legitimate change or mask a deeper problem instead of fixing it.

TRADE-OFF / DECISION

Immediate auto-fix is fast but can hide recurring root causes — track remediation frequency as a signal.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Runbooks and playbooks

IN DEPTH

- A runbook is a precise, often-automated procedure for a known problem (SSM Automation documents encode these).
- A playbook is broader human guidance for novel or major incidents — roles, comms and decision points.
- Codifying responses makes them consistent, fast and improvable after every incident.

WHY IT MATTERS

Runbooks turn hard-won incident knowledge into repeatable automation; playbooks coordinate the human response to the unknown.

FAILURE MODE

Tribal knowledge in one engineer's head fails the moment that person is on leave during an incident.

TRADE-OFF / DECISION

Automated runbooks must be maintained as the system changes, or they confidently do the wrong thing.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Deployment-failure response

IN DEPTH

- CloudWatch alarms can trigger CodeDeploy automatic rollback when a new release misbehaves.
- A failed pipeline stage halts CodePipeline and alerts the team rather than continuing to prod.
- The ECS deployment circuit breaker reverts a failed rollout; blue/green switches traffic back instantly.

WHY IT MATTERS

Wiring release health into automated rollback keeps a bad deployment from becoming a prolonged outage.

FAILURE MODE

Deployments with no health signal or rollback path require a manual, stressful, error-prone recovery.

TRADE-OFF / DECISION

Fast auto-rollback is safe but can oscillate if the health signal is noisy — tune alarm sensitivity.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Troubleshooting methodically across signals

IN DEPTH

- Correlate metrics (what changed), logs (what the app said), traces (where it slowed) and CloudTrail (who changed what).
- Start from the user-facing symptom and the SLO, then narrow by time and by service.
- CloudTrail often reveals that an incident began with a configuration or permission change.

WHY IT MATTERS

A disciplined, signal-driven method finds root cause faster than guessing and prevents fixing symptoms only.

FAILURE MODE

Restarting things at random during an incident can clear the symptom while leaving the cause to recur.

TRADE-OFF / DECISION

Thorough diagnosis takes minutes you may feel you don't have — but mis-diagnosis costs more.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Blameless post-incident review

IN DEPTH

- After recovery, a blameless review reconstructs the timeline and identifies systemic causes.
- Action items harden the system: new alarms, a new runbook, a guardrail, or a design change.
- The fix is codified so the same incident cannot recur silently.

WHY IT MATTERS

Treating incidents as learning — not fault — turns each outage into durable resilience and better automation.

FAILURE MODE

Blaming individuals drives problems underground; people stop reporting near-misses and the system stops improving.

TRADE-OFF / DECISION

Reviews cost time and candour — the payoff is compounding reliability over many incidents.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Orchestrating complex responses with Step Functions

IN DEPTH

- Step Functions coordinate multi-step remediations with retries, branching and error handling.
- State machines make a response auditable and resumable rather than a fragile chain of Lambdas.
- Human-approval states can pause automation for a decision on higher-risk actions.

WHY IT MATTERS

Some incidents need a workflow, not a single action — orchestration gives reliable, observable, multi-step recovery.

FAILURE MODE

A tangle of Lambdas invoking Lambdas becomes impossible to trace or safely modify.

TRADE-OFF / DECISION

Orchestration adds a service to design and cost, justified when the response has real branching and steps.

© 2026 Tertiary Infotech Academy Pte Ltd

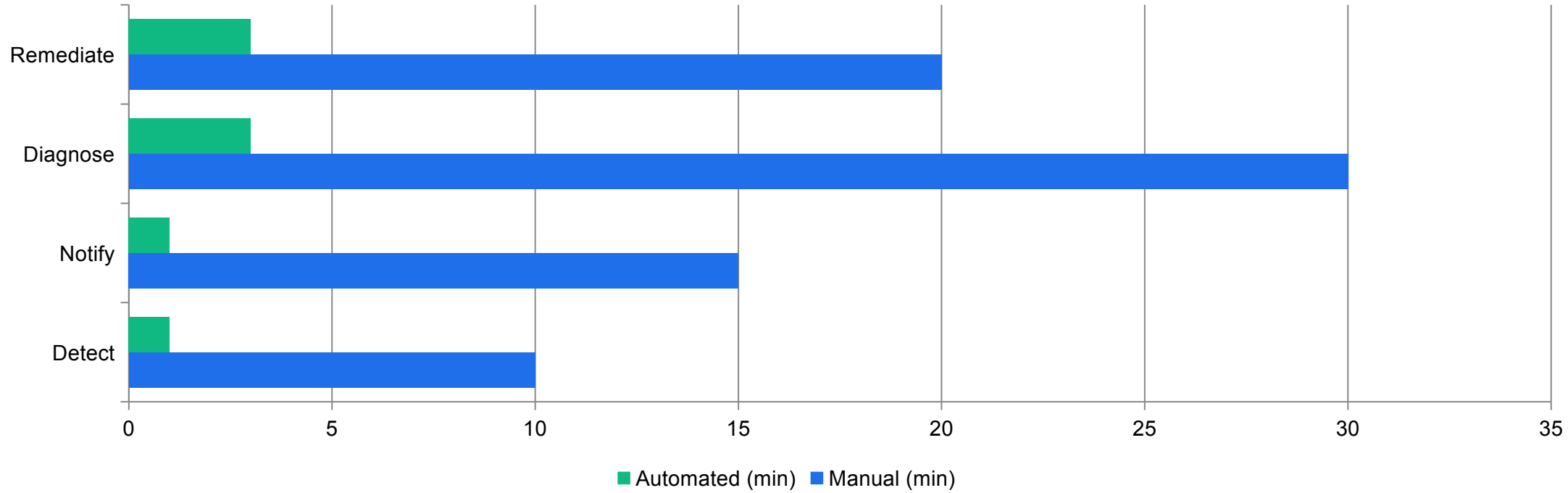
Runbook vs Playbook — Know the Difference

Aspect	Runbook	Playbook
Scope	One known, repeatable fix	Broader incident guidance
Form	Often automated (SSM Automation)	Human roles, comms, decisions
Best for	'Disk full → expand volume'	Novel or major incidents
Outcome	Fast, consistent auto-recovery	Coordinated human response

© 2026 Tertiary Infotech Academy Pte Ltd

WHY AUTOMATE RESPONSE

Automation Cuts MTTR Across Every Phase



WHAT THE DATA SHOWS

Event-driven automation collapses mean-time-to-recover from over an hour to a few minutes — most of the saving is in detection and remediation.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT SLOWS RECOVERY

Incident-Response Failure Modes & Anti-Patterns

1

Over-broad event patterns

Remediation on benign events → precise patterns, test them.

2

Notify-only, no automation

Recovery gated on human reaction → wire the fix.

3

Unscoped remediation IAM

A bad event does damage → least-privilege action roles.

4

Tribal knowledge

Fix lives in one head → codify runbooks.

5

Random restarts

Symptom cleared, cause remains → signal-driven diagnosis.

6

No post-incident learning

Same outage recurs → blameless review + action items.

© 2026 Tertiary Infotech Academy Pte Ltd

PROVEN PRACTICES

Response Patterns Worth Adopting

1. **Detect → route → notify+act**
   EventBridge fans out to people and automation.
2. **Alarm-based rollback**
   Release health auto-reverts bad deploys.
3. **Automated runbooks**
   SSM Automation for known fixes.
4. **Guarded auto-remediation**
   Config → remediation with approval for risky actions.
5. **Orchestrated recovery**
   Step Functions for multi-step responses.
6. **Blameless reviews**
   Every incident hardens the system.

© 2026 Tertiary Infotech Academy Pte Ltd

RESPONSE STRATEGY

Architecture Decision — Automate the Fix or Page a Human?

A — Fully automate

Known cause, safe and reversible fix, frequent occurrence. Fastest recovery — needs tight IAM and validation.

B — Automate with approval

Higher-risk action or ambiguous trigger. Step Functions pauses for a human yes/no before acting.

C — Notify only

Novel or unclear incidents. Page the on-call with rich context; automation isn't safe until the pattern is understood.

© 2026 Tertiary Infotech Academy Pte Ltd

DETECT AND AUTO-REMEDIATE

Scenario — A public S3 bucket appears at 2am

SITUATION

Someone changes a bucket to public. Data exposure risk is immediate; no engineer is awake.

ANALYSIS

- AWS Config's s3-bucket-public-read-prohibited rule detects the non-compliant state within minutes.
- An EventBridge rule on the Config compliance-change event can trigger an SSM Automation remediation.
- The remediation re-applies Block Public Access and SNS notifies the security channel.

DECISION

Wire Config → EventBridge → SSM Automation to auto-remediate and notify, so the exposure window is minutes and unattended — with the action logged for the morning review.

© 2026 Tertiary Infotech Academy Pte Ltd

RELEASE-HEALTH AUTOMATION

Scenario — A deploy degrades error rate

SITUATION

Minutes after a rollout, the error-rate alarm crosses threshold.

ANALYSIS

- The alarm can be an automatic rollback trigger for the CodeDeploy deployment, no human needed.
- EventBridge can also notify the team and open an incident record for context.
- A post-incident action is to move the service to canary so 5% catches this next time.

DECISION

Bind the error-rate alarm to CodeDeploy auto-rollback and notify in parallel; recovery becomes automatic and the follow-up hardens the pipeline.

© 2026 Tertiary Infotech Academy Pte Ltd

EVENT TO RECOVERY

Worked Analysis — Anatomy of an Automated Response

Signal	CloudWatch alarm / Config finding / service event fires.
Route	EventBridge rule matches the event pattern precisely.
Notify	SNS informs responders with context — they watch, not scramble.
Act & verify	Lambda/SSM runbook remediates; a follow-up check confirms healthy state, else escalates.

TAKEAWAY

A good response is a small, testable pipeline — precise trigger, parallel notify-and-act, and a verification step that escalates if the fix didn't hold.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Event-driven incident response

1

PURPOSE

Route precise operational state changes to people and automation with buffering and failure isolation.

2

FEATURE 1

EventBridge filters structured events and supports archives/replay for event buses.

3

FEATURE 2

SNS fans out notifications; SQS buffers consumer work and supports DLQs.

4

FEATURE 3

Lambda performs short response actions under a scoped execution role.

© 2026 Tertiary Infotech Academy Pte Ltd

Event-driven incident response: selection and controls

1

CHOOSE WHEN

Use EventBridge for content routing, SNS for fan-out and SQS when consumers need durable buffering/back-pressure.

2

DEVOPS ENGINEERING

Match exact fields, make actions idempotent, add DLQs, cap retries and require approval for high-impact remediation.

3

FAILURE MODE

A broad event pattern or overprivileged responder can automate the wrong containment action.

4

EVIDENCE

Matched event, delivery attempts, DLQ depth, function request ID, changed resource and approval.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Operations and remediation

1

PURPOSE

Aggregate operational issues, attach context and run controlled investigation or remediation workflows.

2

FEATURE 1

OpsItems connect an issue to resources, related events, runbooks and ownership.

3

FEATURE 2

Automation executes auditable steps with branching, approvals and AWS API integrations.

4

FEATURE 3

Explorer aggregates operational/compliance data across configured accounts and Regions.

© 2026 Tertiary Infotech Academy Pte Ltd

Operations and remediation: selection and controls

1

CHOOSE WHEN

Use Automation for repeatable remediation and OpsCenter when issues need lifecycle, ownership and evidence.

2

DEVOPS ENGINEERING

Start with read-only diagnosis, canary the change, bound targets/errors and preserve before/after state.

3

FAILURE MODE

Automation that deletes diagnostic state before collection can make the incident unexplainable.

4

EVIDENCE

OpsItem, execution steps, approver, target set, output, resource diff and post-check.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Audit and configuration diagnosis

1

PURPOSE

Separate who changed what from what configuration exists now and what desired state declares.

2

FEATURE 1

CloudTrail records control-plane/API activity and identity context; data events require explicit configuration.

3

FEATURE 2

Config records supported resource configuration/history and evaluates rules.

4

FEATURE 3

CloudFormation drift compares supported actual properties with the stack template.

© 2026 Tertiary Infotech Academy Pte Ltd

Audit and configuration diagnosis: selection and controls

1

CHOOSE WHEN

Use CloudTrail for actor/API evidence, Config for resource history/compliance and drift for IaC divergence.

2

DEVOPS ENGINEERING

Centralise trails, protect logs, scope selectors deliberately, aggregate Config and investigate before remediation.

3

FAILURE MODE

No event may mean the trail did not capture that event category —not that no change occurred.

4

EVIDENCE

Event time/identity/request, configuration timeline, compliance evaluation, drift detail and remediation record.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Network troubleshooting

1

PURPOSE

Diagnose connectivity by distinguishing route, policy, name-resolution, load-balancer and application failures.

2

FEATURE 1

Flow Logs show accepted/rejected network-flow metadata, not packet contents.

3

FEATURE 2

Reachability Analyzer evaluates a modeled path through supported network configuration.

4

FEATURE 3

Resolver query logs reveal DNS questions/responses from configured VPCs.

© 2026 Tertiary Infotech Academy Pte Ltd

Network troubleshooting: selection and controls

1

CHOOSE WHEN

Use configuration path analysis first, then flow/DNS/load-balancer evidence to locate the failed layer.

2

DEVOPS ENGINEERING

Check source/destination, DNS, route both ways, security groups, NACLs, endpoints, target health and application listener.

3

FAILURE MODE

Treating every timeout as a security-group issue leads to random changes and larger blast radius.

4

EVIDENCE

Path result, flow-log action, DNS response, route, target health, application log and corrected control.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: SQS buffering, ordering and dead-letter handling

1

PURPOSE

Decouple producers from consumers while making delivery, visibility, idempotency and poison-message behavior explicit.

2

SERVICE BOUNDARY

SQS Standard · FIFO · dead-letter queues

3

PRIMARY DESIGN DECISION

Make the charge operation idempotent, extend visibility while healthy, cap retry attempts and isolate permanent failures in a DLQ.

4

FAILURE BOUNDARY

Increasing visibility indefinitely hides stuck work; relying only on FIFO deduplication does not protect downstream side effects forever.

© 2026 Tertiary Infotech Academy Pte Ltd

SQS buffering, ordering and dead-letter handling: how it works

1

MECHANIC 1

Standard queues scale broadly with at-least-once delivery and best-effort ordering; consumers must tolerate duplicate and reordered messages.

2

MECHANIC 2

FIFO queues preserve order within a message group and deduplicate within the supported window, but throughput and group design still matter.

3

MECHANIC 3

Visibility timeout hides a received message while processing; it must exceed normal work or be extended through a heartbeat.

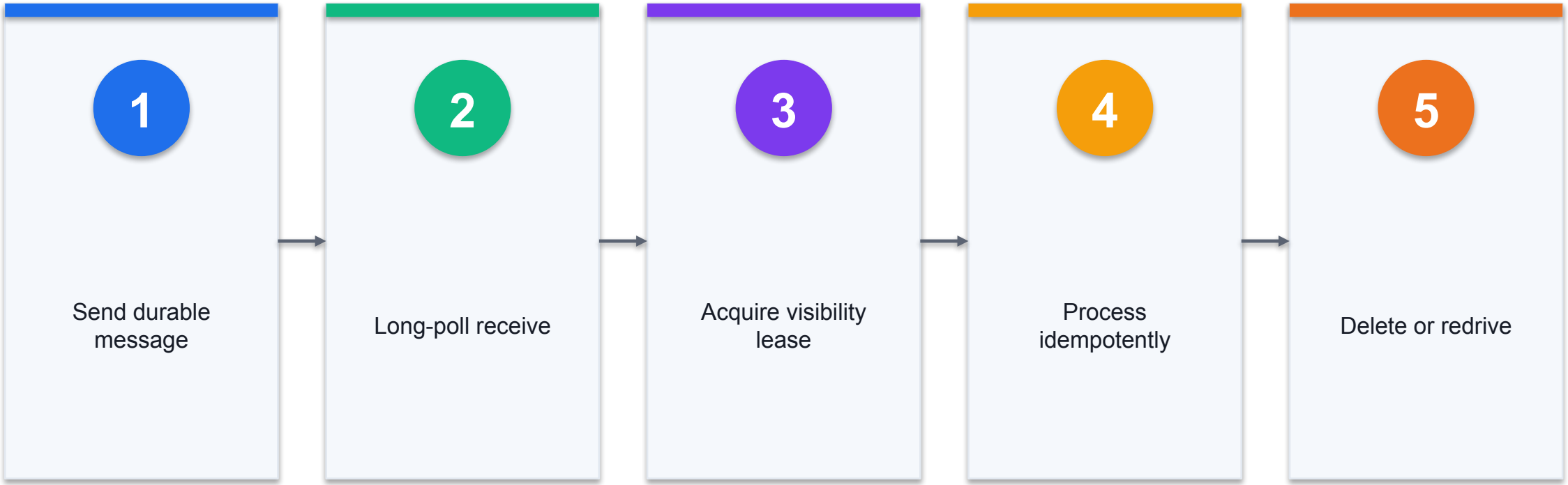
4

MECHANIC 4

Long polling reduces empty receives, while redrive policies move repeatedly failing messages to a DLQ for inspection and controlled replay.

© 2026 Tertiary Infotech Academy Pte Ltd

SQS buffering, ordering and dead-letter handling: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

SQS buffering, ordering and dead-letter handling: configuration decisions

1

QUEUE TYPE

Choose FIFO only when ordered groups/deduplication are real requirements; Standard remains the default for independent work.

2

BATCHING

Batch send/receive/delete for efficiency, but handle per-entry success and failure independently.

3

DLQ

Set max receives from real retry behavior and monitor age/depth; a DLQ is an investigation queue, not a disposal bin.

4

SECURITY

Use queue policies for producer access, KMS where required and VPC endpoints for controlled private API paths.

© 2026 Tertiary Infotech Academy Pte Ltd

SQS buffering, ordering and dead-letter handling: worked architecture decision

1

SCENARIO

A worker occasionally exceeds the visibility timeout, causing two instances to charge the same order.

2

DESIGN DECISION

Make the charge operation idempotent, extend visibility while healthy, cap retry attempts and isolate permanent failures in a DLQ.

3

FAILURE TO AVOID

Increasing visibility indefinitely hides stuck work; relying only on FIFO deduplication does not protect downstream side effects forever.

4

ACCEPTANCE EVIDENCE

Message ID/group, receive count, processing heartbeat, idempotency record, deletion result, DLQ age and redrive record.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: SNS, EventBridge and API event contracts

1

PURPOSE

Select fan-out, event routing or synchronous API integration from contract, delivery and consumer-isolation requirements.

2

SERVICE BOUNDARY

SNS · EventBridge · API Gateway · Lambda

3

PRIMARY DESIGN DECISION

Publish a versioned domain event to EventBridge, filter to per-consumer queues and let each consumer own retry/DLQ behavior.

4

FAILURE BOUNDARY

Direct synchronous calls couple availability and release schedules; a broad event pattern can invoke the wrong automation.

© 2026 Tertiary Infotech Academy Pte Ltd

SNS, EventBridge and API event contracts: how it works

1

MECHANIC 1

SNS publishes one message to multiple subscriptions and supports filtering, retries and protocol-specific delivery behavior.

2

MECHANIC 2

EventBridge routes structured events by content across buses, supports schemas, archives/replay and SaaS or cross-account integration patterns.

3

MECHANIC 3

API Gateway exposes REST, HTTP or WebSocket APIs with stages, integrations, authorization, throttling, logging and deployment controls.

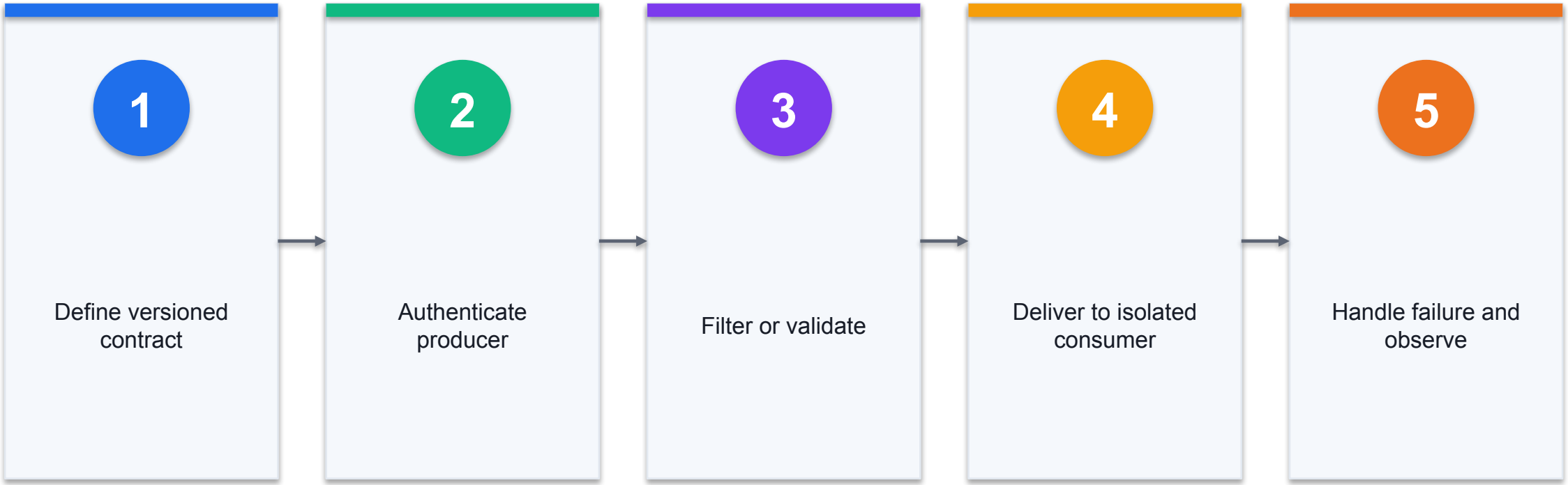
4

MECHANIC 4

Lambda proxy integrations pass a defined event envelope; non-proxy mappings can transform and validate requests before reaching the backend.

© 2026 Tertiary Infotech Academy Pte Ltd

SNS, EventBridge and API event contracts: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

SNS, EventBridge and API event contracts: configuration decisions

1

SNS

Choose for push-based fan-out where subscribers receive the same publication through supported protocols.

2

EVENTBRIDGE

Choose for content-based routing, account decoupling, event discovery and replayable operational events.

3

API GATEWAY

Choose for managed synchronous HTTP/WebSocket entry with authorization, quotas and request/response controls.

4

BUFFER

Place SQS between routing and slow consumers when back-pressure and independent retry are required.

© 2026 Tertiary Infotech Academy Pte Ltd

SNS, EventBridge and API event contracts: worked architecture decision

1

SCENARIO

An order event must reach billing, analytics and notification teams without the producer knowing their endpoints or failure state.

2

DESIGN DECISION

Publish a versioned domain event to EventBridge, filter to per-consumer queues and let each consumer own retry/DLQ behavior.

3

FAILURE TO AVOID

Direct synchronous calls couple availability and release schedules; a broad event pattern can invoke the wrong automation.

4

ACCEPTANCE EVIDENCE

Schema/version, producer identity, matched rule, delivery attempts, consumer queue depth, DLQ record and correlation ID.

© 2026 Tertiary Infotech Academy Pte Ltd

Event-driven operations

1

OUTCOME

Route operational events to the right automated or human response with explicit contracts.

2

ARCHITECTURE

Use precise event patterns, schemas, input transformation, retries, DLQs and archive/replay where required.

3

FAILURE MODE

An overly broad rule can invoke expensive or destructive automation for unrelated events.

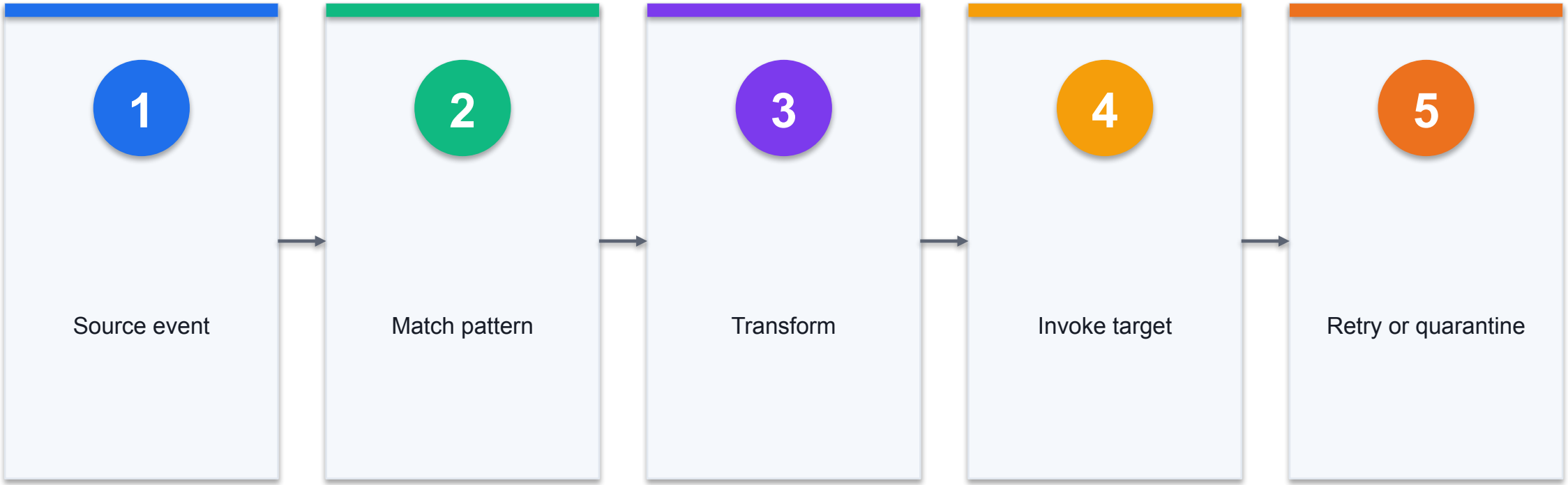
4

EVIDENCE

Event pattern, matched sample, target role, retry policy, DLQ and replay record.

© 2026 Tertiary Infotech Academy Pte Ltd

Event-driven operations: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Event-driven operations: design decision

? SCENARIO

Only failed production deployments should create an incident and trigger rollback diagnostics.

1 DESIGN RESPONSE

Use precise event patterns, schemas, input transformation, retries, DLQs and archive/replay where required.

2 PRIMARY SERVICES

EventBridge · Pipes · Scheduler · Lambda

3 FAILURE MODE

An overly broad rule can invoke expensive or destructive automation for unrelated events.

4 PROOF

Event pattern, matched sample, target role, retry policy, DLQ and replay record.

© 2026 Tertiary Infotech Academy Pte Ltd

Event-driven operations: failure diagnostic

1

RECOGNISE

Observable failure condition: An overly broad rule can invoke expensive or destructive automation for unrelated events.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Source event through Retry or quarantine; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Event pattern, matched sample, target role, retry policy, DLQ and replay record.

© 2026 Tertiary Infotech Academy Pte Ltd

Event-driven operations: worked decision

1

CONSTRAINT

Only failed production deployments should create an incident and trigger rollback diagnostics.

2

DECISION

Use precise event patterns, schemas, input transformation, retries, DLQs and archive/replay where required.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: An overly broad rule can invoke expensive or destructive automation for unrelated events.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Event pattern, matched sample, target role, retry policy, DLQ and replay record.

© 2026 Tertiary Infotech Academy Pte Ltd

Health and audit event correlation

1

OUTCOME

Distinguish provider events, configuration changes and workload symptoms during an incident.

2

ARCHITECTURE

Correlate Health events with API history and resource telemetry using account, Region, resource and time.

3

FAILURE MODE

Treating every symptom as an application defect delays response to service or control-plane events.

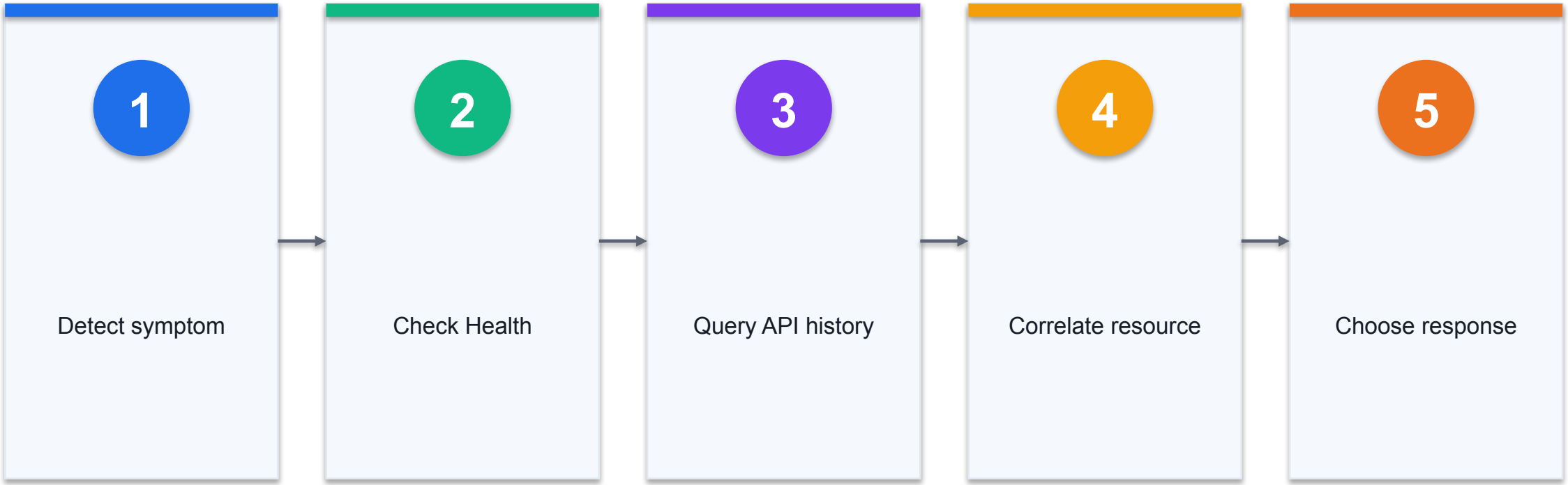
4

EVIDENCE

Health event ARN, CloudTrail event ID, affected entities, timeline and correlated alarm.

© 2026 Tertiary Infotech Academy Pte Ltd

Health and audit event correlation: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Health and audit event correlation: design decision

? SCENARIO

Instances fail in one Region after a maintenance event while a deployment occurred minutes earlier.

1 DESIGN RESPONSE

Correlate Health events with API history and resource telemetry using account, Region, resource and time.

2 PRIMARY SERVICES

AWS Health · CloudTrail · EventBridge

3 FAILURE MODE

Treating every symptom as an application defect delays response to service or control-plane events.

4 PROOF

Health event ARN, CloudTrail event ID, affected entities, timeline and correlated alarm.

© 2026 Tertiary Infotech Academy Pte Ltd

Health and audit event correlation: failure diagnostic

1

RECOGNISE

Observable failure condition: Treating every symptom as an application defect delays response to service or control-plane events.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Detect symptom through Choose response; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Health event ARN, CloudTrail event ID, affected entities, timeline and correlated alarm.

© 2026 Tertiary Infotech Academy Pte Ltd

Health and audit event correlation: worked decision

1

CONSTRAINT

Instances fail in one Region after a maintenance event while a deployment occurred minutes earlier.

2

DECISION

Correlate Health events with API history and resource telemetry using account, Region, resource and time.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Treating every symptom as an application defect delays response to service or control-plane events.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Health event ARN, CloudTrail event ID, affected entities, timeline and correlated alarm.

© 2026 Tertiary Infotech Academy Pte Ltd

Queues, dead-letter queues and redrive

1

OUTCOME

Contain poison messages, preserve evidence and restore flow without replaying successful work.

2

ARCHITECTURE

Set max receives from retry cost, alarm on age and DLQ depth, inspect safely and redrive in controlled batches.

3

FAILURE MODE

Redriving an uncorrected poison message recreates the incident.

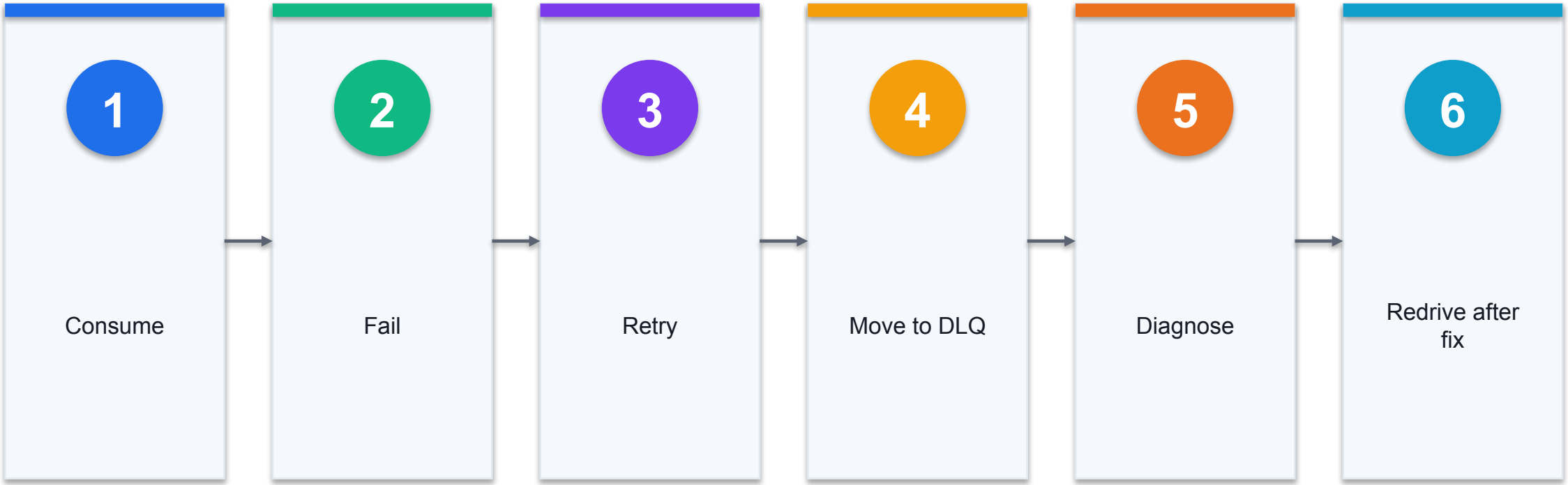
4

EVIDENCE

Message ID, receive count, failure reason, DLQ age, fix version and redrive result.

© 2026 Tertiary Infotech Academy Pte Ltd

Queues, dead-letter queues and redrive: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Queues, dead-letter queues and redrive: design decision

? SCENARIO

A malformed event blocks a consumer while valid orders accumulate behind it.

1 DESIGN RESPONSE

Set max receives from retry cost, alarm on age and DLQ depth, inspect safely and redrive in controlled batches.

2 PRIMARY SERVICES

SQS · SNS · Lambda

3 FAILURE MODE

Redriving an uncorrected poison message recreates the incident.

4 PROOF

Message ID, receive count, failure reason, DLQ age, fix version and redrive result.

© 2026 Tertiary Infotech Academy Pte Ltd

Queues, dead-letter queues and redrive: failure diagnostic

1

RECOGNISE

Observable failure condition: Redriving an uncorrected poison message recreates the incident.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Consume through Redrive after fix; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Message ID, receive count, failure reason, DLQ age, fix version and redrive result.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 5 · APPLIED SCENARIO

Queues, dead-letter queues and redrive: worked decision

1

CONSTRAINT

A malformed event blocks a consumer while valid orders accumulate behind it.

2

DECISION

Set max receives from retry cost, alarm on age and DLQ depth, inspect safely and redrive in controlled batches.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Redriving an uncorrected poison message recreates the incident.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Message ID, receive count, failure reason, DLQ age, fix version and redrive result.

© 2026 Tertiary Infotech Academy Pte Ltd

Automated remediation

1

OUTCOME

Correct known, bounded failure modes quickly while preserving safety and auditability.

2

ARCHITECTURE

Validate preconditions, scope targets, use approvals for high-risk steps, enforce rate controls and verify postconditions.

3

FAILURE MODE

Automation that acts on uncertain diagnosis can deepen an outage.

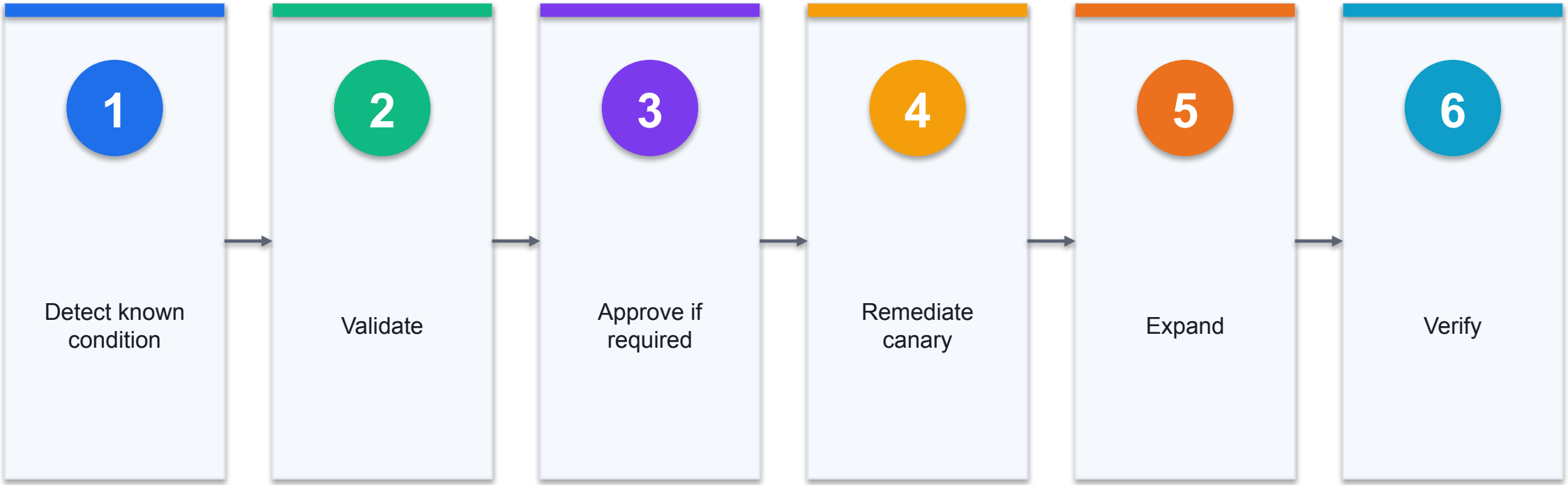
4

EVIDENCE

Trigger evidence, automation execution, approvals, step output, rollback and postcondition check.

© 2026 Tertiary Infotech Academy Pte Ltd

Automated remediation: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Automated remediation: design decision

? SCENARIO

A public security group rule must be removed automatically without touching approved exceptions.

1 DESIGN RESPONSE

Validate preconditions, scope targets, use approvals for high-risk steps, enforce rate controls and verify postconditions.

2 PRIMARY SERVICES

Systems Manager Automation · Config · Lambda

3 FAILURE MODE

Automation that acts on uncertain diagnosis can deepen an outage.

4 PROOF

Trigger evidence, automation execution, approvals, step output, rollback and postcondition check.

© 2026 Tertiary Infotech Academy Pte Ltd

Automated remediation: failure diagnostic

1

RECOGNISE

Observable failure condition: Automation that acts on uncertain diagnosis can deepen an outage.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Detect known condition through Verify; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Trigger evidence, automation execution, approvals, step output, rollback and postcondition check.

© 2026 Tertiary Infotech Academy Pte Ltd

Automated remediation: worked decision

1

CONSTRAINT

A public security group rule must be removed automatically without touching approved exceptions.

2

DECISION

Validate preconditions, scope targets, use approvals for high-risk steps, enforce rate controls and verify postconditions.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Automation that acts on uncertain diagnosis can deepen an outage.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Trigger evidence, automation execution, approvals, step output, rollback and postcondition check.

© 2026 Tertiary Infotech Academy Pte Ltd

Incident coordination and runbooks

1

OUTCOME

Create a shared timeline, ownership model and executable response path during incidents.

2

ARCHITECTURE

Define severity, response plans, contacts, escalation, runbooks and post-incident actions before failure.

3

FAILURE MODE

A technically correct fix can still fail when ownership and communication are unclear.

4

EVIDENCE

Incident record, responder engagement, timeline, runbook execution, decision log and action items.

© 2026 Tertiary Infotech Academy Pte Ltd

Incident coordination and runbooks: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Incident coordination and runbooks: design decision

?

SCENARIO

A critical API outage requires application, platform, security and business responders to coordinate.

1

DESIGN RESPONSE

Define severity, response plans, contacts, escalation, runbooks and post-incident actions before failure.

2

PRIMARY SERVICES

Incident Manager · OpsCenter · Chatbot

3

FAILURE MODE

A technically correct fix can still fail when ownership and communication are unclear.

4

PROOF

Incident record, responder engagement, timeline, runbook execution, decision log and action items.

© 2026 Tertiary Infotech Academy Pte Ltd

Incident coordination and runbooks: failure diagnostic

1

RECOGNISE

Observable failure condition: A technically correct fix can still fail when ownership and communication are unclear.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Declare through Learn; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Incident record, responder engagement, timeline, runbook execution, decision log and action items.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 5 · APPLIED SCENARIO

Incident coordination and runbooks: worked decision

1

CONSTRAINT

A critical API outage requires application, platform, security and business responders to coordinate.

2

DECISION

Define severity, response plans, contacts, escalation, runbooks and post-incident actions before failure.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: A technically correct fix can still fail when ownership and communication are unclear.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Incident record, responder engagement, timeline, runbook execution, decision log and action items.

© 2026 Tertiary Infotech Academy Pte Ltd

Structured troubleshooting

1

OUTCOME

Reduce mean time to recovery through hypothesis-driven diagnosis and controlled changes.

2

ARCHITECTURE

Start from user impact, check recent changes, narrow by dependency, compare healthy baselines and change one variable at a time.

3

FAILURE MODE

Random changes destroy evidence and create new failure modes.

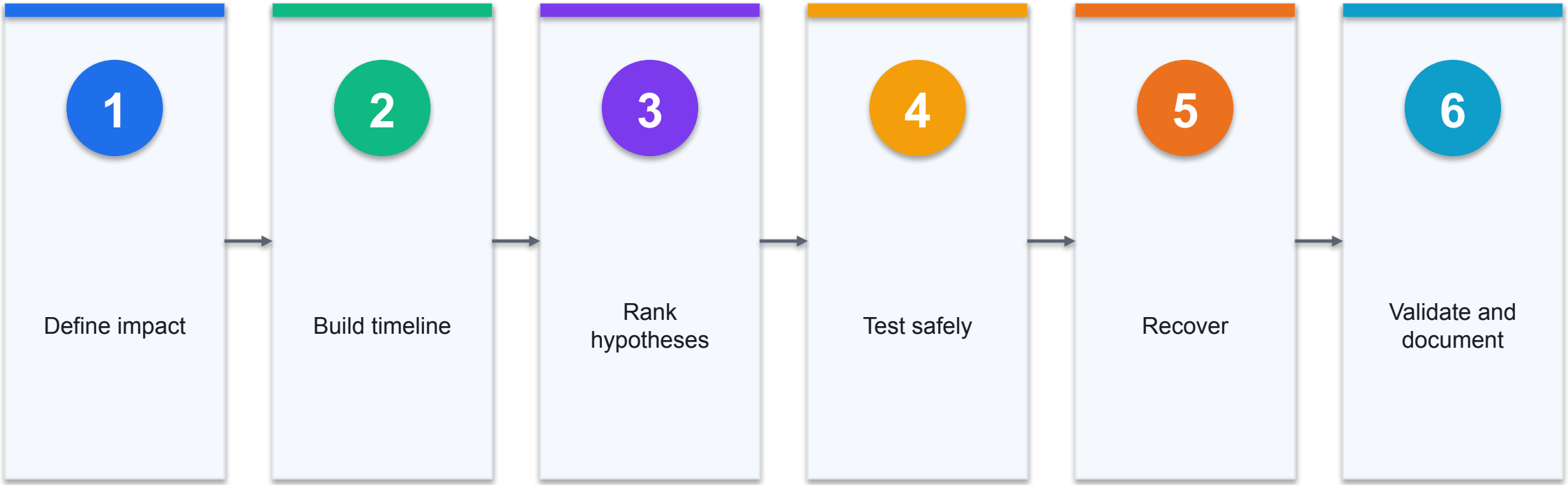
4

EVIDENCE

Impact statement, timeline, hypotheses, evidence, change record, recovery metric and root cause.

© 2026 Tertiary Infotech Academy Pte Ltd

Structured troubleshooting: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Structured troubleshooting: design decision

? SCENARIO

Latency rises after a deployment, but only for requests using one downstream dependency.

1 DESIGN RESPONSE

Start from user impact, check recent changes, narrow by dependency, compare healthy baselines and change one variable at a time.

2 PRIMARY SERVICES

CloudWatch · X-Ray · CloudTrail · Systems Manager

3 FAILURE MODE

Random changes destroy evidence and create new failure modes.

4 PROOF

Impact statement, timeline, hypotheses, evidence, change record, recovery metric and root cause.

© 2026 Tertiary Infotech Academy Pte Ltd

Structured troubleshooting: failure diagnostic

1

RECOGNISE

Observable failure condition: Random changes destroy evidence and create new failure modes.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Define impact through Validate and document; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Impact statement, timeline, hypotheses, evidence, change record, recovery metric and root cause.

© 2026 Tertiary Infotech Academy Pte Ltd

Structured troubleshooting: worked decision

1

CONSTRAINT

Latency rises after a deployment, but only for requests using one downstream dependency.

2

DECISION

Start from user impact, check recent changes, narrow by dependency, compare healthy baselines and change one variable at a time.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Random changes destroy evidence and create new failure modes.

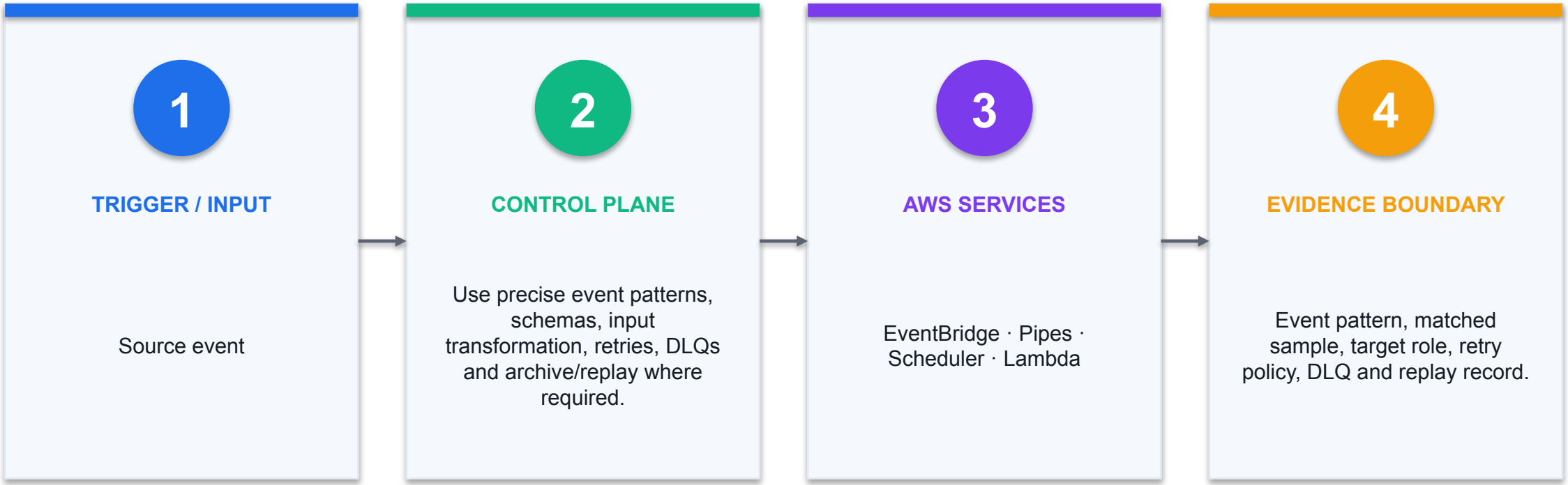
4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Impact statement, timeline, hypotheses, evidence, change record, recovery metric and root cause.

© 2026 Tertiary Infotech Academy Pte Ltd

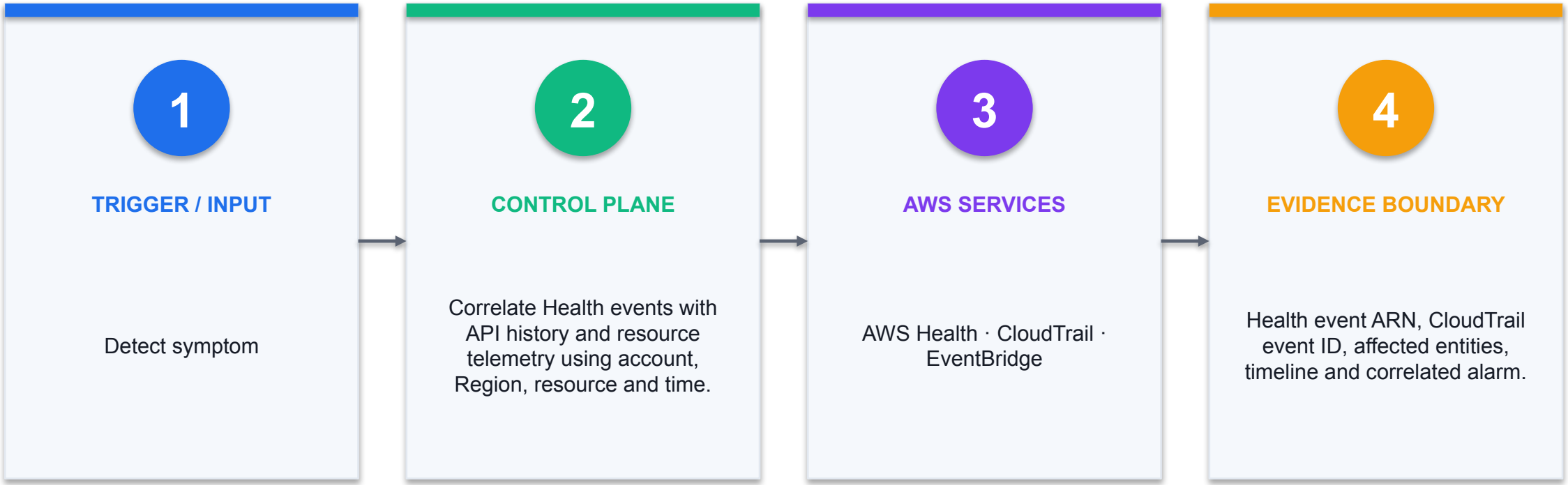
Event-driven operations: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

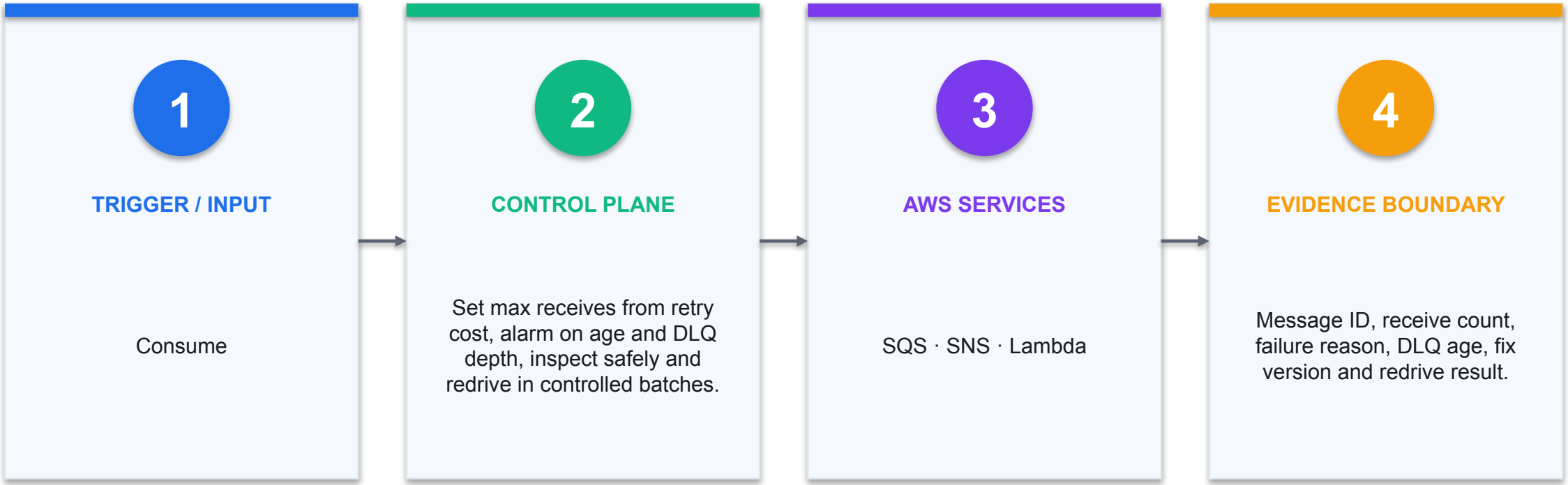
Health and audit event correlation: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

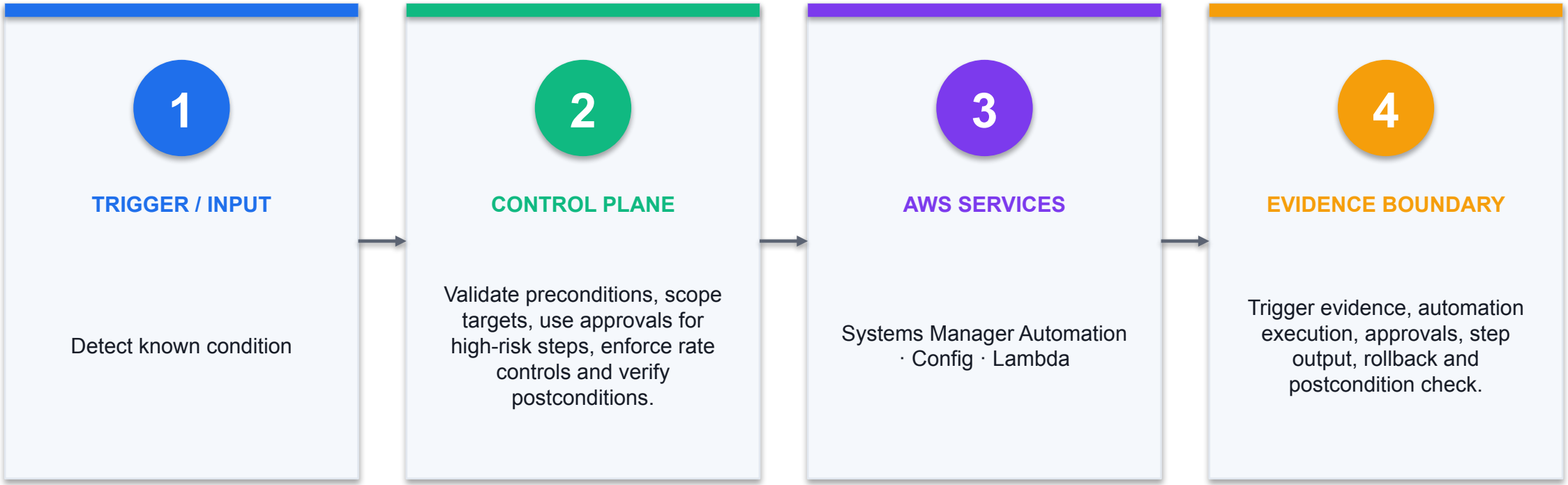
Queues, dead-letter queues and redrive: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

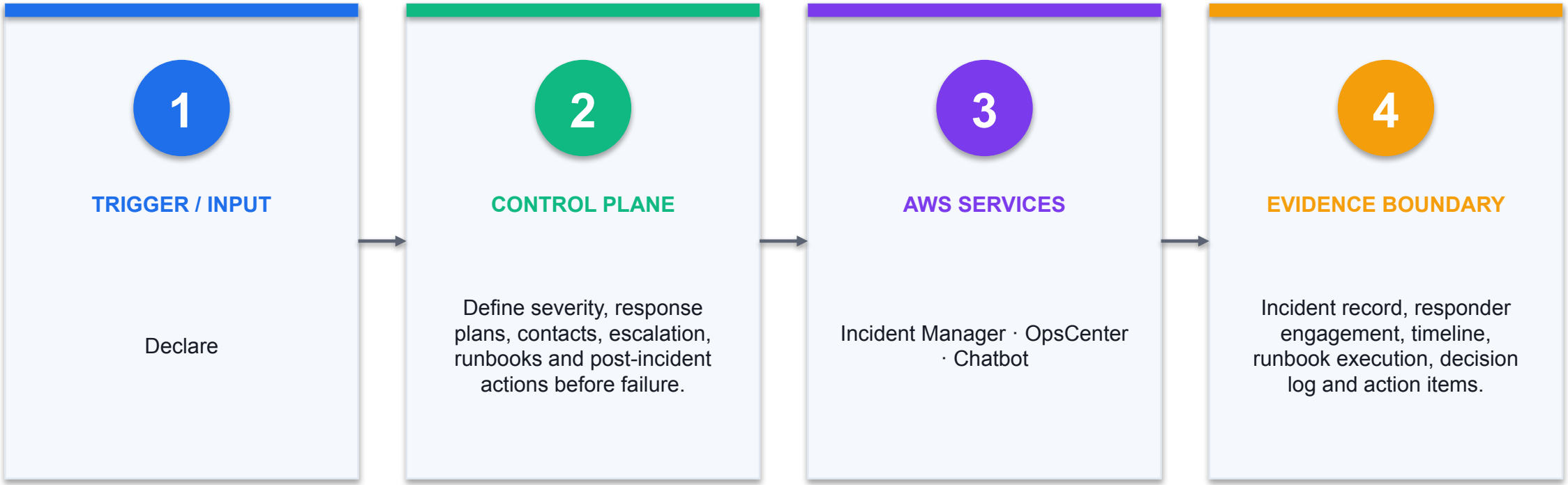
Automated remediation: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

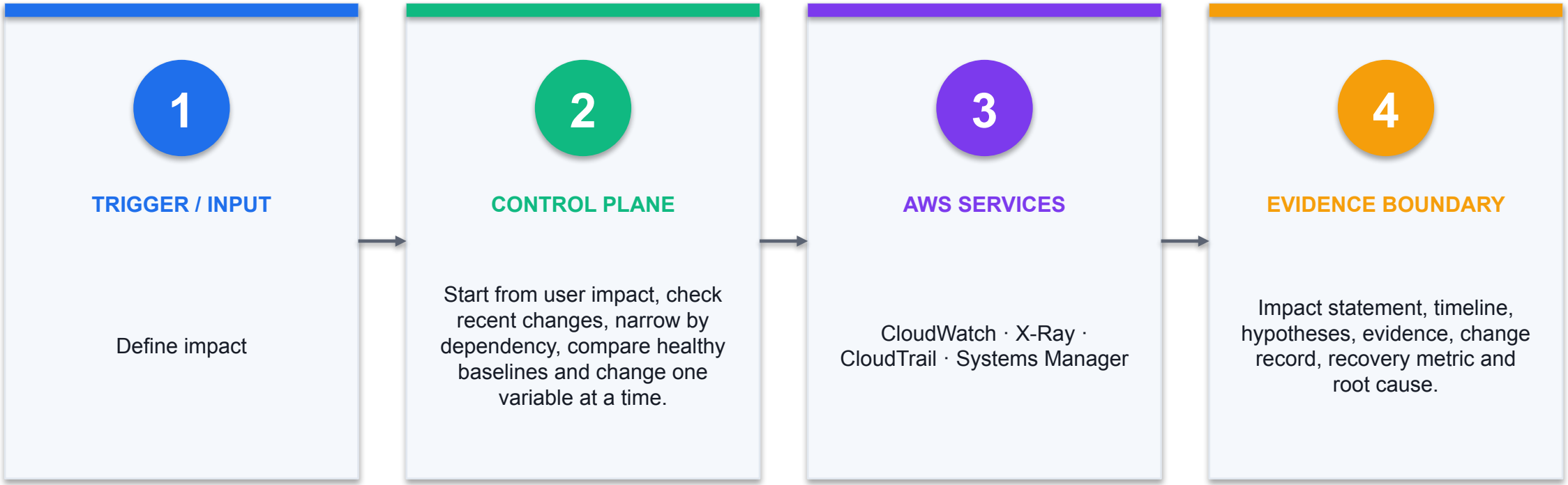
Incident coordination and runbooks: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Structured troubleshooting: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Route Incidents with Amazon EventBridge

LAB 19

Day 4 · LO5 · K7, A5

Your team wants a single 'routing brain' for operational incidents.

YOU'LL BUILD

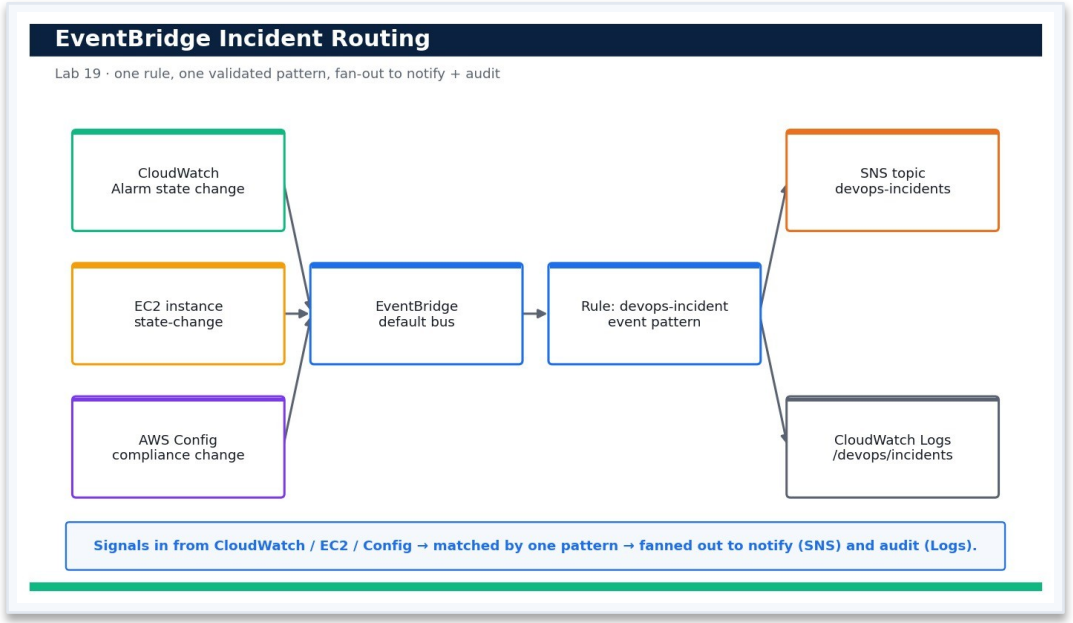
An enabled EventBridge rule on the default bus whose validated event pattern matches CloudWatch Alarm state changes, EC2 state-change notifications.

Tools: Amazon EventBridge, Amazon SNS, Amazon CloudWatch, CloudWatch Logs, IAM

OUTCOME & EVIDENCE

Outcome: Build an EventBridge rule that matches incident signals and fans them out to notify and log targets (LO5, K7, A5).

Evidence: The three results showing true / true / false for the alarm, EC2 and unrelated samples.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Automated Remediation with Lambda and Systems Manager

LAB 20

Day 4 · LO5 · K7, A5

Notification alone does not resolve an incident — the on-call engineer still has to act.

YOU'LL BUILD

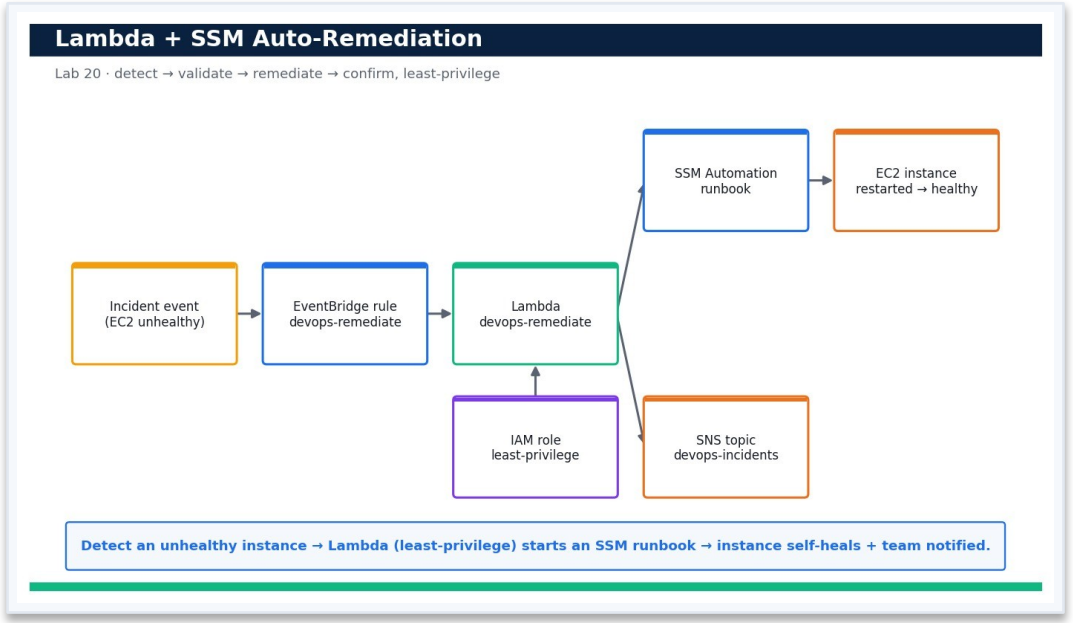
A CloudFormation-deployed auto-remediation stack (EventBridge rule + least-privilege Lambda + SSM Automation runbook + SNS notify) that, on an unhealthy-instance event, restarts the instance automatically.

Tools: AWS Lambda, AWS Systems Manager Automation, Amazon EventBridge, AWS IAM, Amazon CloudWatch Logs + 2 more

OUTCOME & EVIDENCE

Outcome: Wire an EventBridge target to a least-privilege Lambda that triggers an SSM Automation runbook so an unhealthy resource self-heals (LO5, K7, A5).

Evidence: output proving the Lambda role is least-privilege (scoped runbook + tag-conditioned EC2, no broad wildcards).



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN RECAP

Recap — Incident and Event Response

1

Lab 19

Build an EventBridge rule that matches incident signals.

2

Lab 20

Wire an EventBridge target to a least-privilege Lambda that triggers an SSM Automation runbook so an unhealthy resource.

3

Key principle

EventBridge routes events from AWS services and applications to targets that act automatically.

© 2026 Tertiary Infotech Academy Pte Ltd

06

DOMAIN 06

Security and Compliance

Exam weighting 17% · Identity & access at scale · data protection · security monitoring & audit

© 2026 Tertiary Infotech Academy Pte Ltd

Security and Compliance



- Security and compliance as continuous, automated guardrails

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 06 · WEIGHTING 17%

Key Concepts — Security and Compliance

1

Identity & access management

IAM users, roles, policies and permission boundaries enforce least privilege; roles beat long-lived keys.

2

IAM at scale

Organizations SCPs, IAM Identity Center and federation manage access across many accounts.

3

Data protection

KMS encryption, Secrets Manager, S3 Block Public Access and TLS protect data at rest and in transit.

4

Detective controls

CloudTrail, AWS Config, GuardDuty and Security Hub record, assess and score the account's posture.

5

Automated compliance

Config rules and conformance packs continuously check resources against a technical blueprint.

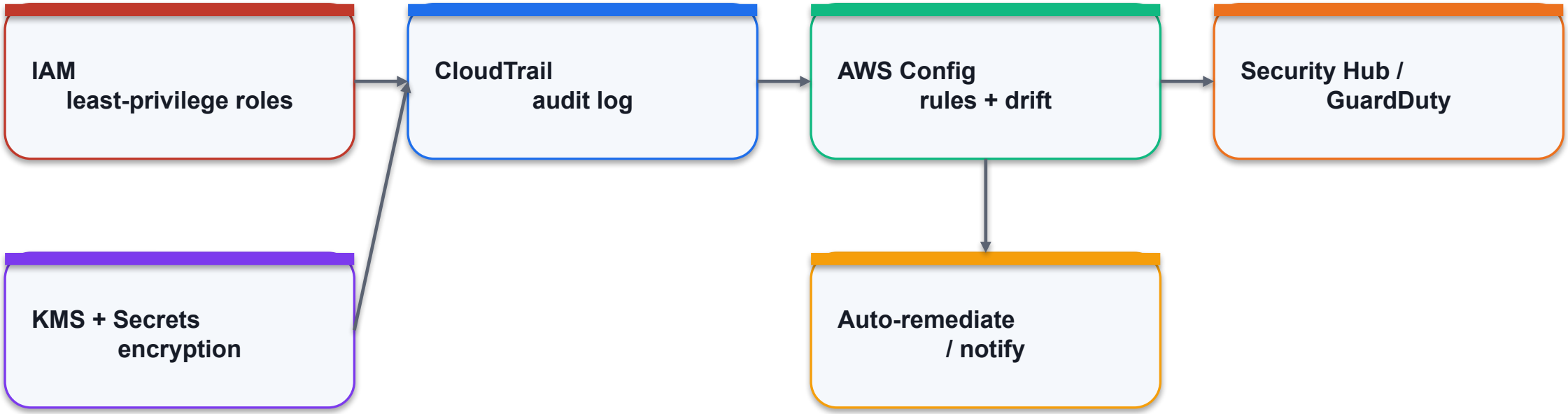
6

Governance blueprint

A landing zone plus guardrails give a documented, auditable security and compliance baseline.

© 2026 Tertiary Infotech Academy Pte Ltd

Security & Compliance Blueprint



- Least-privilege identity + encryption, with CloudTrail, Config and Security Hub providing continuous audit and scoring.

© 2026 Tertiary Infotech Academy Pte Ltd

LAYERED SECURITY

Defence in Depth on AWS

1

Identity

IAM roles, permission boundaries, IAM Identity Center, SCPs.

2

Data protection

KMS encryption, Secrets Manager, S3 Block Public Access, TLS.

3

Detective controls

CloudTrail, AWS Config, GuardDuty, Security Hub.

4

Preventive controls

Security groups, WAF, SCPs and least-privilege policies.

5

Automated compliance

Config rules and conformance packs check continuously.

6

Auditable blueprint

Landing zone + guardrails give a documented baseline.

© 2026 Tertiary Infotech Academy Pte Ltd

LEAST PRIVILEGE

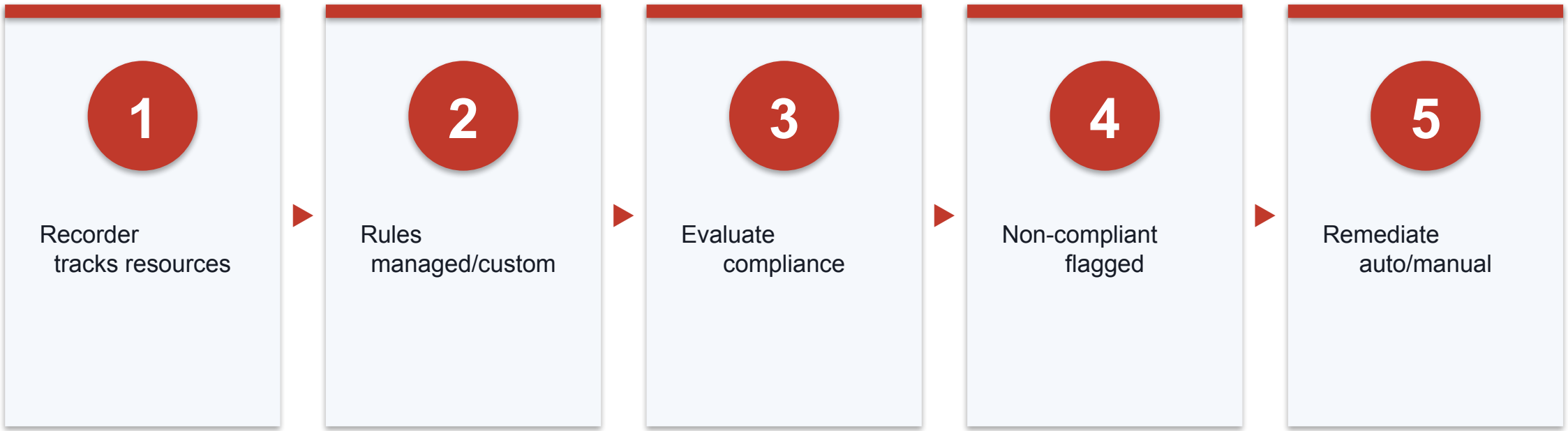
IAM — Users vs Roles vs Policies

Concept	What it is	Best practice
IAM User	Long-lived identity + credentials	Avoid; prefer federation
IAM Role	Temporary, assumable permissions	Use for apps & cross-account
Policy	JSON permissions document	Least privilege; scope by condition
Permission boundary	Max permissions a principal can have	Guardrail for delegated admin

© 2026 Tertiary Infotech Academy Pte Ltd

GUARDRAILS THAT WATCH

AWS Config — Continuous Compliance



© 2026 Tertiary Infotech Academy Pte Ltd

KNOW WHAT HAPPENED

Detective & Governance Services

Service	What it does
CloudTrail	Records every API call for audit and forensics
AWS Config	Tracks resource configuration & compliance over time
GuardDuty	ML threat detection from logs & network traffic
Security Hub	Aggregates findings & scores posture against standards

© 2026 Tertiary Infotech Academy Pte Ltd

PROTECTING DATA

Encryption Everywhere

1

At rest

KMS-managed keys for S3, EBS, RDS, DynamoDB.

2

In transit

TLS for all service and app traffic.

3

Secrets

Secrets Manager with automatic rotation.

4

Key governance

Key policies, grants and rotation with AWS KMS.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Identity and access — least privilege by design

IN DEPTH

- IAM policies grant explicit permissions; roles provide temporary, assumable credentials for apps and cross-account access.
- Permission boundaries cap the maximum permissions a delegated admin can grant.
- Prefer roles and short-lived credentials over long-lived access keys everywhere.

WHY IT MATTERS

Least privilege limits the blast radius of any compromised credential or mistaken policy to exactly what was needed.

FAILURE MODE

A wildcard 'Action: *' policy or shared long-lived keys turn one leaked secret into full account compromise.

TRADE-OFF / DECISION

Tight policies take more effort to author and maintain — Access Analyzer and policy generation reduce that cost.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Identity at scale — Identity Center, federation, SCPs

IN DEPTH

- IAM Identity Center centralises workforce access and single sign-on across many accounts.
- Federation lets your corporate identity provider issue temporary AWS access — no per-user IAM users.
- Service Control Policies set org-wide guardrails no account can exceed.

WHY IT MATTERS

Central identity plus org guardrails make access consistent, auditable and revocable across a whole organisation.

FAILURE MODE

Per-account IAM users multiply credentials to rotate and audit and inevitably drift out of least privilege.

TRADE-OFF / DECISION

Central control needs governance so it doesn't become a bottleneck for teams needing new access.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Data protection — encryption and secrets

IN DEPTH

- KMS manages encryption keys and key policies for data at rest across S3, EBS, RDS and DynamoDB; TLS protects data in transit.
- Secrets Manager stores and rotates credentials; S3 Block Public Access prevents accidental exposure.
- Envelope encryption and key policies control exactly who can use which key.

WHY IT MATTERS

Encrypting everywhere and centralising key and secret governance protect data even if another control fails.

FAILURE MODE

An unencrypted volume or a public bucket exposes data regardless of how strong the network perimeter is.

TRADE-OFF / DECISION

Customer-managed keys give control and auditability but add key-management responsibility and cost.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Detective controls — the audit trail

IN DEPTH

- CloudTrail records every API call for audit and forensics; AWS Config tracks configuration state over time.
- GuardDuty applies ML threat detection to logs and network flows; Security Hub aggregates and scores findings.
- Together they answer 'what happened, when, by whom, and are we compliant?'

WHY IT MATTERS

Detective controls provide the evidence for incident forensics, audits and continuous compliance scoring.

FAILURE MODE

No CloudTrail (or a trail without log-file validation) means an incident can't be reconstructed and audits fail.

TRADE-OFF / DECISION

Full logging and threat detection cost money and generate findings to triage — tune and prioritise by severity.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Automated compliance — Config rules and conformance packs

IN DEPTH

- Config rules continuously evaluate resources against a technical blueprint (managed or custom rules).
- Conformance packs bundle rules and remediations to a standard (e.g. CIS, PCI) and deploy org-wide.
- Non-compliant resources are flagged and can be auto-remediated.

WHY IT MATTERS

Continuous, codified compliance replaces point-in-time audits with always-on assurance and evidence.

FAILURE MODE

Manual quarterly audits miss the days-to-weeks a resource sits non-compliant between checks.

TRADE-OFF / DECISION

Strict conformance packs improve posture but can generate noise — phase in rules and remediations.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Preventive controls and defence in depth

IN DEPTH

- Security groups and network ACLs, WAF, and least-privilege policies stop bad requests before they reach resources.
- SCPs prevent whole classes of action org-wide; VPC design limits lateral movement.
- Layered controls mean no single failure exposes the system.

WHY IT MATTERS

Defence in depth ensures that when one layer fails — and layers do fail — others still protect the data.

FAILURE MODE

Relying on a single perimeter (a firewall) fails completely the moment that one control is bypassed.

TRADE-OFF / DECISION

More layers add complexity and can impede legitimate traffic — design them to be observable and testable.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Governance blueprint — landing zone and guardrails

IN DEPTH

- A landing zone (via Control Tower) gives a documented, auditable security and network baseline.
- Preventive guardrails (SCPs) and detective guardrails (Config rules) are applied to every account.
- New accounts inherit the baseline automatically through account vending.

WHY IT MATTERS

A governance blueprint makes 'secure and compliant' the default state of every account, not a per-project effort.

FAILURE MODE

Ad-hoc, per-project security guarantees nothing is consistent and every audit re-discovers the same gaps.

TRADE-OFF / DECISION

A central blueprint needs a platform team and change process — the payoff is org-wide, provable compliance.

© 2026 Tertiary Infotech Academy Pte Ltd

IN DEPTH

Securing the pipeline itself (DevSecOps)

IN DEPTH

- Scan source and dependencies (SAST/SCA) and infrastructure templates in the build; sign and verify artifacts.
- Scope pipeline and deployment roles to least privilege and isolate environments by account.
- Store no secrets in code; pull them at deploy time from Secrets Manager/SSM.

WHY IT MATTERS

The pipeline has powerful credentials and touches production — a compromised pipeline is a compromised environment.

FAILURE MODE

A pipeline role with broad permissions and secrets in the repo is a direct path from a code leak to full compromise.

TRADE-OFF / DECISION

Security gates add build time — order them fast-to-slow and fail fast to keep the pipeline usable.

© 2026 Tertiary Infotech Academy Pte Ltd

LEAST PRIVILEGE AT SCALE

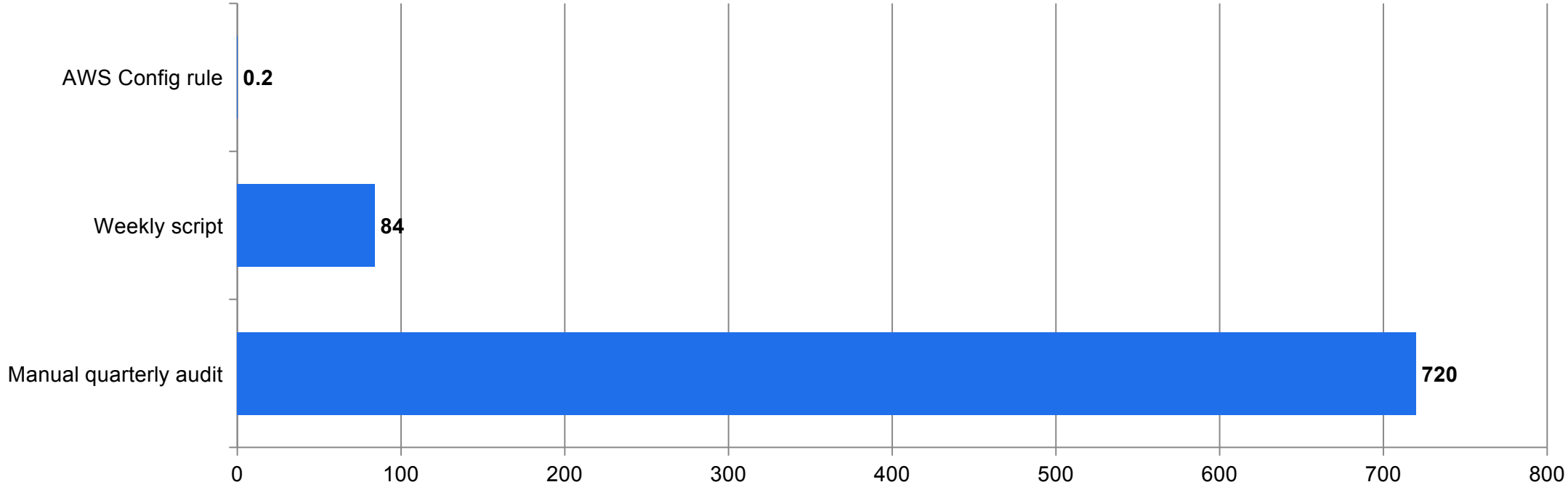
IAM — Users vs Roles vs Federation

Identity	What it is	Best practice
IAM user + keys	Long-lived credential	Avoid; rotate/retire, prefer roles
IAM role	Temporary, assumable	Use for apps and cross-account
Federation / Identity Center	IdP issues short-lived access	Standard for workforce access
Permission boundary	Cap on granted permissions	Guardrail for delegated admins

© 2026 Tertiary Infotech Academy Pte Ltd

CONTINUOUS COMPLIANCE WINS

Time-to-Detect a Public Bucket — Manual vs Continuous



WHAT THE DATA SHOWS

Continuous Config evaluation shrinks the exposure window from months to minutes — the difference between a finding and a breach.

© 2026 Tertiary Infotech Academy Pte Ltd

HOW ACCOUNTS GET COMPROMISED

Security Failure Modes & Anti-Patterns

1

Wildcard IAM policies

One leaked key = full access → least privilege + Access Analyzer.

2

Long-lived access keys

Hard to rotate/audit → roles and federation.

3

Public S3 / open SGs

Accidental exposure → Block Public Access, Config rules.

4

No CloudTrail / validation

Incidents unauditable → org trail + log-file validation.

5

Secrets in code

Leak on repo access → Secrets Manager/SSM.

6

Over-privileged pipeline

Code leak → prod compromise → scope deploy roles.

© 2026 Tertiary Infotech Academy Pte Ltd

PROVEN PRACTICES

Security Patterns Worth Adopting

1. **Least privilege everywhere**
   Roles, boundaries, scoped policies, no wildcards.

2. **Encrypt at rest and in transit**
   KMS + TLS by default on every store.

3. **Continuous compliance**
   Config rules + conformance packs, auto-remediate.

4. **Central logging account**
   Immutable CloudTrail + Security Hub aggregation.

5. **Guardrails by SCP**
   Prevent whole action classes org-wide.

6. **DevSecOps**
   Scan code, deps and IaC in the pipeline.

© 2026 Tertiary Infotech Academy Pte Ltd

PREVENT, DETECT, REMEDIATE

Architecture Decision — Where Do Compliance Controls Live?

A — Preventive (SCP / policy)

Stop the action from ever happening org-wide. Strongest — but blunt; can block legitimate edge cases.

B — Detective (Config / GuardDuty)

Allow, but continuously detect and score. Flexible and auditable — leaves a (short) exposure window.

C — Automated remediation

Detect then auto-fix (Config → SSM). Closes the window fast — needs careful scoping and logging.

© 2026 Tertiary Infotech Academy Pte Ltd

EVIDENCE ON DEMAND

Scenario — Passing an external security audit

SITUATION

An auditor asks you to prove that all volumes are encrypted and no bucket is public, historically and now.

ANALYSIS

- Point-in-time console screenshots don't prove continuous compliance over the audit period.
- AWS Config records configuration history and rule-compliance over time — that history is the evidence.
- CloudTrail with log-file validation proves the audit record itself wasn't tampered with.

DECISION

Present Config compliance history and conformance-pack results as continuous evidence, backed by a validated CloudTrail — turning audit from a scramble into a query.

© 2026 Tertiary Infotech Academy Pte Ltd

LEAST PRIVILEGE IN PRACTICE

Scenario — An app needs one new permission

SITUATION

A service fails with AccessDenied writing to a new S3 prefix. The quick fix is to grant s3:* on all buckets.

ANALYSIS

- A wildcard grant fixes the error but massively widens blast radius if the role is ever compromised.
- IAM Access Analyzer can generate a policy scoped to exactly the actions and resources the app used.
- The correct grant is s3:PutObject on that specific bucket/prefix, nothing more.

DECISION

Grant the single scoped permission the app actually needs (not a wildcard), using Access Analyzer to right-size the policy — keeping least privilege intact under time pressure.

© 2026 Tertiary Infotech Academy Pte Ltd

Worked Analysis — Building a Compliance Blueprint

Requirement: encrypt all data at rest	Config rule encrypted-volumes + S3 default encryption; KMS keys with scoped policies.
Requirement: no public data	S3 Block Public Access org-wide + s3-bucket-public-read-prohibited rule.
Requirement: full audit trail	Org CloudTrail to an encrypted, access-controlled logging account with validation.
Requirement: least-privilege identity	Roles + Identity Center + permission boundaries + Access Analyzer reviews.

TAKEAWAY

A compliance blueprint maps each written requirement to a concrete, testable AWS control — so 'we are compliant' is something you can prove, continuously.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Identity and temporary access

1

PURPOSE

Grant human and workload access through least privilege, federation and short-lived sessions.

2

FEATURE 1

Identity policies, resource policies, permission boundaries and SCPs intersect to determine effective access.

3

FEATURE 2

STS issues temporary credentials for roles and federation.

4

FEATURE 3

IAM Identity Center assigns workforce access across accounts/applications using permission sets.

© 2026 Tertiary Infotech Academy Pte Ltd

Identity and temporary access: selection and controls

1

CHOOSE WHEN

Use roles and federation for humans/workloads; use resource policies for cross-principal resource access where supported.

2

DEVOPS ENGINEERING

Constrain trust policies, require context such as ExternalId/tags where appropriate, analyse access and remove unused permission.

3

FAILURE MODE

An Allow can still be blocked by an explicit deny, permission boundary, session policy or SCP.

4

EVIDENCE

Principal/session, evaluated policy layers, Access Analyzer finding, CloudTrail event and last-accessed data.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Encryption and secrets

1

PURPOSE

Protect data and credentials through governed keys, rotation and workload-time retrieval.

2

FEATURE 1

KMS keys enforce cryptographic permissions through key and IAM policies and emit CloudTrail evidence.

3

FEATURE 2

Secrets Manager stores/rotates secrets and versions; Parameter Store manages hierarchical parameters and SecureString values.

4

FEATURE 3

ACM provisions and renews supported public/private certificates for integrated services.

© 2026 Tertiary Infotech Academy Pte Ltd

Encryption and secrets: selection and controls

1

CHOOSE WHEN

Use Secrets Manager for rotating credentials, Parameter Store for configuration/parameters and KMS for control of encryption keys.

2

DEVOPS ENGINEERING

Use envelope encryption, scope decrypt by key/context, retrieve at runtime, rotate and never log secret values.

3

FAILURE MODE

Encrypting data does not help if broad principals can decrypt it or plaintext is copied into pipeline logs.

4

EVIDENCE

Key policy/grant, encryption context, rotation/version stage, access event and failed-decrypt alert.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Preventive governance

1

PURPOSE

Set permission ceilings and mandatory account controls while leaving workload teams clear deployment paths.

2

FEATURE 1

SCPs limit available permissions but do not grant permissions.

3

FEATURE 2

Control Tower preventive/proactive/detective controls apply managed governance patterns.

4

FEATURE 3

Config rules evaluate resource configuration and can trigger scoped remediation.

© 2026 Tertiary Infotech Academy Pte Ltd

Preventive governance: selection and controls

1

CHOOSE WHEN

Use preventive controls for actions that must never occur and detective controls where context or exception handling is required.

2

DEVOPS ENGINEERING

Test in a canary OU, protect break-glass paths, document exemptions and monitor policy/control drift.

3

FAILURE MODE

A deny deployed at the root without testing can block security and recovery operations across all accounts.

4

EVIDENCE

Policy version, target hierarchy, simulation/test, exception, Config evaluation and remediation result.

© 2026 Tertiary Infotech Academy Pte Ltd

AWS foundations: Threat detection and edge protection

1

PURPOSE

Detect suspicious activity, vulnerabilities and sensitive-data exposure, then apply consistent edge/network protections.

2

FEATURE 1

GuardDuty analyses AWS data sources for threats; Inspector scans supported compute/container functions for vulnerabilities.

3

FEATURE 2

Macie discovers sensitive data in S3; Security Hub normalises/aggregates findings and workflow status.

4

FEATURE 3

WAF filters Layer 7 requests, Shield mitigates DDoS, and Firewall Manager applies policies across accounts.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 6 · SERVICE DECISION

Threat detection and edge protection: selection and controls

1

CHOOSE WHEN

Select the control from threat surface: identity/network behaviour, vulnerable software, sensitive data, application requests or organization-wide policy.

2

DEVOPS ENGINEERING

Delegate administration, tune findings/rules, enrich ownership, automate only well-understood containment and preserve evidence.

3

FAILURE MODE

Enabling every managed rule in block mode without observation can create a security-induced outage.

4

EVIDENCE

Finding ID/severity, affected resource, rule sample, workflow status, owner, containment and closure evidence.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: IAM, STS and Cognito authorization boundaries

1

PURPOSE

Separate workforce, workload and customer identity while reasoning through every policy layer that shapes effective permission.

2

SERVICE BOUNDARY

IAM · STS · IAM Identity Center · Cognito

3

PRIMARY DESIGN DECISION

Create a customer role with narrow S3 permissions, a vendor principal in the trust policy and a unique ExternalId condition; log every assumed session.

4

FAILURE BOUNDARY

Granting the vendor identity directly across resources is hard to revoke and audit; a trust policy alone grants no S3 action.

© 2026 Tertiary Infotech Academy Pte Ltd

IAM, STS and Cognito authorization boundaries: how it works

1

MECHANIC 1

IAM evaluation combines identity and resource policies, with explicit denies, permission boundaries, session policies and SCPs limiting the final result.

2

MECHANIC 2

A role trust policy controls who may assume the role; the role permission policy controls what the resulting session may do.

3

MECHANIC 3

STS sessions are temporary and can carry session tags for attribute-based access; ExternalId helps mitigate confused-deputy risk in third-party access.

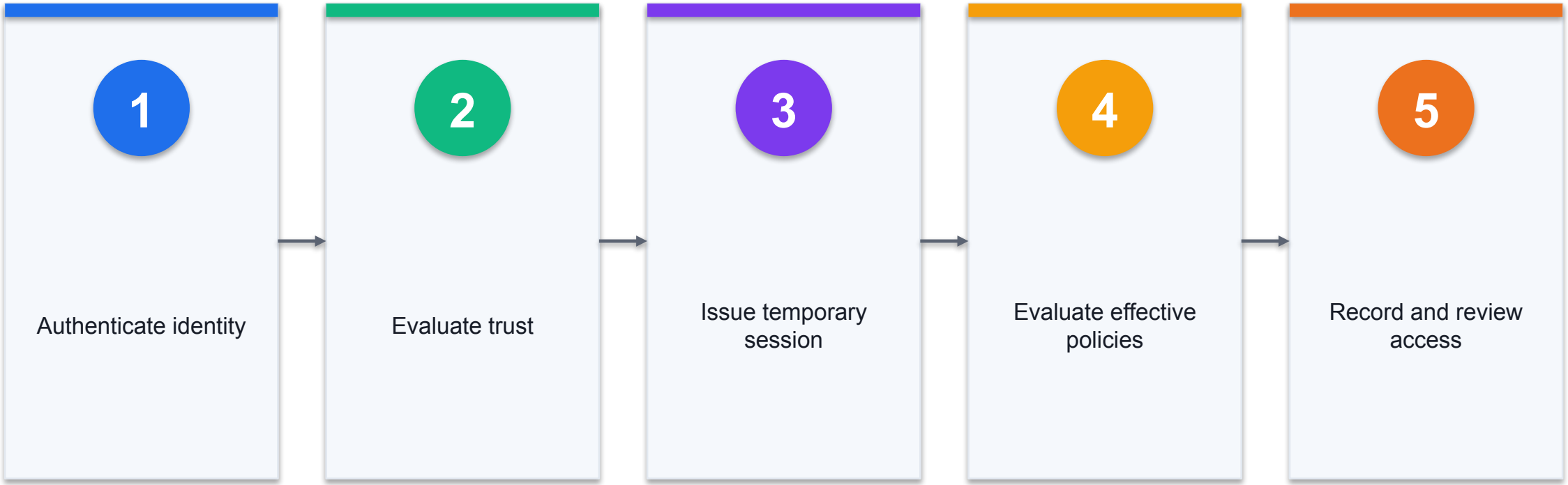
4

MECHANIC 4

Cognito user pools authenticate application users, while identity pools exchange supported identities for scoped AWS credentials.

© 2026 Tertiary Infotech Academy Pte Ltd

IAM, STS and Cognito authorization boundaries: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

IAM, STS and Cognito authorization boundaries: configuration decisions

1

WORKFORCE

Use IAM Identity Center and account permission sets for centrally managed employee access.

2

WORKLOAD

Attach roles to compute and restrict trust to the intended service, account and conditions.

3

CUSTOMER

Use user pools for application authentication and identity pools only when clients need scoped AWS credentials.

4

ANALYSIS

Use policy simulation, Access Analyzer and last-accessed data to validate and reduce permissions.

© 2026 Tertiary Infotech Academy Pte Ltd

IAM, STS and Cognito authorization boundaries: worked architecture decision

1

SCENARIO

A SaaS vendor needs read access to one customer prefix without receiving reusable customer credentials.

2

DESIGN DECISION

Create a customer role with narrow S3 permissions, a vendor principal in the trust policy and a unique ExternalId condition; log every assumed session.

3

FAILURE TO AVOID

Granting the vendor identity directly across resources is hard to revoke and audit; a trust policy alone grants no S3 action.

4

ACCEPTANCE EVIDENCE

Trust policy, permission policy, ExternalId control, session ARN/tags, Access Analyzer result and CloudTrail AssumeRole event.

© 2026 Tertiary Infotech Academy Pte Ltd

Developer deep dive: KMS, secrets and application-edge protection

1

PURPOSE

Protect data, credentials and public application paths with controls whose permissions, rotation and failure behavior are observable.

2

SERVICE BOUNDARY

KMS · Secrets Manager · ACM · WAF · Shield

3

PRIMARY DESIGN DECISION

Use staged secret rotation with connection refresh, encrypt under a scoped key, deploy WAF in count mode first and promote the validated rule to block.

4

FAILURE BOUNDARY

A key policy that excludes recovery administrators can lock out data; immediate blocking can create a security-induced outage.

© 2026 Tertiary Infotech Academy Pte Ltd

KMS, secrets and application-edge protection: how it works

1

MECHANIC 1

KMS performs cryptographic operations under key and IAM policies; envelope encryption keeps large data operations outside KMS while protecting data keys.

2

MECHANIC 2

Encryption context can bind decrypt permission to non-secret request attributes and is recorded with supported audit events.

3

MECHANIC 3

Secrets Manager stores versioned secret values with staging labels and can automate rotation; applications should cache retrieval and handle rotation overlap.

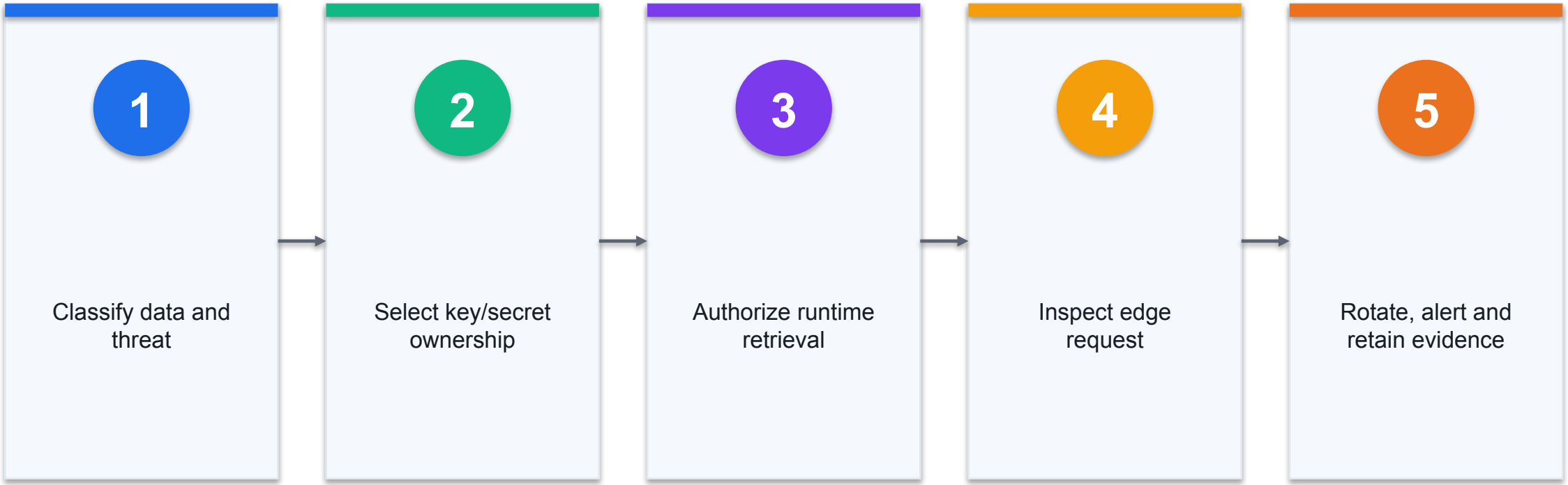
4

MECHANIC 4

ACM manages supported certificates, WAF evaluates Layer 7 rules and Shield provides DDoS protections; Firewall Manager applies policies across governed accounts.

© 2026 Tertiary Infotech Academy Pte Ltd

KMS, secrets and application-edge protection: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

KMS, secrets and application-edge protection: configuration decisions

1

KEY OWNERSHIP

Choose AWS-owned, AWS-managed or customer-managed keys from policy, separation, rotation and cross-account requirements.

2

GRANTS

Use grants for delegated service use where appropriate and retire them with the consuming resource lifecycle.

3

ROTATION

Support overlapping credential versions and test both rotation and emergency revocation paths.

4

WAF ROLLOUT

Observe managed rules in count mode, inspect sampled requests and introduce blocking with exception ownership.

© 2026 Tertiary Infotech Academy Pte Ltd

KMS, secrets and application-edge protection: worked architecture decision

1

SCENARIO

A public API must rotate its database secret without downtime and block a new malicious request pattern safely.

2

DESIGN DECISION

Use staged secret rotation with connection refresh, encrypt under a scoped key, deploy WAF in count mode first and promote the validated rule to block.

3

FAILURE TO AVOID

A key policy that excludes recovery administrators can lock out data; immediate blocking can create a security-induced outage.

4

ACCEPTANCE EVIDENCE

Key policy/grant, encryption-context event, secret version stages, rotation result, WAF samples, blocked-rate metric and exception record.

© 2026 Tertiary Infotech Academy Pte Ltd

Identity at scale

1

OUTCOME

Provide workforce and workload access through federation, short-lived credentials and least privilege.

2

ARCHITECTURE

Use permission sets, roles, session controls and access analysis; avoid long-lived IAM user keys.

3

FAILURE MODE

Broad shared roles erase accountability and expand blast radius.

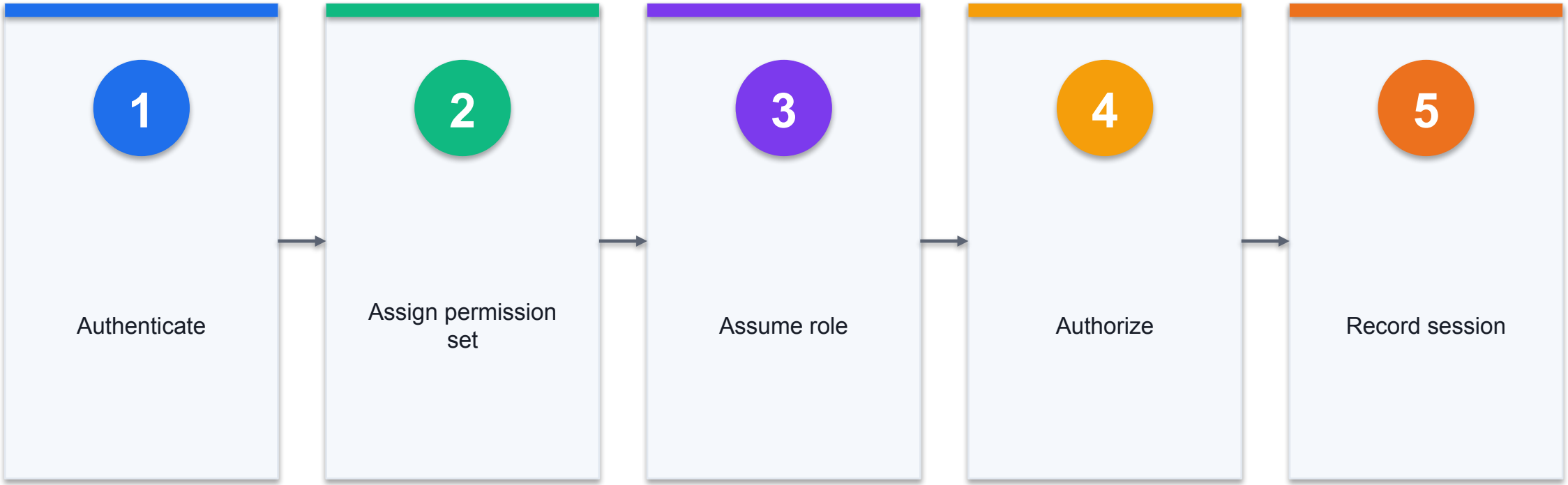
4

EVIDENCE

Identity source, permission-set version, role session, access finding and CloudTrail principal.

© 2026 Tertiary Infotech Academy Pte Ltd

Identity at scale: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Identity at scale: design decision

? SCENARIO

Engineers need auditable access to development and production accounts with different privileges.

1 DESIGN RESPONSE

Use permission sets, roles, session controls and access analysis; avoid long-lived IAM user keys.

2 PRIMARY SERVICES

IAM Identity Center · IAM · STS

3 FAILURE MODE

Broad shared roles erase accountability and expand blast radius.

4 PROOF

Identity source, permission-set version, role session, access finding and CloudTrail principal.

© 2026 Tertiary Infotech Academy Pte Ltd

Identity at scale: failure diagnostic

1

RECOGNISE

Observable failure condition: Broad shared roles erase accountability and expand blast radius.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Authenticate through Record session; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Identity source, permission-set version, role session, access finding and CloudTrail principal.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 6 · APPLIED SCENARIO

Identity at scale: worked decision

1

CONSTRAINT

Engineers need auditable access to development and production accounts with different privileges.

2

DECISION

Use permission sets, roles, session controls and access analysis; avoid long-lived IAM user keys.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Broad shared roles erase accountability and expand blast radius.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Identity source, permission-set version, role session, access finding and CloudTrail principal.

© 2026 Tertiary Infotech Academy Pte Ltd

Organizations and preventive guardrails

1

OUTCOME

Set organization-wide permission boundaries while preserving account autonomy inside approved limits.

2

ARCHITECTURE

Design OU structure from governance needs, test SCP impact, use deny guardrails carefully and keep break-glass access.

3

FAILURE MODE

An SCP does not grant permission; it limits what identity and resource policies may allow.

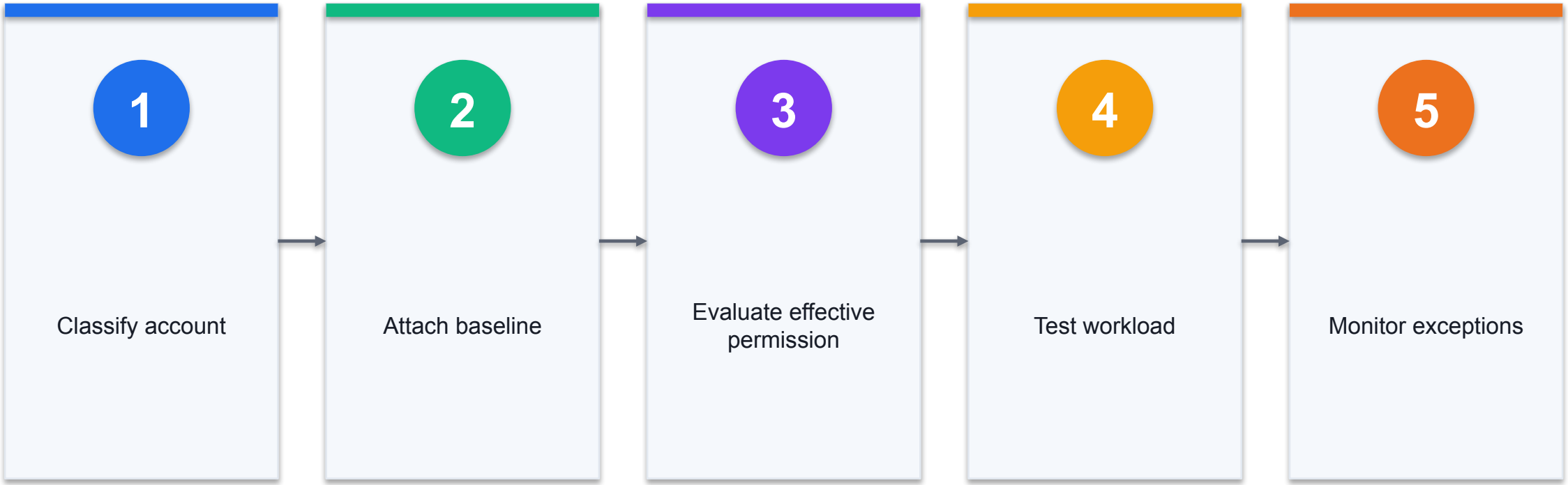
4

EVIDENCE

OU hierarchy, SCP version, policy simulator result, exception and affected-account inventory.

© 2026 Tertiary Infotech Academy Pte Ltd

Organizations and preventive guardrails: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Organizations and preventive guardrails: design decision

? SCENARIO

Production accounts must never disable CloudTrail or create unencrypted storage.

1 DESIGN RESPONSE

Design OU structure from governance needs, test SCP impact, use deny guardrails carefully and keep break-glass access.

2 PRIMARY SERVICES

Organizations · SCPs · Control Tower

3 FAILURE MODE

An SCP does not grant permission; it limits what identity and resource policies may allow.

4 PROOF

OU hierarchy, SCP version, policy simulator result, exception and affected-account inventory.

© 2026 Tertiary Infotech Academy Pte Ltd

Organizations and preventive guardrails: failure diagnostic

1

RECOGNISE

Observable failure condition: An SCP does not grant permission; it limits what identity and resource policies may allow.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Classify account through Monitor exceptions; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with OU hierarchy, SCP version, policy simulator result, exception and affected-account inventory.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 6 · APPLIED SCENARIO

Organizations and preventive guardrails: worked decision

1

CONSTRAINT

Production accounts must never disable CloudTrail or create unencrypted storage.

2

DECISION

Design OU structure from governance needs, test SCP impact, use deny guardrails carefully and keep break-glass access.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: An SCP does not grant permission; it limits what identity and resource policies may allow.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: OU hierarchy, SCP version, policy simulator result, exception and affected-account inventory.

© 2026 Tertiary Infotech Academy Pte Ltd

Secrets, keys and data protection

1

OUTCOME

Protect data and credentials through encryption, controlled key use and automated rotation.

2

ARCHITECTURE

Separate key administration from use, scope resource policies, rotate secrets safely and plan multi-Region recovery.

3

FAILURE MODE

Encrypting with a key that the recovery account cannot use makes backups operationally unavailable.

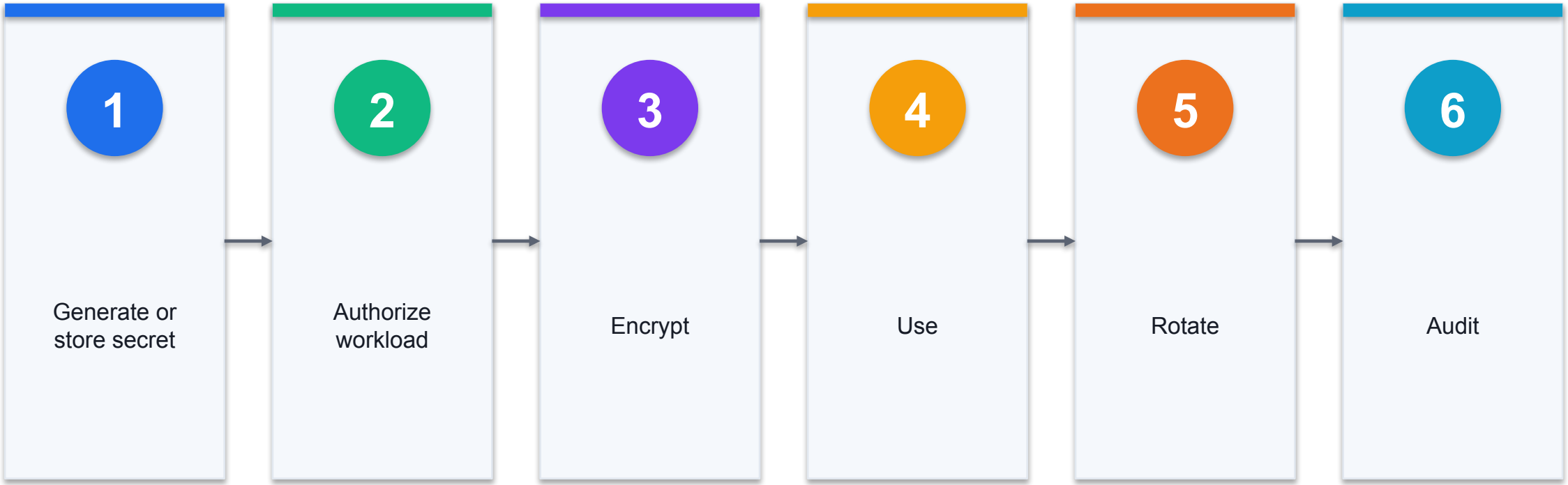
4

EVIDENCE

Key policy, grant, encryption context, secret version stages, rotation result and access trail.

© 2026 Tertiary Infotech Academy Pte Ltd

Secrets, keys and data protection: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Secrets, keys and data protection: design decision

? SCENARIO

A database credential must rotate without downtime and remain recoverable in the DR Region.

1 DESIGN RESPONSE

Separate key administration from use, scope resource policies, rotate secrets safely and plan multi-Region recovery.

2 PRIMARY SERVICES

KMS · Secrets Manager · Parameter Store · ACM

3 FAILURE MODE

Encrypting with a key that the recovery account cannot use makes backups operationally unavailable.

4 PROOF

Key policy, grant, encryption context, secret version stages, rotation result and access trail.

© 2026 Tertiary Infotech Academy Pte Ltd

Secrets, keys and data protection: failure diagnostic

1

RECOGNISE

Observable failure condition: Encrypting with a key that the recovery account cannot use makes backups operationally unavailable.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Generate or store secret through Audit; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Key policy, grant, encryption context, secret version stages, rotation result and access trail.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 6 · APPLIED SCENARIO

Secrets, keys and data protection: worked decision

1

CONSTRAINT

A database credential must rotate without downtime and remain recoverable in the DR Region.

2

DECISION

Separate key administration from use, scope resource policies, rotate secrets safely and plan multi-Region recovery.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Encrypting with a key that the recovery account cannot use makes backups operationally unavailable.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Key policy, grant, encryption context, secret version stages, rotation result and access trail.

© 2026 Tertiary Infotech Academy Pte Ltd

Config rules and conformance

1

OUTCOME

Continuously evaluate resource configuration against explicit controls and remediate safely.

2

ARCHITECTURE

Use managed/custom rules, aggregators, evidence retention and remediation with exception tagging.

3

FAILURE MODE

A compliant snapshot is not proof that the resource stayed compliant between evaluations.

4

EVIDENCE

Configuration timeline, rule evaluation, conformance score, exception, remediation execution and recheck.

© 2026 Tertiary Infotech Academy Pte Ltd

Config rules and conformance: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Config rules and conformance: design decision

? SCENARIO

Auditors require evidence that all production buckets remained encrypted and blocked public access.

1 DESIGN RESPONSE

Use managed/custom rules, aggregators, evidence retention and remediation with exception tagging.

2 PRIMARY SERVICES

AWS Config · Conformance Packs · Systems Manager

3 FAILURE MODE

A compliant snapshot is not proof that the resource stayed compliant between evaluations.

4 PROOF

Configuration timeline, rule evaluation, conformance score, exception, remediation execution and recheck.

© 2026 Tertiary Infotech Academy Pte Ltd

Config rules and conformance: failure diagnostic

1

RECOGNISE

Observable failure condition: A compliant snapshot is not proof that the resource stayed compliant between evaluations.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Record configuration through Re-evaluate; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Configuration timeline, rule evaluation, conformance score, exception, remediation execution and recheck.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 6 · APPLIED SCENARIO

Config rules and conformance: worked decision

1

CONSTRAINT

Auditors require evidence that all production buckets remained encrypted and blocked public access.

2

DECISION

Use managed/custom rules, aggregators, evidence retention and remediation with exception tagging.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: A compliant snapshot is not proof that the resource stayed compliant between evaluations.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Configuration timeline, rule evaluation, conformance score, exception, remediation execution and recheck.

© 2026 Tertiary Infotech Academy Pte Ltd

Threat detection and investigation

1

OUTCOME

Combine findings, vulnerability context and behavior graphs to prioritize credible threats.

2

ARCHITECTURE

Delegate administration, centralize findings, normalize severity, enrich with ownership and automate only well-understood responses.

3

FAILURE MODE

A high volume of unowned findings becomes an unworkable compliance queue.

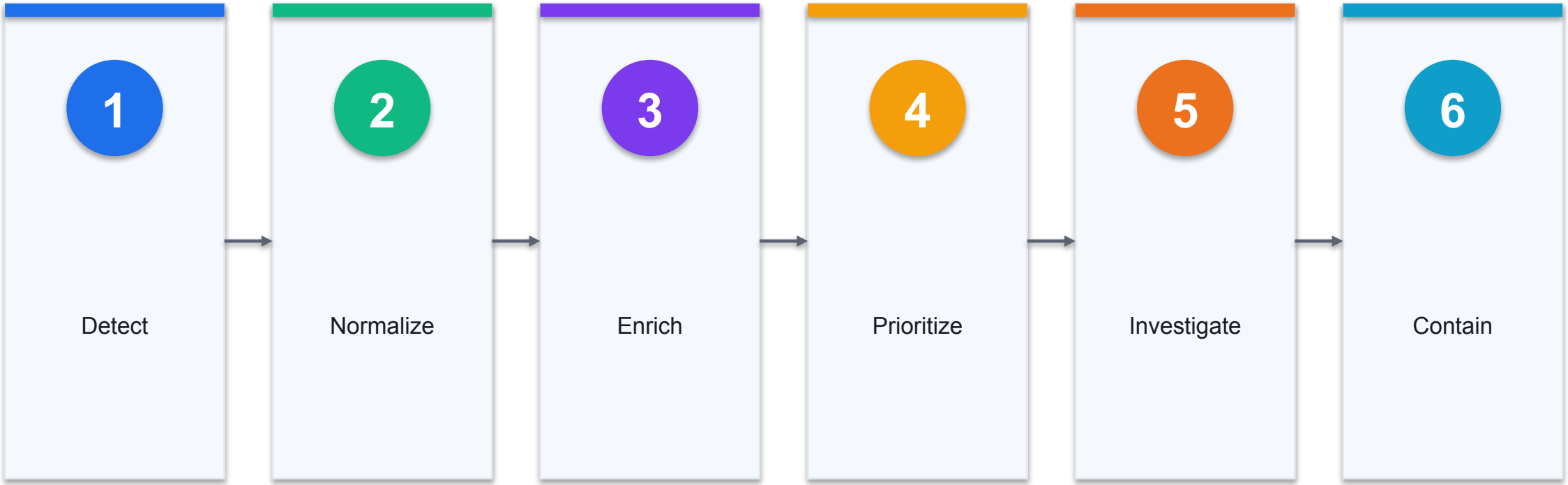
4

EVIDENCE

Finding ID, affected resource, severity, vulnerability, graph evidence, workflow status and response.

© 2026 Tertiary Infotech Academy Pte Ltd

Threat detection and investigation: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Threat detection and investigation: design decision

? SCENARIO

An exposed instance shows anomalous network activity and a critical package vulnerability.

1 DESIGN RESPONSE

Delegate administration, centralize findings, normalize severity, enrich with ownership and automate only well-understood responses.

2 PRIMARY SERVICES

GuardDuty · Security Hub · Detective · Inspector

3 FAILURE MODE

A high volume of unowned findings becomes an unworkable compliance queue.

4 PROOF

Finding ID, affected resource, severity, vulnerability, graph evidence, workflow status and response.

© 2026 Tertiary Infotech Academy Pte Ltd

Threat detection and investigation: failure diagnostic

1

RECOGNISE

Observable failure condition: A high volume of unowned findings becomes an unworkable compliance queue.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Detect through Contain; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Finding ID, affected resource, severity, vulnerability, graph evidence, workflow status and response.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 6 · APPLIED SCENARIO

Threat detection and investigation: worked decision

1

CONSTRAINT

An exposed instance shows anomalous network activity and a critical package vulnerability.

2

DECISION

Delegate administration, centralize findings, normalize severity, enrich with ownership and automate only well-understood responses.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: A high volume of unowned findings becomes an unworkable compliance queue.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Finding ID, affected resource, severity, vulnerability, graph evidence, workflow status and response.

© 2026 Tertiary Infotech Academy Pte Ltd

Edge and network protection

1

OUTCOME

Apply consistent layered protection to public endpoints and network paths.

2

ARCHITECTURE

Use managed rules in count mode first, central policies, rate limits, logging and exception workflows.

3

FAILURE MODE

Blocking without observing false positives can turn a security control into an outage.

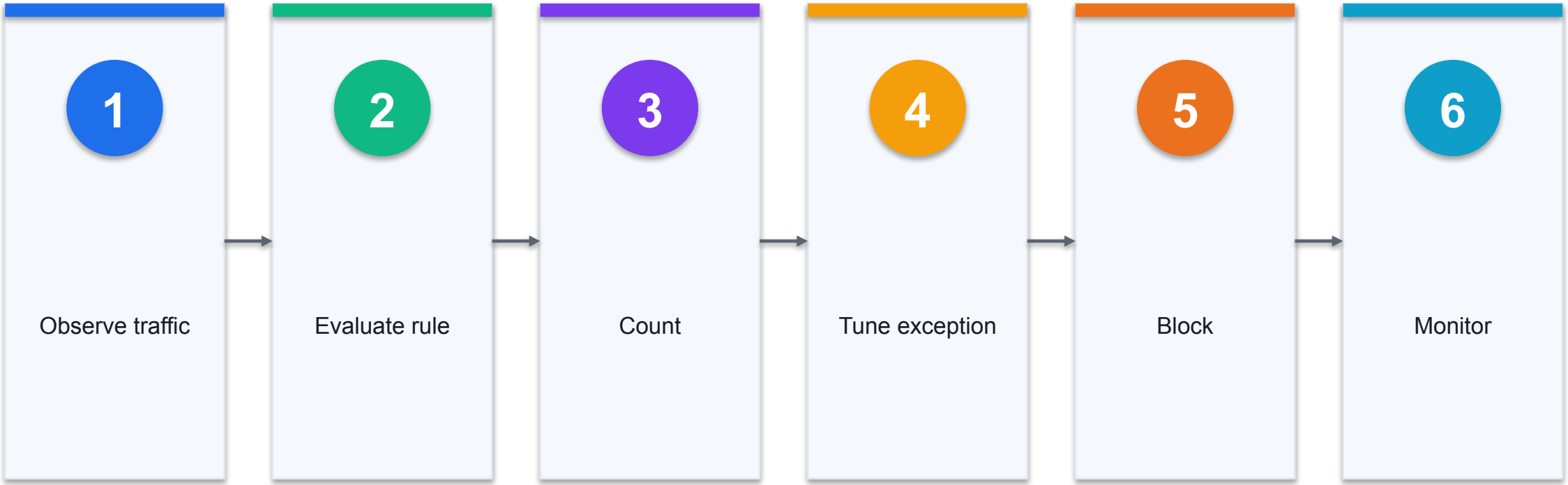
4

EVIDENCE

Policy version, sampled request, rule action, log destination, exception and rollout scope.

© 2026 Tertiary Infotech Academy Pte Ltd

Edge and network protection: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Edge and network protection: design decision

? SCENARIO

A company must deploy baseline WAF controls to every internet-facing application across accounts.

1 DESIGN RESPONSE

Use managed rules in count mode first, central policies, rate limits, logging and exception workflows.

2 PRIMARY SERVICES

WAF · Firewall Manager · Shield · Network Firewall

3 FAILURE MODE

Blocking without observing false positives can turn a security control into an outage.

4 PROOF

Policy version, sampled request, rule action, log destination, exception and rollout scope.

© 2026 Tertiary Infotech Academy Pte Ltd

Edge and network protection: failure diagnostic

1

RECOGNISE

Observable failure condition: Blocking without observing false positives can turn a security control into an outage.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Observe traffic through Monitor; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Policy version, sampled request, rule action, log destination, exception and rollout scope.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 6 · APPLIED SCENARIO

Edge and network protection: worked decision

1

CONSTRAINT

A company must deploy baseline WAF controls to every internet-facing application across accounts.

2

DECISION

Use managed rules in count mode first, central policies, rate limits, logging and exception workflows.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: Blocking without observing false positives can turn a security control into an outage.

4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Policy version, sampled request, rule action, log destination, exception and rollout scope.

© 2026 Tertiary Infotech Academy Pte Ltd

Security blueprint and audit evidence

1

OUTCOME

Translate risk and compliance obligations into components, controls, ownership and evidence flows.

2

ARCHITECTURE

Document trust boundaries, data classification, preventive/detective controls, logging, retention and incident responsibilities.

3

FAILURE MODE

A service list without data flows, control intent and ownership is not an actionable blueprint.

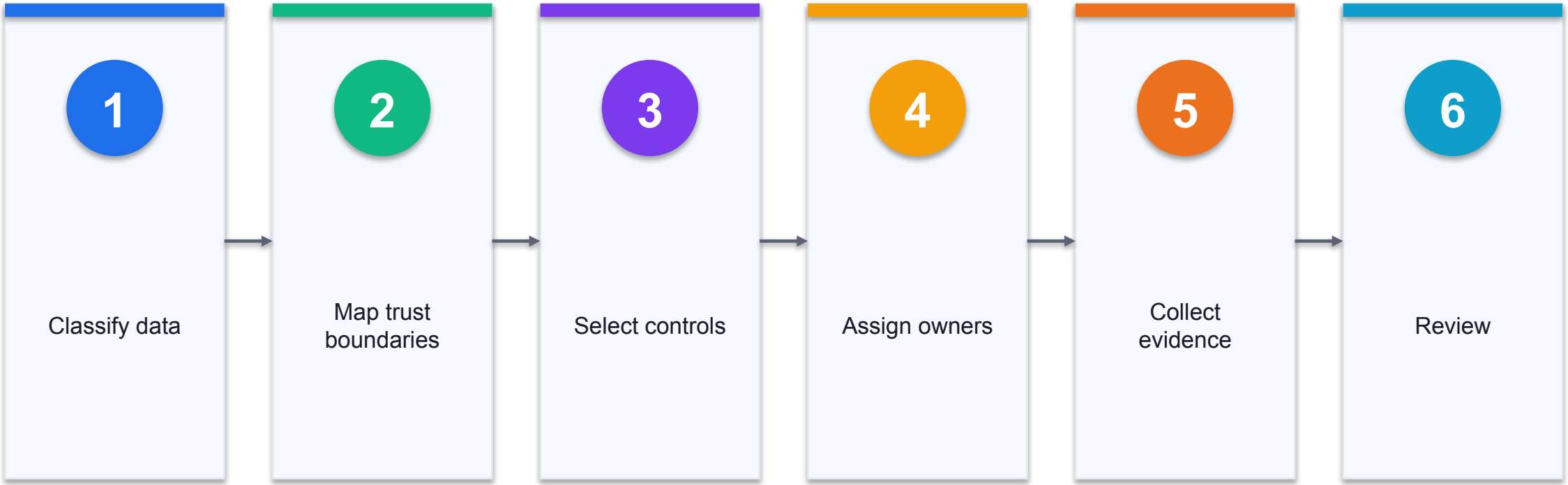
4

EVIDENCE

Architecture diagram, control matrix, evidence source, owner, retention rule, exception and review cadence.

© 2026 Tertiary Infotech Academy Pte Ltd

Security blueprint and audit evidence: operating flow



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Security blueprint and audit evidence: design decision

? SCENARIO

A regulated workload needs a technical blueprint that developers, security and auditors can all execute.

1 DESIGN RESPONSE

Document trust boundaries, data classification, preventive/detective controls, logging, retention and incident responsibilities.

2 PRIMARY SERVICES

Security Hub · Audit Manager · Artifact · Tagging

3 FAILURE MODE

A service list without data flows, control intent and ownership is not an actionable blueprint.

4 PROOF

Architecture diagram, control matrix, evidence source, owner, retention rule, exception and review cadence.

© 2026 Tertiary Infotech Academy Pte Ltd

Security blueprint and audit evidence: failure diagnostic

1

RECOGNISE

Observable failure condition: A service list without data flows, control intent and ownership is not an actionable blueprint.

2

CONTAIN

Pause promotion, preserve evidence and reduce impact without destroying diagnostic state.

3

DIAGNOSE

Trace the workflow from Classify data through Review; compare intended configuration with runtime state.

4

RECOVER

Restore the last known-good state, then prove recovery with Architecture diagram, control matrix, evidence source, owner, retention rule, exception and review cadence.

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 6 · APPLIED SCENARIO

Security blueprint and audit evidence: worked decision

1

CONSTRAINT

A regulated workload needs a technical blueprint that developers, security and auditors can all execute.

2

DECISION

Document trust boundaries, data classification, preventive/detective controls, logging, retention and incident responsibilities.

3

WHY ALTERNATIVES LOSE

Reject options that do not change the stated failure boundary: A service list without data flows, control intent and ownership is not an actionable blueprint.

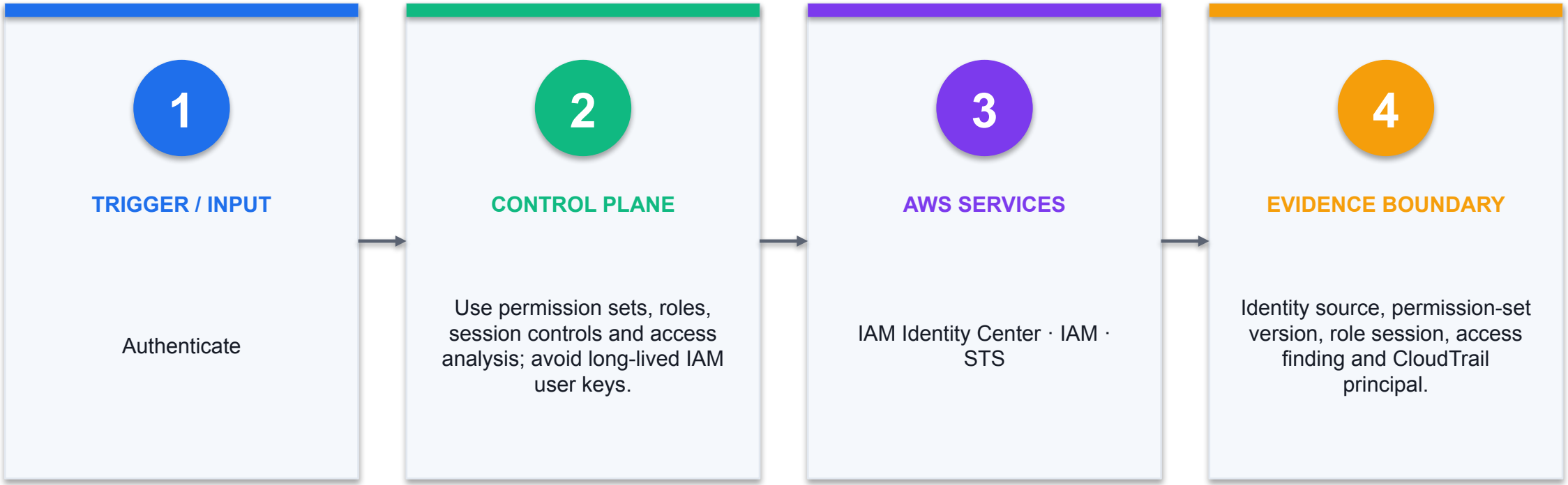
4

ACCEPTANCE TEST

The decision is complete only when this proof exists: Architecture diagram, control matrix, evidence source, owner, retention rule, exception and review cadence.

© 2026 Tertiary Infotech Academy Pte Ltd

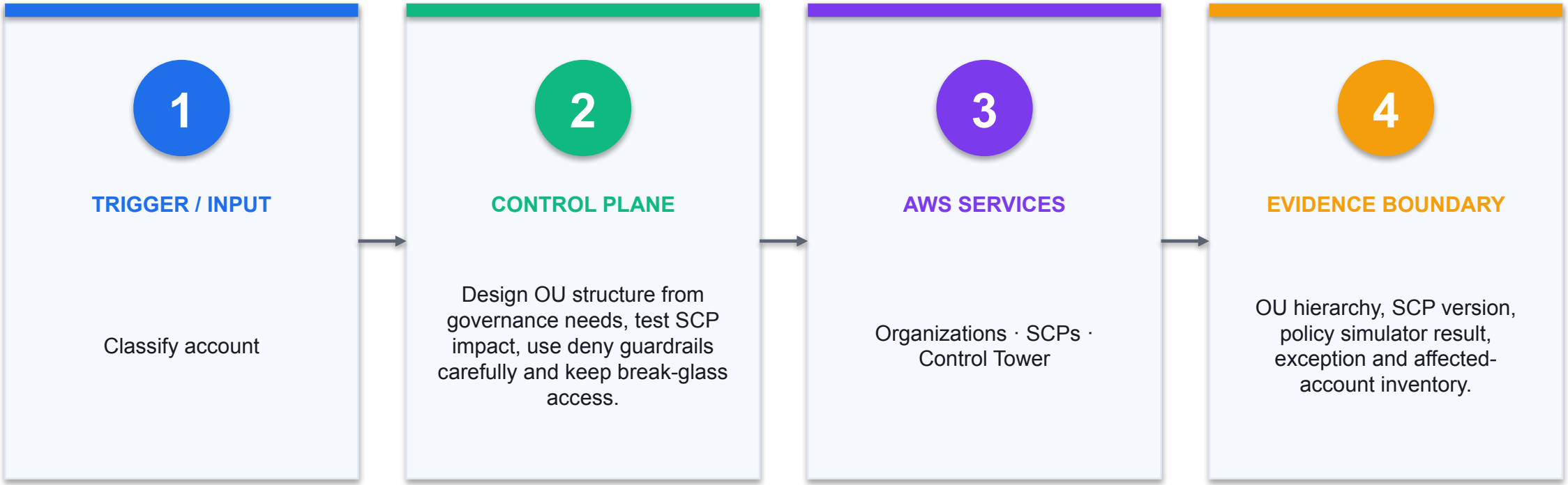
Identity at scale: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

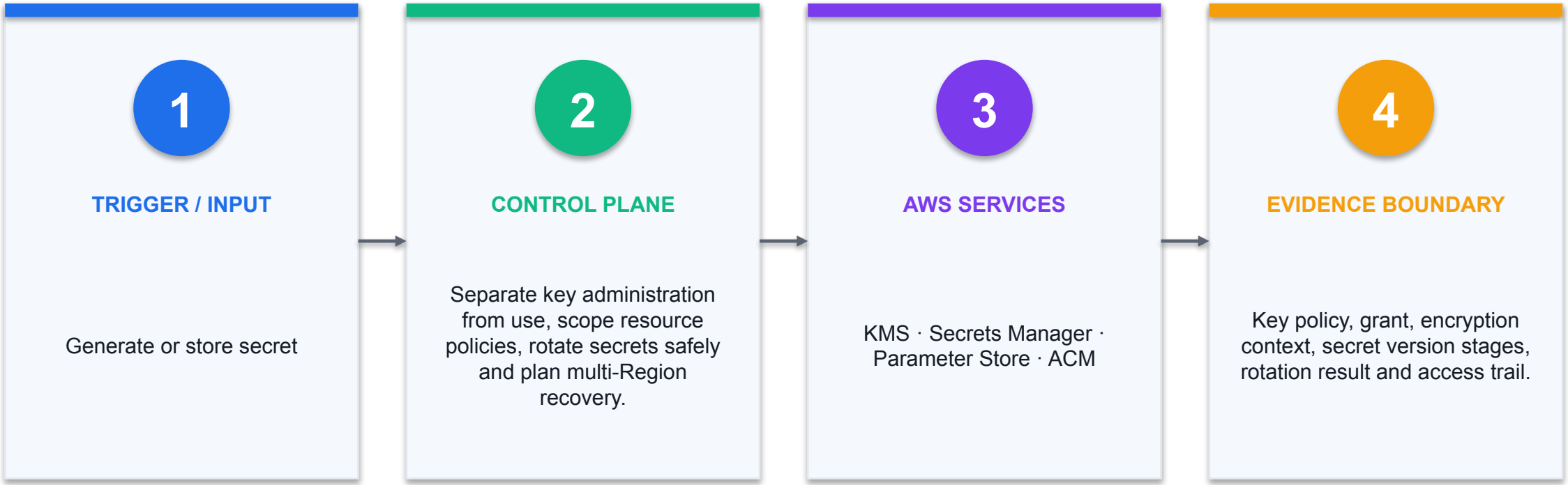
Organizations and preventive guardrails: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

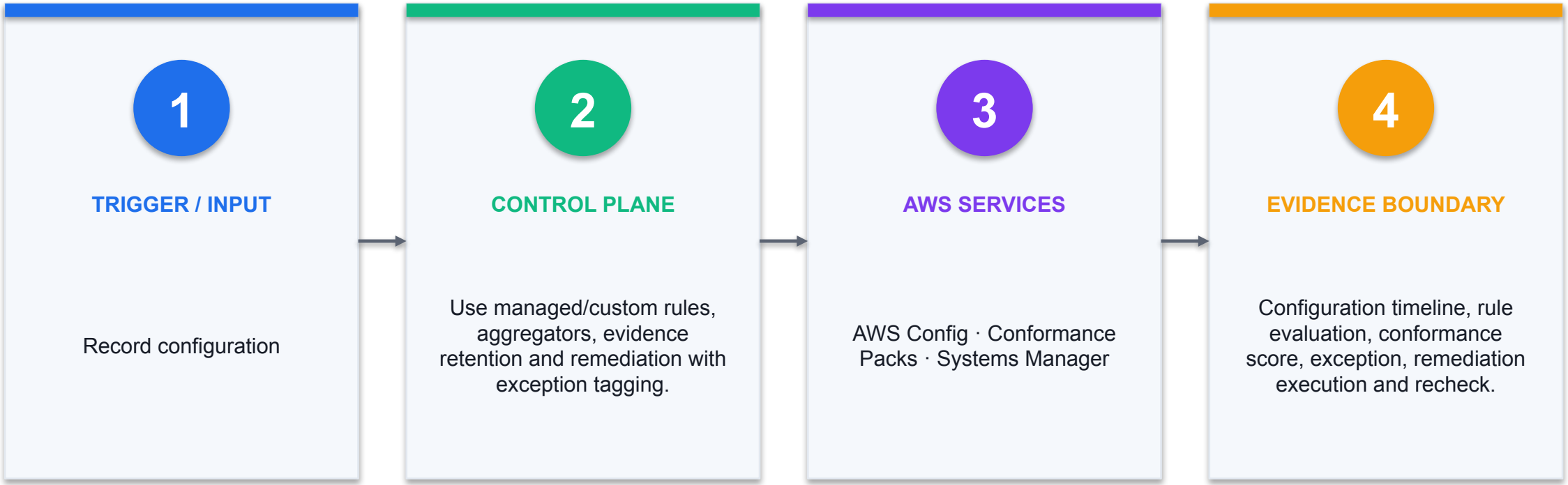
Secrets, keys and data protection: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

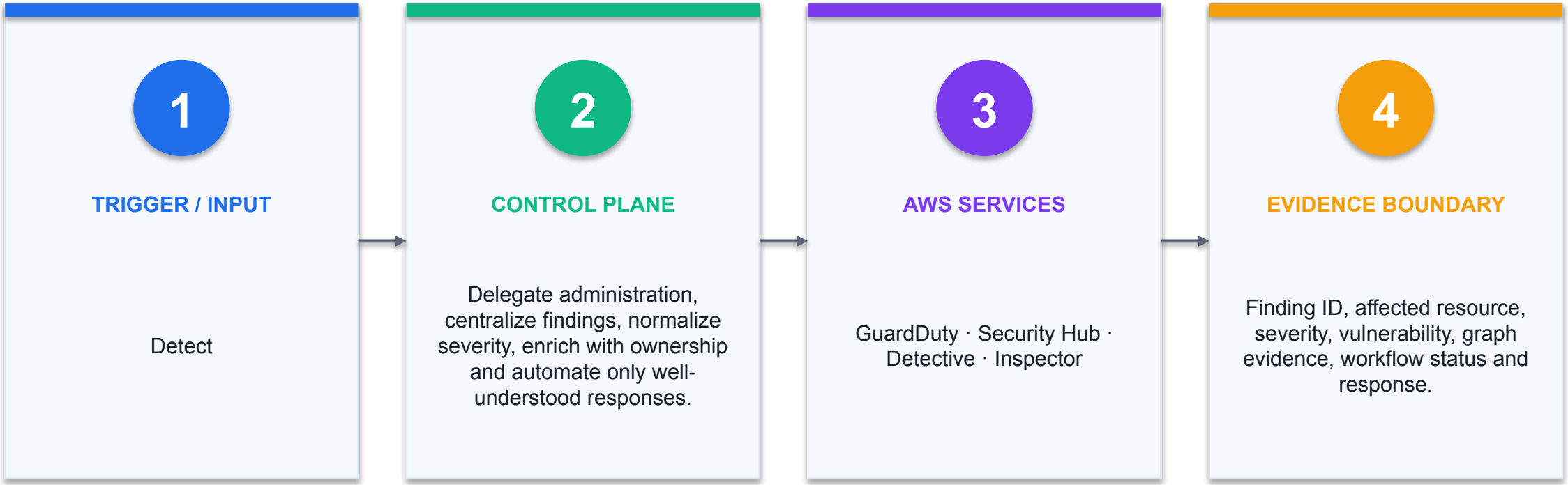
Config rules and conformance: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

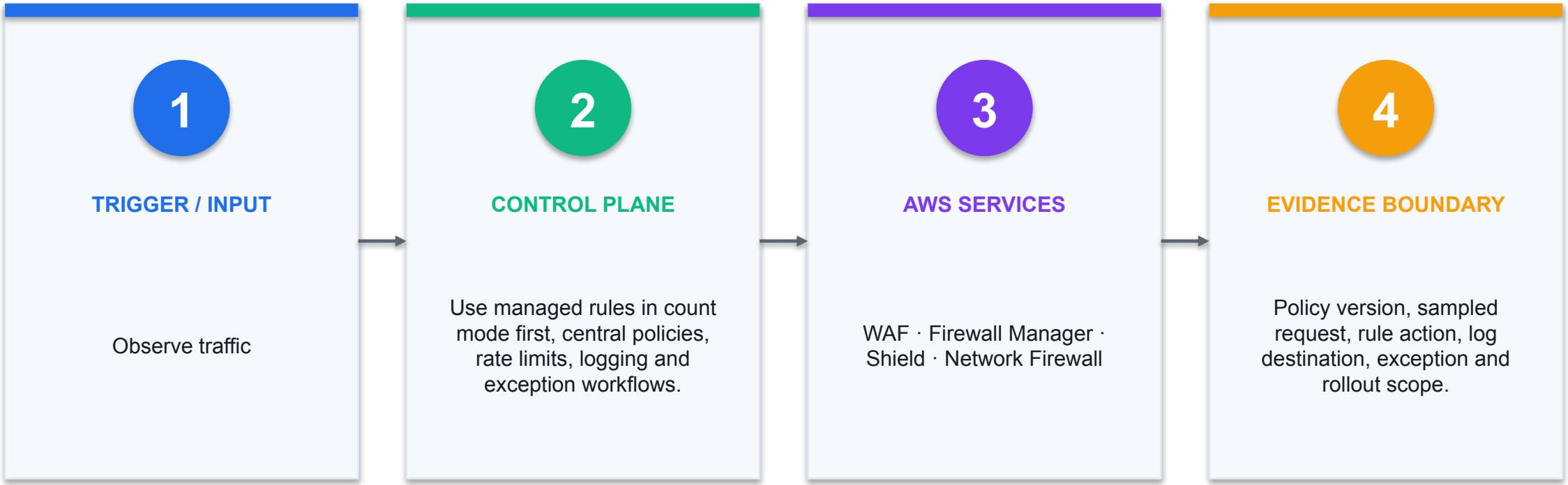
Threat detection and investigation: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

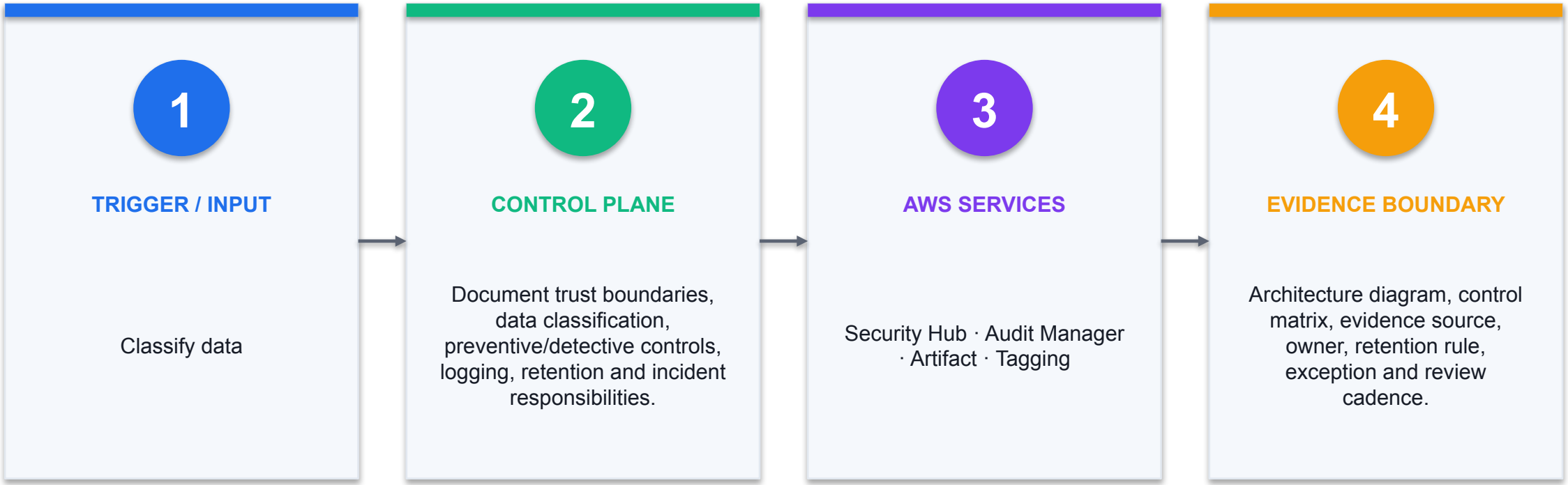
Edge and network protection: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Security blueprint and audit evidence: architecture map



OPERATING PRINCIPLE Treat the stages as one controlled system; evidence from each stage informs the next decision.

© 2026 Tertiary Infotech Academy Pte Ltd

Enforce IAM Least Privilege and Analyse Policies for Over-Permissive Access

LAB 21

Day 5 · LO6 · K6, A4

Broad permissions are the single most common cloud security failure.

YOU'LL BUILD

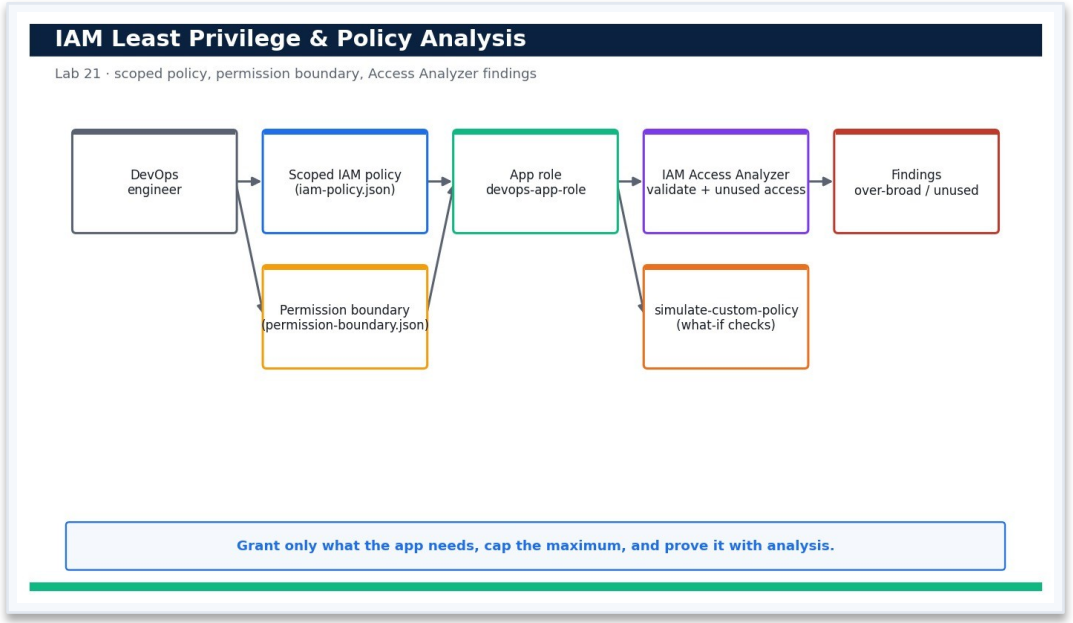
A least-privilege IAM policy for the app role, capped by a permission boundary, that passes Access Analyzer validation with no security warnings and grants nothing beyond the app's real needs.

Tools: AWS IAM, IAM Access Analyzer, AWS STS, AWS CloudFormation

OUTCOME & EVIDENCE

Outcome: Write a scoped IAM policy, cap it with a permission boundary, and prove least privilege with IAM Access Analyzer.

Evidence: The output showing zero SECURITY_WARNING findings for iam-policy.json.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Protect Data with Secrets Manager and Customer-Managed KMS Encryption

LAB 22

Day 5 · LO6 · K6, A4

Sensitive data must be encrypted with keys you control and secrets must never live in code.

YOU'LL BUILD

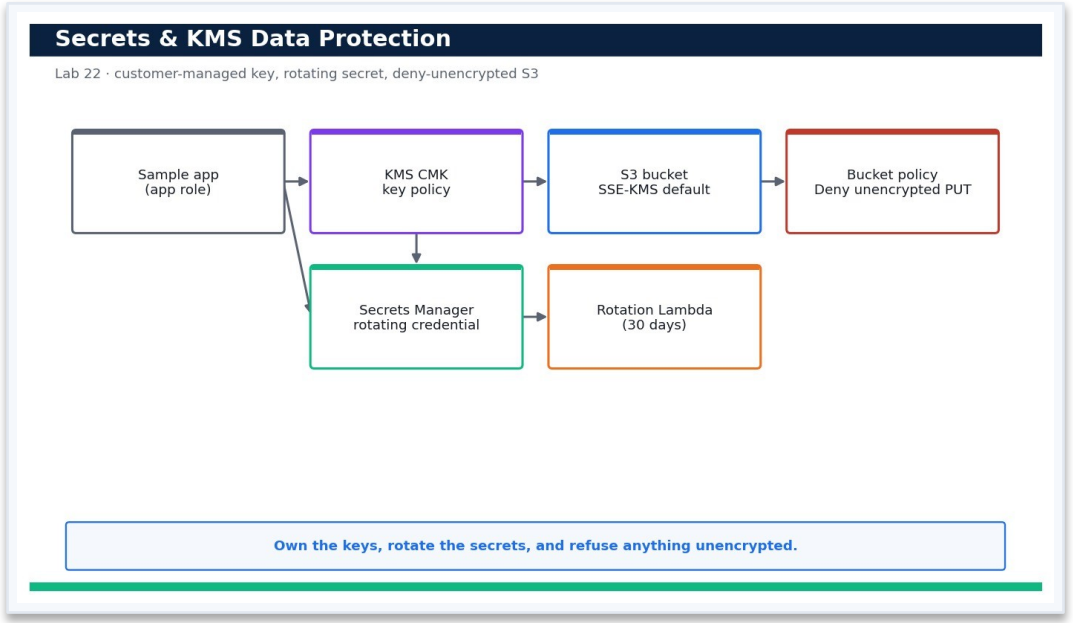
A customer-managed KMS key with a scoped key policy, a rotating Secrets Manager secret, and an S3 bucket that encrypts by default with your key and rejects any unencrypted upload.

Tools: AWS KMS, AWS Secrets Manager, Amazon S3, AWS Lambda (rotation), AWS CloudFormation

OUTCOME & EVIDENCE

Outcome: Create a customer-managed KMS key with a key policy, store a secret with rotation, and enforce encryption on S3.

Evidence: describe-key output showing KeyManager CUSTOMER, KeyState Enabled and rotation enabled for alias/devops-app.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Produce CloudTrail Audit Evidence and Answer Who-Did-What-When

LAB 23

Day 5 · LO6 · K6, A4

Auditors and incident responders need a tamper-evident record of every API call.

YOU'LL BUILD

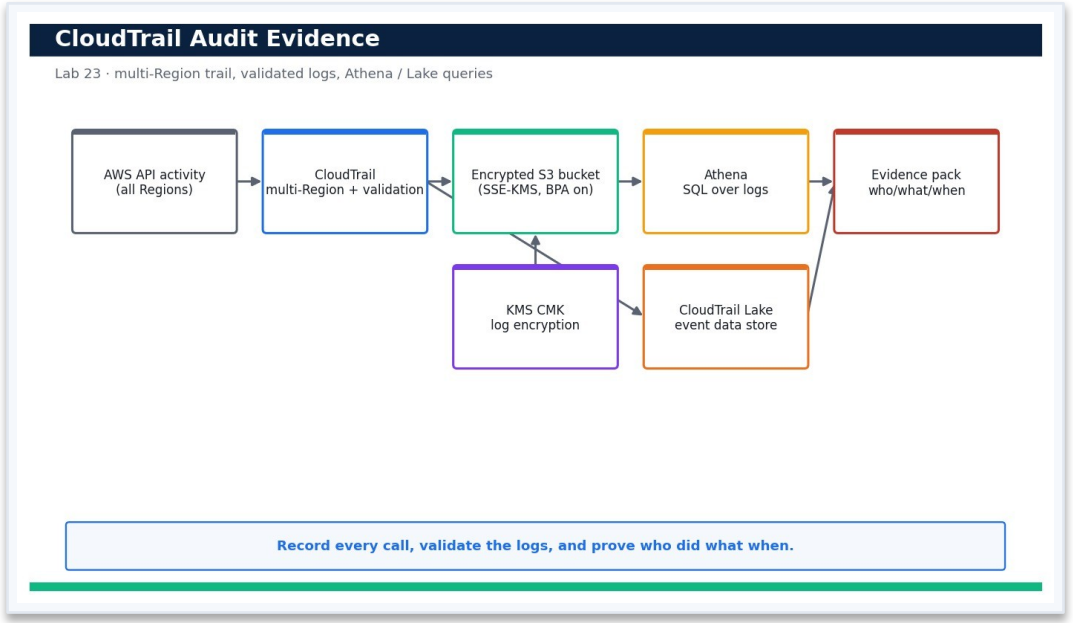
A validated, multi-Region CloudTrail delivering encrypted logs to S3, plus Athena/CloudTrail Lake queries that answer who-did-what-when and an exported evidence pack.

Tools: AWS CloudTrail, Amazon S3, Amazon Athena, CloudTrail Lake, AWS KMS

OUTCOME & EVIDENCE

Outcome: Stand up a multi-Region CloudTrail with log-file validation and query it to export audit evidence.

Evidence: describe-trails output showing multi-Region true, log-file validation true and a KMS key on devops-trail.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Run Continuous Compliance with AWS Config and Security Hub

LAB 24

Day 5 · LO6 · K6, A4

Compliance is not a one-off audit — it must run continuously.

YOU'LL BUILD

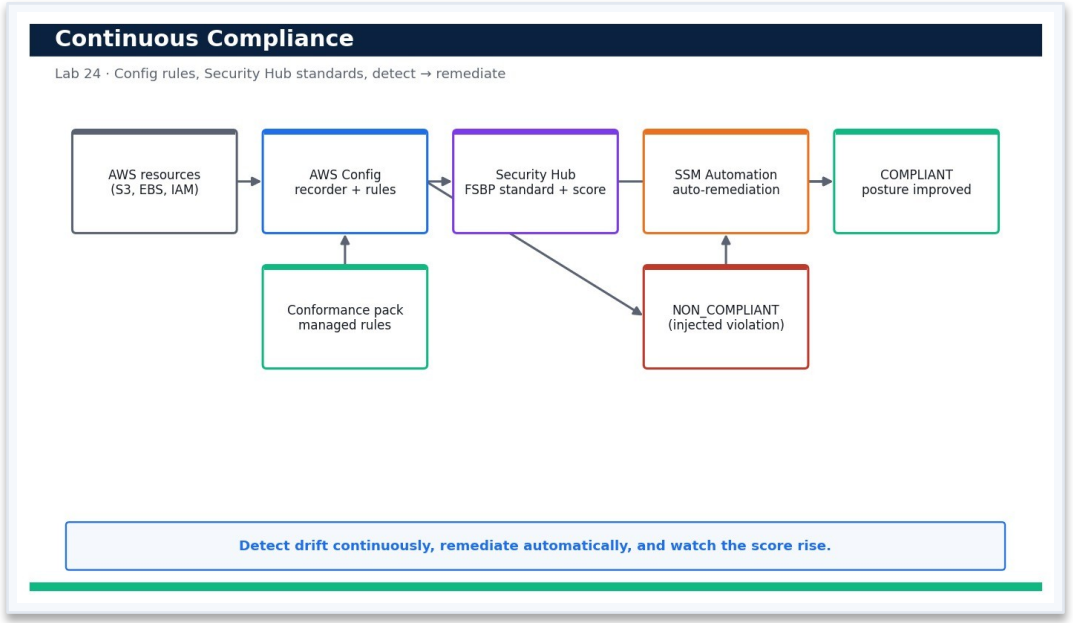
A continuously evaluating AWS Config + Security Hub setup that detects an injected violation as NON_COMPLIANT, auto-remediates it to COMPLIANT, and shows the posture score improve.

Tools: AWS Config, AWS Security Hub, AWS Systems Manager Automation, Amazon S3, AWS CloudFormation + 1 more

OUTCOME & EVIDENCE

Outcome: Enable AWS Config managed rules and Security Hub standards, inject a violation and remediate to COMPLIANT.

Evidence: describe-configuration-recorder-status showing recording true and get-enabled-standards showing FSBP enabled.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

Capstone — Blueprint, Well-Architected Review and Evidence Pack for the Whole Delivery System

LAB 25

Day 5 · LO1–LO6 · K1, K2, K3, K4, K5, K6, K7, A1, A2, A3, A4, A5, A6

This capstone ties the entire course together.

YOU'LL BUILD

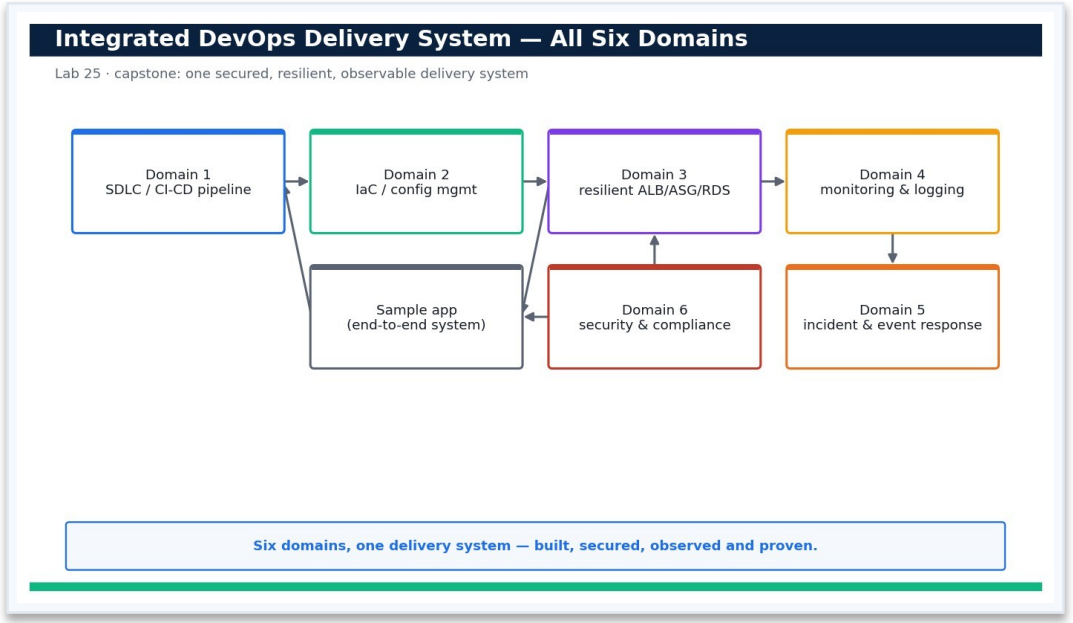
A completed technical blueprint, a Well-Architected pillar review with scores and actions, and an evidence pack that proves the whole secured/resilient/observable delivery system against LO1–LO6.

Tools: AWS CodePipeline/CodeBuild/CodeDeploy, CloudFormation, EC2/ALB/Auto Scaling, RDS, CloudWatch + 9 more

OUTCOME & EVIDENCE

Outcome: Synthesise Labs 1–24 into one secured, resilient, observable delivery system with a blueprint, an operational review and an evidence pack.

Evidence: The completed blueprint.md with all six domain sections filled and cross-referenced to the labs.



Reference architecture

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN RECAP

Recap — Security and Compliance

1

Lab 21

Write a scoped IAM policy, cap it.

2

Lab 22

Create a customer-managed KMS key with a key policy, store a secret with rotation.

3

Lab 23

Stand up a multi-Region CloudTrail with log-file validation and query it to export audit evidence.

4

Lab 24

Enable AWS Config managed rules and Security Hub standards, inject a violation and remediate to COMPLIANT.

5

Lab 25

Synthesise Labs 1–24 into one secured, resilient, observable delivery system.

6

Key principle

IAM users, roles, policies and permission boundaries enforce least privilege; roles beat long-lived keys.

© 2026 Tertiary Infotech Academy Pte Ltd

PUTTING IT ALL TOGETHER

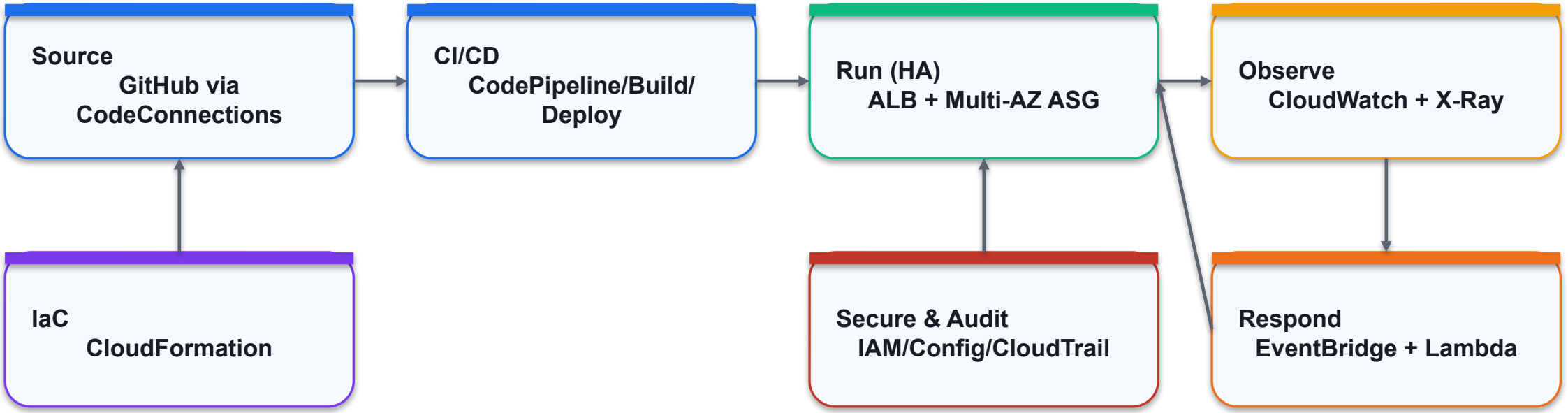
Cross-Domain Integration

The six domains are one system — a single sample application delivered, run and secured end to end

© 2026 Tertiary Infotech Academy Pte Ltd

ALL SIX DOMAINS AS ONE SYSTEM

The Sample Application, End to End



▶ Domains 1–6 map onto one delivery-to-operations loop: build it (1), codify it (2), make it resilient (3), watch it (4), respond (5) and secure it (6).

© 2026 Tertiary Infotech Academy Pte Ltd

ONE CONNECTED SYSTEM · 5 LABS × 5 DAYS

How the 25 Labs Connect

1

Day 1 → Day 2

The CI/CD pipeline built across Labs 1–5 deploys the Infrastructure as Code defined in Labs 6–10.

2

Day 2 → Day 3

The network and app stacks become the highly available, auto-scaling, DR-ready foundation (Labs 11–15).

3

Day 3 → Day 4

The running fleet emits the metrics, logs and events monitored and routed in Labs 16–20.

4

Day 4 → Day 5

Alarms and events drive automated response, then least-privilege, encryption and audit (Labs 21–24).

5

Labs → Capstone

Lab 25 integrates all six domains into one secured, resilient, observable blueprint + evidence pack.

6

One system

Each lab produces an artifact the next builds on — 25 connected labs across 5 days, not isolated exercises.

© 2026 Tertiary Infotech Academy Pte Ltd

TRACING ONE EVENT ACROSS ALL SIX DOMAINS

Capstone Scenario — A change, a failure, a recovery

SITUATION

A developer merges a change that passes tests but degrades checkout latency once at production scale.

ANALYSIS

- Domain 1: the pipeline built and promoted an immutable artifact with a canary stage.
- Domain 3/4: canary p99 latency breaches the SLO; the CloudWatch alarm fires within minutes.
- Domain 5: the alarm triggers CodeDeploy auto-rollback and notifies the on-call via EventBridge/SNS.
- Domain 6: CloudTrail records exactly who changed what, feeding the blameless review.
- Domain 2: the durable fix is encoded in the template/config and re-applied, not hand-patched.

DECISION

No 2am heroics: the system detects, rolls back and records the event automatically. The follow-up hardens the canary threshold and adds a regression test.

© 2026 Tertiary Infotech Academy Pte Ltd

MAPPING A NEED ONTO THE DOMAINS

From Business Requirement to Six-Domain Design

'Ship weekly without downtime'	Domain 1 CI/CD + blue/green; Domain 3 multi-AZ so releases don't drop traffic.
'Recover an AZ loss in minutes'	Domain 3 Multi-AZ ASG + RDS failover; Domain 4 alarms to confirm health.
'Know within a minute when it breaks'	Domain 4 golden-signal alarms; Domain 5 event-driven notify + remediate.
'Prove we are compliant'	Domain 6 Config + CloudTrail; Domain 2 guardrails as code in every account.

TAKEAWAY

Real requirements cut across domains — a competent DevOps engineer composes controls from all six into one coherent design.

© 2026 Tertiary Infotech Academy Pte Ltd

A RECURRING DEVOPS TRADE-OFF

Decision — Build It Yourself or Use a Managed Service?

A — Managed service

Default choice. Less undifferentiated heavy lifting (e.g. RDS vs self-run DB) — at higher unit cost and less control.

B — Self-managed on EC2

Only when you need control the managed option can't give (custom kernel, licensing). More ops burden and patching you own.

C — Serverless-first

For spiky, event-driven work. Pay-per-use and minimal ops — within the service's limits and cold-start profile.

© 2026 Tertiary Infotech Academy Pte Ltd

WRAP-UP

Course Summary & Next Steps

© 2026 Tertiary Infotech Academy Pte Ltd

LEARNING OUTCOMES

What You Achieved

1

LO1

Develop an AWS CI/CD pipeline proof of concept.

2

LO2

Identify the technical and practical requirements in an AWS cloud infrastructure setup.

3

LO3

Develop a resilient cloud solution based on AWS cloud components.

4

LO4

Implement regular AWS monitoring and perform necessary troubleshooting.

5

LO5

Demonstrate solutions to incidents and business problems on the AWS cloud.

6

LO6

Prepare a technical blueprint to address security and compliance.

© 2026 Tertiary Infotech Academy Pte Ltd

NEXT STEPS

Continue Your AWS DevOps Journey

1

Rebuild the labs

Redo all six labs from memory until the AWS workflows are automatic.

2

Automate more

Add tests, canary deploys and IaC to your own projects.

3

Well-Architected reviews

Run Well-Architected reviews on real workloads.

4

Keep learning

Explore ECS/EKS, Step Functions, and multi-account governance.

© 2026 Tertiary Infotech Academy Pte Ltd

WRAP-UP

Assessment

- Written Assessment (SAQ) — 1 hour. Practical Performance (PP) — 2 hours.
- Open book: slides, Learner Guide and approved materials only.
- Remember to take the Assessment digital attendance (TRAQOM).
- Submit your completed answers on the LMS at https://lms-tms.tertiaryinfotech.com/.

© 2026 Tertiary Infotech Academy Pte Ltd

ON ASSESSMENT DAY

Assessment Flow

1. TRAQOM survey — scan the QR code on the LMS
2. Assessment digital attendance — scan the SSG QR
3. Sit WA (SAQ) then PP — open book
4. Submit your answers on the LMS
5. Sign the Assessment Summary Record

© 2026 Tertiary Infotech Academy Pte Ltd

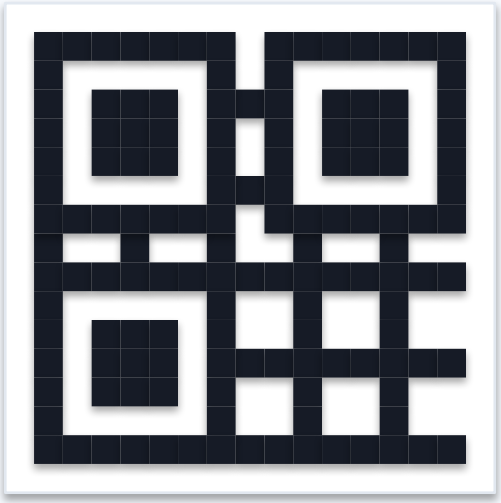
Digital Attendance (Mandatory)

Take the SSG digital attendance at three points on every funded day:

AM	PM	ASSESSMENT
Start of the morning session	Start of the afternoon session	Before the Day 5 assessment

1. The trainer/administrator displays the SSG digital-attendance QR code from the portal.
2. Scan the QR code with your mobile phone camera and submit your attendance.
3. It is mandatory for AM, PM and the Assessment sessions on WSQ-funded courses.

SCAN TO SUBMIT ATTENDANCE



SSG digital-attendance QR — displayed live from the portal by the trainer

Minimum 75% attendance on the SSG Digital Attendance record is required to be eligible for assessment and funding.

© 2026 Tertiary Infotech Academy Pte Ltd

WELL DONE

Thank You!

You can now plan, automate, operate and secure DevOps solutions on AWS across the full delivery lifecycle.

© 2026 Tertiary Infotech Academy Pte Ltd